

CME295 学习笔记

Transformers & LLMs · Stanford 2025 秋

非官方个人笔记 · 根据公开课视频字幕整理 · 图为自绘示意

2026-09-24

CME295 学习笔记

Stanford CME295: Transformers & Large Language Models (2025 秋) ·
讲者 Afshine Amidi、Shervine Amidi 非官方个人笔记，根据 YouTube 公开
课视频的字幕整理 · [播放列表](#) · [课程大纲](#)

一句话：这门课把大模型从底到顶走一遍——模型长什么样（第 1–3 讲），怎么训出来、怎么调到听话（第 4–5 讲），怎么想问题、怎么动手做事（第 6–7 讲），怎么量它好不好（第 8 讲），最后回顾并看趋势（第 9 讲）。CS329A 默认你已经懂这些，这门课正好把那些默认的词一个个讲清楚。

课程地图

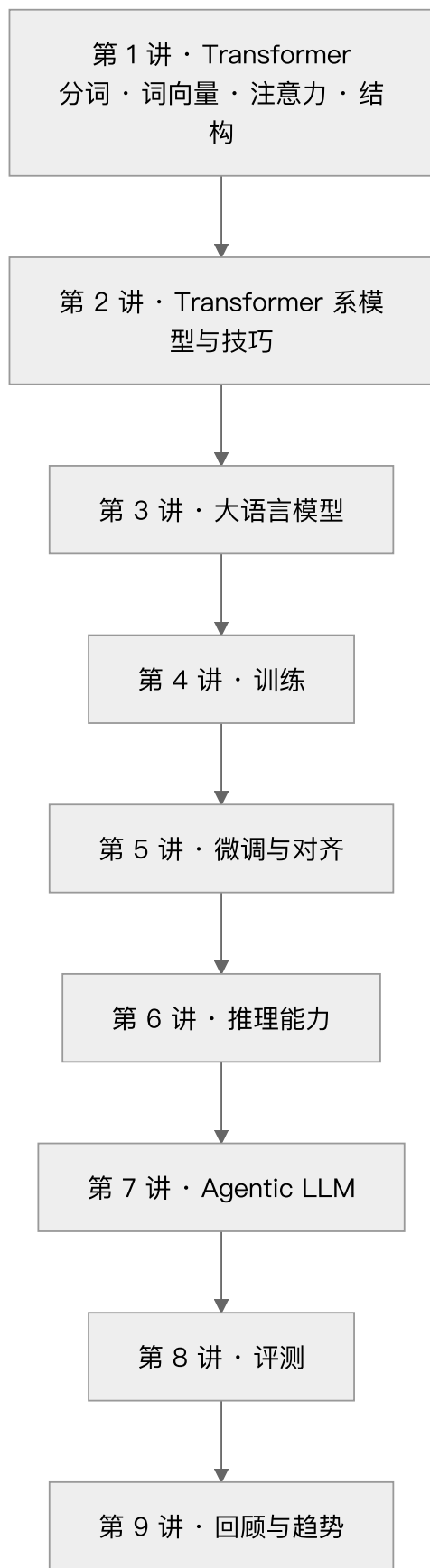


图 0-1 | 九讲的顺序就是一个模型从结构到落地的顺序（自绘示意）

各讲索引

讲	主题	视频
第 1 讲	Transformer：NLP 背景、分词、词向量、RNN、注意力、完整结构	YouTube
第 2 讲	Transformer 系模型与技巧：位置嵌入、注意力变体、几类模型家族	YouTube
第 3 讲	大语言模型：从 Transformer 到 LLM	YouTube
第 4 讲	LLM 训练	YouTube
第 5 讲	LLM 微调：偏好数据、奖励模型、RLHF 与 PPO、Best-of-N、DPO	YouTube
第 6 讲	LLM 推理能力	YouTube
第 7 讲	Agentic LLM	YouTube
第 8 讲	LLM 评测：人工评分、规则指标、LLM-as-a-judge、benchmark、agent 评测	YouTube
第 9 讲	回顾与当前趋势	YouTube

和 CS329A 怎么对照

这门课	CS329A 里对应的内容	关系
第 5 讲 微调与对齐	第 3 讲验证器、第 6 讲 RL	这里讲 RLHF、PPO、DPO 的基础；那边讲研究前沿
第 6 讲 推理能力	第 2 讲测试时计算、第 6 讲 GRPO	同一批想法，这里从头解释
第 7 讲 Agentic LLM	第 4 讲工具反馈、第 5 讲规划、第 7 讲 Search-o1	这里讲 RAG、工具调用、agent 循环的机制
第 8 讲 评测	第 8 讲智能体评测	这里讲指标和 judge 的基本功；那边讲长程任务的评测

怎么用这份笔记

- 每讲的结构是固定的：一句话 → 时间轴 → 核心内容（带图和公式） → 关键图表速查 → 提到的工作 → 术语对照 → 字幕勘误 → 带走的问题。赶时间可以只读"一句话"、图和"带走的问题"。

- **时间戳都能点**，直接跳到视频里对应的位置。图下面黄色的「▶ 看原幻灯片」会跳到老师讲那一页 PPT 的时刻。
- **图是重新画的示意图**，用来解释机制，不是课件截图。
- **公式单独成块**，后面跟一句话解释每个符号。
- "小注" 是补充的课外事实或对口误的更正；"应为 / 应出自" 表示这是根据内容推断的，老师没有明说。
- 这门课的英文字幕多为自动生成，人名和缩写错得不少，每讲末尾的"字幕勘误"列了会影响理解的那些。想对照原文：在播放器「设置 → 字幕 → 自动翻译 → 中文（简体）」可以开中文字幕。

CME295 第 1 讲 | Transformer

Stanford CME295: Transformers & Large Language Models (2025 秋) · 第 1 讲, 2025 年 9 月 26 日 视频: <https://www.youtube.com/watch?v=Ub3GoFaUcDs> (1:41:59, 自带英文 CC, 质量不错, 个别词仍有误) 讲者: Afshine Amidi (主讲) · Shervine Amidi (1:29:30 起的详细例子)。两人是双胞胎, 都毕业于 Centrale Paris, 之后一个去 MIT、一个读 Stanford ICME 硕士, 又先后在 Uber、Google 任职, 现在都在 Netflix 做 LLM 课程大纲: <https://cme295.stanford.edu/syllabus/> · 教材是两位讲者自己写的 Super Study Guide: Transformers & Large Language Models, GitHub 上另有浓缩版 VIP cheatsheet (已译成多种语言)

小注: 大纲页现在挂的是 2026 秋季版课表, 讲次和主题已经重排, 和这套 2025 秋录像的九讲不一一对应。本笔记里说的“第 N 讲”都指 2025 秋录像的编号。

一句话: 模型只认数字, 所以处理文本的第一步是把它切成 token、再把 token 变成向量; 这一讲沿着 one-hot → word2vec → RNN / LSTM → attention 的路线, 说明每一步补了上一步的什么缺陷, 最后落到 2017 年的 Transformer——丢掉循环结构, 让每个 token 通过 query / key / value 直接“看”全句——并用一个英译法的例子, 把 encoder-decoder 从输入到输出完整走了一遍。

时间轴

时间	内容
0:05	开场: 两位讲者的背景; 课程由来 (workshop → 正式课程)、两个目标、先修要求
3:54	课程安排: 上课时间、录像、考核方式、教材; 问答 (不考编程、没有作业)
9:40	两个提示 (幻灯片标出处、缩写多); NLP 的三类任务
14:12	分类: 情感分析; accuracy / precision / recall / F1 与类别不平衡
16:48	token 级分类: NER, 指标按 token 或实体类别统计
17:50	生成: 机器翻译与 WMT; BLEU / ROUGE 依赖参考答案; perplexity
21:03	NLP 简史: RNN → LSTM → word2vec → Transformer → LLM
22:57	Tokenization: 任意切、词级、子词级、字符级各自的得失
28:27	三种切法小结; OOV
30:28	词表示: one-hot 编码、cosine similarity, one-hot 为什么不行
33:13	问答: 比相似度为什么不看向量长度

时间	内容
35:39	问答：词表该多大，多语言怎么办
37:49	word2vec: CBOW、skip-gram 与代理任务的思想
40:28	玩具网络走查：V = 6、d = 2，预测下一个词
46:34	问答：V 是什么、unknown token；何时停训、EOS、隐层维度怎么定；同形异义词
53:23	RNN：隐状态逐词传递；三类任务各怎么用
59:09	RNN 的毛病：长距离依赖、LSTM 的 cell state、梯度消失
1:03:17	小结：word2vec 与 RNN 各缺什么；RNN 还很慢
1:04:52	attention (2014)：预测时直接回看输入
1:06:47	Transformer (2017) 与 self-attention：每个 token 直连全句
1:09:02	query / key / value；写成矩阵，对 GPU 友好
1:12:12	问答：key 和 value 到底是什么
1:13:53	架构：encoder 与 decoder；cross-attention 小测验；masked self-attention
1:20:11	位置编码；从输入到输出的高层走查
1:22:46	问答：head 是什么；multi-head 与卷积核的类比
1:25:23	label smoothing 及问答
1:29:53	详细例子 (Shervine)：切分、BOS / EOS、embedding 加位置编码
1:31:42	encoder 的 self-attention：三个投影矩阵、公式逐步解读、为什么除以 $\sqrt{d_k}$
1:34:56	multi-head 拼接与 W_O；问答：各 head 为什么不会学成一样
1:37:31	FFN 的隐层更宽；encoder、decoder 各叠 N 层
1:38:36	解码：BOS $\rightarrow$ masked self-attention $\rightarrow$ cross-attention $\rightarrow$ FFN $\rightarrow$ softmax $\rightarrow$ 回填，直到 EOS

核心内容

1. 课程定位与这一讲的主线



图 1-1 | 本讲的主线：每一步都在补上一步的缺陷（自绘示意）·

- **课程由来与目标：**两位讲者 2020 年起专做 NLP（自然语言处理），这门课原本是每年一次的 workshop；ChatGPT 之后需求大涨，2025 年春季第一次作为 Stanford 正式课程开设，这学期是第二次。目标有两个：弄懂让 LLM（large language model，大语言模型）运转起来的底层机制，也就是 Transformer；弄懂 LLM 是怎么训练的、用在哪里。
- **面向谁、要什么基础：**想当 research scientist 或 ML scientist 的人、想在自己的项目里用 LLM 并了解其坑的人、想把 LLM 搬到本领域的外行，都适合。先修只要 ML 基础（知道模型怎么训练、神经网络是什么）和一点线性代数（会矩阵乘法）。考核是期中、期末各 50%，没有作业，不考编程，只考课上的概念。
- **两个阅读提示：**每张幻灯片底部都标了出处，方便顺藤摸瓜；这个领域缩写极多，讲者希望学完后你脑子里有一张“缩写 → 含义”的对照表（本笔记末尾的术语表就是干这个的）。
- **这一讲的主线（图 1-1）：**先把文本切开（tokenization），再把每一块变成向量（embedding）；此后所有的演进，都是为了让这些向量“知道自己的上下文”，同时还要算得快。

2. NLP 的三类任务，以及各自怎么评测

NLP 就是对文本做计算。讲者按“输出长什么样”把任务分成三个桶：

类别	输入 → 输出	课上的例子	数据	指标
classification（分类）	一段文本 → 一个标签	情感分析；意图识别（“帮我定个明天的闹钟” → 创建闹钟）；语种识别；主题分类	IMDb 影评、Amazon 商品评论、推文	accuracy、precision、recall、F1
multi-classification（讲者的叫法）	一段文本 → 每个 token 一个标签	NER（命名实体识别，标出哪些词是地点、时间等）；词性标注；依存 / 成分句法分析	课上没提	同上，但按 token 或按实体类别统计
generation（生成）	文本 → 长度事先未知的文本	机器翻译；问答（ChatGPT、Gemini 这类助手）；摘要；写代码、写诗	WMT：成对的双语句子，如欧洲议会的英法、英德语料	BLEU、ROUGE、perplexity

- **分类为什么要好几个指标：**accuracy 是预测对的比例；precision 是“预测为正的里面有多少真是正”；recall 是“真正的正例里找回了多少”；F1 是这两者的调和平均。类别极不平衡时（比如 99% 都是正

例)，一个永远猜多数类的模型 accuracy 也有 99%，却毫无用处，这时就得看 precision 和 recall。

- **生成任务为什么难评：**同一句话有很多种正确译法，成对的数据也更难拿到。BLEU、ROUGE 这类基于规则的指标，都是拿模型输出去和人工写的参考译文（reference）比对，分数越高越好；代价是必须先有参考答案，而标注又贵又慢。讲者预告：借助 LLM，后面可以摆脱参考答案，走向 reference-free 的评测（应在第 8 讲 LLM evaluation）。perplexity 只看模型自己输出的概率，衡量模型对这段文本有多“意外”，越低越好。

小注：讲者说的 multi-classification，更通行的名字是 sequence labeling 或 token classification（序列标注）。BLEU 全称 Bilingual Evaluation Understudy（Papineni et al., 2002），ROUGE 全称 Recall-Oriented Understudy for Gisting Evaluation（Lin, 2004）；讲者顺带调侃这两个名字正好凑成法语的“蓝”和“红”。perplexity 的定义是“每个 token 平均负对数似然”取指数。

- **一段简史：**RNN 的想法 1980 年代就有，LSTM 出现在 1990 年代，但那时既没有互联网规模的数据，也没有足够的算力；2013 年 word2vec 让“有意义的词向量”流行起来；2017 年的 Transformer 论文是今天所有主流模型的地基；此后在算力和数据两头不断放大，就成了 2020 年代的 LLM。

3. Tokenization：先决定把文本切成多大的块

- **为什么需要：**模型只能处理数字，所以第一个问题是——一句话该怎么切，才能一块块送进模型？切出来的最小单位叫 token，切的过程叫 tokenization。最随意的做法是按任意单位切；课上的例句 a cute teddy bear is reading 就一直把 teddy bear 整个当成一个 token。更系统的切法有三种：

切法	好处	代价
词级 (word-level)	最简单直观	bear 和 bears、run 和 runs 被当成毫不相干的两个 token，要各学一个向量，还得想办法让它们相近；词形变化多，词表很大；没见过的词只能标成未知，OOV 风险高
子词级 (subword-level)	利用词根：bear 和 bears 共用 bear 这个片段；OOV 风险低（仍可能发生）	序列变长
字符级 (character-level)	不怕拼写和大小写错误；基本不存在 OOV	序列长得多，计算慢得多；单个字母的向量很难说代表了什么含义

切法	好处	代价
ter-level)		

- **序列长度为什么是成本**：模型的计算量随序列长度增长，token 越多，处理越慢。所以切得细并不免费。
- **OOV (out of vocabulary)**：推理时遇到训练时没见过的 token。通常的处理是在词表里留一个 unknown token 的位置，凡是认不出的都映射到它、共用同一个向量——信息就这样丢了。子词切法是“利用词根”和“少撞 OOV”之间的折中，因此成了默认选择。
- **问答：词表该多大**：看任务。只做一种语言（如英语）时，通常是几万的量级；现在的模型要同时覆盖多语言和代码，词表常到几十万。中文等非拉丁文字道理相同，只是基本字符换成了该语言自己的字符。

小注：常见的子词算法有 BPE、WordPiece、Unigram（这一讲没有点名）。Transformer 原论文的英德翻译用的是 BPE，源语言和目标语言共用约 37,000 个 token 的词表，正好落在“几万”这个量级。

4. 词表示：从 one-hot 到 word2vec

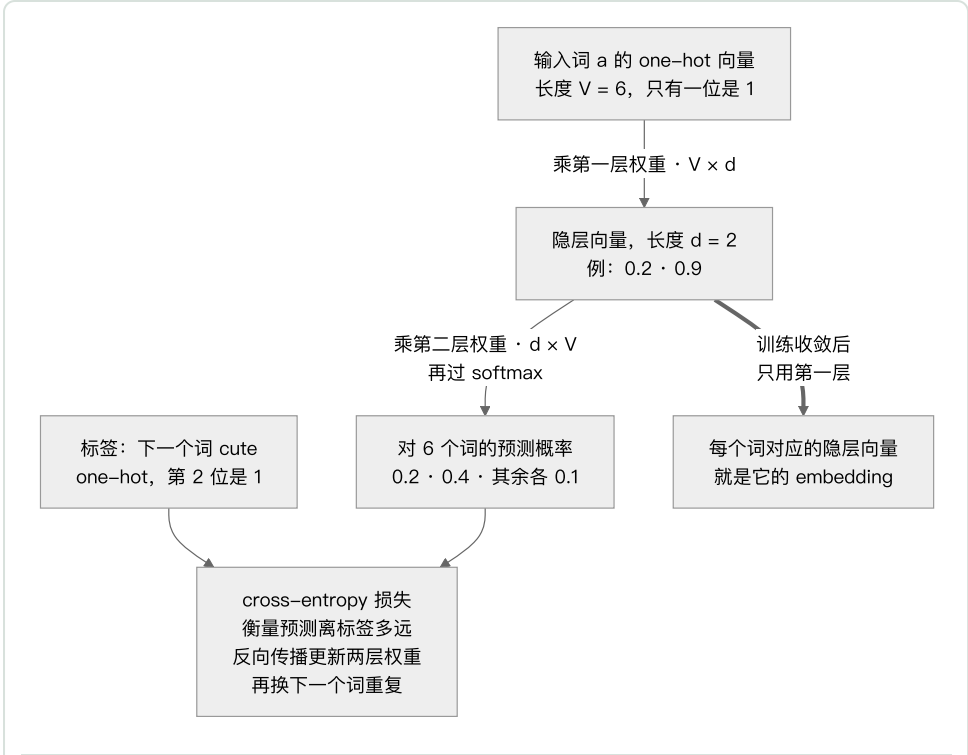


图 1-2 | 用代理任务学词向量：课上的玩具网络（自绘示意）·

▶ 看原幻灯片 42:13 · 出处：Mikolov et al., 2013

- **one-hot 为什么不行：**最朴素的表示法，是给词表里每个 token 一个长度为 V （词表大小）的向量，只有自己那一位是 1。比如词表只有 soft、teddy bear、book 三个词，它们就是 (1,0,0)、(0,1,0)、(0,0,1)。可我们想要的是“意思相近的 token，向量也相近”，好让模型能比较它们；而 one-hot 向量两两正交，任何两个词的相似度都是 0。理想情况应该是 teddy bear 和 soft 的相似度高，和 book 接近 0。常用的相似度度量是 cosine similarity：

$$\cos(u, v) = \frac{u \cdot v}{\|u\| \|v\|}$$

u 、 v 是两个 token 的向量，分子是点积，分母是两个向量长度的乘积；结果只反映夹角：同向接近 1（相似），正交为 0（无关），反向为 -1（相反）。

- **问答：为什么不看向量长度：**有学生注意到幻灯片上用的是点积，而不是完整的 cosine。讲者坦言没有特别好的答案：这些都只是度量方式，没有哪个完美；通常关心的是方向（夹角），长度里有没有信息，要看向量是怎么训出来的；直接用点积的也有。
- **word2vec (2013) 的思路：用代理任务学向量。**不再人为指定向量，而是从大量文本里学。办法是设一个 proxy task（代理任务）：CBOW（continuous bag of words）用周围的词预测中间的目标词，skip-gram 反过来，用目标词预测周围的词。预测得准不准并不是目的；关键在于，一个能根据上下文猜词的模型，必然已经掌握了语言的某些规律，而这些规律就沉淀在它学到的向量里。它当年出名，是因为学出的向量可以直观解读：king 之于 queen，Paris 之于 France 正如 Berlin 之于 Germany。
- **玩具例子**（图 1-2，代理任务简化成“预测下一个词”）：词表 6 个词，隐层维度 $d = 2$ 。输入 a 的 one-hot，乘第一层权重得到隐层向量 (0.2, 0.9)，再乘第二层权重、过 softmax，得到 6 个词的概率。真实的下一个词是词表里的第 2 个词 cute，于是拿预测和它的 one-hot 标签算 cross-entropy 损失（衡量预测离标签多远），再用反向传播（backpropagation：算出损失对每个权重的梯度，顺着它逐层修正权重）让预测向标签靠拢。接着输入 cute、目标 teddy bear，如此遍历全部文本。训练完，一个词的 embedding 就是它的 one-hot 乘第一层权重得到的隐层向量，相当于从权重矩阵里取出属于它的那一行。
 - softmax：把一组任意实数变成一组都为正、总和为 1 的数（先取指数，再各自除以总和），可以当概率用。后面 attention 里还会用到。

- d 远小于 V : V 是几万到几十万, d 通常只有几百 (讲者举的例子是 768)。

- **问答**

- 何时停止训练: 盯着代理任务的损失随 epoch (完整过一遍训练集算一个 epoch) 的变化, 收敛了就可以停; 但代理任务不是最终目的, 最后还得看下游任务的效果。
- 隐层维度怎么定: 是权衡。下游任务越复杂, 需要的向量越大; 向量越大, 计算量、延迟和成本越高。实践中靠经验和前人的结果, 量级在几百到几千。
- 生成什么时候停: 词表里有特殊 token, 模型生成出 EOS (end of sequence) 就停。

- **留下的缺陷:** 每个 token 只有一个固定向量, 和它出现在哪句话里无关, 也不含词序。学生举的例子是 bank——“河岸”和“银行”拼写相同、意思不同, word2vec 分不开。讲者说, 这正是后面要解决的问题: 让表示带上上下文。

小注: 真实的 word2vec 训练的是 CBOW 或 skip-gram, 并不是预测下一个词; 后者是讲者为了举例做的简化 (他自己也说明了)。真正以“预测下一个 token”为训练目标的是后面的 LLM 预训练 (应在第 4 讲 LLM training 展开)。

5. RNN 与 LSTM: 有了词序, 却记不远、跑不快

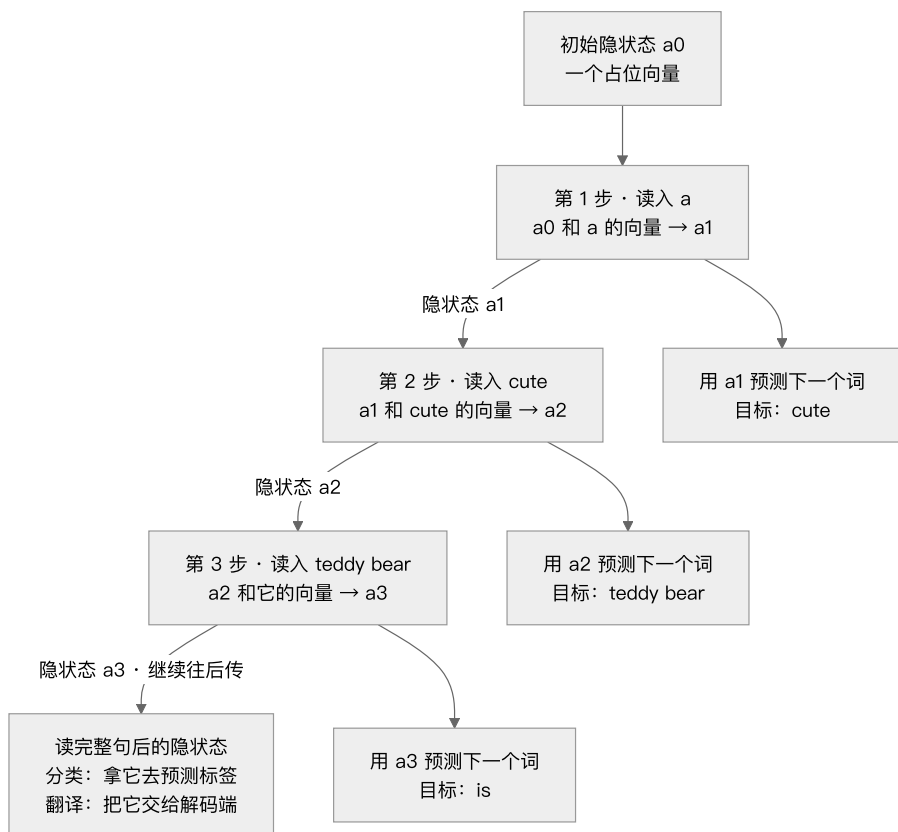


图 1-3 | RNN 按词序展开：第一个词的信息要穿过每一步的隐状态，才能影响到后面（自绘示意）· [▶ 看原幻灯片 54:56](#)

- **为什么需要：**有了词向量，怎么表示一整句话？最简单是把词向量取平均，可这样词序就丢了，每个词的向量也仍然和上下文无关。RNN（recurrent neural network，循环神经网络）就是为“按顺序读”而设计的。
- **怎么工作（图 1-3）：**维护一个隐状态（hidden state，记作 a 或 h ，也叫 activation 或 context vector），可以理解成“到目前为止读过的内容的摘要”。每一步，RNN 单元接收上一步的隐状态和当前 token 的向量，做几次矩阵乘法，输出新的隐状态，并可以用它预测下一个词。一句话就这样从左到右、一个 token 一个 token 地读完。
- **三类任务怎么用：**分类——取读完最后一个 token 后的隐状态，投影到标签空间；token 级分类——取每个 token 位置上的表示，分别投影；生成——先把源文本整个读完，把最后的隐状态（此时叫 context vector）交给解码端，由它逐词生成输出。
- **三个毛病（这也是今天很少再听到 RNN 的原因）**
 1. 长距离依赖（long-range dependency）：整句话的意思只能挤在一个隐状态向量里，读得越远，早先的信息越容易被冲掉。LSTM

(long short-term memory) 在隐状态之外多维护一个 cell state (记作 c)，专门留住值得记住的东西；有改善，但仍不完美。

2. 梯度消失 (vanishing gradient)：梯度是损失对参数的导数，决定每个参数该往哪边改、改多少。训练时，最后一步的误差要沿时间一路传回第一步，见下式。

3. 慢：要算第 t 步，必须先算完前面所有步的隐状态，无法并行；序列一长，训练就非常久。

$$\frac{\partial \mathcal{L}_T}{\partial a_1} = \frac{\partial \mathcal{L}_T}{\partial a_T} \prod_{t=2}^T \frac{\partial a_t}{\partial a_{t-1}}$$

$\mathcal{L}_T$ 是最后一步预测的损失， a_t 是第 t 步的隐状态；误差从第 T 步传回第 1 步，要连乘 T-1 个“相邻两步隐状态之间的导数”。这些因子普遍小于 1，乘积就趋于 0，靠前的部分几乎得不到更新（梯度消失）；普遍大于 1，则越乘越大（梯度爆炸）。课上只讲了这个直觉，没有写推导，这里用的是标准的链式法则形式。

小注：RNN 每一步用的是同一组权重 (recurrent 指的就是同一个单元被反复套用)，所以连乘的是一串性质相近的因子，才会一边倒地趋于 0 或发散——这一点课上没有明说。LSTM 应为 Hochreiter & Schmidhuber (1997)。

6. 从 attention 到 self-attention: query、key、value

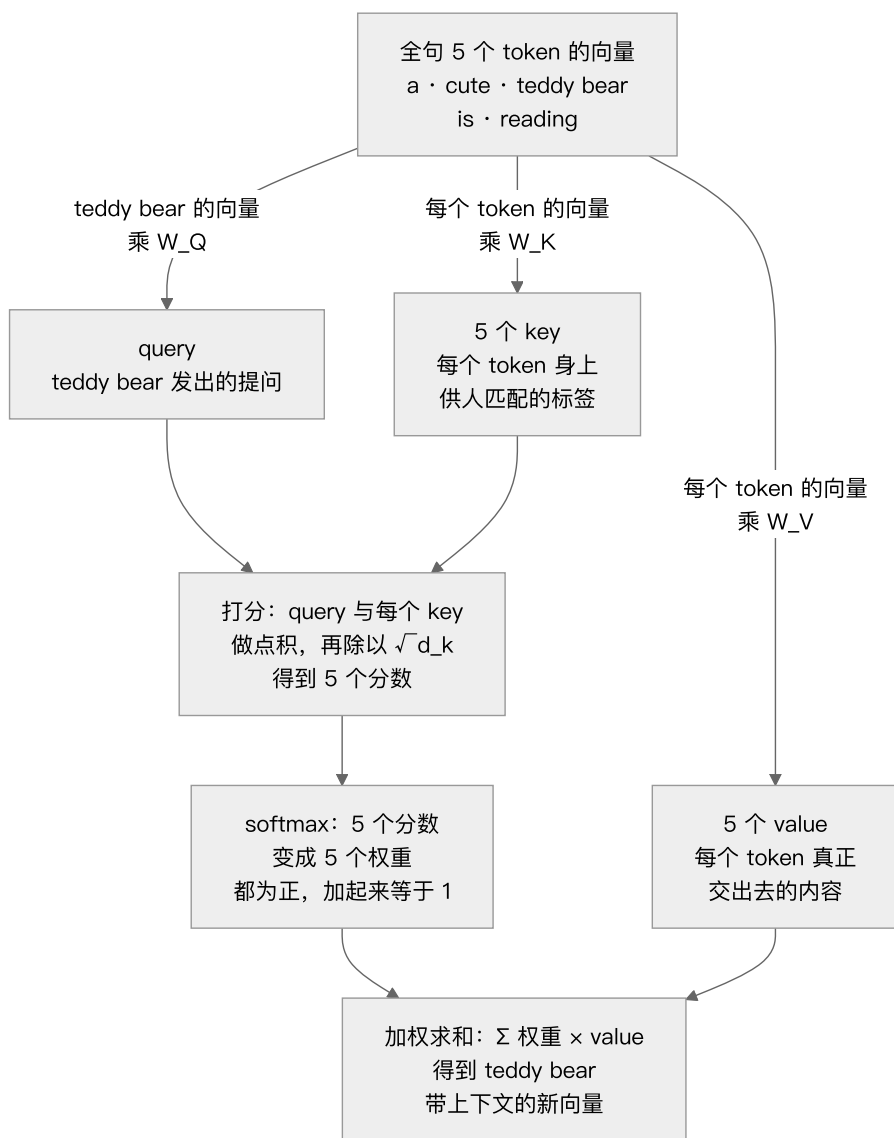


图 1-4 | self-attention 怎样算出一个 token 的新表示 (自绘示意)。

► 看原幻灯片 1:09:37 · 出处: Vaswani et al., 2017

- **attention (2014) 要解决什么:** 用 RNN 做翻译时, 解码端生成每个词, 都只能依赖那一个 context vector。直觉上, 要写出译文的下一个词, 最好能直接“瞄一眼”原文里对应的那部分。attention 就是在“当前要预测的东西”和“输入的每个位置”之间建立直连, 信息不必再穿过长长的隐状态链, 长距离依赖的问题因此缓解。
- **Transformer (2017) 更进一步:** 论文标题 Attention Is All You Need 已经表明态度——干脆不要循环结构, 让每个 token 同时、直接地连到句中所有 token, 这就是 self-attention (自注意力, “自”指一句话内部自己对自己做 attention)。作者在机器翻译上做实验, 效果很好。

小注：2014 年的那篇应为 Bahdanau et al., 2014，它把定长向量称为 encoder-decoder 翻译模型的瓶颈，让模型在生成每个目标词时自动去源句里搜索相关部分。Transformer 论文摘要里的成绩：WMT 2014 英德 28.4 BLEU、英法 41.8 BLEU，后者只在 8 块 GPU 上训练了 3.5 天。

- **query / key / value 各是什么**（图 1-4）：想用句中其他 token 来表达某个 token（比如 teddy bear），就让它发出一个 query（提问）；句中每个 token 各有一个 key（供人匹配的标签）和一个 value（真正交出去的内容）。拿 query 和每个 key 比相似度，得到一组权重，再按权重对所有 value 求加权平均，结果就是 teddy bear 的新表示。
 - 可以想成一次“软”查字典：普通字典只返回完全匹配的那一条；attention 则按匹配程度，把所有条目的内容混合后返回。（这个类比是笔记加的。）
 - 三者都不是人为设定的：把 token 的向量分别乘上三个投影矩阵 W_Q 、 W_K 、 W_V 就得到它们，而这三个矩阵是模型学出来的。
 - 这样算出的表示因句而异：bank 在“河岸”和“抢银行”两句话里会得到不同的向量，第 4 节留下的问题就此解决。
- **写成矩阵，一次算完整句**：把所有 token 的 query、key、value 分别堆成矩阵 Q 、 K 、 V ，整句的 self-attention 就是几次矩阵乘法。讲者强调这正合硬件的胃口——GPU 最擅长的就是矩阵运算。

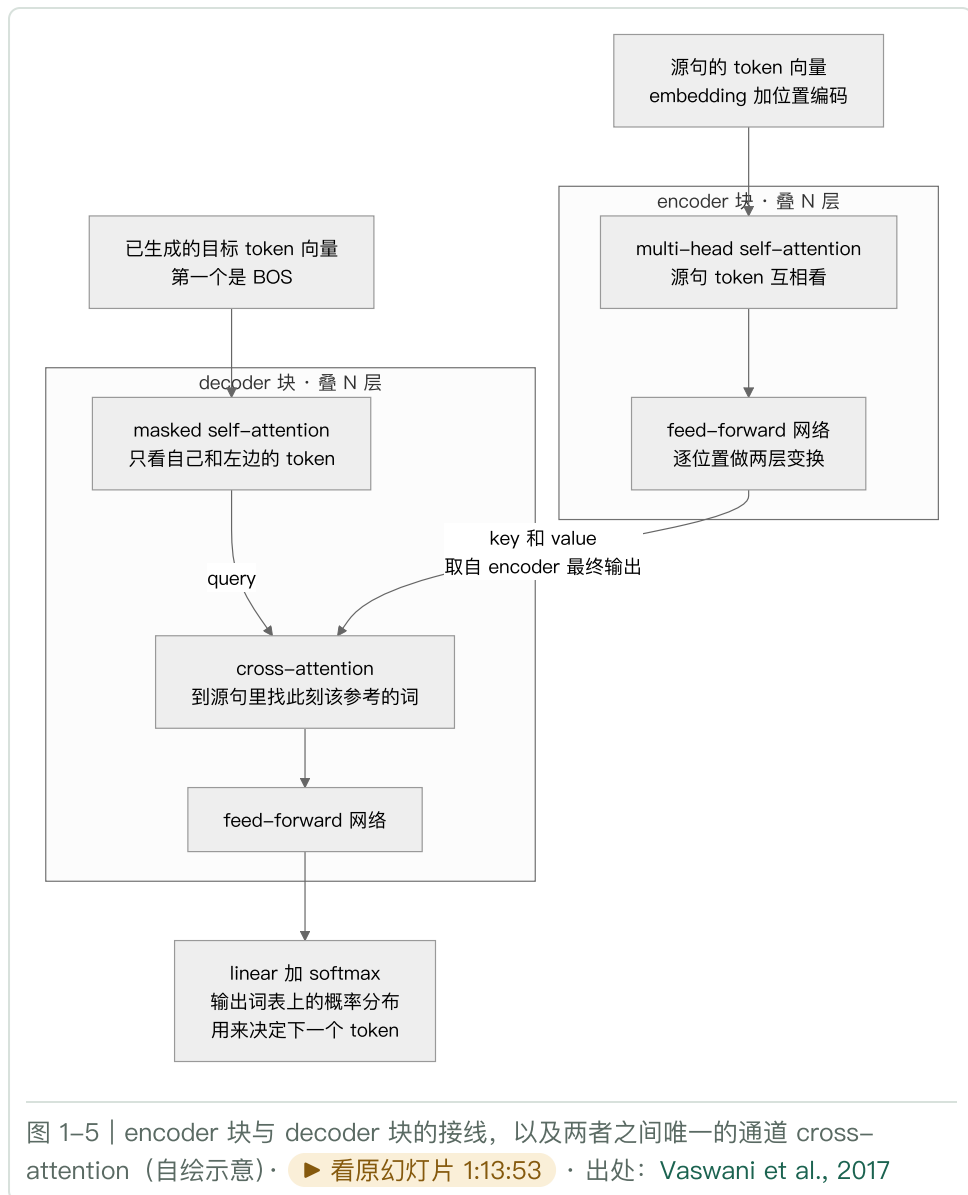
$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Q 、 K 、 V 分别是 query、key、value 组成的矩阵，每行对应一个 token； d_k 是 key 向量的维度；softmax 按行做。

- **逐步读这个式子**（Shervine 在详细例子里的讲法）
 1. 输入是 $n \times d_{\text{model}}$ 的矩阵 X ： n 个 token，每个是 d_{model} 维的向量。 $Q = XW_Q$ ， $K = XW_K$ ， $V = XW_V$ 。
 2. QK^T 是 $n \times n$ 的矩阵：第 i 行是第 i 个 query 和所有 key 的点积，也就是 token i 给每个 token 打的原始分。
 3. 除以 $\sqrt{d_k}$ ，再按行做 softmax：每一行变成一个概率分布，即 token i 分给各个 token 的注意力权重。
 4. 乘 V ：每一行得到 value 的加权和，就是 token i 的新表示。
- **为什么要除以 $\sqrt{d_k}$** ：点积的数值会随向量维度变大而变大，需要拉回来。顺带一提，query 要和 key 做点积，所以两者维度必须相等（ $d_q = d_k$ ）。

小注：原论文的解释更具体——假设 q 、 k 的各个分量相互独立、均值 0、方差 1，那么点积的方差就是 d_k ；数值过大时 softmax 会落进梯度极小的区域，所以用 $1/\sqrt{d_k}$ 把它缩回去。

7. Transformer 架构：encoder、decoder 与三处 attention



- **两半各管什么：**原论文做的是翻译。encoder（编码器，原论文架构图的左半）读源语言句子，输出每个源 token 带上下文的表示；decoder（解码器，右半）一个 token 一个 token 地写出目标语言。
- **encoder 块：**一层 multi-head self-attention（源句所有 token 互相看），再接一层 feed-forward 网络（FFN）。
- **decoder 块：**三个子层。

- 1. masked self-attention: 在已经译出的 token 之间做 self-attention，弄清“已译出的哪些词对预测下一个词有用”。叫 masked（带掩码），是因为只许看自己和左边——右边的词还没生成，无从看起。这种只看过去的 attention 也叫 causal（因果）。
- 2. cross-attention: query 来自 decoder（上一子层的输出），key 和 value 来自 encoder 的输出。讲者在课上让大家猜 decoder 那根箭头是 Q、K、V 里的哪一个——是 query，因为此刻在提问的是 decoder：原文里哪些词和我现在要写的词有关？
- 3. FFN。

• **输出：**最后一层 decoder 的输出经过 linear 投影加 softmax，变成一个长度等于词表大小的概率向量，用来决定下一个 token。

attention 的位置	query 来自	key、value 来自	能看到什么
encoder 的 self-attention	源句	源句	源句的全部 token
decoder 的 masked self-attention	已生成的译文	已生成的译文	自己和左边的 token
decoder 的 cross-attention	decoder	encoder 的最终输出	源句的全部 token

• **multi-head（多头）：**同一套 attention 计算并行做 h 次，每次用一组各自学出来的投影矩阵（一组 W_Q 、 W_K 、 W_V 就叫一个 head）；把 h 个结果按列拼起来，再乘一个输出矩阵 W_O ，投影回原来的维度 d_{model} 。目的是多给模型一些自由度，让不同的 head 学到 token 之间不同类型的关联。讲者的类比：像 CNN 里同一层放多个卷积核。

$$\text{MultiHead}(X) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) W_O, \quad \text{head}_i = \text{Attention}(XW_Q^{(i)}, XW_K^{(i)}, XW_V^{(i)})$$

X 是输入的 token 矩阵，h 是 head 的个数，上标 (i) 表示第 i 个 head 自己的那组投影矩阵， W_O 把拼接后的结果投影回 d_{model} 维。

- **问答：各个 head 会不会学成一样：**没有任何约束强制它们不同，靠的是梯度下降——训练目标是把下一个词预测好，重复学同一种关联没有收益，实践中各 head 会收敛到不同的表示。也有论文专门研究给 head 加约束会怎样。
- **FFN 在干什么：**对每个位置的向量单独做一次两层的全连接变换。和 word2vec 的小隐层相反，这里的隐层比输入输出更宽，为的是给模型更多自由度，学出更丰富的特征。
- **叠 N 层：**encoder 块和 decoder 块各重复 N 次；最后一层 encoder 的输出，送进每一层 decoder 的 cross-attention。

- **位置编码** (position encoding): self-attention 是直连, 不像 RNN 那样按顺序读, 模型本身并不知道谁在前谁在后。所以要在 token 的 embedding 上再加一个表示位置的向量; 原论文用一组正弦、余弦函数生成它, 和 embedding 逐元素相加。讲者说今天不展开 (推断: 留给第 2 讲)。

小注: 原论文 base 模型的配置是 $N = 6$ 、 $d_{\text{model}} = 512$ 、 $h = 8$ (每个 head 的 $d_k = d_v = 64$)、FFN 隐层 $d_{\text{ff}} = 2048$; 课上随口举的 768 是后来 BERT-base 的维度。另外, 原架构里每个子层外面都包着 residual connection 加 layer normalization, 即 $\text{LayerNorm}(x + \text{Sublayer}(x))$, 这一讲没有讲到。

8. 端到端走一遍：训练和推理时数据怎么流

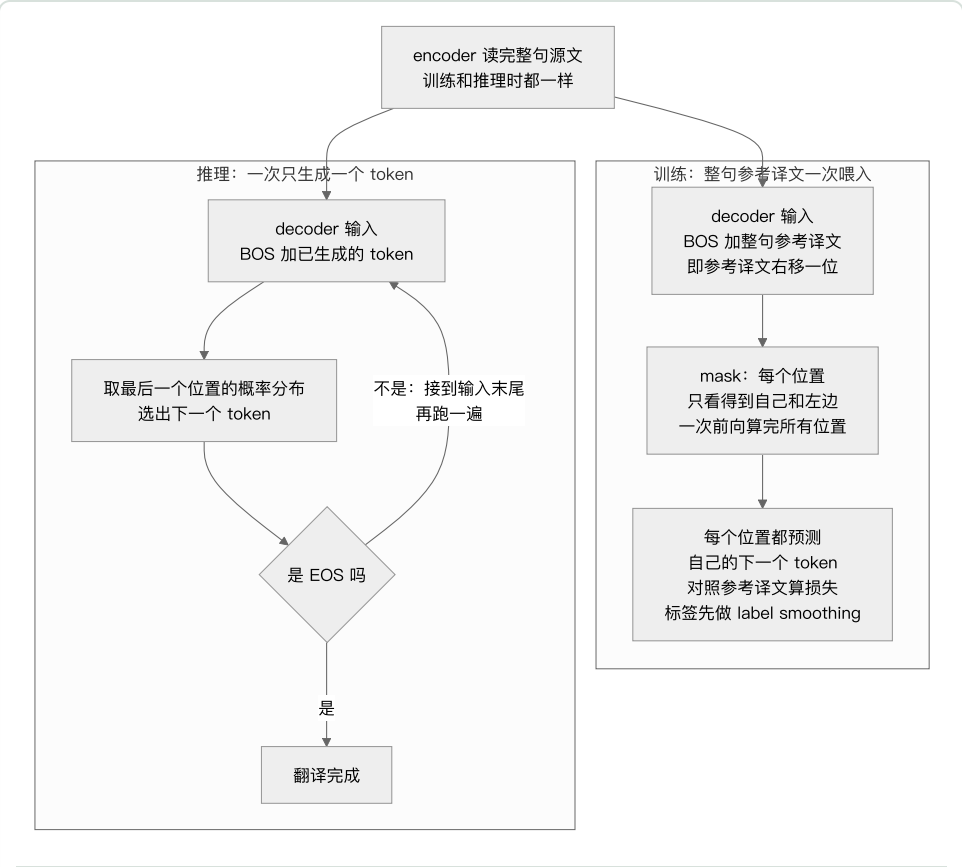


图 1-6 | 同一个 encoder-decoder, 训练时并行、推理时逐词循环; 推理一侧是课上讲的, 训练一侧按原论文补充 (自绘示意) · [▶ 看原幻灯片 1:38:36](#) · 出处: Vaswani et al., 2017

- **输入侧** (例句 a cute teddy bear is reading): 先切成 token, 首尾加上 BOS (begin of sequence) 和 EOS (end of sequence) 两个特殊 token; 每个 token 查到自己学出来的 embedding, 加上位置编码, 整句就成了一个 $n \times d_{\text{model}}$ 的矩阵。

- **encoder**: 矩阵依次穿过 N 个 encoder 块 (self-attention → FFN)，出来的是每个源 token 带上下文的表示，形状不变。
- **decoder 的推理循环** (图 1-6 的推理一侧): 从只含 BOS 的序列开始。BOS 先过 masked self-attention (此时只有它自己可看)，其输出作为 query 进 cross-attention 去查 encoder 的输出，再过 FFN；这样走完 N 层，经 linear 加 softmax 得到词表上的概率分布，选出第一个词。把它接到 decoder 输入的末尾，再跑一遍，得到下一个词……直到生成 EOS。课上的例句最后译成 un ours en peluche mignon lit。

小注：课上只讲了推理时的逐词循环。训练时并不循环（图 1-6 的训练一侧，按原论文补充）：把“BOS 加参考译文”整句一次喂给 decoder，相当于参考译文右移一位；mask 保证每个位置只看得到左边，于是所有位置的“下一个词”可以在一次前向里并行预测、一起算损失。这才是 mask 不可或缺的地方，也是 Transformer 训练比 RNN 快的原因。至于怎样从概率分布里选词，课上没有展开（推断：采样策略留给后面的课）。

- **label smoothing** (标签平滑): 原论文用的一个训练技巧。动机是，语言里“下一个词”往往不止一个正确答案 (what a great 后面接 day、lecture、book 都说得通)，而 one-hot 标签却要求模型把 100% 的概率押在一个词上。做法是把标签改软：

$$y_k = \begin{cases} 1 - \varepsilon, & k = c \\ \frac{\varepsilon}{V - 1}, & k \neq c \end{cases}$$

y_k 是训练标签在词表第 k 个 token 上的取值， c 是真实的下一个 token 的编号， ε 是一个小常数， V 是词表大小。

- **效果与问答**: 模型被训练得不那么笃定，而作者观察到 BLEU 反而更好。问答里有学生联想到 explore / exploit，还有一问听不清，从回答推断是：softmax 的输出本来就不是 0 和 1，这不是已经“免费”做到了吗？讲者澄清：softmax 管的是模型的输出，label smoothing 改的是拿来和输出比较的标签，也就是训练目标本身，两者不是一回事。

小注：label smoothing 应出自 Szegedy et al., 2015，原式是 $(1-\varepsilon) \cdot \delta + \varepsilon \cdot u(k)$ ，其中 δ 是 one-hot， $u(k)$ 是标签上的一个先验分布，通常取均匀分布——学生问到的幻灯片上的 u 应该就是它（推断）；和讲者口述的版本只差在真实类别是否也分到一份 ε 。Transformer 论文取 $\varepsilon = 0.1$ ，并说明这会让 perplexity 变差，但 accuracy 和 BLEU 变好。

关键图表速查（点时间戳跳到原幻灯片）

图	看什么	跳转	出处
NLP 简史时间线	80 年代 RNN → 90 年代 LSTM → 2013 word2vec → 2017 Transformer → 2020 年代 LLM；早年的瓶颈是数据和算力	21:03	课程自制
三种 tokenizer 的优缺点小结	词级、子词级、字符级各自的 pro 和 con；OOV 风险和序列长度此消彼长	28:27	课程自制
one-hot 的相似度对比理想的相似度	一边全是 0（两两正交），另一边 teddy bear 与 soft 高、与 book 低	32:42	课程自制
word2vec 玩具网络	$V = 6$ 、 $d = 2$ 的两层网络逐词走查；讲者说的“绿色单元”就是最后留下来的 embedding	42:13	word2vec
RNN 展开图	隐状态沿时间步往后传；每步的蓝色模块吃进“上一个隐状态加当前词”	54:56	课程自制
RNN 用于三类任务	分类取最后的隐状态；token 级分类取各位置；生成把最后的隐状态交给解码端	58:02	课程自制
attention 的直连	译文每生成一个词，都有连线直接指回原文的相关位置，不再只靠一个 context vector	1:05:24	应出自 Bahdanau et al., 2014
self-attention: teddy bear 连向全句	一个 token 同时和句中所有 token 直接相连，中间不经过任何隐状态	1:08:01	Vaswani et al., 2017
query / key / value 与矩阵公式	query 去和每个 key 比，按相似度加权取 value；整句写成 $\text{softmax}(QK^T)V$	1:09:37	同上
Transformer 架构全图	左 encoder 右 decoder；找出三处 attention，以及从 encoder 指向 decoder 的两根箭头（key 和 value）	1:16:33	同上
label smoothing	one-hot 标签被改成 $1-\epsilon$ 与 $\epsilon/(V-1)$	1:26:26	应出自 Szegedy et al., 2015
QK^T 矩阵逐行解读	Q 的每行是一个 query， K^T 的每列是一个 key；乘积第 i 行就是 token i 对所有 token 的打分	1:32:46	Vaswani et al., 2017
h 个 head 拼接再乘 W_O	h 份输出按列拼起来， W_O 把它投影回 d_{model} 维	1:34:56	同上
解码循环	BOS 进 decoder → 概率分布 → 选词 → 回填到输入 → 直到 EOS	1:39:06	同上

提到的工作

名称	在本讲里的作用
Transformer: Attention Is All You Need (Vaswani et al., 2017)	本讲主角: self-attention、encoder-decoder 架构、multi-head、位置编码、label smoothing
word2vec (Mikolov et al., 2013): CBOW 与 skip-gram	用代理任务学词向量的代表作; king-queen 类比
RNN (1980 年代)	按词序逐个处理、靠隐状态记忆的序列模型; Transformer 要取代的对象
LSTM (应为 Hochreiter & Schmidhuber, 1997)	给 RNN 加 cell state, 缓解遗忘
attention (2014; 应为 Bahdanau et al.)	在 RNN 翻译模型上让解码端直接回看输入; self-attention 的前身
label smoothing (应出自 Szegedy et al., 2015)	Transformer 论文用的训练技巧
accuracy / precision / recall / F1	分类任务的指标
BLEU / ROUGE / perplexity	生成任务的传统指标; 前两者依赖参考答案
WMT (Workshop on Machine Translation)、欧洲议会语料	机器翻译的标准平行语料; Transformer 的实验数据
IMDb、Amazon reviews、推文 (X posts)	情感分类常用数据
ChatGPT、Gemini	问答 / 助手类生成任务的例子; ChatGPT (2022) 带火了这门课
CNN 的多个卷积核	理解 multi-head 的类比
Super Study Guide、VIP cheatsheet	课程教材与速查表 (讲者自著)

术语对照

English	中文
NLP (natural language processing)	自然语言处理
NER (named entity recognition)	命名实体识别: 标出文本里的地点、时间、人名等
part-of-speech tagging	词性标注
dependency / constituency parsing	依存 / 成分句法分析
precision / recall / F1	精确率 / 召回率 / 两者的调和平均
reference-based / reference-free metric	需要 / 不需要参考答案的指标
perplexity	困惑度: 模型对文本有多“意外”, 越低越好
token / tokenization / tokenizer	词元 (文本切分后的最小单位) / 切分 / 切分器

English	中文
subword	子词：比词小、比字符大的片段
vocabulary	词表：模型认识的全部 token
OOV (out of vocabulary)	未登录词：训练时没见过的 token
one-hot encoding (OHE)	独热编码
embedding	嵌入：token 对应的稠密向量
cosine similarity	余弦相似度
proxy task	代理任务：目的不在任务本身，而在顺带学到的表示
CBOW / skip-gram	用上下文预测中心词 / 用中心词预测上下文
softmax	把一组实数变成概率分布的函数
cross-entropy	交叉熵：衡量预测分布离标签多远的损失
backpropagation	反向传播
epoch	完整过一遍训练集
downstream task	下游任务：学到的表示最终要服务的任务
RNN (recurrent neural network)	循环神经网络
hidden state / context vector	隐状态 / 上下文向量
LSTM (long short-term memory) / cell state	长短期记忆网络 / 细胞状态
long-range dependency	长距离依赖
vanishing / exploding gradient	梯度消失 / 梯度爆炸
backpropagation through time	沿时间反向传播
attention / self-attention	注意力 / 自注意力
query / key / value	查询 / 键 / 值
projection matrix	投影矩阵
encoder / decoder	编码器 / 解码器
multi-head attention / head	多头注意力 / 头
masked (causal) self-attention	带掩码的（因果）自注意力：只看自己和左边
cross-attention	交叉注意力：decoder 去查 encoder 的输出
feed-forward network (FFN)	前馈网络
position encoding	位置编码
BOS / EOS	序列开始 / 序列结束的特殊 token
label smoothing	标签平滑

字幕勘误

这一讲的 CC 整体可靠，值得留意的只有几处：“Under Study” → Understudy (BLEU 全称的最后一个词)；1:08:58 主讲人问时间时喊的名字被写成“Afshine”，应为 Shervine (发问的正是 Afshine 本人)；1:34 前后的“has the dimension” → as the dimension (意思是“随着维度变大”)；1:35 前后的“each times” → h times (做 h 次，h 是 head 数)；1:38 前后的“setups encoded” → set of encoded (推断)；51:55 前后的“a pool of” → 应为 a couple of (推断)。另外 multi-classification 确实是讲者的原话，不是识别错误 (通行叫法见第 2 节小注)。

带走的问题

1. QK^T 是 $n \times n$ 的矩阵。序列长度翻倍，self-attention 的计算量和显存怎么变？回头看第 3 节，这对“子词还是字符级”的取舍意味着什么？
2. 假如去掉位置编码，再把输入句子的 token 顺序打乱，encoder 的输出会怎样变化？由此说明位置编码补上的到底是什么。
3. 今天的 LLM 大多只有 decoder，没有 encoder，也没有 cross-attention。那“原文”（提示词）放在哪里，由哪一种 attention 负责读它？（推断：第 2、3 讲讲到各类 Transformer 模型时会回答）
4. 训练时 decoder 看到的是参考译文，推理时看到的是自己生成的 token。这种不一致会带来什么问题？mask 在训练和推理中各起什么作用？
5. label smoothing 让 BLEU 变好，却让 perplexity 变差，两个指标为什么会朝相反方向动？讲者对“各 head 不会学成一样”的解释是梯度下降没有动力去重复——这个论证充分吗，如果很多 head 其实是冗余的，意味着什么？

CME295 第 2 讲 | Transformer 系模型与技巧 (Transformer-Based Models & Tricks)

Stanford CME295: Transformers & Large Language Models (2025 秋) · 第 2 讲, 2025 年 10 月 3 日 视频: <https://www.youtube.com/watch?v=yT84Y5zCnaA> (1:47:19, 自带英文字幕, 质量不错, 仍有少量专名和近音词识别错误) 讲者: Afshine Amidi (前一小时: 位置嵌入、归一化、attention 变体) · Shervine Amidi (后 45 分钟: 模型谱系与 BERT) 课程大纲: <https://cme295.stanford.edu/syllabus/>

一句话: 2017 年的 Transformer 骨架至今没变, 换过的是三个部件和一种"只留一半"的用法——位置信息从"加在输入上的向量"搬进了 attention 内部 (T5 bias、ALiBi, 直到今天几乎通用的 RoPE); 归一化从 post-norm LayerNorm 变成 pre-norm RMSNorm; attention 用 sliding window 压 $O(n^2)$, 用 GQA / MQA 共享 K、V 投影来压 KV cache。模型则分成 encoder-decoder (T5)、encoder-only (BERT) 和 decoder-only (今天的 LLM) 三支; 后半场把 BERT 的输入构造、MLM + NSP 预训练、微调, 以及 DistilBERT、RoBERTa 两个改进讲透。

时间	内容
0:06	开场: 换了录音设置; 期末考试日期待定
1:30	回顾第 1 讲: self-attention 公式、encoder-decoder 架构
3:44	multi-head 再解释: 每个 head 一套投影; attention map 的例子; 问答
9:32	本讲结构: 前半讲变过的部件, 后半讲模型谱系
10:37	位置嵌入总览: 为什么需要; learned embedding 及其两个局限
15:36	sinusoidal embedding: 公式; 点积只取决于相对距离
22:50	位置向量热力图: 低维高频、高维低频; 和 learned 效果相当但能外推
25:56	为什么把位置信息搬进 attention 公式; T5 bias、ALiBi
31:02	RoPE: 把 query 和 key 各转一个角度
32:59	黑板推导: 旋转矩阵确实把向量转了 θ 角
37:13	RoPE 的两个好处: 打分只含相对距离、长程衰减
40:39	问答: θ 是什么、怎么推广到 d 维、衰减曲线的来历
43:42	Layer normalization: Add & Norm、LayerNorm 的公式
46:00	post-norm $\rightarrow$ pre-norm; LayerNorm $\rightarrow$ RMSNorm

时间	内容
48:13	问答：归一化的直觉；和 BatchNorm 的区别
50:39	稀疏注意力：O(n ²)、Longformer、问答（实现上怎么省）
53:07	sliding window；局部层与全局层交替；Mistral 与感受野类比
55:38	共享注意力头：为什么共享的是 K、V；和 KV cache 的关系
58:29	MQA、GQA、MHA 三档
1:00:00	问答：用在哪些 attention 层；怎么选
1:02:42	换 Shervine：encoder-decoder 一支与 T5 家族
1:04:45	T5 的训练目标：span corruption 与 sentinel token
1:07:58	encoder-only：不能生成，但表示可用于分类
1:09:33	decoder-only：cross-attention 消失；为什么它成了主流
1:11:38	BERT 深入：名字的含义、双向性
1:13:50	同年的 ELMo；芝麻街式的命名
1:15:54	[CLS] 与 [SEP]；两阶段训练：预训练（MLM + NSP）与微调
1:20:17	这套方法的优缺点
1:22:19	从 Transformer 到 BERT：WordPiece、特殊 token
1:25:30	输入 embedding：token + position + segment；问答
1:29:02	MLM 的 80 / 10 / 10；NSP
1:31:06	论文记号 L / H / A；cased 与 uncased；参数量级
1:33:24	微调：分类头、冻结还是全量；情感分类与问答两个例子
1:34:56	例子走查：一句话的情感分类从头到尾
1:38:21	问答：分类头是什么；为什么只用 [CLS]；[CLS] 的 Q、K、V
1:41:26	BERT 的价值与三个局限
1:43:30	蒸馏与 DistilBERT
1:46:18	RoBERTa；下课

核心内容

1. 回顾与本讲地图：骨架没变，换过的是部件

- **回顾 self-attention**：每个 token 拿自己的 query 去和所有 token 的 key 比相似度，再按相似度对 value 加权求和（公式见第 1 讲）。整件事就是几次大矩阵乘法，正好是硬件最擅长的运算。
- **multi-head 到底是什么**（上节课被问得最多）：每个 head 是一套独立的 Q、K、V 投影矩阵，相当于多给模型一次机会，去学"按什么标准找相关的词"。讲者用原论文附录里的 attention map 举例：看 its

这个词的 query 和哪些 key 的点积大，被点亮的是 law 和 application——正是 its 指代和修饰的对象；不同 head 点亮的侧重不一样。

- **问答：**各个 head 走的是不同的 MLP 吗？不是 MLP，是各自的投影矩阵。所有 head 并行计算，各出一个结果，拼接起来再过一次输出投影。整个过程只有投影、矩阵乘法和 softmax，高度并行。
- **本讲地图：**讲者的判断是，2017 年的架构出人意料地耐用，今天的模型大体上都还建在它上面，只有少数部件变了。前一小时逐个讲变过的三个部件，后 45 分钟讲模型谱系并深入 BERT。下表是全讲的索引：

部件	2017 原版	今天常见的做法	课上给的理由
位置信息	sinusoidal 或 learned 向量，加在输入上	RoPE，作用在 attention 的 q、k 上	让相对距离直接进入打分
归一化	post-norm + LayerNorm	pre-norm + RMSNorm	RMSNorm 效果相当、参数更少、更快；挪位置的原因课上没展开
attention	全局；每个 head 一套 K、V 投影	局部层与全局层交替；GQA 共享 K、V	压 $O(n^2)$ 的计算；压 KV cache
整体结构	encoder-decoder	decoder-only	算力投在 decoder 上最划算；next-token 目标最容易放大

2. 位置嵌入（上）：从"加在输入上"到"写进 attention 公式"

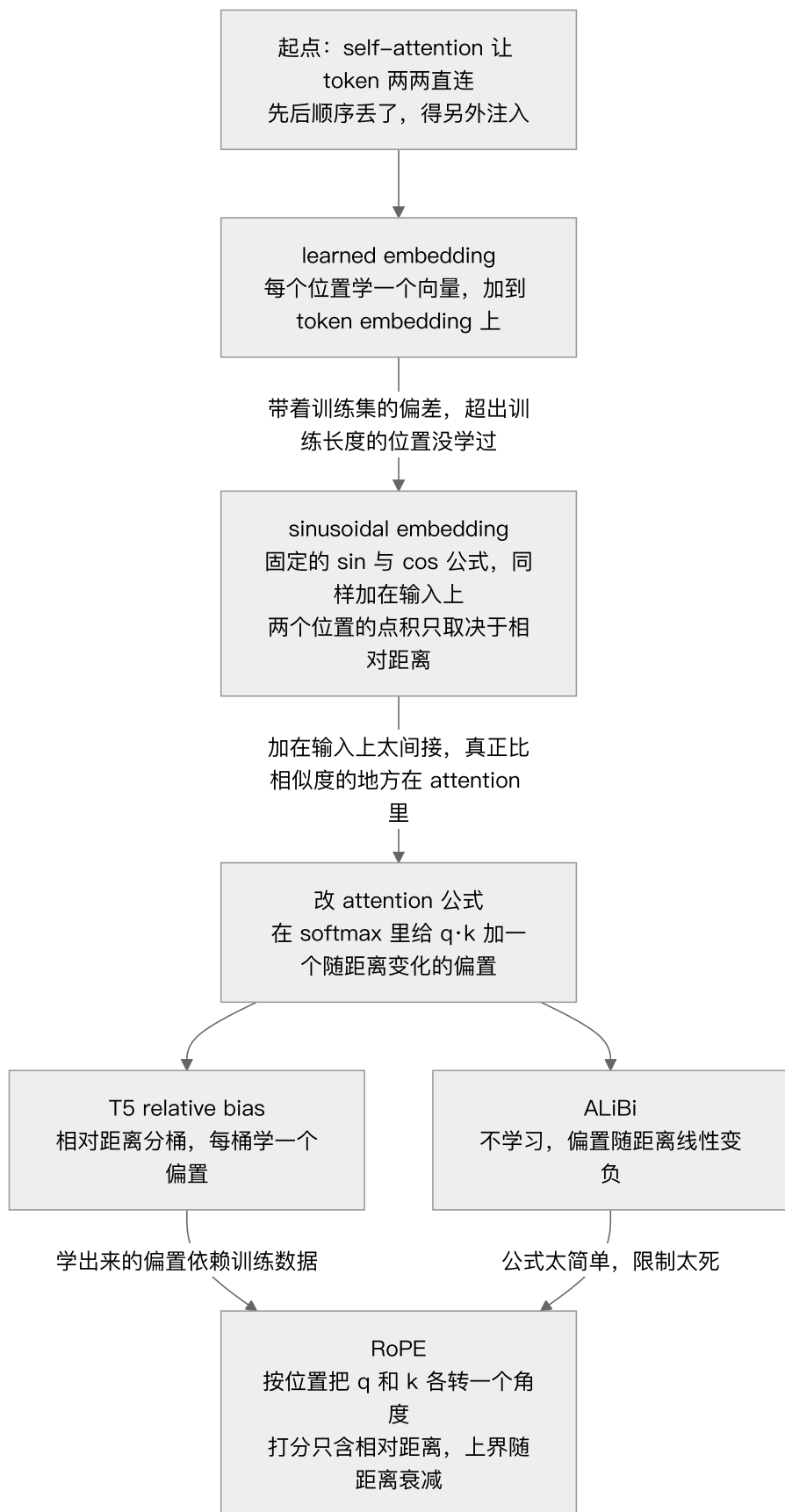


图 2-1 | 位置信息注入方式的演进: 每一步在补上一步的什么短板 (自绘示意)·

- **为什么需要：**RNN 一个一个地处理 token，先后顺序天然就在计算过程里；self-attention 让所有 token 两两直连，"谁在前谁在后"这条信息丢了。所以得把"第几个位置"变成某种数值信号，再注入模型。
- **做法一：learned position embedding。**给每个位置配一个可训练的向量，和 token embedding 相加，用梯度下降学出来。好处是交给数据自己学。两个局限：一是学到什么取决于训练集——如果训练文本里第 2 个位置总出现某类内容，这种偏差会被学进去；二是只能学到训练时见过的最大长度（比如 512），推理时超出的位置没有向量可用。
- **做法二：sinusoidal embedding。**同样是每个位置一个 d_{model} 维的向量（要和 token embedding 相加，维度必须一致），但不学，直接按公式算：

$$PE_{(m, 2i)} = \sin(\omega_i m), \quad PE_{(m, 2i+1)} = \cos(\omega_i m), \quad \omega_i = 10000^{-2i/d_{\text{model}}}$$

m 是位置， i 是"第几对维度"， ω_i 是这对维度的频率： i 小则频率高， i 大则频率低。

- **为什么偏偏是 sin 和 cos：**我们想要的性质是"离得近的位置更相似"，而在 embedding 的世界里衡量相似靠点积（cosine similarity 就是点积除以两个模长）。把两个位置的向量做点积，逐对维度套用 $\cos(a - b) = \cos a \cdot \cos b + \sin a \cdot \sin b$ ：

$$PE_m \cdot PE_n = \sum_i [\sin(\omega_i m) \sin(\omega_i n) + \cos(\omega_i m) \cos(\omega_i n)] = \sum_i \cos(\omega_i (m - n))$$

PE_m 、 PE_n 是位置 m 、 n 的向量；右边说明两者的相似度只取决于相对距离 $m - n$ ，并且在 $m = n$ 时最大（每一项都是 $\cos 0 = 1$ ）。

- 讲者自己补了一句： $\cos$ 是周期函数，"越远越不相似"并不严格单调，只是大趋势。
- **热力图怎么读：**纵轴是位置（约 50 个），横轴是向量的各个维度。低维的值随位置快速上下摆动（高频），高维变化很慢（低频），对应 ω_i 随 i 增大而减小。
- 原论文两种做法都试过，效果相当；sinusoidal 的优势是任意长度的位置向量都算得出来，不受训练长度限制。
- **到 2025 年还在用吗：**讲者的回答是"算是"。"远的 token 应该不如近的相似"这个想法留下了，注入方式变了。理由：真正比较 token 相似度的地方是 attention 层里的 $q \cdot k$ ，把位置向量加在最底层的输入上，

对那里的影响是间接的。更直接的做法是改 attention 公式本身，在 softmax 里面加一个与相对位置有关的偏置：

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}} + B\right)V, \quad B_{mn} = b(m - n)$$

B 是一个偏置矩阵，第 m 行第 n 列的值只由相对距离 $m - n$ 决定；其余符号同第 1 讲。

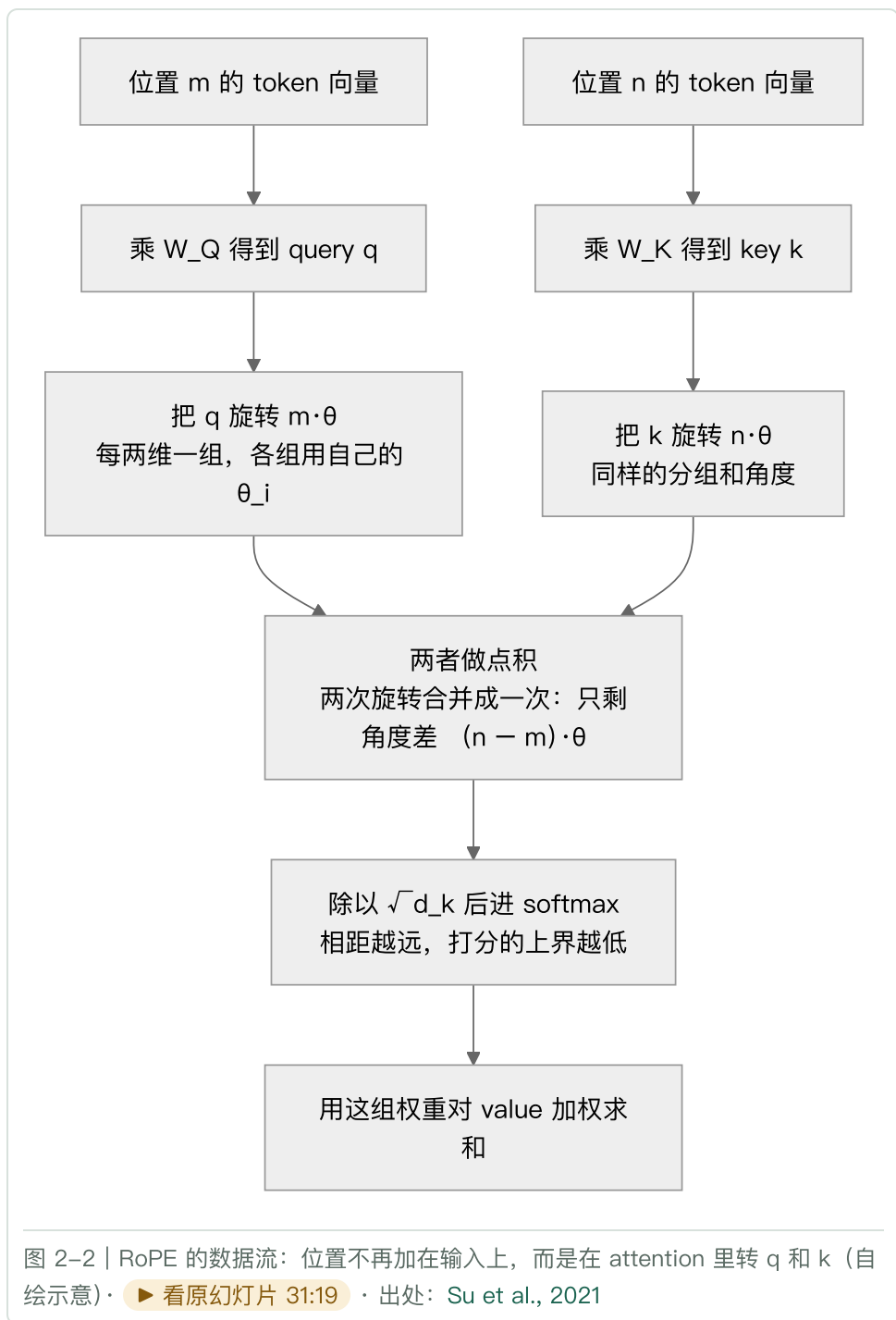
- **两种偏置**

- *T5 relative position bias*: 把 $m - n$ 分桶，每个桶学一个标量偏置。
- *ALiBi* (出自 Train Short, Test Long 一文): 不学习，用一个关于相对距离的确定公式，离得越远偏置越负。

小注：ALiBi 的具体形式是“负的斜率 × 相对距离”，每个 head 一个固定斜率（按等比数列取，不训练）；BLOOM、MPT 用的就是它。

- **问答**：往 softmax 里加偏置，会不会破坏“概率加起来等于 1”？不会。softmax 里面放什么都行，它最后都会归一化；把偏置理解成“离得越远就越负”的一项即可。
- **两者的不足，引出 RoPE**：学出来的偏置仍然依赖训练数据的分布（训练时和推理时，“近的 token 怎么个相似法”可能不同）；ALiBi 没有可学的部分，但只是距离的一个简单函数，限制太死。

3. 位置嵌入（下）：RoPE——把 query 和 key 各转一个角度



- **做法：**先看二维的情形。位置 m 的 query 旋转一个与 m 成正比的角度，位置 n 的 key 旋转一个与 n 成正比的角度，然后照常做点积。旋转就是左乘一个 2×2 的旋转矩阵。
- **黑板上的小证明：**把二维向量写成“模长 r 乘以 $(\cos \phi, \sin \phi)$ ”，左乘旋转矩阵后用和角公式整理，得到“r 乘以 $(\cos(\theta+\phi), \sin(\theta+\phi))$ ”——模长不变，角度多了 θ ，所以这个矩阵确实是“转 θ 角”。中间的乘法讲者留作练习。

$$R_\theta = \begin{pmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{pmatrix}, \quad (R_{m\theta} q)^\top (R_{n\theta} k) = q^\top R_{(n-m)\theta} k$$

R_θ 是旋转 θ 角的矩阵， q 、 k 是位置 m 、 n 上的 query 和 key；等式说的是：两边各转各的，点积里只剩下角度差 $(n - m)\theta$ 。

- **好处一：打分只含相对距离。**这正是 sinusoidal 那一节想要的性质，但这次它直接出现在 attention 的打分里，不用再隔着好几层间接起作用。对照上一节末尾的两个不足：它没有要学的位置参数，谈不上过拟合训练集；形式上又不像 ALiBi 那样只是距离的一条直线。
- **好处二：长程衰减。**RoFormer 论文的附录证明： $q \cdot k$ 打分的上界随相对距离增大而下降。曲线带着小幅振荡，并不完美，但趋势是"越远、上界越低"，和"近的更相关"的直觉一致。
- **问答**
 - θ 是学出来的吗？不是，是固定的。 d 维向量被切成 $d/2$ 个二维小块，第 i 块用自己的 θ_i ； θ_i 是 i 和 d 的函数，和前面 sinusoidal 的 ω_i 大致是同一个东西。
 - 维度对得上吗？整体的旋转矩阵是 $d \times d$ 的，和 q 、 k 的维度一致，矩阵乘法才成立。
 - 衰减曲线怎么来的？是数学上推出来的上界，不是实验曲线；推导在论文附录，课上不展开。
- 讲者在这部分花的时间最多，理由是：今天大多数模型都用 RoPE，而它的直觉并不显然；前面 sinusoidal 那段三角恒等式就是在为它铺路。

小注：RoFormer 里 $\theta_i = 10000^{-(2(i-1)/d)}$ ， i 从 1 到 $d/2$ ，和 sinusoidal 的频率是同一组；整体旋转矩阵是按二维小块排成的块对角阵。RoPE 在每一层 attention 里作用于 q 和 k ，value 不旋转，输入上也不再加位置向量。LLaMA、Qwen、Mistral、DeepSeek 等模型都用它；因为位置藏在角度里，后来的长上下文扩展方法（position interpolation、YaRN 等）都是在缩放这些角度上做文章。

4. 归一化：post-norm LayerNorm $\rightarrow$ pre-norm RMSNorm

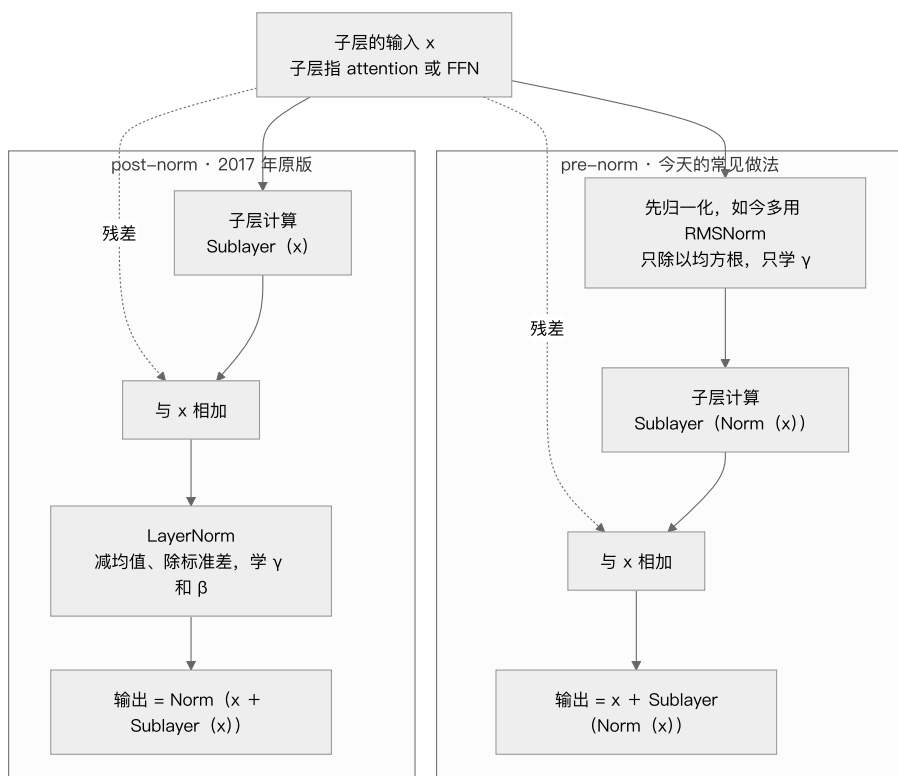


图 2-3 | 归一化放在哪: post-norm 与 pre-norm 的数据流对比 (自绘示意)。

► 看原幻灯片 46:00

- **原版的 Add & Norm:** 每个子层 (attention 或 FFN) 后面, 把子层的输入和输出相加, 再做 LayerNorm。作用是让训练更稳、收敛更快。
- **LayerNorm 做什么:** 对单个向量, 减去它各分量的均值、除以标准差, 把分量拉回一个规范的范围; 再乘一个可学的缩放 γ 、加一个可学的平移 β 。

$$\text{LayerNorm}(x) = \gamma \odot \frac{x - \mu}{\sigma} + \beta, \quad \text{RMSNorm}(x) = \gamma \odot \frac{x}{\sqrt{\frac{1}{d} \sum_{j=1}^d x_j^2}}$$

x 是一个 d 维的激活向量, μ 和 σ 是它 d 个分量的均值与标准差, γ 、 β 是可学的逐维缩放和移动, $\odot$ 表示逐元素相乘。

- **变化一: 位置从 post-norm 挪到 pre-norm。** 原版是 Norm(x + Sublayer(x)); 现在是 x + Sublayer(Norm(x)), 归一化放在向量进入子层之前。

小注: 为什么要挪, 课上没展开。通常的解释是: pre-norm 下残差支路从输入到输出是一条不被归一化打断的通路, 深层网络的梯度更稳; Xiong et al., 2020 分析过 post-norm 离不开学习率 warmup, 而 pre-norm 可以不用。GPT-2 起就已经是 pre-norm。

- **变化二：LayerNorm 换成 RMSNorm。**不减均值，只除以各分量的均方根，也只学 γ 。收敛性质相当，但要学的参数更少、算得更快。
- **问答：归一化的直觉。**层与层之间流动的向量叫 activation。它在不同层、不同分量上可能忽大忽小，变化太剧烈时各层的权重就很难学；归一化把它拉回一个不太离谱的范围。讲者给了一个可以去查的关键词：internal covariate shift。

小注：这个词出自 BatchNorm 原论文 (Ioffe & Szegedy, 2015)。后来 Santurkar et al., 2018 认为归一化起作用主要是因为让优化曲面更平滑，原来的解释至今有争议——把它当成查资料的入口就好。

- **问答：和 BatchNorm 的区别。**BatchNorm 沿另一个维度归一化：对同一个分量，跨 batch 里的其他样本来算均值和方差。Transformer 系模型普遍用 LayerNorm，一是经验上更好，二是 BatchNorm 依赖 batch，会造成训练和推理时的不一致。

5. 稀疏注意力：用局部窗口压 $O(n^2)$

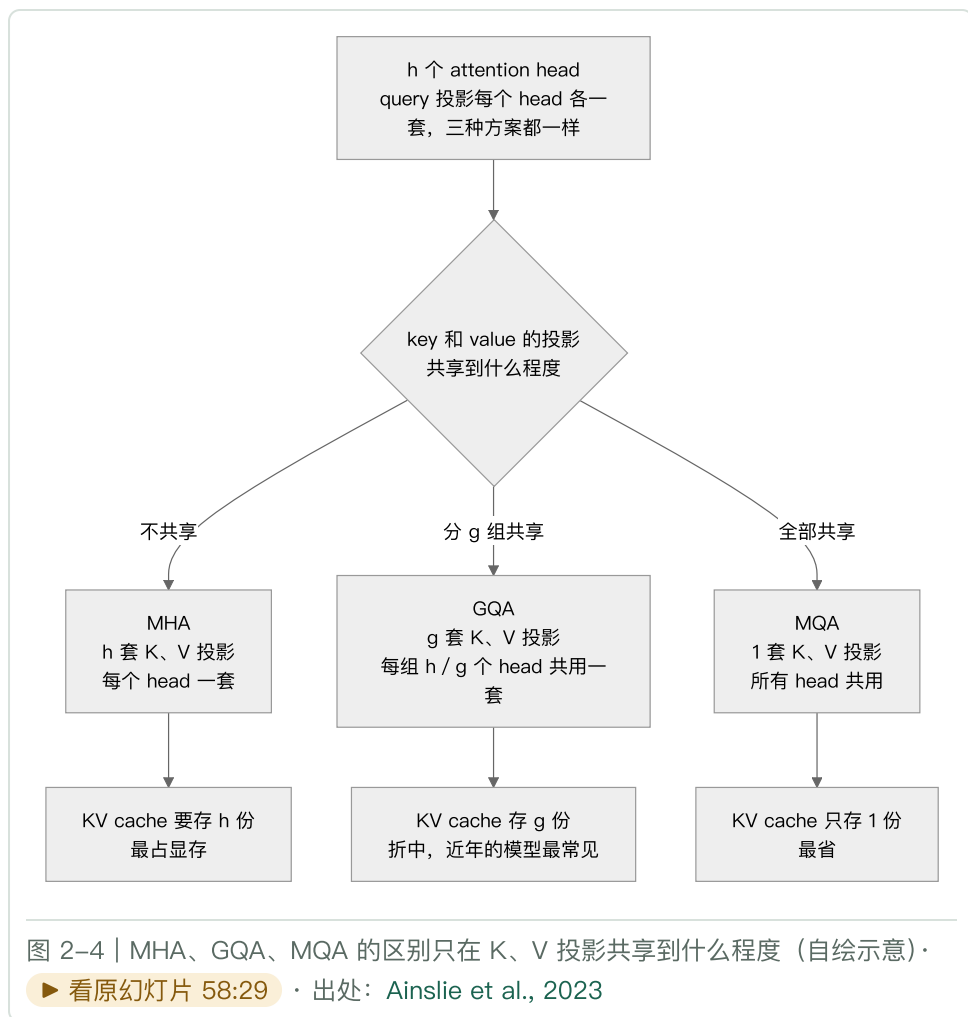
- **问题：**self-attention 让 n 个 token 两两交互，交互矩阵是 $n \times n$ ，计算量 $O(n^2)$ ，序列一长就吃不消。目标是在不掉效果的前提下，把它近似成更便宜的形式。
- **Longformer (2020)：**限制 attention 的作用范围——每个 token 只和自己邻域里的 token 交互。今天这种局部 attention 一般叫 sliding window attention。
- **问答：**是不是先算完整 attention 矩阵，再把窗口外的部分遮掉？那样就什么也没省。实际的实现会用分块 (tiling) 之类的技巧，压根不去算那个巨大的 $n \times n$ 矩阵，只算需要的部分。

小注：tiling 这个词最出名的出处是 FlashAttention (Dao et al., 2022)：把 Q、K、V 切块、逐块计算，不把 $n \times n$ 矩阵整个放进显存。它算是精确的 attention，和“只看窗口”是两件可以叠加的事。

- **今天的用法：**有些层用 local attention，有些层用 global attention，交替堆叠；比例因模型而异，没有固定配方。幻灯片上的窗口画得很小，真实模型里的窗口在几千 token 的量级。
- **感受野的类比**（学生问这像不像卷积）：以 Mistral 为例，它每一层都是 sliding window。单看一层，token 只能看到窗口内的邻居；但邻居在上一层又看过自己的邻居，信息就这样逐层向远处传。问“这个 token 实际上和哪些 token 发生过交互”，和 CV 里问“这个输出值看到了哪些输入像素”是同一个问题，也就是 receptive field。

小注: Mistral 7B 论文里窗口 $W = 4096$ 、共 32 层, 信息每层最多前进 W 个 token, 理论上的注意力跨度约 131K。层间交替的例子: Gemma 2 是 local 与 global 隔层交替, Gemma 3 提高到 5 层 local 配 1 层 global, 以压 KV cache。

6. 共享注意力头: MHA、GQA、MQA



- **做什么:** 和上一节正交的另一种省法——不再让每个 head 都有自己的 K、V 投影矩阵, 而是让若干个 head 共用一套; query 的投影仍然每个 head 一套。
- **为什么共享的是 K、V 而不是 Q:** 直觉上, query 是在"发问", 保留多种问法有好处。更核心的原因在解码: 每生成一个新 token, 都要让它去 attend 此前的所有 token, 于是前面那些 token 的 key 和 value 会被一遍遍用到。第 3 讲要讲的 KV cache 就是把这些 key、value 存起来免得重算, 而我们不希望这个缓存太大。head 之间共享 K、V 投影, 要存的份数就少了, 省的是显存。

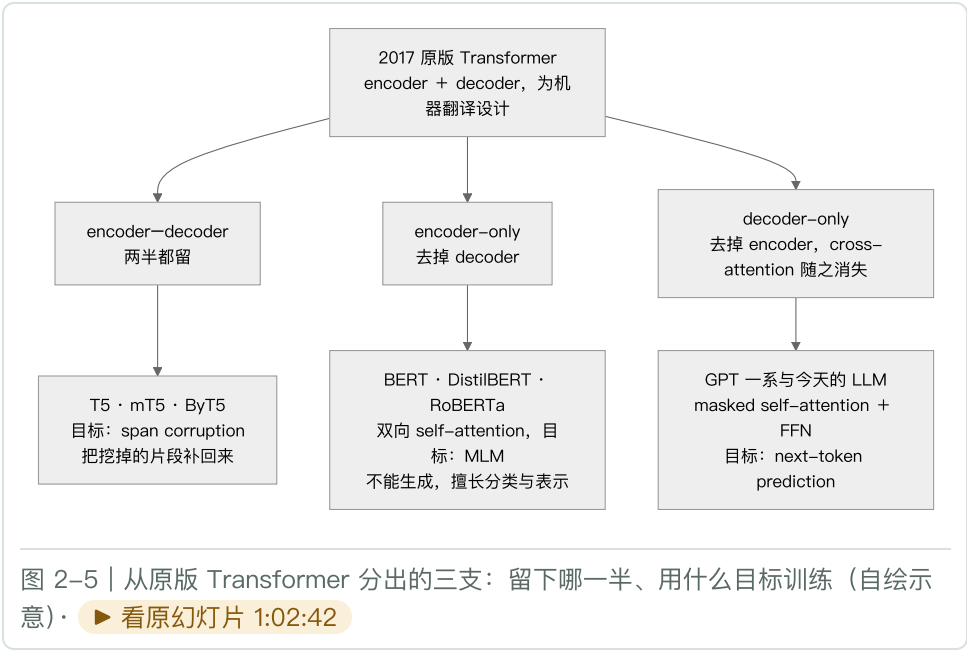
- 三档：MHA（每个 head 各一套，即原版）、GQA（h 个 head 分成 g 组，每组 h/g 个 head 共用一套）、MQA（所有 head 共用一套，最极端）。

小注：KV cache 的大小约等于 $2 \times \text{层数} \times \text{KV head 数} \times \text{每个 head 的维度} \times \text{序列长度} \times \text{每个数的字节数}$ ，所以 KV head 数砍到几分之一，缓存就小到几分之一。例：Llama 2 70B 是 64 个 query head 配 8 个 KV head，Mistral 7B 是 32 配 8。MQA 出自 [Shazeer, 2019](#)。

问答

- self-attention 和 cross-attention 都能用吗？技术上任何 attention 层都能用；但它主要在 decoder 的 masked self-attention 上起作用，而今天的 LLM 本来就是 decoder-only。
- 该选哪个？看效果、看对延迟和成本有多敏感、看模型多大、输入多长，没有标准答案。不过近年的模型多数在共享，GQA 最常见，但也不是全部。

7. 模型谱系：encoder-decoder、encoder-only、decoder-only



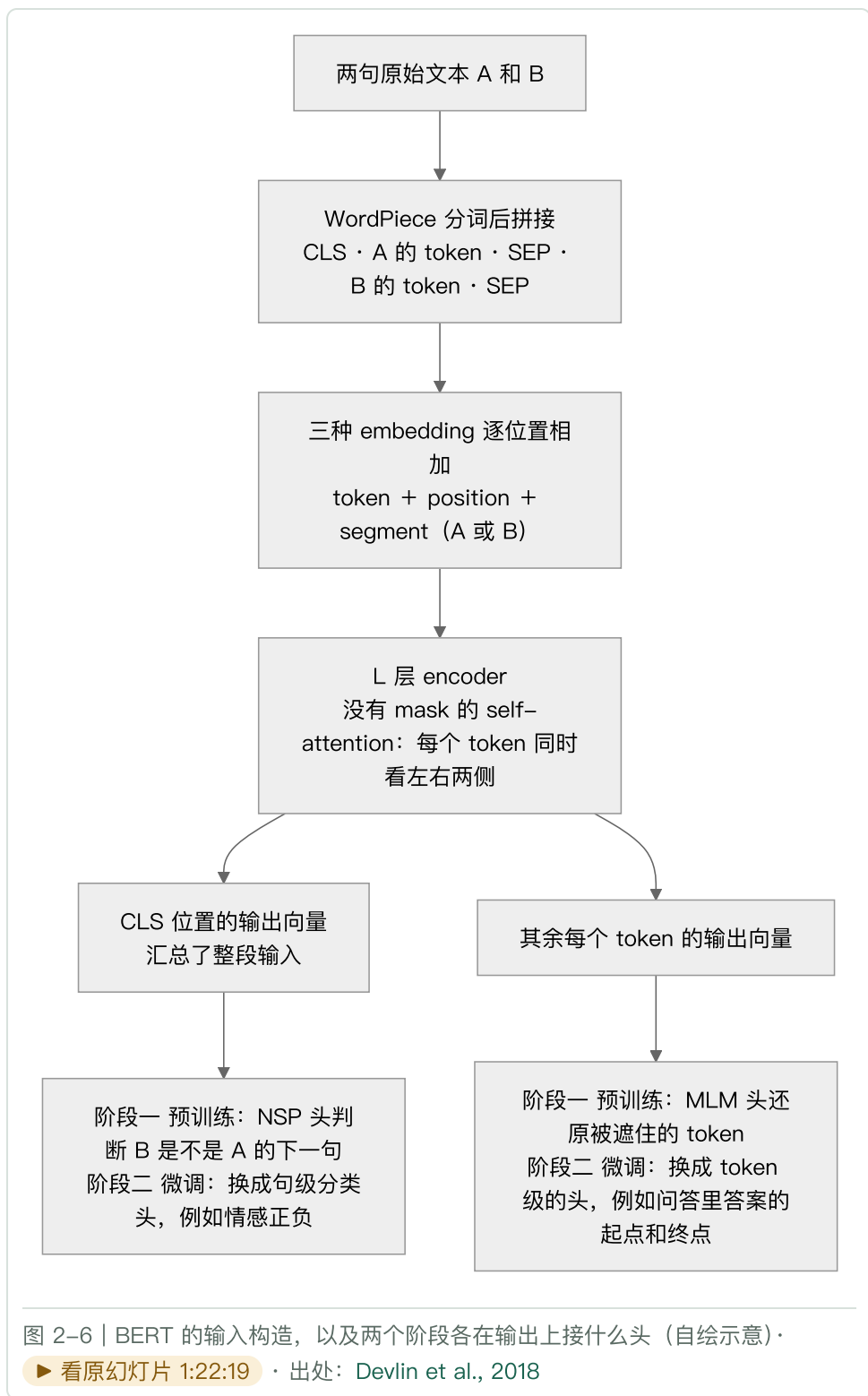
- encoder-decoder**：原版之后，这一支的代表是 T5 家族。T5 的名字来自一串 T：Text-to-Text Transfer Transformer。mT5 是多语言版（训练数据和词表都为多语言重做）；ByT5 干脆不要 tokenizer，直接在字节上操作，词表从约 3 万缩到 $2^8 = 256$ 。
- T5 的训练目标：span corruption**。原版 Transformer 的 decoder 做的是 next-token prediction；T5 换了目标：把输入句子里的若干片段（每段一个或多个 token）挖掉，各用一个 sentinel token 占

位，交给 encoder；decoder 依次输出"第一个 sentinel、它对应的原文、第二个 sentinel、它对应的原文....."，以多出来的最后一个 sentinel 收尾。比如输入 `my teddy bear [X] reading`，目标就是 `[X] is cute and [Y]`。训练时用 teacher forcing：整个目标序列一次喂给 decoder，所有位置并行预测。

小注：T5 论文的默认设置是挖掉 15% 的 token、片段平均长 3 个 token；词表是 32k 的 SentencePiece。mT5 覆盖 101 种语言。

- **encoder-only**：把 decoder 整个去掉。代价是不能生成文本——讲者说这恰恰是重点：encoder 输出的表示拿来做什么任务（情感分类、token 级分类），这些事过去要靠各自专门的模型。代表是 BERT，以及它的两个改进 DistilBERT、RoBERTa。
- **decoder-only**：把 encoder 去掉。没有 encoder 的输出可供 cross-attention，这个模块也就一并消失，每个 block 只剩 masked self-attention 和 FFN。
- **为什么最后是 decoder-only 胜出**：早期的假设是 encoder 对得到好的表示很重要，所以完整的 encoder-decoder 更流行；后来大家发现，同样的算力预算投在 decoder-only 上最划算。它的训练目标 next-token prediction 也最简单、最容易放大和泛化——span corruption 还得专门去挖空，更"定制"；而且"接着往下写"和今天的主要应用（聊天助手）天然对齐。这一支留到后面几讲。

8. BERT（上）：输入怎么拼，预训练学什么



- **名字:** Bidirectional Encoder Representations from Transformers。Encoder 好理解，就是 Transformer 去掉 decoder 的那一半。Bidirectional 指的是：encoder 的 self-attention 没有 mask，每个 token 的输出表示同时看过左右两侧的全部 token；GPT 那一支用 causal mask，每个 token 只能看自己和之前的 token。BERT 论文花了不少篇幅强调这个区别。

- **同年的 ELMo**：用多层双向 LSTM 同样得到了双向的、带上下文的词表示，想法很新，但带着 RNN 的老问题——循环结构难以放大，于是被同年出现的 Transformer 系工作盖过了风头。（ELMo 和 BERT 都是《芝麻街》里的角色，讲者顺便调侃了研究者凑缩写的爱好。）
- **三个特殊 token**：[CLS] 放在序列最前面，是个占位符，它的输出向量会汇总整段输入的信息，专门拿去做分类；[SEP] 用来分隔两句话；[PAD] 把序列补到固定长度，因为训练按 batch 进行，batch 是定长的矩阵。
- **tokenizer**：WordPiece——在训练语料上学一套合并规则，朝着使语料似然最大的方向把小单元逐步合并成词表项，词表约 3 万。讲者顺带给了量级感：这类模型的词表一般在 10^4 到 10^5 之间，ByT5 的 256 是特例。
- **输入是三种 embedding 相加**：token embedding（查表）、position embedding（前面讲过的）、新引入的 segment embedding。后者只有两个可学向量：第一句的 token 都加 A，第二句的都加 B。作者的假设是它有助于判断句子的先后关系；讲者预告，这个假设后来受到了挑战。

小注：讲者说他“不太确定”BERT 的 position embedding 是不是写死的公式——实际是 learned 的绝对位置向量，位置表只有 512 个，上下文也就封顶在 512。

- **预训练目标一：MLM (masked language model)**。随机选出一部分 token 作为预测对象：其中 80% 换成 [MASK]，10% 保持原样，10% 换成一个随机的词；模型要在这些位置还原出原来的 token。要猜一个被遮住的词就得看它左右两边的上下文，双向 attention 正是为此服务的。

小注：BERT 论文里被选作预测对象的 token 占 15%。之所以不全换成 [MASK]，是因为微调和使用时输入里根本没有 [MASK]，全换会让预训练和下游对不上。

- **预训练目标二：NSP (next sentence prediction)**。从语料里取两句话拼在一起，一半情况下它们确实前后相连，另一半不是；在 [CLS] 的输出上接一个分类头，判断 B 是不是 A 的下一句。学生问：预测下一句，不是 decoder 的活吗？——这里不生成任何东西，只是二分类。
- 两个目标的标签都能从无标注文本里自动构造出来（哪两句相邻、哪个词被遮住，自己都知道），所以是自监督。

- **论文记号与规模**：BERT 论文把层数记作 L （原版的 N ）、embedding 维度记作 H （即 d_{model} ）、head 数记作 A （即 h ）。Hugging Face 上的 BERT 常见 `cased` 和 `uncased` 两种版本，区别在预处理时是否统一转成小写。参数量在 1 亿上下。讲者推荐把这篇论文和 *Attention Is All You Need* 一起读，他印象中它的引用数在 17 万左右。

小注：BERT-base 是 $L=12$ 、 $H=768$ 、 $A=12$ ，约 1.1 亿参数；BERT-large 是 $L=24$ 、 $H=1024$ 、 $A=16$ ，约 3.4 亿。讲者回忆说“原版 Transformer 是 12 层”，这一点记岔了：原版 base 的 encoder 和 decoder 各 6 层；BERT-base 的尺寸是照着 GPT-1 定的，方便对比。

9. BERT（下）：微调、例子走查与问答

- **微调**：保留预训练好的 encoder，在上面接一个很小的任务头（线性层或一个小 FFN），用少量标注数据训练。可以冻结预训练权重、只训练新加的头，也可以全部一起训——取决于愿意花多少算力，以及目标任务和预训练任务差多远。

小注：BERT 论文里默认的微调方式是所有参数一起更新；冻结 encoder、只把输出当特征用的做法，论文在 NER 上试过，效果略低但接近。“预训练 → 微调”这条流水线也是第 4、5 讲 LLM 训练与调优的原型。

- **两类下游任务**：句级分类（如情感），只用 `[CLS]` 的输出；token 级任务（如抽取式问答），给每个 token 的输出都接上头，预测它是不是答案的起点、是不是终点。
- **例子走查**：给 “this teddy bear is so cute” 做情感分类（`uncased` 版本）。转小写 → WordPiece 切分 → 前面加 `[CLS]`、后面加 `[SEP]`，再用 `[PAD]` 补齐 → 每个位置的 token、position、segment 三个向量相加 → 过 encoder → 只取 `[CLS]` 的输出向量 → 接分类头，输出正面或负面。
- **问答**
 - 分类头具体是什么？从输出 embedding 的维度映射到任务类别（如正、负）的小网络；带一个隐藏层的话就是两个矩阵。
 - 为什么把其他 token 的输出都扔掉？句级分类里只是约定用 `[CLS]`：经过多层 self-attention，其他 token 的信息已经混进了它的表示。换成 token 级任务，每个输出都要用。
 - `[CLS]` 的 Q、K、V 怎么来？和其他 token 完全一样：它有自己的 embedding，同样被投影成 q 、 k 、 v 参与 attention。把它当成一

个普通 token 就行。

- **优点：**预训练只要无标注文本；得到的是带上下文的 embedding（ELMo 也是）；微调只需要很少的标注数据和很少的新参数，当年就超过了各任务的 SOTA；换个头就能做任意分类任务。讲者说，业界今天做情感检测这类分类任务，用 BERT 类模型仍然很常见。
- **缺点：**不能做生成；"预训练 + 微调"的两段式流程，相比一次训练到位的传统方法是额外的负担。
- **三个局限：**上下文只有 512（可以借前半场的局部 attention 等技巧去扩）；BERT-base 有 1.1 亿参数，延迟偏高，能不能更小更快？MLM 和 NSP 两个目标真的都有用吗，能不能简化？后两个问题分别引出 DistilBERT 和 RoBERTa。

10. 两个改进：DistilBERT 与 RoBERTa

- **蒸馏 (distillation) 的想法：**讲者引了 Hinton、Vinyals、Dean 的一个观点——模型输出的整个概率分布 (soft targets) 几乎装着它学到的全部知识，比只有对错的硬标签信息量大得多。于是有了 teacher-student 的做法：让小模型 (student) 去拟合大模型 (teacher) 的输出分布，而不是直接拟合硬标签。损失函数用 KL 散度：

$$\text{KL}(y_T \parallel y_S) = \sum_c y_{T,c} \log \frac{y_{T,c}}{y_{S,c}}$$

y_T 、 y_S 是 teacher 和 student 在各类别 c 上的输出概率；讲者的解读是：在一个由 teacher 描述的世界里，用 student 来建模有多糟。

- 一个有意思的特例：如果 y_T 退化成 one-hot 的硬标签，KL 就变回普通的交叉熵 $-\log y_S$ 。

小注：Hinton 等人的蒸馏论文还引入了 temperature：把 logits 除以大于 1 的 T 再做 softmax，让分布更"软"，小概率类别里的信息才不会被淹没；DistilBERT 沿用了这一点。

- **DistilBERT：**把层数减半，再用蒸馏把效果保住，结果性能几乎不掉。讲者评价这篇论文只有几页，影响却很大。

小注：DistilBERT 摘要里的数字是体积小 40%、快 60%、保留 97% 的语言理解能力。训练时除了蒸馏损失，还保留了 MLM 损失，并加了一个对齐隐藏向量方向的 cosine 损失；student 用 teacher 每隔一层的权重来初始化。

- **RoBERTa**：实验发现去掉 NSP 几乎不掉点，于是直接去掉；把遮盖改成动态的，同一段文本每个 epoch 换一种遮法；并指出 BERT 其实训练得远远不够，于是加大数据的规模和多样性，同样的 benchmark 上成绩明显提高。

小注：RoBERTa 摘要的结论是 BERT 训练得严重不足，调好之后能追平或超过此后发表各个模型；论文里数据从 16GB 加到 160GB，同时用了更大的 batch 和更长的训练。

关键图表速查（点时间戳跳到原幻灯片）

图	看什么	跳转	出处
attention map： its 在看谁	its 这一行被点亮的是 law 和 application；两个 head 点亮的侧重不同	4:51	Vaswani et al., 2017 附录
sinusoidal 点积的推导	逐对维度套 $\cos(a - b)$ 恒等式，最后只剩 $m - n$	18:29	同上
位置向量热力图	纵轴约 50 个位置、横轴是维度：低维一侧随位置快速摆动，高维一侧变化很慢	22:50	同上
带偏置的 attention 公式	softmax 里多出来的那一项；T5 是学出来的桶，ALiBi 是确定的线性式	28:14	T5、ALiBi
RoPE 的二维示意	q 按位置 m 转、k 按位置 n 转；旋转矩阵的四个元素	31:19	RoFormer
RoPE 长程衰减曲线	上界随相对距离下降，带小幅振荡；是推导出的上界，不是实验点	39:24	同上（附录）
post-norm 与 pre-norm、RMSNorm	Norm 方块相对子层和加法的位置变了；47:04 起对比 RMSNorm：不减均值、没有 β	46:00	RMSNorm 应为 Zhang & Sennrich, 2019
局部窗口的交互矩阵	每个 token 只和邻域交互， $n \times n$ 的交互矩阵里只有一小部分需要算	51:30	Longformer
Mistral 的感受野示意	逐层往上，一个 token 间接够得着的范围一层层变宽	54:10	Mistral 7B
MHA、GQA、MQA 三档	query 的头数不变，K、V 的套数从 h 到 g 再到 1	58:29	应为 Ainslie et al., 2023
span corruption 例子	encoder 输入里的 sentinel 占位；decoder 输出"sentinel、被挖内容"交替的序列	1:05:16	T5
BERT 输入的三种 embedding	token、position、segment 逐位置相加；segment 只有 A、B 两种	1:25:30	BERT
情感分类走查	从小写化、WordPiece、加特殊 token，到只取 [CLS] 输出接分类头	1:34:56	同上

图	看什么	跳转	出处
蒸馏与 KL 散度	teacher 和 student 的输出分布；标签为 one-hot 时退化成交叉熵	1:44:41	Hinton et al., 2015

提到的工作

名称	在本讲里的作用
Attention Is All You Need (Vaswani et al., 2017)	全讲的基准：sinusoidal 与 learned 位置向量、post-norm、MHA、encoder-decoder；attention map 的例子出自它的附录
T5 (Raffel et al., 2019)	两次出场：relative position bias；encoder-decoder 一支的代表与 span corruption
ALiBi (Press et al., 2021, 即 Train Short, Test Long)	不学习的线性距离偏置
RoFormer (Su et al., 2021)	提出 RoPE；附录里有长程衰减上界的推导
LayerNorm (应为 Ba et al., 2016)	原版 Add & Norm 用的归一化
RMSNorm (应为 Zhang & Sennrich, 2019)	今天常用的归一化：只除以均方根、只学 γ
BatchNorm 与 internal covariate shift (Ioffe & Szegedy, 2015)	问答里的对照和关键词
Longformer (Beltagy et al., 2020)	用局部窗口近似全局 attention
Mistral 7B (Jiang et al., 2023)	每层都用 sliding window 的例子，引出感受野类比
MQA (应为 Shazeer, 2019)、GQA (应为 Ainslie et al., 2023)	共享 K、V 投影的两档方案
KV cache	共享 K、V 的动机；第 3 讲展开
mT5、ByT5	T5 的多语言版与字节级版
BERT (Devlin et al., 2018)	下半场的主角：encoder-only、MLM + NSP、预训练加微调
GPT (OpenAI, 2018)	对照：causal mask，只看左侧
ELMo (Peters et al., 2018)	同年的双向 LSTM 上下文词向量
WordPiece	BERT 的 tokenizer
Hugging Face	模型库里 BERT 的 cased 与 uncased 版本
Distilling the Knowledge in a Neural Network (Hinton, Vinyals, Dean, 2015)	蒸馏与 soft targets 的来源；讲者说那句话出自他们的一次讲座
DistilBERT (Sanh et al., 2019)	层数减半 + 蒸馏
RoBERTa (Liu et al., 2019)	去掉 NSP、动态掩码、更多数据

名称	在本讲里的作用
RNN、LSTM（第 1 讲）	对照：顺序处理自带位置信息，但难以放大
CNN 的 receptive field	理解多层 sliding window 的类比

术语对照

English	中文
position embedding / positional encoding	位置嵌入 / 位置编码
learned position embedding	可学习的位置嵌入（每个位置一个训练出来的向量）
sinusoidal embedding	正弦位置编码（按 sin、cos 公式算，不训练）
absolute / relative position	绝对位置 / 相对位置（两个 token 相隔多远）
relative position bias	相对位置偏置（加在 softmax 里的一项）
bucketize	分桶（把相近的距离归到同一档）
ALiBi (Attention with Linear Biases)	线性偏置注意力
RoPE (Rotary Position Embedding)	旋转位置编码
rotation matrix	旋转矩阵
long-term decay	长程衰减（相距越远，打分上界越低）
extrapolation	长度外推（推理时用到比训练更长的序列）
layer normalization (LayerNorm)	层归一化（对单个向量的各分量做）
post-norm / pre-norm	后置 / 前置归一化（Norm 在子层之后还是之前）
RMSNorm	均方根归一化
activation	激活（层与层之间流动的向量）
internal covariate shift	内部协变量偏移
batch normalization	批归一化（跨 batch 里的样本做）
sparse attention	稀疏注意力（只算一部分 token 对）
sliding window / local attention	滑动窗口 / 局部注意力
global attention	全局注意力（所有 token 两两交互）
receptive field	感受野（一个输出实际"看到"的输入范围）
tiling	分块计算
projection matrix	投影矩阵（W_Q、W_K、W_V）
MHA / GQA / MQA	多头 / 分组查询 / 多查询注意力
KV cache	键值缓存（解码时存下已算过的 key、value）
cross-attention	交叉注意力（decoder 去看 encoder 的输出）

English	中文
masked (causal) self-attention	带掩码的（因果）自注意力，只能看自己和左侧
bidirectional	双向（左右两侧的上下文都看）
span corruption	片段破坏（挖掉若干片段让模型补回）
sentinel token	哨兵 token（给被挖掉的片段占位）
teacher forcing	教师强制（训练时把正确的目标序列直接喂给 decoder）
tokenizer-free / byte-level	不用分词器 / 字节级
segment embedding	段嵌入（标记 token 属于第一句还是第二句）
MLM (masked language model)	掩码语言模型
NSP (next sentence prediction)	下一句预测
pre-training / fine-tuning	预训练 / 微调
self-supervised	自监督（标签由数据自身构造）
contextual embedding	带上下文的词向量（同一个词在不同句子里向量不同）
classification head	分类头（接在 encoder 输出上的小网络）
cased / uncased	区分大小写 / 统一转小写
distillation	蒸馏
teacher / student	教师模型 / 学生模型
soft targets / hard labels	软目标（完整概率分布） / 硬标签
KL divergence	KL 散度
dynamic masking	动态掩码（每个 epoch 重新选遮盖位置）
undertrained	训练不足

字幕勘误

"two days LLMs""two days' applications" → today's LLMs、today's applications；"a mistral" → Mistral（模型名）；"another message" → another method（指 ALiBi）；"sign of this" → sine；"o of several, thousands" → 窗口大小在几千 token 的量级；"Group Query Attention" → 论文写法是 Grouped-Query Attention；"three-key encoder only models" → three key encoder-only models；"over hurdled" → 应为 overhead 或 a hurdle（推断）；"like case, uncased" → cased、uncased；"data lot" → the data a lot；"LongFormer" 即 Longformer。另有一处是讲者口误而非字幕错误："further together" 应为 farther apart。

带走的问题

1. sinusoidal 向量的点积只取决于相对距离，可它被加到 token embedding 上之后，还要过 W_Q 、 W_K 两个投影才轮到算 $q \cdot k$ ——这个性质到了 attention 打分里还剩多少？对照 RoPE 把旋转放在投影之后，这个先后次序为什么关键？
2. RoPE 的 θ_i 是固定的，模型在训练长度之外的角度组合上从没见过数据。要把上下文从 8K 扩到 128K，为什么"缩放角度"这类办法行得通？长程衰减对超长上下文是帮助还是妨碍？
3. 每层都用 sliding window，理论感受野有"层数 $\times$ W"；但信息每过一层都要被转述一次。对"十万个 token 之前的一句原话"这种需要精确取回的内容，靠逐层传递够吗？这和交替插入 global 层的设计有什么关系？
4. GQA 把 KV cache 缩到 g/h，代价是什么？为什么共享 K、V 投影几乎不掉效果，而 Q 要保持每个 head 一套——head 的多样性主要住在哪里？
5. decoder-only 赢在目标简单、容易放大。但检索、分类、打分这些"只要表示、不要生成"的环节，双向的 encoder 仍有优势。你手上的 agent 系统里，哪些环节其实更适合一个 BERT 类的小模型？

CME295 第 3 讲 | 大语言模型 (Transformers & Large Language Models)

Stanford CME295: Transformers & Large Language Models (2025 秋) · 第 3 讲 (2025 年 10 月 10 日) 视频: <https://www.youtube.com/watch?v=Q5baLehv5So> (1:48:45, 英文字幕是自动生成的, 人名和术语有识别错误) 讲者: Afshine Amidi (前半: LLM 定义、MoE、解码) · Shervine Amidi (1:07:07 起: 提示与推理优化) 课程大纲: <https://cme295.stanford.edu/syllabus/>

一句话: LLM 就是在参数、数据、算力三个维度上一起放大的 decoder-only Transformer。这一讲沿着"模型长什么样 → 怎么吐出回答 → 怎么问它 → 怎么跑得快"走了一遍: MoE 让参数变多而每个 token 的计算量不变; 解码策略和 temperature 决定怎样从"下一个 token 的概率分布"里取词; in-context learning、chain of thought、self-consistency 不改权重就能改善回答; KV cache、GQA、PagedAttention、MLA 和 speculative decoding 让生成更快、更省显存。

时间轴

时间	内容
00:00	开场与公告: 幻灯片会在每周四晚提前放到课程网站
1:02	回顾前两讲: encoder-decoder (T5)、encoder-only (BERT)、decoder-only (GPT) 三类模型
3:43	LLM 的定义: language model 加三个"大"; BERT 为什么不算; decoder-only 骨架与代表模型
7:37	MoE 的动机 (四位专家的比喻); expert、gate / router 与加权求和公式
12:43	dense MoE 与 sparse MoE (top-k); FLOPs 是什么
15:47	MoE 放在 LLM 的哪里: FFN 参数最多, 换成一组 FFN expert, 按 token 路由
20:46	routing collapse 与负载均衡的辅助损失; noisy gating (24:21 起)
25:21	问答: 辅助损失里哪一项可导; gate 的输出怎么来
27:31	问答: expert 越多总参数越大, active parameters 不变; Switch Transformer; sample efficiency
29:41	问答: expert 与注意力头无关; 每层各有一套 expert 和 router; 推理时一个 token 怎么走 (31:15 起)
34:48	Mixtral 论文里的路由上色图; MoE 小结

时间	内容
36:35	回答是怎么生成的：next-token prediction 与输出概率分布
38:34	greedy decoding 和它的两个问题
41:36	beam search：保留 K 条路径；序列 log 概率、偏好短序列与长度归一化（43:39 起）
46:36	sampling；top-k 与 top-p（49:19 起）
50:53	概率从哪来：linear 层加 softmax
52:24	temperature：公式与直觉；板书推导 $T \rightarrow 0$ 和 $T \rightarrow \infty$ （55:40 起）
1:02:23	确定性：随机性只来自采样； $T = 0$ 在实践中仍可能不一致
1:04:53	guided decoding：以生成 JSON 为例
1:07:07	下半场（Shervine）：context length 的几个叫法与量级；context rot（1:09:03 起）
1:11:37	prompt 的四个组成部分
1:14:09	in-context learning：zero-shot 与 few-shot；给示例不一定更好（1:16:50 起）
1:18:35	chain of thought；拿推理文本来调试（1:20:27 起）
1:21:59	self-consistency；问答：怎么抽答案、并行采样的延迟
1:25:00	推理优化总览：exact 与 approximate 两类
1:27:32	KV cache；问答：训练时用不用；用 GQA 减少 K/V 的份数（1:31:09 起）
1:33:09	PagedAttention：整段预留显存的浪费、三种浪费、按 block 管理
1:36:30	MLA：把 K、V 压成一个低维 latent 再缓存
1:41:20	speculative decoding：draft 模型起草，target 模型一次验证
1:46:28	multi-token prediction；收尾

核心内容

1. 什么叫 LLM：decoder-only，外加三个"大"

- **先回顾**（第 2 讲）：Transformer 衍生出三类模型——encoder-decoder（T5，文本进、文本出）；encoder-only（BERT，不生成文本，但产出的向量很有用，取 CLS token 的向量就能做分类，实际里常拿来给句子和文档编码，讲者说后面的课还会用到）；decoder-only（GPT，去掉 encoder，也就不再需要 cross-attention，文本进、文本出）。
- **language model** 的定义：给 token 序列分配概率的模型。落到实现上，就是给定前文，输出"下一个 token 是词表里每个词"的概率。
- **large** 指三件事一起放大：

- 参数：几千亿已经不稀奇；一般至少十亿（1B）量级才叫 LLM。
- 数据：按预训练用的 token 数算，从几千亿到几万亿，最大的已经到几十万亿。
- 算力：训练和运行都要一批 GPU；不过现在有不少优化能让模型在消费级 GPU 上跑（讲者说后面会讲）。这三个“大”具体怎么训出来，是第 4 讲（LLM training）的内容。
- **这个名字很新**：2018—19 年还没有公认的定义，早期有人把 BERT 也算作 LLM。按现在已经稳定下来的用法，LLM 专指规模很大、能生成文本的模型；BERT 是 encoder-only，不生成文本，所以不算。
- **骨架**：decoder-only 的每个 block 只剩 masked self-attention、FFN（前馈网络）和 add & norm。GPT、Llama（Meta）、Gemma（Google）、DeepSeek、Mistral、Qwen 都是这个结构；讲者估计现在九成以上的 LLM 是 decoder-only。

2. Mixture of Experts：参数变多，每个 token 的计算量不变

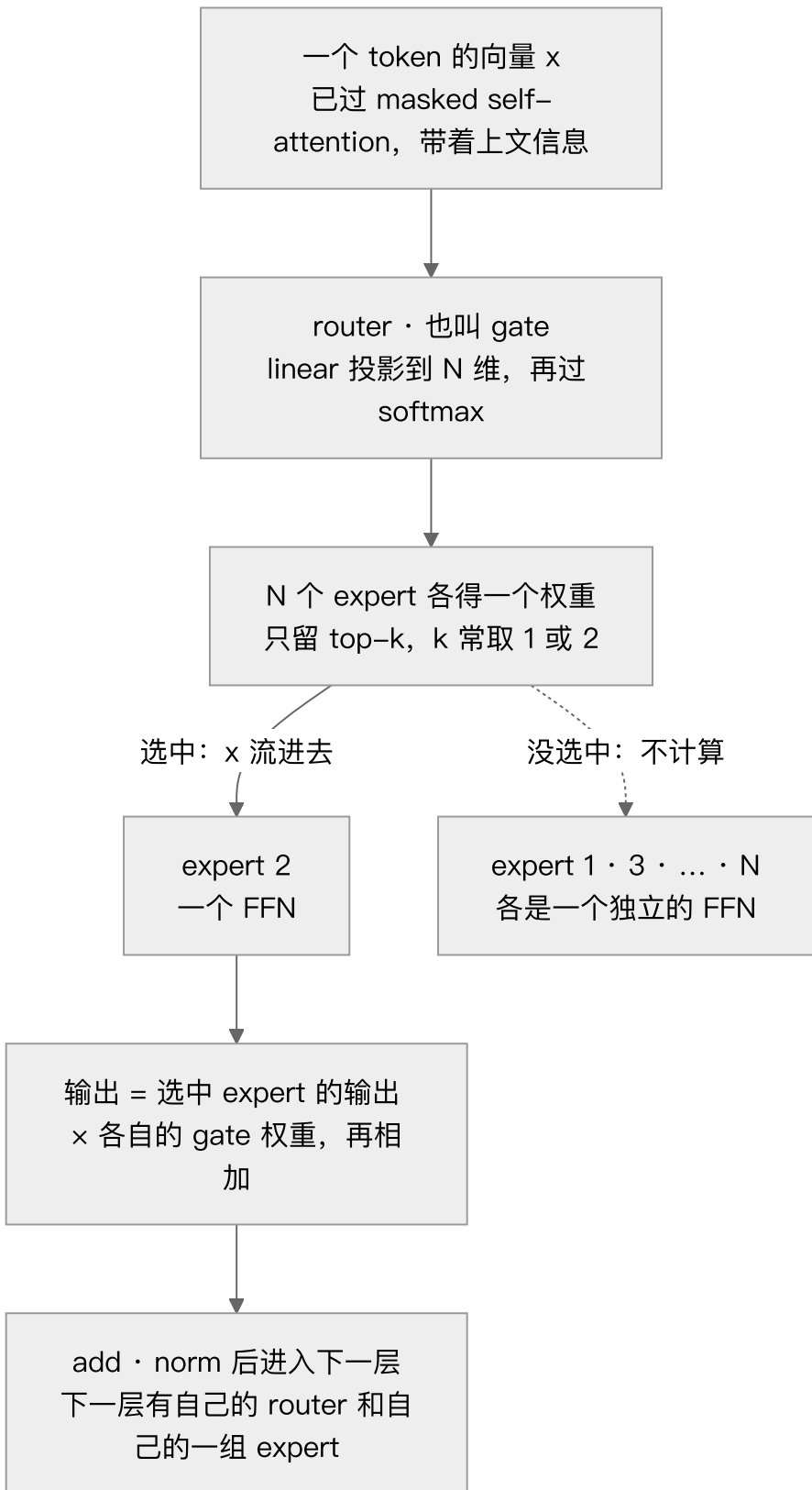


图 3-1 | 一个 token 穿过 sparse MoE 层的路径 (自绘示意) ·

► 看原幻灯片 18:45 · 出处: Fedus et al., 2021

- **为什么要有它**: 模型有几千亿参数, 按原来的做法, 每预测一个 token, 全部参数都要参与一次前向计算, 训练和推理都很贵。讲者的

比喻：屋里坐着数学家、物理学家、化学家和历史学家，你有一道数学题，并不需要四个人都答一遍。MoE 的想法就是：给定输入，只让模型的一部分参与计算。

- **两个部件**：N 个 expert（各是一个子网络）和一个 gate（也叫 router，本身是个小网络）。gate 看输入 x，给每个 expert 打一个权重；这一层的输出是各 expert 输出的加权和。gate 和 expert 一起训练，照常前向、算 loss、反向传播。

$$\hat{y} = \sum_{i=1}^N G(x)_i E_i(x) \quad \text{sparse:} \quad \hat{y} = \sum_{i \in \text{TopK}(G(x))} G(x)_i E_i(x)$$

$\hat{y}$ 是这一层的输出， $E_i(x)$ 是第 i 个 expert 对输入 x 的输出， $G(x)_i$ 是 gate 给它的权重；sparse 版只对权重最高的 k 个 expert 求和，其余的根本不计算。

- **dense 与 sparse**：dense MoE 不限制参与的 expert 数，权重是一组 0 到 1 之间、像概率分布一样的数——所有 expert 都得算，只是数学家的话分量更重，计算一点没省。sparse MoE 只取权重最高的 k 个（k 是超参数，常取 1 或 2），这才省下计算。计算量用 FLOPs（floating-point operations，一次前向里加法、乘法这类浮点运算的总次数）衡量：sparse MoE 的 FLOPs 比 dense 低。
- **放在 LLM 的哪里**：decoder block 里有 masked self-attention、FFN、add & norm 三处，参数最多的是 FFN：它把 d_{model} 维的向量升到 d_{ff} 维再降回来，参数量约 $2 \times d_{\text{model}} \times d_{\text{ff}}$ （外加 bias），而 d_{ff} 通常比 d_{model} 大得多（第 1 讲）。所以现在的 LLM 把每层的 FFN 换成“一个 router 加一组 FFN expert”，用 sparse 的方式，每次只激活其中一两个。

小注：讲者对比时用的是单个注意力头的投影矩阵（ $d_{\text{model}} \times d_k$ ）。把所有头加起来，注意力的四个投影矩阵约 $4 \times d_{\text{model}}^2$ ；按常见的 $d_{\text{ff}} = 4 \times d_{\text{model}}$ ，FFN 约 $8 \times d_{\text{model}}^2$ ，大致占一个 block 参数的三分之二——结论不变。

- **路由是逐 token、逐层的**（图 3-1）：token 过完 masked self-attention、带上了上文信息之后，向量先进 router（投影到 N 维再 softmax，得到对 N 个 expert 的概率），取 top-k，只有被选中的 expert 真的做计算。同一句话里相邻的两个 token 可以去不同的 expert；同一个 token 在第 1 层去了 expert 3，到第 2 层可能去 expert 1。每层有自己的 router 和自己的一组 expert，权重不共享。

- **总参数与 active parameters**: expert 加得越多, 总参数越大, 模型容量也越大; 但每个 token 只经过 k 个 expert, 一次前向真正用到的参数 (active parameters) 基本不变。这就是 MoE 模型能做到万亿参数的原因——讲者推荐阅读的 Switch Transformer 做到了一万多亿。论文还显示这类模型更 sample efficient: 达到同样效果所需的训练时间更短。

小注: Switch Transformer 最大的 Switch-C 是 1.6 万亿参数、2,048 个 expert; 摘要里说同等算力下预训练最多快 7 倍。代价在课上只带了一句"都是 trade-off": 所有 expert 都得放进显存, 显存占用跟着总参数走, 速度才跟着 active parameters 走。

- **Mixtral 的路由上色图**: Mistral 团队在论文里把一段文本的每个 token 按"在这一层被分到哪个 expert"涂上颜色。讲者让大家看的是颜色混得很匀, 说明各 expert 的使用大致均衡; 如果整段一个颜色, 就是下一节说的 routing collapse。

小注: Mixtral 8x7B 每层 8 个 expert, 每个 token 选 2 个, 总参数 47B, active 13B。论文的路由分析没有发现 expert 按主题 (数学、生物、哲学.....) 分工, 规律更偏句法: 代码里的缩进 token 总去同一个 expert, 相邻 token 也常被分到同一个 expert。"数学家、历史学家"只是帮助理解动机的比喻, 别按字面理解。

- **问答**: expert 和注意力头没有关系。多头注意力的结果在注意力层末尾已经拼接并投影回 d_model 维, 之后才进 MoE; router 是每层一个, 不是每个头一个。

3. 训练 MoE 的麻烦: routing collapse 与负载均衡

$$\mathcal{L}_{\text{aux}} = \alpha \cdot N \cdot \sum_{i=1}^N f_i P_i$$

N 是 expert 的个数, α 是控制这一项分量的超参数, f_i 是这一批 token 里被分到 expert i 的比例, P_i 是 router 给 expert i 的平均概率。

- **问题**: 训练时很可能出现 router 总是选那一两个 expert、其余的从不被激活的情况, 叫 routing collapse。没被选中的 expert 白占参数, MoE 的容量优势就没了。
- **办法**: 在原来的 loss 上加一个辅助项 (上式)。讲者说不必抠公式细节, 记住它的作用是把 f_i 和 P_i 都推向均匀分布, 让各 expert 的使用量差不多。它和主 loss 一样按 mini-batch 计算: 一批 token 过完模型, 统计这两个量, 再一起反向传播。

小注：这个式子应出自 Switch Transformer，论文里 $\alpha = 0.01$ 。均匀路由时 $f_i = P_i = 1/N$ ，求和得 $1/N$ ，再乘 N 恰好是 1——乘 N 是为了让这一项的大小不随 expert 个数变化。课上有学生追问 f_i 怎么求导，讲者当场没有答案；论文的回答是 f_i （来自 argmax 的计数）不可导，梯度只通过 P_i 传回 router。

- **其他手段**：noisy gating——给 gate 的打分加一点噪声，让别的 expert 也有机会被选中。有学生问能不能用 dropout，讲者说当然可以叠加，noisy gating 的思路和它相近。

小注：noisy top-k gating 出自 Shazeer et al., 2017。那篇论文也描述了 collapse 的成因：早期碰巧被多选的 expert 得到更多训练、变得更好，于是更常被选，是一个自我强化的过程。

4. 回答是怎么生成的：greedy、beam search 与采样

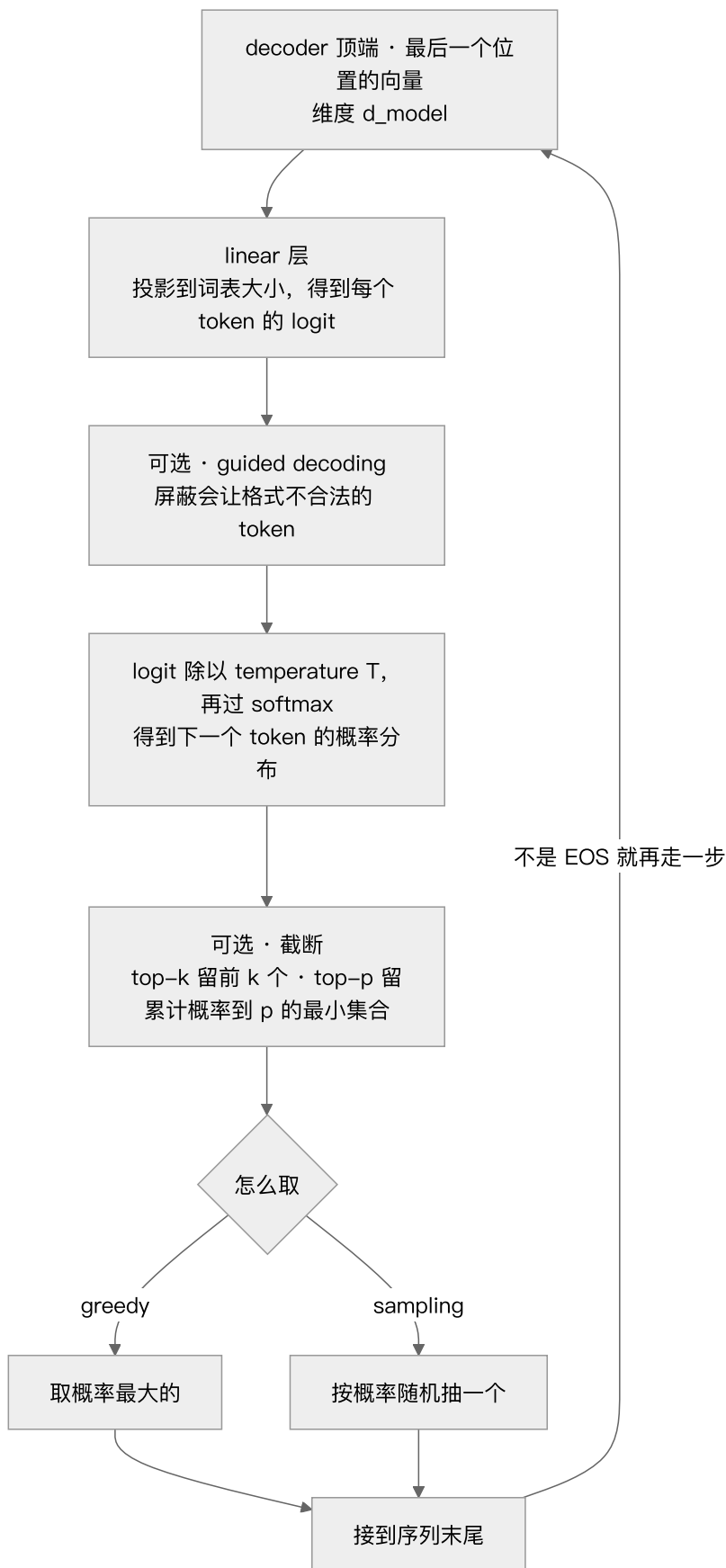


图 3-2 | 生成一个 token 的完整一步：从隐向量到选出下一个 token（自绘示意，把第 4、5 两节的部件串在一起）· [▶ 看原幻灯片 50:53](#)

- **生成是一个循环**：LLM 每次只做一件事——给定目前的序列，输出"下一个 token"在整个词表上的概率分布。选一个 token 接到末尾，再喂回去，直到出现 EOS（结束符）。"怎么选"就是解码策略。
 - **概率从哪来**：decoder 顶端，最后一个位置的向量（ d_{model} 维）先过一个 linear 层投影到词表大小，得到每个 token 的打分（logit），再过 softmax 变成和为 1 的概率。
 - **greedy decoding**：每步取概率最大的 token。两个问题：
 1. 整个前向计算是确定的，于是同样的输入永远得到同样的输出，没有多样性——而 ChatGPT、Gemini 每次的回答都不太一样。
 2. 每步最优不等于整句最优。我们要的是概率高的整条序列；第一步选了 0.8 的 token，后面可能一路都是低概率，反倒是从 0.2 那个 token 出发的路径后面步步都高，整条序列的概率更大。
 - **beam search**：同时保留 K 条概率最高的路径（K 叫 beam size 或 beam width）。每一步把这 K 条都往前扩展一个 token，在所有候选里再留下最好的 K 条，最后取总分最高的那条。序列的分数是各步 log 概率之和。
 - 偏好短句：每个概率都小于 1，越乘越小，序列越长分数越低。所以实际会做长度归一化：把分数除以 token 数的某次幂。
 - 局限：要维护 K 条路径，计算和存储都更多；而且它找的仍是"模型认为最可能"的那一句，没有多样性。它适合机器翻译这类答案比较确定的任务，LLM 对话一般不用。
 - **sampling**：按概率分布随机抽下一个 token。概率高的常被抽中，概率低的偶尔也会被抽中，多样性就来了。麻烦在尾部：词表里大量概率极低的 token 理论上也可能被抽到。所以先截断再抽：
 - top-k：只在概率最高的 k 个 token 里抽。
 - top-p：按概率从高到低累加，取累计概率刚超过阈值 p 的最小集合，在里面抽。
- 小注：top-p 又叫 nucleus sampling，应出自 [Holtzman et al., 2019](#)；top-k sampling 应出自 [Fan et al., 2018](#)。两者的差别：top-p 的候选个数随分布变化，模型很有把握时只剩两三个候选，拿不准时候选自动变多。
- **问答**：这些都只是推理时的事。训练时不采样，而是拿输出分布和真实的下一个 token（hard label）比较，算 loss。

5. temperature、确定性与 guided decoding

$$P(i) = \frac{\exp(x_i/T)}{\sum_j \exp(x_j/T)}$$

x_i 是 linear 层给第 i 个 token 的 logit, T 是 temperature, 分母对词表里所有 token 求和; $T = 1$ 就是普通的 softmax。

- **它做什么**: T 是 softmax 里除在 logit 下面的一个数。 T 小, 分布变尖, 概率集中到最高的几个 token 上; T 大, 分布变平, 原本排不上号的 token 也有了机会。写创意文案用高 T ; 想要稳定、接近"模型认为最好"的回答用低 T 。
- **板书的推导**: 设 k 是 logit 最大的 token, 分子分母同除以 $\exp(x_k / T)$, 每一项就成了 $\exp((x_j - x_k) / T)$, 指数都不大于 0。 $T \rightarrow 0$ 时, $j \neq k$ 的指数趋向负无穷, 这些项全部消失, $P(k) \rightarrow 1$, 等价于 greedy; $T \rightarrow \infty$ 时, 所有指数趋向 0, 每项趋向 1, P 变成"1 除以词表大小"的均匀分布。
- **随机性只有一个来源**: Transformer 的前向计算里没有任何随机的东西, 唯一的随机是"按分布抽 token"这一步。所以只要 $T > 0$, 同一输入每次的输出都可能不同; 理论上 $T = 0$ 就完全确定。但实践中 $T = 0$ 也可能得到不同结果: 浮点加法的结果和相加顺序有关, GPU 上数量级相差很大的数以不同顺序归约, 结果会有细微差别。讲者推荐了一篇选读文章 [Defeating Nondeterminism in LLM Inference](#)。

小注: 这篇文章 (Horace He 与 Thinking Machines Lab, 2025 年 9 月) 比课上的转述更进一步: "并发加浮点"只解释了一半, 主因是推理 kernel 不具备 batch invariance——同一条请求的数值结果随 batch 大小变化, 而 batch 大小取决于服务器当时同时在处理多少别人的请求。文中给出了 batch-invariant 的 RMSNorm、矩阵乘法和 attention 实现。做 agent 评测想要可复现的话, 只设 $T = 0$ 是不够的。

- **guided decoding**: 想让输出严格符合某种格式 (比如 JSON)。朴素做法是在 prompt 里提要求, 拿到后校验, 不合法就重来。guided decoding 则在生成的每一步把"会让输出不合法"的 token 直接屏蔽: JSON 的第一个 token 只能是左花括号, 接着只能是属性名.....; 同时有多个合法 token 时, 再按平常的策略 (采样等) 在里面选。哪些 token 合法怎么判定, 讲者只给了关键词: finite-state machine 和 context-free grammar。

小注: 代表工作如 Outlines (Willard 与 Louf, 2023), 把正则表达式或 JSON schema 编译成词表上的状态机。各家 API 的 structured output、

6. 提示：context length、prompt 的结构与 in-context learning

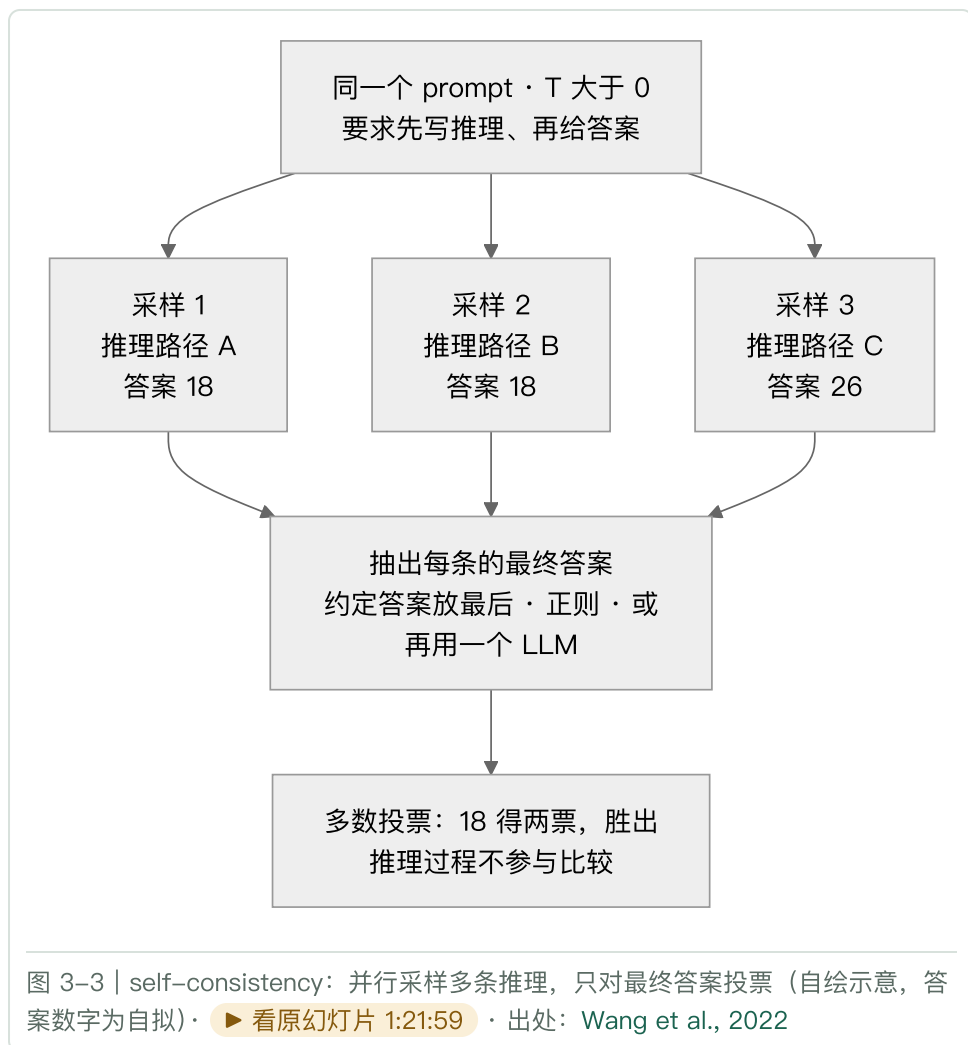
- **context length**（也叫 context size、window size）：模型一次前向能接收的 token 数，也就是 self-attention 实际看到的序列长度。现在的模型在几万、几十万到上百万 token 的量级（Gemini 等已到百万级）。长度一上去，self-attention 的 n^2 开销就成了问题，第 2 讲那些降低注意力复杂度的技巧就是为此准备的（问答）。
- **长不等于好——context rot**：在 needle in a haystack 测试里（把答案埋进越来越长的文本，再提问），上下文越长，模型把埋着的信息找出来的能力越差；文本里混进干扰项（distractor，和答案相似但不不对的内容）时掉得更厉害。上下文里放什么同样重要。实践含义：检索类问题里，尽量只把相关的那一段喂给模型，而不是全塞进去。

小注：出处是向量数据库公司 Chroma 2025 年 7 月的技术报告（Hong、Troynikov、Huber），测了 18 个模型，不是同行评审论文。报告还发现：问题和答案的语义相似度越低，性能随长度掉得越快。

- **prompt 的四个部分**（没有正式理论，是一个好用的心智模型）：
context（设定场景、角色、日期）、instructions（要做什么任务）、input（任务作用的对象）、constraints（格式、长度、安全等限制）。以 ChatGPT 为例：系统写入的“你是 ChatGPT、现在是几月几号几点”是 context；用户的提问是 instructions 加 input；用户看不见的安全规则属于 constraints。
- **in-context learning**：不改任何权重，只靠上下文里的内容让模型完成任务。learning 这个词在这里名不副实，模型的权重什么都没学。分两类：
 - zero-shot：只给任务本身。
 - few-shot：在真正的输入前放几组“输入 → 输出”的示例。
- **示例通常有用，但不总是更好**：示例能把模型引到你想要的任务上，代价是要花功夫收集，上下文变长、计算更多。讲者特意强调“通常”二字：模型的推理能力变强之后，把 instructions 写好——用自然语言讲清楚任务该怎么做——效果可以追平甚至超过给示例。原因是示例是一个有限集合，模型会往示例上靠，遇到和示例分布不同的输入反而不好泛化。这方面的文献还在发展，讲者举了 Plan-and-Solve：让模型先自己列计划再解题。

小注：Plan-and-Solve (Wang et al.) 是 ACL 2023 的论文，不算新。它是一种 zero-shot 提示，摘要里说在数学推理上全面超过 zero-shot CoT，与 8-shot CoT 相当。

7. Chain of thought 与 self-consistency



- **chain of thought (CoT)**: 要求模型在给答案前先写出推理过程 (rationale)，准确率会提高。讲者的解释：逼模型直接吐一个数字，它未必能把题目里的条件和答案联系起来；写出完整的推理链，答案是怎么来的就清楚了。它可以和 few-shot 结合：示例里写上“推理加答案”，模型就会照这个格式先推理再作答。

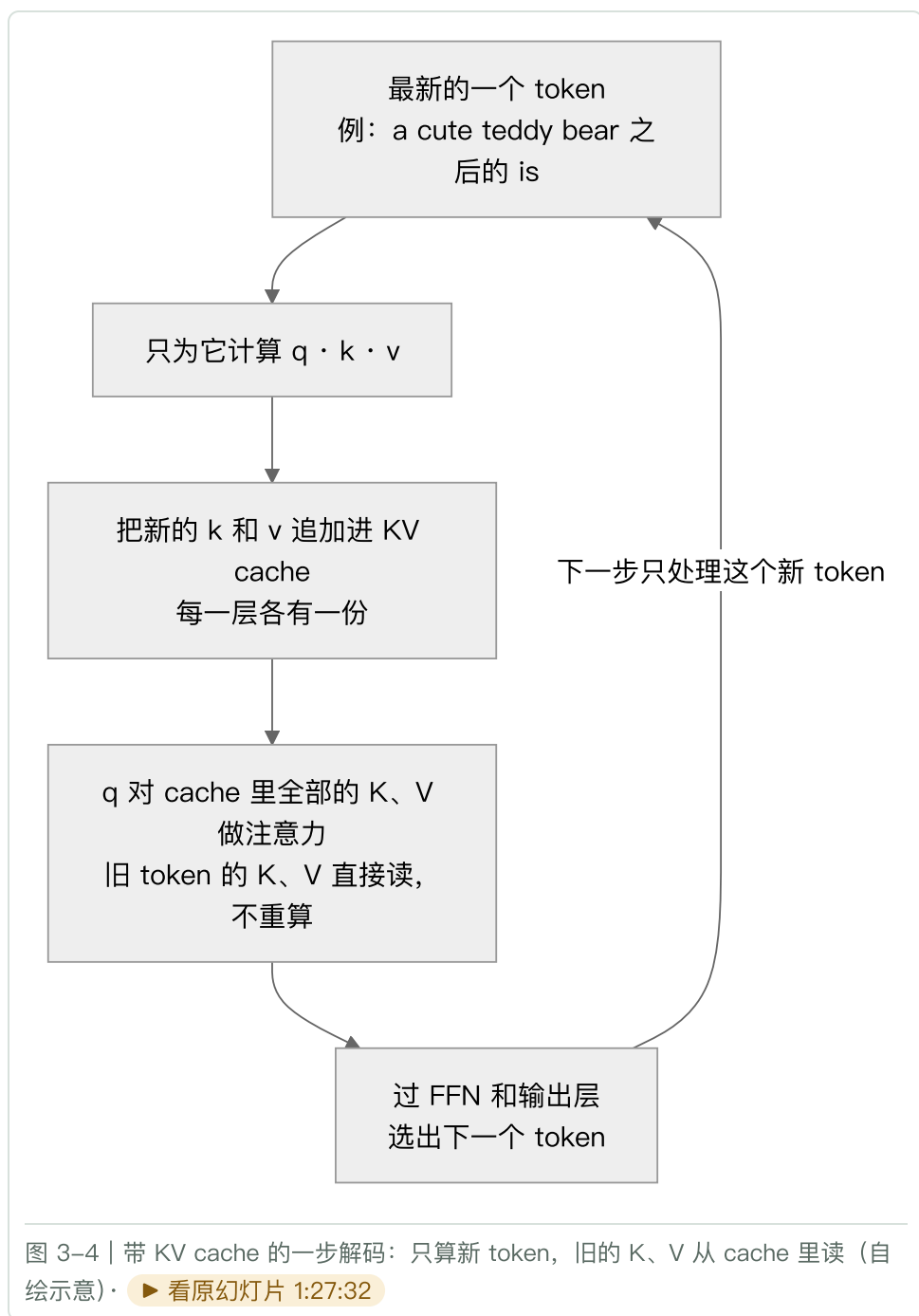
小注：一个常见的补充解释——模型生成每个 token 的计算量是固定的，把中间步骤写出来，等于给难题分配了更多计算，后面的 token 还能直接引用前面的中间结果。出处是 Wei et al., 2022；第 6 讲 (LLM reasoning) 从这里接着往下讲。

- **顺带的好处是能调试**：LLM 出错时没法像传统模型那样去翻权重，能看的只有它吐出的 token。推理文本能帮你追到根因：问"这只熊明年几岁"，如果推理里写着"现在是 2019 年"，就知道是上下文里的日期给错了。
- **代价**：要多生成 token，更慢、更贵；一般可以接受。
- **self-consistency** (图 3-3)：在 CoT 的基础上，同一个 prompt 采样多次 ($T > 0$ ，每次的推理路径不同)，从每条回答里抽出最终答案，多数投票。不比较推理过程，只数答案。

小注：论文摘要里的幅度：GSM8K +17.9%、SVAMP +11.0%、AQuA +12.2%。

- **问答**
 - 怎么知道它真的更好：需要带标准答案的 benchmark 来验证；原论文用的是算术和数学推理类数据集。
 - 怎么从一大段文字里抽出答案：在 prompt 里要求"最终答案放在最后"然后取末尾；用正则；或者再用一个 LLM 来抽。
 - 多次采样是串行的吗：并行。各条采样互不进入对方的上下文，总延迟约等于最慢的那条（算力当然是 N 份）。

8. 推理优化（上）：KV cache，以及怎么让它少占显存



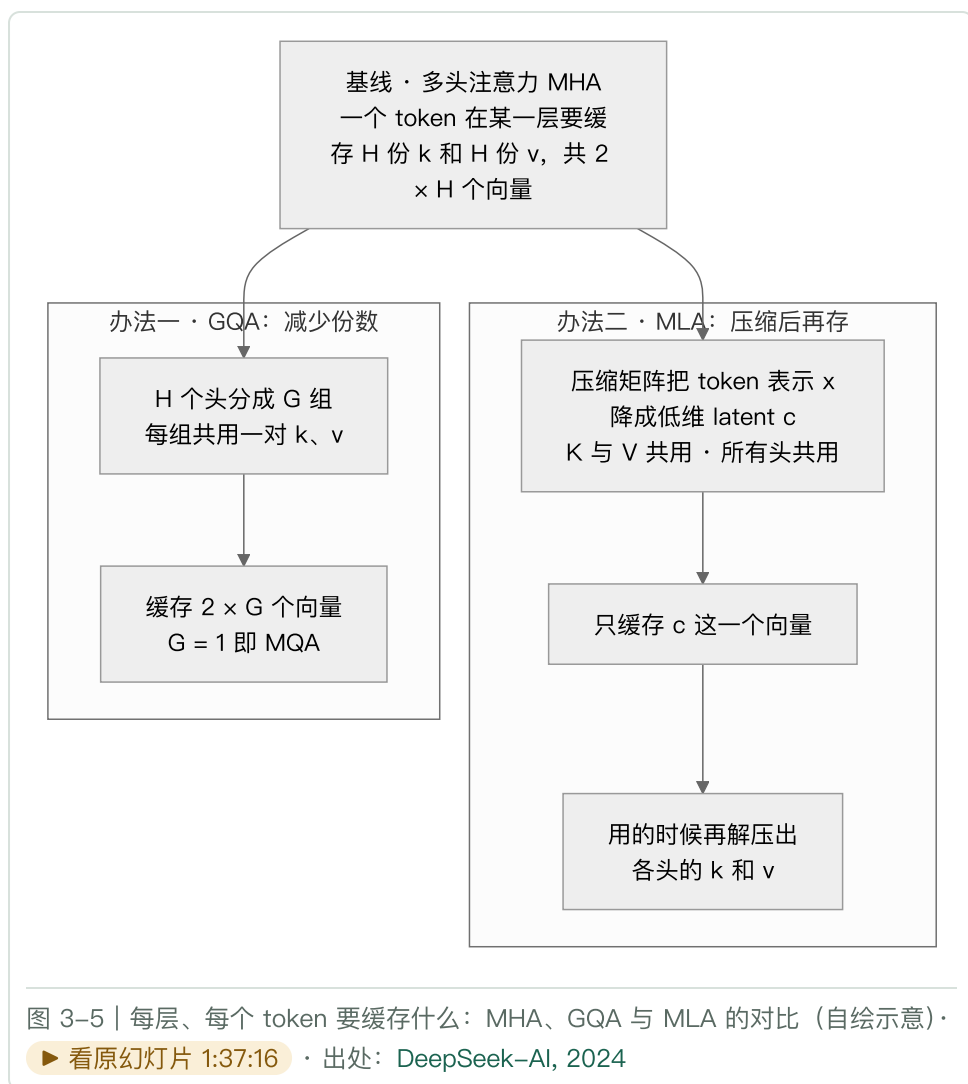
- **两类思路**：Shervine 把推理优化分成 exact（计算结果完全不变，只是去掉冗余、把显存管好、或者把式子换个等价写法）和 approximate（允许结果略有不同，换更低的成本：改架构、压缩表示、在预测 token 上做文章）。讲的时候按位置走：先 attention 层，再输出层。
- **KV cache 为什么成立**：生成第 t 个 token 时，当前 token 的 query 要和前面所有 token 的 key、value 做注意力。前面那些 token 的 K 和 V 在之前的步骤里已经算过，之后也不会再变（causal mask 下每个位置只看得到自己和前面的 token，后来新增的 token 影响不到

它)，再算一遍纯属浪费。于是把每层的 K、V 存起来，每步只为最新的 token 算 q、k、v，把新的 k、v 追加进 cache，q 直接对 cache 里的全部 K、V 做注意力（图 3-4）。旧 token 的 query 以后用不到，所以没有"Q cache"。

- **训练时用不用**（学生问）：不用。训练是 teacher forcing：整条序列一次喂进去，所有位置并行计算，不存在一步一步生成，也就没有可复用的东西。
- **cache 本身成了负担**：每层、每个 token 要存 H 份 key 和 H 份 value（H 是头数）。第一个办法来自第 2 讲：grouped-query attention（GQA）让若干个 query 头共用一组 K/V；组数 $G = H$ 就是原始的多头注意力， $G = 1$ 就是 multi-query attention。现在的 LLM 大多取一个折中的 G。
- **PagedAttention**（推理框架 vLLM 背后的技术）：推理服务器同时服务很多请求。朴素做法是每来一个请求，就按最大 context length 预留一整段连续显存，因为不知道什么时候才生成到 EOS。这样浪费很大，显存很快就装不下新请求。论文把浪费分成三种：reserved（给将来的 token 预留、眼下空着的位置）、internal fragmentation（按最大长度多留、到结束也用不上的位置）、external fragmentation（内存分配器在各段之间留下的空隙）。PagedAttention 借用操作系统分页的思路：把 KV cache 切成固定大小的 block（论文里一个 block 放 16 个 token），用多少分配多少，block 在物理显存里不必连续，另有一张表记录"第几个 token 的 K、V 在哪个 block"。碎片大幅减少，同一张卡能同时服务更多请求。

小注：论文里的数字——当时的推理系统中，KV cache 显存只有 20.4%—38.2% 真正存着 token 状态；vLLM 在相同延迟下把吞吐提高 2—4 倍。讲者口头对 reserved 和 internal fragmentation 的解释有些含糊，这里按论文的定义写。

9. MLA：把 K 和 V 压成一个低维向量再缓存



$$c_t = W^{\text{down}} x_t, \quad k_t = W_K^{\text{up}} c_t, \quad v_t = W_V^{\text{up}} c_t$$

x_t 是某层第 t 个 token 的表示， c_t 是压缩后的 latent 向量（维度远小于所有头的 key 拼起来的维度）， W^{down} 是 K、V 和所有头共用的压缩矩阵， W_K^{up} 和 W_V^{up} 是各自的解压矩阵；cache 里只存 c_t 。

- **动机**：GQA 减少的是 K/V 的份数，每一份的维度没变。DeepSeek 的 Multi-head Latent Attention (MLA) 换了一个角度：让要缓存的表示本身更紧凑。
- **三步**：
 1. 低秩分解：把"token 表示 $\rightarrow$ key"的投影矩阵拆成两段，先降到一个低维的中间空间，再升回来。
 2. K 和 V 共用压缩矩阵：同一个 token 的 key 和 value 由同一个 latent 解压出来；解压矩阵各用各的，所以 K 和 V 仍然可以不同。

3. 所有头共用：不再是 H 个头各存一份，每个 token 在每层只缓存一个 latent 向量。

- **效果**：要存的东西少了很多。DeepSeek-V2 论文还报告效果比标准多头注意力更好，讲者的解读是共享的低秩表示起了类似正则化的作用——收益不只在硬件上。
- **问答**：latent 的维度是设计时定下的超参数，对所有 token 都一样；理论上可以不同，实践中一般统一。

小注：讲者把名字说成了 multi-latent attention，全称是 Multi-head Latent Attention。DeepSeek-V2（总参数 236B，每个 token 激活 21B，本身也是 MoE）摘要里的数字：KV cache 比 DeepSeek 67B 少 93.3%，最大生成吞吐是它的 5.76 倍。课上略去了一个细节：RoPE 位置编码和这种低秩压缩不兼容，论文另外缓存了一小段带位置信息的 key (decoupled RoPE)。

10. 推理优化（下）：speculative decoding 与 multi-token prediction

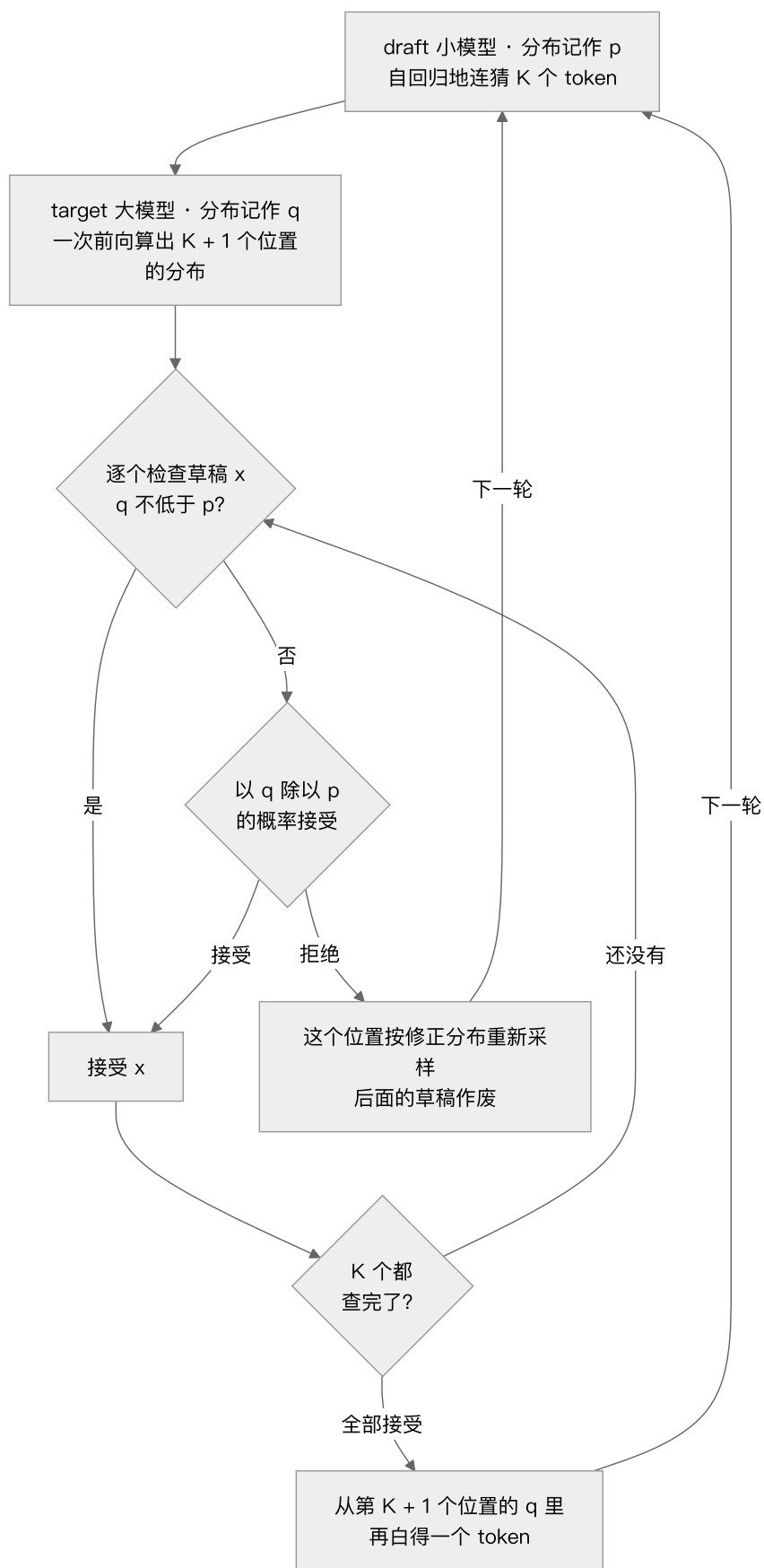


图 3-6 | speculative decoding 的一轮：起草、一次验证、逐个接受或拒绝（自绘示意）· [▶ 看原幻灯片 1:42:22](#) · 出处：Chen et al., 2023

$$P_{\text{accept}}(x) = \min\left(1, \frac{q(x)}{p(x)}\right), \quad p_{\text{resample}}(x) = \frac{\max(0, q(x) - p(x))}{\sum_y \max(0, q(y) - p(y))}$$

p 是 draft 模型在这个位置给出的分布， q 是 target 模型的分布， x 是 draft 猜的 token；草稿被拒绝时，从右边这个" q 比 p 多出来的部分"归一化后的分布里重新抽一个。

- **前提：推理是 memory-bound。**生成时每一步的耗时主要花在搬运权重和 cache 上，算术本身不是瓶颈。所以大模型一次前向同时处理 $K + 1$ 个 token，和只处理 1 个 token 的耗时差不多。只要有人能便宜地猜出后面 K 个 token，大模型就能一次验完。
- **流程（图 3-6）：**
 1. 小的 draft 模型自回归地连猜 K 个 token（它小，所以快）。
 2. 把"前文加 K 个草稿 token"一次喂给大的 target 模型，得到每个位置上 target 的分布 q ，外加第 $K + 1$ 个位置的分布——这是白得的。
 3. 按顺序检查每个草稿 token x ：target 给它的概率不低于 draft 给的，直接接受；否则以 $q(x) / p(x)$ 的概率接受。第一次被拒绝时，这个位置按上式右边的修正分布重新采样，后面的草稿作废。
 4. 如果 K 个全部被接受，再从第 $K + 1$ 个位置的 q 里抽一个 token。于是大模型每做一次前向，产出 1 到 $K + 1$ 个 token。
- **为什么输出分布和只用大模型完全一样：**这不是拍脑袋的配方。用全概率公式把"接受"和"拒绝后重采样"两个分支写开：接受分支贡献 $\min(p, q)$ ，重采样分支恰好补上 $q - \min(p, q)$ ，加起来就是 q 。讲者把它留作练习，说论文里的证明只有几行。有学生指出这很像 rejection sampling，讲者确认了。

小注：幻灯片用的记号（draft、 K 、第 $K + 1$ 个位置的 q ）应出自 Chen et al., 2023 (DeepMind, Chinchilla 70B 上解码加速 2–2.5 倍)；同期独立提出的还有 Leviathan et al., 2022 (T5-XXL 上 2–3 倍)。接受规则的具体式子按论文写出，课上只口头描述。

- **multi-token prediction (MTP)：**把 draft 模型并进主模型。在最后一层 decoder 的表示上接多个输出头，第 1 个头预测下一个 token，第 2 个头预测再下一个……训练目标从 next-token prediction 改成一次预测多个 token。推理时，后面的头充当 draft，第 1 个头就是主模型，拿草稿回来做接受或拒绝。论文里的验证是 greedy 式的；因为架构和训练目标都变了，上面那个"分布严格一致"的性质不再直接成

立。讲者让大家记住两点：训练目标变了；draft 和 target 合在一个模型里，不用另外维护一个小模型。

小注：应为 Gloeckle et al., 2024 (Meta)：共享主干上接 n 个独立输出头，摘要里说用 4-token 预测训练的模型推理最多快 3 倍。DeepSeek-V3 也用了 MTP 训练目标，不过它的预测模块是顺序串联的，不是并列的头。

11. 本讲的推理优化放在一起看

技术	作用位置	省的是什么	模型或输出变不变
KV cache	attention	旧 token 的 K、V 不再重算（用显存换计算）	不变
GQA	attention 结构	K/V 的份数	改架构，训练时就得这样设计
PagedAttention	显存管理	预留和碎片造成的显存浪费	不变
MLA	attention 结构	每个 token 缓存的向量个数与维度	改架构
speculative decoding	输出层	大模型前向的次数	输出分布不变
multi-token prediction	训练目标与输出头	同上，且不需要单独的 draft 模型	改训练目标；greedy 验证，没有分布一致的保证

小注：最后一列是按讲者给的 exact / approximate 标准做的归类，属于笔者推断；幻灯片上的分组以原片为准。

关键图表速查（点时间戳跳到原幻灯片）

图	看什么	跳转	出处
MoE 的基本结构	gate 指向被选中的 expert，输入只流进那一个；紧接着是 dense 与 sparse 两种写法的公式	9:23	应出自 Shazeer et al., 2017
FFN 换成 MoE	decoder block 里 FFN 的位置被"router 加一组 expert"取代；路由以 token 为单位	18:45	Switch Transformer
负载均衡辅助损失	α 、 N 、 f_i 、 P_i 各是什么；目标是两个量都趋向均匀	21:16	应出自 Switch Transformer
Mixtral 路由上色图	某一层里每个 token 按所去的 expert 上色；颜色混得匀，说明没有 routing collapse	34:48	Mixtral of Experts
beam search 示例	$K = 2$ 时每一步保留哪两条路径；最后按序列 log 概率取最高的	41:36	—

图	看什么	跳转	出处
top-k 与 top-p	同一个分布上，两种截断方式各留下哪些 token	49:19	—
带 temperature 的 softmax	T 出现在公式的哪里；55:40 起是板书推导 T $\rightarrow 0$ 和 T $\rightarrow \infty$ 两个极限	52:24	—
guided decoding 生成 JSON	每一步只有哪些 token 合法；有多个合法 token 时回到普通采样	1:04:53	—
context rot	上下文越长、干扰项越多，needle in a haystack 的检索表现越差	1:09:03	Chroma 技术报告
self-consistency	多条推理路径、各自的最终答案、多数投票	1:21:59	Wang et al., 2022
KV cache 示意	生成到 is 这个 token 时，哪些 K、V 直接从 cache 读，哪些要新算	1:27:32	—
PagedAttention 显存布局	先看朴素做法里三种浪费各在哪，再看按 block 分配之后的样子 (1:35:14 起)	1:34:13	Kwon et al., 2023
MLA 的压缩与解压	压缩矩阵先在 K 和 V 之间共用，再在所有头之间共用，最后每层每个 token 只剩一个缓存向量	1:37:16	DeepSeek-V2
speculative decoding 流程	draft 逐个生成的草稿怎样一次送进 target；接受与拒绝的规则；多出来的第 K + 1 个 token	1:42:22	应为 Chen et al., 2023

提到的工作

名称	在本讲里的作用
T5 · BERT · GPT	回顾：三类 Transformer 模型的代表
Llama、Gemma、DeepSeek、Mistral、Qwen	decoder-only LLM 的例子
sparse MoE 与 noisy top-k gating (Shazeer et al., 2017)	MoE 基本形式与 noisy gating 的出处（课上未点名，应出自）
Switch Transformer (Fedus et al., 2021)	推荐阅读：万亿参数的 sparse MoE、sample efficiency；负载均衡损失应出自这里
Mixtral of Experts (Jiang et al., 2024)	路由上色图的来源
beam search	保留 K 条路径的搜索式解码，机器翻译常用
top-k sampling (Fan et al., 2018) · top-p / nucleus sampling (Holtzman et al., 2019)	两种截断采样（课上未点名，应出自）

名称	在本讲里的作用
Defeating Nondeterminism in LLM Inference (Thinking Machines Lab, 2025)	选读: T = 0 为什么仍然不确定
finite-state machine / context-free grammar; Outlines	guided decoding 的实现思路; Outlines 是笔者补充的代表工作
Gemini	百万级 context length 的例子
Cursor	讲者提到代码助手里多用 context length 这个叫法
Context Rot (Chroma, 2025) · needle in a haystack	上下文越长检索越差的证据, 以及所用的测试方法
ChatGPT	prompt 四部分的例子; "每次回答都不同"的例子
GPT-3 (Brown et al., 2020)	zero-shot / few-shot 这套说法的出处 (课上未点名)
Plan-and-Solve Prompting (Wang et al., 2023)	"写好 instructions 可以不输示例"的例子
Chain-of-Thought Prompting (Wei et al., 2022)	先写推理再作答
Self-Consistency (Wang et al., 2022)	多次采样加多数投票
MQA (Shazeer, 2019) · GQA (Ainslie et al., 2023)	第 2 讲的内容, 这里用来减小 KV cache
PagedAttention / vLLM (Kwon et al., 2023)	按 block 管理 KV cache 显存
MLA / DeepSeek-V2 (2024)	低秩压缩 K、V
speculative decoding (Leviathan et al., 2022; Chen et al., 2023)	小模型起草, 大模型验证, 输出分布不变
multi-token prediction (Gloeckle et al., 2024; DeepSeek-V3)	draft 内置于主模型 (课上所指论文为推断)

术语对照

English	中文
language model	语言模型: 给 token 序列分配概率的模型
large language model (LLM)	大语言模型
decoder-only	只保留 decoder 的 Transformer
FLOPs (floating-point operations)	浮点运算次数, 衡量一次前向的计算量 (不是每秒次数)
mixture of experts (MoE)	混合专家
expert	专家: MoE 层里的一个子网络, 在 LLM 里就是一个 FFN

English	中文
gate / router	门控 / 路由器：决定 token 去哪个 expert 的小网络
dense MoE / sparse MoE	稠密 MoE（所有 expert 都算） / 稀疏 MoE（只算 top-k 个）
active parameters	激活参数：一次前向真正用到的参数
routing collapse	路由坍塌：router 总选少数几个 expert
load balancing loss / auxiliary loss	负载均衡损失 / 辅助损失
noisy gating	带噪声的门控
sample efficiency	样本效率：达到同样效果所需的训练量更少
next-token prediction	预测下一个 token
decoding	解码：从概率分布里选出 token 的过程
greedy decoding	贪心解码
beam search / beam width	束搜索 / 束宽（保留的路径数 K）
length normalization	长度归一化
top-k sampling / top-p (nucleus) sampling	在前 k 个里采样 / 在累计概率到 p 的最小集合里采样
logit	softmax 之前的原始打分
temperature	温度：softmax 里除在 logit 下面的系数
guided (constrained) decoding	引导解码 / 约束解码
finite-state machine / context-free grammar	有限状态机 / 上下文无关文法
context length / context window	上下文长度 / 上下文窗口
context rot	上下文腐化：输入越长表现越差
needle in a haystack	大海捞针测试
distractor	干扰项
in-context learning	上下文学习（不改权重）
zero-shot / few-shot	零样本 / 少样本
chain of thought (CoT)	思维链
rationale	推理依据
self-consistency	自洽性：多次采样后对答案做多数投票
KV cache	键值缓存
teacher forcing	训练时把真实序列整条喂入，各位置并行预测
multi-query / grouped-query attention (MQA / GQA)	多查询 / 分组查询注意力

English	中文
PagedAttention	分页注意力：按固定大小的 block 管理 KV cache
internal / external fragmentation	内部碎片 / 外部碎片
Multi-head Latent Attention (MLA)	多头潜在注意力
low-rank	低秩：用两个小矩阵的乘积代替一个大矩阵
latent	潜向量：压缩后的中间表示
speculative decoding	投机解码
draft model / target model	草稿模型 / 目标模型
rejection sampling	拒绝采样
memory-bound	瓶颈在访存而不在计算
multi-token prediction (MTP)	多 token 预测

字幕勘误

"Shervin / Sherwin" → Shervine；"Aashish" → Afshine；"Guan" → Qwen；"Deep Seek" → DeepSeek；"mass self-attention" → masked self-attention；"writer router" → router；"flops" → FLOPs；"upper probabilities" → output probabilities；"LM inference" → LLM inference；"context rots" → context rot；"cursor" → Cursor；"your plan and solve" → Plan-and-Solve；"rational" → rationale；"key vacation caching" → KV caching；"external values H and 1" → extreme values（ $G = H$ 和 $G = 1$ 两个极端）；"page detention" → PagedAttention；"vllm" → vLLM；"eight copies"、"eight projection matrices" → H copies、H projection matrices（H 是头数，不是 8）；"H search embeddings" → H such embeddings；"Q K + 1" → q 的第 K + 1 个位置。另有两处是讲者口误，不是识别错误：MLA 被说成 multi-latent attention；讲接受规则时把 target 说成了 draft（应为 target 给的概率不低于 draft 给的就接受）。

带走的问题

- MoE 的比喻是"按学科分工的专家"，但路由发生在每一层、每个 token 上，Mixtral 的分析也没看到按主题分工。那 expert 学到的到底是什么？总参数涨、active 参数不变，部署时的瓶颈会转移到哪里（显存、跨卡通信）？

2. 你的 agent 评测想要"同一输入得到同一输出"。只设 $T = 0$ 为什么不够？随机性是从哪里溜进来的，其中哪些你能控制？（第 8 讲 LLM evaluation 会再碰到可复现性。）
3. 什么时候 few-shot 示例会帮倒忙？如果要在自己的任务上比较"加示例"和"把 instructions 写细"，你会怎么设计对照，又怎么把 context rot 的代价算进去？
4. 自己推一遍：speculative decoding 的输出分布为什么和只用 target 模型完全一样？加速比取决于哪些量（草稿接受率、K、draft 模型自身的耗时）？ $T = 0$ 时接受规则退化成什么？
5. KV cache 的大小约等于"层数 $\times$ token 数 $\times$ 每个 token 的缓存量"。GQA、MLA、PagedAttention 各动的是哪一项？上下文到了百万 token，哪一项先撑不住？

CME295 第 4 讲 | LLM 训练 (LLM Training)

Stanford CME295: Transformers & Large Language Models (2025 秋) · 第 4 讲, 2025 年 10 月 17 日 (期中考试前的最后一讲) 视频: https://www.youtube.com/watch?v=VIA_jt_3Qc4 (1:47:27; 英文字幕为自动生成, 缩写和专名错得较多, 见文末勘误) 讲者: Afshine Amidi (开场到 1:02:23: 预训练与训练优化) · Shervine Amidi (1:02:23 起: SFT、评测、LoRA / QLoRA) 课程大纲: <https://cme295.stanford.edu/syllabus/>

一句话: LLM 的训练分两大步——先在数千亿到十几万亿 token 的原始文本上做"预测下一个 token"的预训练 (约 10^{25} FLOPs, 模型大小和数据量按 Chinchilla 的"每个参数约 20 个 token"搭配), 为此得靠数据并行加 ZeRO、模型并行、FlashAttention 和混合精度, 把参数、激活、梯度、优化器状态塞进每张只有 80 GB 的 GPU; 再用小几个数量级的高质量"指令—回答"数据做 SFT (loss 只算回答部分), 把只会续写的 base model 变成助手, 算力不够就用 LoRA / QLoRA 只训练一个低秩增量——而"调得好不好", 至今没有哪一个数字说得清。

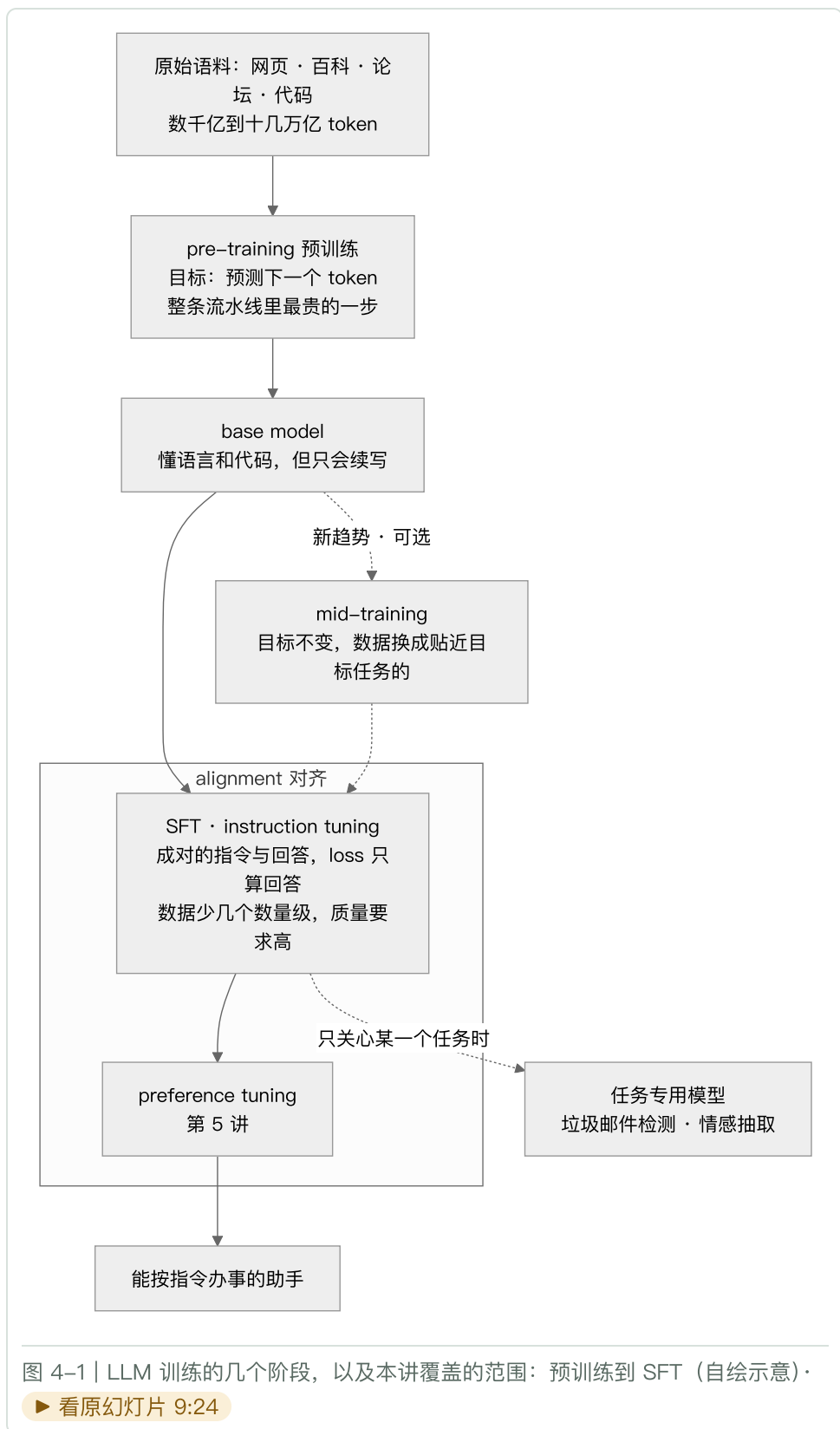
时间轴

时间	内容
0:00	课务: 下周期中 (第 1—4 讲, 闭卷); 12 月 10 日期末 (第 5—9 讲)
4:11	回顾第 3 讲: MoE、三种解码方式与 temperature、KV cache 等推理优化
7:19	从"一任务一模型"到 transfer learning; LLM 的两阶段范式
10:24	预训练: 预测下一个 token; Common Crawl 等数据来源; GPT-3 300B、Llama 3 15T token
13:26	两个记号: FLOPs (计算量) 与 FLOPS (硬件速度)
16:34	Scaling laws: 算力、数据、参数越多越好; 大模型更 sample efficient
18:37	固定算力怎么分: IsoFLOP 曲线、Chinchilla 的 20 倍经验值; GPT-3 训练不足
20:09	问答: 架构影响不大; 跨代模型之间有没有迁移, 没法一概而论
21:55	预训练的难处: 成本、knowledge cutoff、知识编辑、照搬训练数据
24:49	训练优化总览: 矩阵乘法与 GPU / TPU
26:34	训练一步要存什么: 激活、梯度、Adam 的两个 moment
30:13	显存有限: H100 每张 80 GB, 只能多卡分摊
31:09	Data parallelism: 切 batch、每卡一份模型、梯度取平均; 通信代价

时间	内容
33:51	ZeRO：优化器状态、梯度、参数逐级分片
35:51	Model parallelism：expert / tensor / pipeline 三种
38:26	FlashAttention：GPU 的两种内存 HBM 与 SRAM
40:39	标准 attention 的三趟 HBM 读写；softmax 按行归一化带来的困难
43:17	tiling，以及分块 softmax 的缩放因子技巧
47:25	问答：切的是整行还是网格；缩放因子是精确迭代算出来的
49:30	第二个想法 recomputation；结果表：多算了，却更快更省
52:37	Quantization：浮点数的 sign / exponent / mantissa；FP32、FP16、FP64、BF16
55:14	同一张 GPU，精度越低算得越快
56:00	Mixed precision training：权重 FP32，forward / backward 用 FP16
58:52	问答：是否所有层都降精度；各家自己重做 scaling law（Llama 3 的 405B）；量化的数值范围
1:02:31	Shervine 接手：base model 只会续写（泰迪熊例子）
1:05:29	SFT 的定义；目标函数：loss 只算输出部分
1:09:21	Instruction tuning 的数据：类别、人工与合成、安全（拒答与 hedging）
1:15:25	问答：指令没写明细节，模型怎么泛化
1:17:28	数量级：13k 条对 1,000 万条，都远小于预训练；调优后的泰迪熊回答
1:20:00	SFT 的难处：数据贵、prompt 分布要对得上
1:22:02	问答：会不会背出训练样本；temperature 与多样性；归根到底靠数据
1:26:10	评测：MMLU、GSM8K 等 benchmark 的分类
1:28:16	training on the test task：要比，就得处境相同
1:30:50	Chatbot Arena 及其问题：不稳、可操纵、事实性、人群偏差、安全偏差
1:36:04	alignment = SFT + preference tuning；新出现的 mid-training
1:37:53	LoRA：冻结 W0，只训练低秩的 B 和 A
1:41:17	LoRA 加在哪：attention 还是 FFN；学习率 10 倍、怕大 batch
1:44:24	问答：rank 要不要搜
1:45:16	QLoRA：NF4、double quantization、显存省约 16 倍

核心内容

1. 大图：从"一任务一模型"到"预训练 + 调优"



- **这一讲的位置:** 第 1 讲讲 self-attention 和 Transformer, 第 2 讲讲它的改进与变体, 第 3 讲定义了 LLM (MoE、解码方式、KV cache 等推理优化); 这一讲回答"这些模型是怎么训出来的"。

- **以前的做法**：一个任务训一个模型。要检测垃圾邮件就专门训一个，要抽取情感再训一个，各自走一遍 train / validation / test。可这些任务并非互不相干——都得先读懂文字。
- **transfer learning**（迁移学习）：不从零开始，先拿一个已经学到通用能力的 pre-trained model，再针对自己的任务调权重。这个想法由来已久，LLM 的训练范式就建在它上面（图 4-1）：
 - 第一阶段 pre-training：在海量数据上训练，只求学会语言和代码本身的结构；
 - 第二阶段 tuning：从预训练好的权重出发继续训练，让模型适配具体用途。
- **阶段的名字**（Shervine 在 1:36:04 补充）：SFT 加上第 5 讲的 preference tuning，合称 alignment（对齐）。最近还冒出一个 mid-training，夹在预训练和 SFT 之间——目标函数和预训练一样，数据换成贴近目标任务的那一类。
- **本讲分工**：Afshine 讲预训练，以及“这么大的模型怎么训得动”（第 2—7 节）；Shervine 讲调优的第一步 SFT、怎么评、怎么省着调（第 8—10 节）。

2. 预训练：目标、数据，和两个长得几乎一样的记号

- **目标**只有一个：给定前文，预测下一个 token。第 3 讲说过，LLM 是 text-to-text 模型，九成以上是 decoder-only，一个 token 一个 token 地往后生成；预训练就是拿人类写下的一切文字来练这件事。它是整个训练流程里最贵的一步，算力、时间、钱都是。
- **数据**：能找到的都算——英文、其他语言、各种编程语言的代码。论文里最常见的名字是 Common Crawl（持续抓取的公开网页，讲者说每月约 30 亿页）；此外有 Wikipedia，有 Reddit 这类社交媒体上的对话，代码来自 GitHub、Stack Overflow 等。
- **规模**按 token 数算，量级是数千亿到数万亿、乃至十几万亿：GPT-3 用了 300B token，Llama 3 用了 15T。
- **FLOPs 与 FLOPS**：到处都在用，先分清。
 - FLOPs = floating-point operations，浮点运算的次数，是计算量的单位。训练一个 LLM 大约是 10^{25} FLOPs 的量级。它大致正比于“参数个数 × 训练 token 数”，没有通用的精确公式，因为还取决于架构：MoE 模型每次只激活一部分参数（第 3 讲），同样的规模下算得更少。

- FLOPS (全大写) = floating-point operations per second, 每秒能做多少次浮点运算, 是硬件速度的指标, GPU 规格表上都会标。
- 有的论文会把两者混着写, 只能看上下文。

小注: 最常用的近似是 $C \approx 6 \cdot N \cdot D$ (Kaplan et al., 2020: 每个参数、每个 token, forward 约 2 次运算, backward 约 4 次)。代进去: GPT-3 是 $6 \times 175B \times 300B \approx 3 \times 10^{23}$; Llama 3 405B 是 $6 \times 405B \times 15T \approx 4 \times 10^{25}$, 和课上说的 10^{25} 量级对得上。

3. Scaling laws 与 Chinchilla: 算力固定, 钱该花在模型上还是数据上

- Kaplan et al., 2020 做了大量实验, 分别改变三个量: 算力、数据集大小、模型参数量。结论是任何一个变大, 模型预测下一个 token 的能力都稳定变好。于是 2019 到 2024 年间, 大家一路把模型往大里做。
- 论文的另一个观察: 大模型更 **sample efficient**——处理同样多的 token, 大模型达到的效果更好。
- 但算力有限。Chinchilla 论文 (Hoffmann et al., 2022) 换了一个问法: 算力预算固定, 模型大小和训练 token 数怎么搭配最划算? 做法是固定一个预算 (图上同一种颜色的一条曲线), 在这个预算内训练大小不同的模型——模型越大, 喂得起的 token 越少——每条曲线都有一个最低点; 把各预算下的最低点放在一起看, 得到一条经验关系: **训练 token 数约为参数量的 20 倍时, 算力花得最值。**
- 按这个标准, GPT-3 (175B 参数只配了 300B token, 不到 2 倍; 按 20 倍算要 3.5T) 属于严重 undertrained。

$$C \approx k \cdot N \cdot D, \quad D_{\text{opt}} \approx 20 N$$

C 是训练的总计算量 (FLOPs), N 是模型参数个数, D 是训练用的 token 数, k 是随架构而变的常数 (见第 2 节小注); 第二个式子就是 Chinchilla 的经验结论: 预算 C 给定时, 让 D 约等于 20 倍的 N。

小注: Chinchilla 论文摘要——训练了 400 多个模型 (70M 到 16B 以上参数、5B 到 500B token), 结论是模型大小和 token 数应当同比例放大 (模型翻一倍, token 也翻一倍); 据此训练的 Chinchilla 只有 70B 参数, 但数据是 280B 的 Gopher 的 4 倍, 算力相同而效果全面更好。"20 倍"是从论文的最优配置表里读出来的经验值 (70B 对 1.4T token)。

• 问答

- 这些规律依赖具体架构吗？讲者：现在说 LLM，基本就是 decoder-only 的 Transformer；论文自己也说架构细节影响不大，起决定作用的是 token 数和模型大小。
- 新一代模型会不会从上一代的权重出发？多数模型闭源，不公开这些，没法一概而论。
- 换一套设置，这个配比还成立吗（59:22 的提问）？各家会在自己的设置上重做一遍：先用小模型、少量数据拟合出自己的关系，再外推到大规模。Llama 3 论文专门有一节讲这件事，405B 这个数就是这么定出来的。
- 预训练的几个难处
 - 贵：起步数百万美元，多则数千万、上亿；耗时长；现在的论文也会报告环境成本。
 - **knowledge cutoff**（知识截止日期）：数据只收到某一天为止，base model 自己无从知道此后发生的事。各家的 model card 都会写这一行（课上翻的是 GPT-5 的页面：9 月 30 日）。
 - 事后往权重里注入或修改知识很难：没有干净的办法只改一处而不让别处退步。
 - 模型可能原样吐出训练时见过的内容，有抄袭风险。

小注：OpenAI 的模型页面上，GPT-5 的 knowledge cutoff 是 2024 年 9 月 30 日，而模型 2025 年 8 月才发布——两者隔了将近一年，这段时间里发生的事模型一概不知。

4. 训练一步，显存花在哪

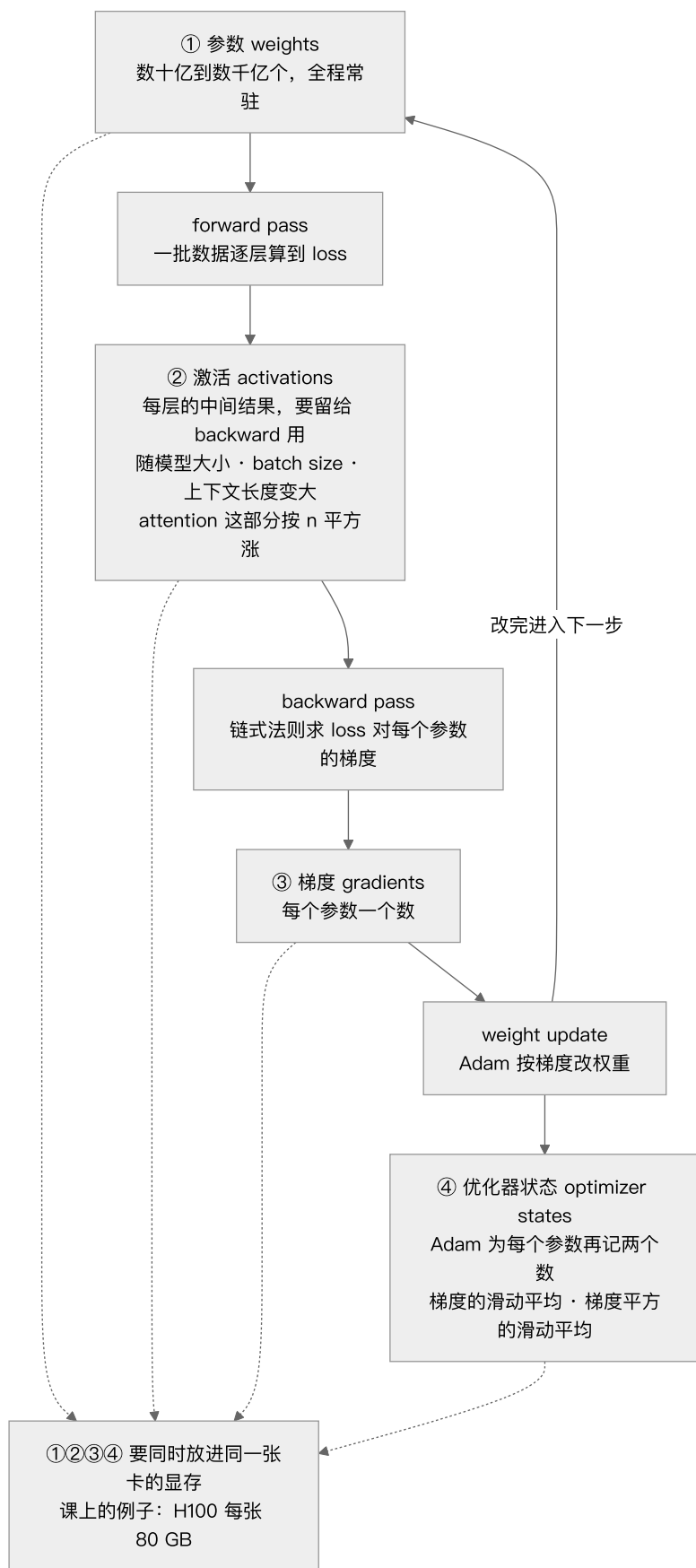


图 4-2 | 训练一步的三个环节，以及显存里常驻的四类东西（自绘示意）·

- **为什么要关心显存：**Transformer 的计算几乎全是矩阵乘法，这正是 GPU 擅长的（Google 用自研的 TPU）。但一张卡的显存有限——课上翻的 H100 规格表上是每张 80 GB——而训练时要同时放下的东西远不止模型本身。
- **一步训练的三个环节（图 4-2）**
 1. *forward pass*: 一批数据逐层算过去，得到 loss（预测和标签差多少）。途中每层算出的中间值叫 **activations**（激活），算 loss 要用，backward 时链式法则也要用（第 6 节会回到这一点），所以得留在显存里。它有多大，取决于模型大小、batch size 和上下文长度——self-attention 是 $O(n^2)$ ，序列一长这部分涨得很快。
 2. *backward pass*: 求 loss 对每个参数的 **gradient**（梯度），也就是“往哪个方向调，loss 会变小”。每个参数一个梯度，同样要存。
 3. *weight update*: 按梯度改权重。常用的 Adam 优化器还要为每个参数维护两个量——first moment 和 second moment，即梯度和梯度平方的滑动平均。这些就是 **optimizer states**（优化器状态），也要存。

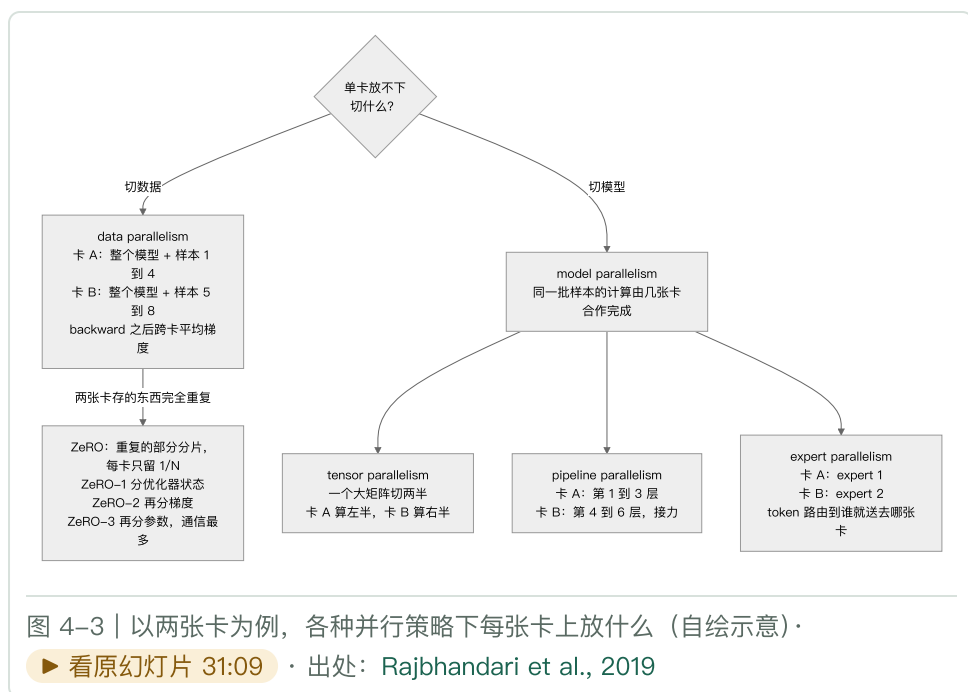
$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t, \quad v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$$

g_t 是第 t 步算出的梯度， m_t 和 v_t 是梯度与梯度平方的滑动平均（即 first / second moment）， β_1 、 β_2 是控制“记多久”的衰减系数；每个参数都有自己的 m 和 v ，所以光优化器状态就是参数量的两倍。

小注：ZeRO 论文给过一个记账方式（混合精度 + Adam）：每个参数占 2 字节的 FP16 权重 + 2 字节的 FP16 梯度 + 12 字节（FP32 的权重副本、 m 、 v 各 4 字节）= 16 字节。一个 7.5B 参数的模型光这几项就要约 120 GB，一张 80 GB 的卡已经放不下，还没算激活。

- **结论：**单卡装不下，只能用多张卡把负载分摊出去。接下来几节的方法，都在回答“怎么分”和“怎么省”。

5. 多卡并行：切数据，还是切模型



- **Data parallelism** (DP, 数据并行): 把一个 batch 切成几份分给各张卡, 每张卡都放一份完整的模型, 各自独立做 forward 和 backward。省下的是和 batch size 挂钩的那部分显存。
 - 梯度怎么办 (学生问): 各卡算出的梯度不同, 更新之前要跨卡通信、取平均, 保证每张卡上的模型始终一致。
 - 代价有两条: 每张卡仍得放得下一整份模型 (学生答出来的); 卡间通信让训练变慢。
- **ZeRO** (Zero Redundancy Optimization): DP 下每张卡存着一模一样的参数、梯度、优化器状态, 纯属重复。ZeRO 把它们分片 (shard / partition), 每张卡只保管自己那一份:
 - ZeRO-1: 只分优化器状态, 这一项就能省掉很多;
 - ZeRO-2: 梯度也分;
 - ZeRO-3: 参数也分, 卡上不再有任何冗余信息。
 - 分得越细, 通信越多。选哪一级, 看模型有多大、对训练时间有多敏感; 放得下就不必分。

小注: ZeRO 论文 Figure 1 的例子 (7.5B 参数、64 张卡): 每卡 120 GB → ZeRO-1 31.4 GB → ZeRO-2 16.6 GB → ZeRO-3 1.9 GB。PyTorch 里的 FSDP 是同一思路的实现。

- **Model parallelism** (模型并行): 连同同一个 batch 内部的计算也拆到多张卡上。讲者不想念目录, 只点了三种, 要求知道大意即可:

- *tensor parallelism*: 把一次大矩阵乘法切开，每张卡算一块，单卡要放的东西就小了；
- *pipeline parallelism*: 按层切，一张卡负责第 1—3 层，下一张负责第 4—6 层，依次接力；
- *expert parallelism*: MoE 模型（第 3 讲）的各个 expert 放在不同的卡上。

小注：真正的大规模训练是把这几种叠起来用的。Llama 3 论文里，405B 同时用了 tensor、pipeline、context、data 四种并行（论文称 4D parallelism）。

6. FlashAttention：少搬数据，结果一位不差

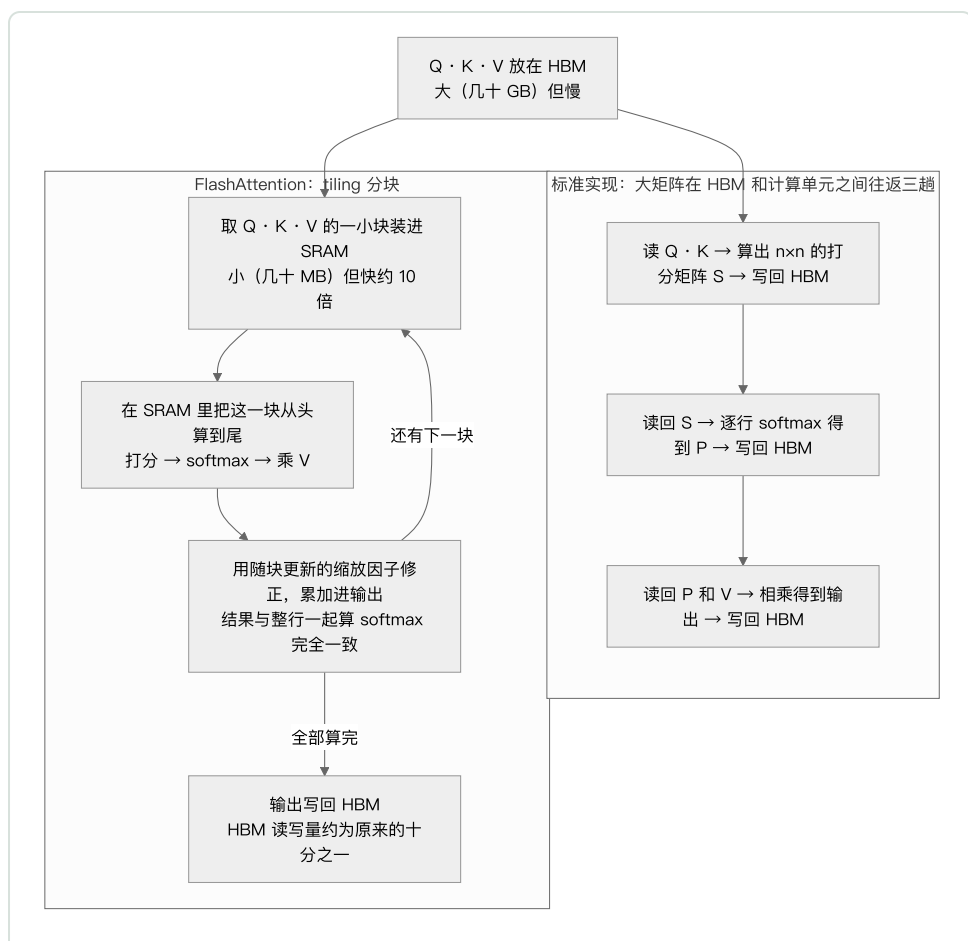


图 4-4 | 标准 attention 与 FlashAttention 的数据搬运对比（自绘示意）·

► 看原幻灯片 41:41 · 出处：Dao et al., 2022

- **动机：GPU 里有两种内存**（这项工作 2022 年出自 Stanford）
 - HBM (high bandwidth memory): 规格表上写的那个“显存”，几十 GB，大但相对慢；

- SRAM：和计算单元在同一块芯片上，只有几十 MB，但快一个数量级（讲者给的量级：HBM 每秒几 TB，SRAM 每秒几十 TB）。

- **标准 attention 慢在哪**：第 1 讲的公式拆开是三步——

$$S = \frac{QK^\top}{\sqrt{d_k}} \in \mathbb{R}^{n \times n}, \quad P = \text{softmax}_{\text{row}}(S), \quad O = PV$$

Q、K、V 各有 n 行（n 是序列长度，每行是一个 token 的向量），d_k 是 key 的维度，S 是 n×n 的打分矩阵，P 是对 S 逐行做 softmax 的结果，O 是输出。

- 不做优化时，Q、K、V 放在 HBM 里，三步每一步都是“从 HBM 读进来、算、写回 HBM”（图 4-4 左）。GPU 算得极快，时间大多耗在这几趟搬运上——瓶颈是 IO，不是计算。
- **为什么看上去没法分块**：softmax 是按行归一化的（每行加起来等于 1），似乎必须先拿到整行才能算。
- **核心技巧**：其实不必。把一行切成几块，各自做 softmax，再各乘一个缩放因子，就等于整行一起做的 softmax：

$$\text{softmax}([S_1 \ S_2]) = [\alpha_1 \text{softmax}(S_1) \ \alpha_2 \text{softmax}(S_2)], \quad \alpha_i = \frac{\sum_j e^{S_{i,j}}}{\sum_j e^{S_{1,j}} + \sum_j e^{S_{2,j}}}$$

S_1、S_2 是同一行打分切成的两块，α_i 是第 i 块的指数和占整行指数和的比例——softmax 的分母是整行共用的一个数，分块之后只需把这个分母修正回来。

小注：α 的表达式课上没有写（讲者说公式在论文里，不要求记），这里按 softmax 的定义补出。实际算法边算边维护“到目前为止的指数和”，每来一块就把已有的结果重新缩放一次，所以最后的结果是精确的。

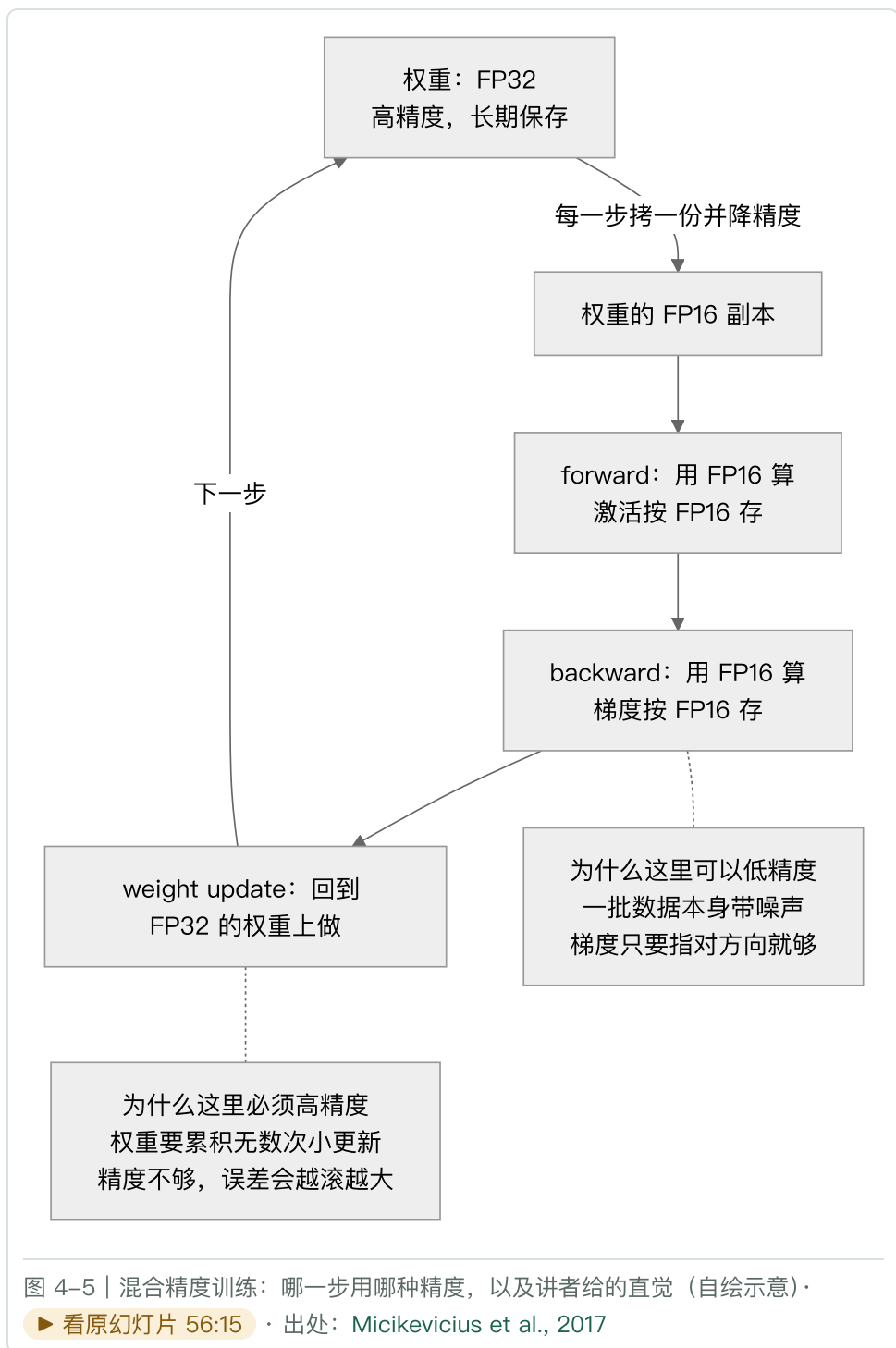
- **tiling（分块）**：于是可以把 Q、K、V 切成小块，一块块送进 SRAM，在里面把“打分 → softmax → 乘 V”从头到尾算完，累加到输出里，最后才写回 HBM（图 4-4 右）。对 HBM 的读写大幅减少。
- 讲者反复强调这是 **exact attention**：没有任何近似，只是换了计算的组织方式。学生问缩放因子是不是估出来的——不是，是迭代着精确算出来的。幻灯片为了好画，一次切的是整行整列；实际可以把矩阵切成任意网格。
- **第二个想法：recomputation（重计算）**。backward 时链式法则要用到 forward 的激活（第 4 节）。既然 attention 现在算得很快，那就不存、或者少存这些激活，backward 时现场再算一遍。

- **结果：**论文的对比表里，FlashAttention 的计算量（GFLOPs）反而更多——因为重算；但 HBM 读写从 40.3 降到 4.4，将近 10 倍，运行时间也更短。通常"重计算"是拿时间换显存，这里时间和显存两头都赚。

小注：论文 Figure 2 (GPT-2 medium, A100)：标准 attention 66.6 GFLOPs、HBM 读写 40.3 GB、41.7 ms；FlashAttention 75.2 GFLOPs、4.4 GB、7.3 ms。摘要里的端到端训练加速：BERT-large 15%，GPT-2（序列长 1K）3 倍。

- FlashAttention-2、-3 是同一思路针对新一代 GPU 的再适配——每代硬件的长处短处不同，总有再优化的空间。

7. 量化与混合精度训练



- **问题:** 权重都是浮点数。小数点后那么多位, 真的都需要吗? 能不能降一点精度来省显存, 而效果基本不变? **Quantization** (量化) 就是把数从一种精度的表示换成另一种 (通常更低的) 精度。
- **浮点数怎么存:** 一串 bit 分三段——1 位符号 (sign)、若干位指数 (exponent, 决定能表示多大多小的范围)、若干位尾数 (mantissa, 决定刻度有多细)。课上的表列了四种格式:

格式	总位数	符号 / 指数 / 尾数	特点
FP64（双精度）	64	1 / 11 / 52	刻度最细，也最慢
FP32（单精度）	32	1 / 8 / 23	传统的默认精度
FP16（半精度）	16	1 / 5 / 10	内存减半，但范围窄、刻度粗
BF16（bfloat16, brain float）	16	1 / 8 / 7	范围和 FP32 一样，刻度更粗

小注：各段的位数是这几种格式的标准定义，幻灯片的表里有，讲者没有逐个念。

- **低精度不只省显存，还更快：**同一张 GPU 规格表上，算力（FLOPS）随数据格式而变——讲者念的数是 FP64 约 34 teraFLOPS，换成 FP32 大致翻倍，更低的精度还要更高。
- **Mixed precision training（混合精度训练）**（图 4–5）：权重保持 FP32；forward 和 backward 里的运算全部用 FP16；权重更新仍然在 FP32 上做。论文作者发现效果几乎不降，显存和速度却都赚了。
- **为什么这样分工**（讲者给的直觉）：一批数据本身就带噪声，算出来的梯度只是告诉权重"大致往哪个方向走"，小数点后很多位没有意义；而权重要长期累积无数次小更新，精度不够的话，量化误差会一步步滚大。

小注：原论文摘要——保留一份 FP32 的权重副本（master weights）来累积更新，每一步再舍入成 FP16 去做 forward / backward；另配 loss scaling（先把 loss 放大再反传，防止很小的梯度在 FP16 下变成 0）；显存占用接近减半。loss scaling 课上没提。现在更常用 BF16：指数位和 FP32 一样多，数值范围相同，一般不需要 loss scaling。

- **问答**
 - 所有层、所有权重都这样处理吗？不一定，变体很多，有些部分更敏感，需要保留高精度。
 - 低精度能表示的范围不够怎么办？量化有多种方案，讲者点了两个名字：zero-point quantization 和 absmax quantization；第 10 节 QLoRA 里的 NF4 也是冲着这个问题设计的。

8. SFT 与 instruction tuning：把"续写机"变成助手

- **base model 的问题**（Shervine 的泰迪熊例子）：问预训练好的模型"泰迪熊脏了，能放进洗衣机吗？"，它不会回答你。它学的只是"接

下来最可能出现什么文字", 所以可能接着再抛一个问题 (学生猜的), 或者续一句关于泰迪熊材质的话——只是在模仿语料里的模式, 不是助手。

- **SFT** (supervised fine-tuning, 监督微调): supervised 指需要标签, 数据是成对的"输入、期望输出"; fine-tuning 指从预训练好的权重出发继续训练, 而不是从头来。
- **目标函数的关键区别**: 形式上还是预测下一个 token, 但 loss 只算在输出部分。输入是给定的条件, 我们不想让模型学着复述用户的话, 而是学"在这个输入之后, 什么样的回答最好"。

$$\mathcal{L}_{\text{SFT}} = - \sum_{t=1}^T \log p_{\theta}(y_t \mid x, y_{<t})$$

x 是指令 (prompt), y_1 到 y_T 是标准回答的各个 token, θ 是模型参数; 求和只跑在回答的 token 上, 指令部分不计 loss。预训练则是从 BOS 开始, 语料的每个位置都算。

	预训练	SFT
数据	原始文本, 没有人工标注	"指令、回答"对, 精挑细选
loss 算在哪	每一个 token 位置	只算回答部分
数据量 (课上的数)	GPT-3 300B token · Llama 3 15T token	GPT-3 约 13k 条 · Llama 3 约 1,000 万条
学到什么	语言、代码和其中携带的知识	以什么方式回应指令

- **instruction tuning** (指令微调): SFT 可以拿来适配你自己的某个任务; 当目的是让通用模型学会回应指令、成为有用的助手时, 这一类 SFT 叫 instruction tuning, 它是今天各家 LLM 训练的主要阶段之一。
- **数据长什么样**
 - 类别: 写故事、写诗、列清单、解释概念.....覆盖用户可能提出的各类请求, 每条指令配一个标准回答; 还有数学 (证明怎么一步步写) 和高质量的代码。
 - 来源: 早期几乎全靠人写——专业的语言学家按一套细则, 写出流畅、有帮助的回答。现在可以让已有的强模型生成候选回答, 再由人或另一个 LLM 审核, 整理数据集快了不少。
 - **安全**: 目标是 helpful 而且 harmless。数据配比 (data mixture) 里通常有一部分专门教模型不要复述预训练语料里的有害内容: 对有害请求拒答, 或者用 hedging 的方式留有余地、不下一刀切的断

言。拒答也可以在模型外面用正则拦，但不可扩展，所以通常是训进模型里的。

- **数量级**：GPT-3 约 13k 条样本，Llama 3 约 1,000 万条。就算每条 1,000 个 token，也比预训练小好几个数量级。心智模型：预训练量大而粗，学语言的通用规律；SFT 量小而精，把模型的目标对准你要的用途。

小注：13k 这个数应出自 InstructGPT (Ouyang et al., 2022) ——它在 GPT-3 上用标注员写的示范做监督微调，SFT 数据集约 13k 条训练 prompt；GPT-3 原论文本身没有 SFT 阶段。

- 调完之后再问泰迪熊：模型会正面回答——最好别机洗，建议手洗。
- **SFT 的四个难处**
 1. 高质量数据贵：要人参与，要符合一整套准则。好在数据集可以复用、逐步补充。
 2. 分布要对得上：SFT 的 prompt 分布应当贴近真实使用时的分布。学生问：训练样本只说“写个故事”，没说什么体裁，模型怎么泛化？讲者：靠预训练积累的知识——SFT 教会的是“写故事”这件事本身，细节模型能自己补。但差得太远就可能失灵，比如训练里都是课本式的故事，用户要的却是电影剧情式的。
 3. 评估难（第 9 节）。
 4. 计算开销大（第 10 节）。
- **问答**
 - 把某条 SFT 训练输入原样再喂给模型，会得到训练时的那个故事吗？多半不会逐字相同——采样时 temperature 不为零；风格可能相近，取决于预训练见过什么。temperature 越高，越容易走到输出分布里不那么常见的区域（第 3 讲）。
 - 想让回答更多样，除了调 temperature 呢？讲者说答案简单但难办：在这个类别里放更多、铺得更开的数据。数据点在分布空间里散得开，模型才抓得住要学的要点，而不是反复背同一个故事。

9. 调完怎么评：没有一个数字说得清

- **benchmark 的分类**：通用语言理解（MMLU, Massive Multitask Language Understanding, 几十个子任务）、推理、数学推理（GSM8K）、代码生成……新 benchmark 不断冒出来，因为大家会冲着已有的去优化，总会留下新的缺口。

小注：MMLU 共 57 个学科；GSM8K 是 Grade School Math 8K，约 8.5K 道小学水平的数学应用题——讲者对 G 代表什么没有把握，说成了高中水平。

- **training on the test task**（注意不是 test set）：同一类模型在某个 benchmark 上的分数有时会突然跳一截，却说不出原因。一篇论文（应为 Dominguez-Olmedo et al., 2024）指出：训练数据里有没有“和这个测试任务同类的数据”影响很大——不少 benchmark 本身就附带一份 auxiliary training set。要公平地比两个模型，必须保证它们在这一点上处境相同：要么都在测试任务上训练过，要么都没有；否则比出来的不是模型本身的能力。

小注：论文的做法是评测前先把每个模型都在同一份任务相关数据上微调一遍；这样处理之后，所谓“涌现能力”也变成了随规模逐渐出现，而不是突然出现。

- **分数高不等于好用**：benchmark 一出来，训练数据就会越做越像它，结果模型处处高分，用户却感觉不到差别。这不怪模型，也不怪 benchmark——“模型对你有多大价值”本来就很难压成一个数。
- **Chatbot Arena**：用户提问，两个模型各答一份，用户选更好的；网站用成对比较的结果算出排名，相当于给用户的“vibes”打了个分。问题同样不少：
 - 不稳：新模型刚上榜时碰到哪些对手带有随机性，头几场对排名影响很大。
 - 可操纵：有论文表明，只要能从回答里认出是哪个模型（最简单的办法是问一句“你是谁”），恶意投票者就可以专给目标模型投票。
 - 用户判断不了事实性：“用冷水机洗”听上去具体、可操作，用户会觉得有帮助，可它对不对，用户并不知道；专家整理的 benchmark 则有明确的准则判定事实对错。
 - 投票人群不等于真实用户群：比如回答里的 emoji，大众多半喜欢，领域专家可能反感。
 - 安全偏差：用户不喜欢被拒答，投票天然偏向有问必答的模型，哪怕拒答才是产品想要的行为。

小注：讲者没报论文题目，可能是 Min et al., 2025（用水印或分类器认出目标模型，几百张新票就能改变排名）或 Huang et al., 2025（识别模型的准确率超过 95%，约一千票可以操纵排名）。

- **结论：**评测要多个角度合起来看，最后取决于你自己的用途——看清每个模型的长处短处，再对照自己的场景。第 8 讲专门讲评测。

10. LoRA 与 QLoRA：只训练一个低秩增量

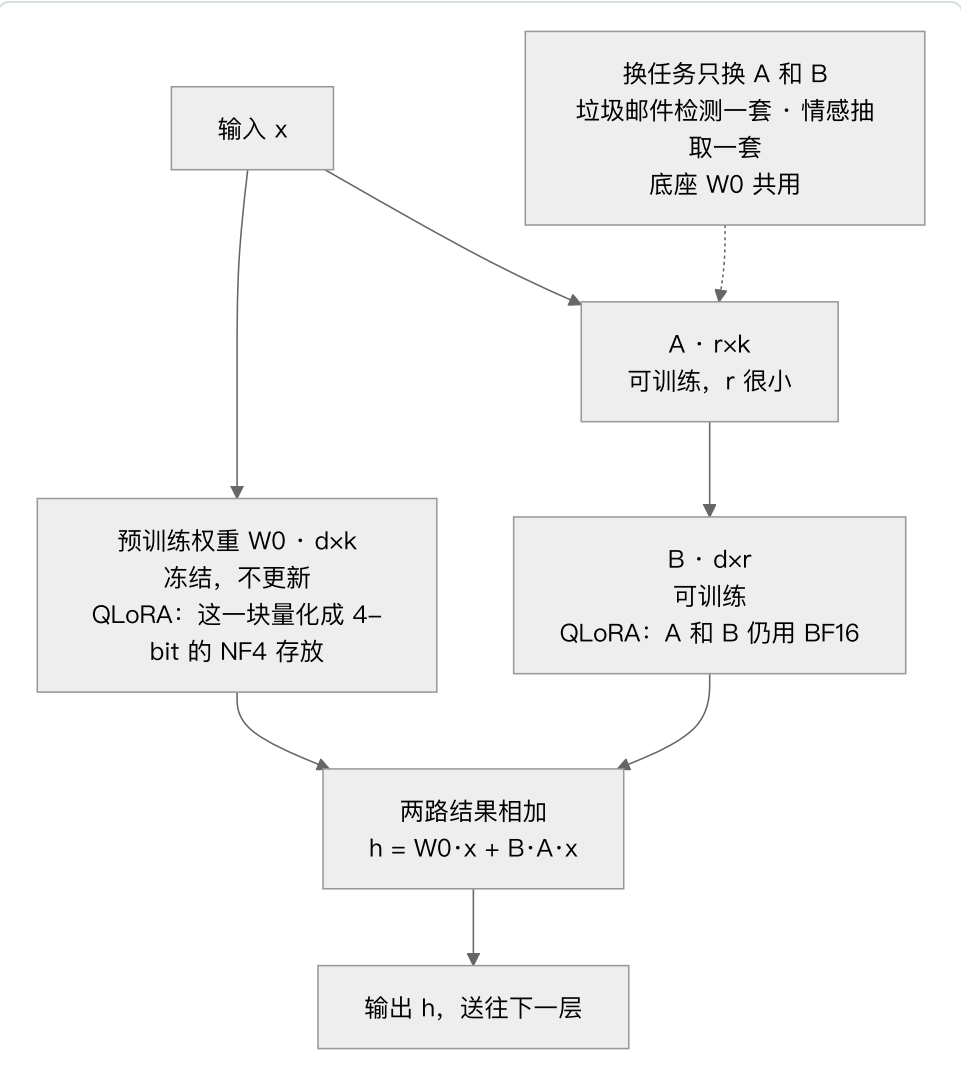


图 4-6 | LoRA 的两路 forward, 以及 QLoRA 改了哪一块 (自绘示意) ·

► 看原幻灯片 1:38:11 · 出处: Hu et al., 2021

- **动机：**全量微调要更新每一个权重，计算开销大。LoRA (low-rank adaptation) 是最常用的参数高效微调 (parameter-efficient fine-tuning) 方法。
- **做法：**不直接改权重矩阵，而是把它写成"冻结的预训练权重 + 一个低秩的增量"——

$$W = W_0 + BA, \quad B \in \mathbb{R}^{d \times r}, \quad A \in \mathbb{R}^{r \times k}, \quad r \ll \min(d, k)$$

W_0 是冻结的预训练权重 (d 行 k 列), B 和 A 是新加的、唯一要训练的两个矩阵, r 是它们的秩 (rank); 可训练参数从 $d \times k$ 个变成 $r \times (d +$

k) 个。

- W 的维度通常是几百到几千, r 通常只有十来个以内 (讲者说 4 就很常见)。举例: $d = k = 4096$ 、 $r = 8$ 时, 全是约 1,680 万个参数, LoRA 只有约 6.6 万个, 少 256 倍。forward 时两路分别算, 结果相加 (图 4-6)。

小注: 联系第 4 节—— W_0 不更新, 就不必为它存梯度和 Adam 的两份状态, 这才是省显存的大头。LoRA 论文摘要的数字: 相对 GPT-3 175B 的 Adam 全量微调, 可训练参数少 10,000 倍, 显存需求少 3 倍; 和 adapter 不同, 它不增加推理延迟。

- **任务专属的 A、B:** 同一个 base model, 垃圾邮件检测训一套 A、B, 情感抽取再训一套; 底座共用, 按需切换。
- **加在哪些层:** 原论文只加在 attention 的权重矩阵上, 后来发现那不是收益最大的位置。讲者引了"几周前"的一篇博客 (应为 Thinking Machines 的 [LoRA Without Regret](#), 2025 年 9 月): 加在 feed-forward 层收益最大。现在通常两处都加, 但主要增益来自 FFN。
- **两个经验性质** (同一起来源; 讲者说没有严格的理论解释, 只转述了研究者的猜测)
 - 学习率要比全量微调高, 经验值约 10 倍——猜测是秩很小, 要探索的空间需要更大的步子;
 - batch size 大了效果变差——猜测是"两个矩阵的乘积"的训练动力学和一个完整矩阵不同。
- 其他参数高效的方法: prefix tuning、adapters, 课程教材里有, 用得少, 没展开。
- **问答:** r 要不要 grid search? 可以, 但它就是个设计选择, 通常前人已经试过, 直接挑一个常见值即可——参数量已经降了几个数量级, 再抠一点意义不大。
- **QLoRA** (最后两分钟): 在 LoRA 的基础上, 把冻结的 W_0 量化成 4-bit 存放, A、B 仍用 BF16 训练。
 - **NF4** (4-bit NormalFloat): 假设权重服从正态分布, 按分位数 (quantile) 而不是等宽的区间来划分取值, 让每个取值对应大致同样多的权重——有限的几个 bit 用得最充分。
 - **double quantization:** 量化会产生一批"量化常数" (用来在量化值和原值之间来回换算), 把这些常数再量化一次。额外省得不多, 但值得知道。

。效果：讲者给的数是显存节省约 16 倍。

小注：QLoRA 摘要——65B 参数的模型可以在单张 48 GB 的 GPU 上微调，效果与 16-bit 全量微调持平。论文正文里，16-bit 全量微调 65B 需要 780 GB 以上显存， $780 \div 48 \approx 16$ ，应是"16 倍"的来源。论文的第三个技巧 paged optimizers（应对显存尖峰）课上没提。

11. 放在一起看：每个手段省的是哪一块

手段	对准哪一块	课上提到的代价
data parallelism	和 batch size 挂钩的显存，分摊到多卡	每卡仍要放下整份模型；梯度聚合的通信让训练变慢
ZeRO-1 / 2 / 3	各卡重复保存的优化器状态 / 梯度 / 参数	分得越细，通信越多
tensor / pipeline / expert parallelism	单卡放不下的模型本身	课上没展开
FlashAttention	attention 的 HBM 读写；配合 recomputation 少存激活	多做一些 FLOPs，但总时间反而更短
mixed precision	激活和梯度的字节数，顺带算得更快	权重和更新仍要留在 FP32
LoRA	要训练的参数量，少几个数量级	学习率要重调（约 10 倍）；大 batch 下效果变差
QLoRA	冻结权重 W0 自身占的显存 (4-bit)	课上没展开

关键图表速查（点时间戳跳到原幻灯片）

图	看什么	跳转	出处
Scaling laws 三联图	算力、数据集大小、参数量各自增大时，loss 都稳定下降	17:04	Kaplan et al., 2020
IsoFLOP 曲线	一种颜色一个算力预算；每条曲线的最低点就是该预算下最划算的模型大小	18:37	Hoffmann et al., 2022
最优配置表	参数量与 token 数大约 1 : 20；拿 GPT-3 的 175B / 300B 对照	19:39	同上
训练一步的示意	依次点出要存的三样：activations、gradients、Adam 的两个 moment	27:06	—
H100 规格表	GPU memory 一行：80 GB；精度下的算力	30:13	NVIDIA 规格表
ZeRO 分片示意	三级：优化器状态 → 梯度 → 参数，每一级每卡占用都更少	34:22	ZeRO

图	看什么	跳转	出处
GPU 内存层级	HBM 大而慢、SRAM 小而快，带宽差约 10 倍	39:01	FlashAttention
分块 softmax 等式	整行 softmax = 各块 softmax 乘各自的缩放因子	44:51	同上
tiling 动画	绿色是从 SRAM 读，蓝色是从 HBM 读；输出矩阵被一块块填满	45:52	同上
FlashAttention 结果表	GFLOPs 变多，HBM 读写 40.3 → 4.4，运行时间反而更短	50:35	同上
浮点格式表	sign / exponent / mantissa 各占几位；FP16 和 BF16 差在哪	53:40	—
混合精度流程	权重 FP32, forward / backward FP16, 重新回到 FP32	56:15	Micikevicius et al., 2017
SFT 的 loss 位置	指令部分不算 loss，黄色的回答部分才算	1:11:10	—
LoRA 分解图 / QLoRA	W0 冻结，旁路 B·A；QLoRA (1:45:57) 里 W0 换成 NF4	1:38:11	LoRA · QLoRA

提到的工作

名称	在本讲里的作用
transfer learning	两阶段范式的思想来源：不从零开始，先预训练再调
Common Crawl	预训练语料的代表：持续抓取的公开网页，每月约 30 亿页
GPT-3 (Brown et al., 2020)	175B 参数、300B token；按 Chinchilla 标准训练不足的例子
Llama 3 (2024)	15T token 的预训练；自己重做 scaling law 定出 405B；SFT 约 1,000 万条
Scaling Laws for Neural Language Models (Kaplan et al., 2020)	算力、数据、参数与 loss 的关系；大模型更 sample efficient；架构细节影响小
Chinchilla (Hoffmann et al., 2022)	固定算力下的最优配比：token 数约为参数量的 20 倍
GPT-5 的 model card	knowledge cutoff 那一行的示例
NVIDIA H100 · Google TPU	显存 80 GB、各精度算力的规格表示例；TPU 是 Google 自研的替代
Adam (Kingma & Ba, 2014)	优化器；它的 first / second moment 就是要额外存的优化器状态
ZeRO (Rajbhandari et al., 2019)	给数据并行去冗余：三级分片
expert / tensor / pipeline parallelism	模型并行的三种，只讲大意

名称	在本讲里的作用
FlashAttention (Dao et al., 2022)	IO-aware 的精确 attention: tiling 加 recomputation
FlashAttention-2 · FlashAttention-3	同一思路针对新一代 GPU 的再适配
Mixed Precision Training (Micikevicius et al., 2017)	权重 FP32、计算 FP16 的混合精度做法
zero-point / absmax quantization	问答里点名的两种处理数值范围的量化方案
InstructGPT (Ouyang et al., 2022)	课上"GPT-3 约 13k 条 SFT 样本"应出自这里
MMLU · GSM8K	通用知识与数学推理 benchmark 的代表
Training on the Test Task (Dominguez-Olmedo et al., 2024)	解释 benchmark 分数的突然跳升; 公平比较要求处境相同 (应为这篇)
Chatbot Arena (Chiang et al., 2024)	用户成对投票的排行榜; 课上讲了它的五类问题
Min et al., 2025 或 Huang et al., 2025	"排行榜可以被操纵"的论文; 讲者没报题目 (推断)
LoRA (Hu et al., 2021)	参数高效微调的主角
LoRA Without Regret (Thinking Machines, 2025 年 9 月)	"几周前的博客"应为这篇: FFN 上收益最大、学习率约 10 倍、怕大 batch
prefix tuning · adapters	其他参数高效微调方法, 只点了名
QLoRA (Dettmers et al., 2023)	W0 量化成 NF4, 加 double quantization

术语对照

English	中文
pre-training / fine-tuning	预训练 / 微调 (从已有权重出发继续训练)
transfer learning	迁移学习
base model	基座模型: 只做过预训练、只会续写的模型
next-token prediction	预测下一个 token
knowledge cutoff	知识截止日期
FLOPs / FLOPS	浮点运算次数 (计算量) / 每秒浮点运算次数 (硬件速度)
scaling laws	规模定律: loss 随算力、数据、参数变化的经验规律
compute-optimal	算力最优: 预算固定时最划算的模型与数据配比
sample efficient	样本效率高: 看同样多的数据学得更好
undertrained	训练不足: 相对模型大小, 数据喂得太少
forward / backward pass	前向 / 反向传播
activations	激活: 每层的中间计算结果

English	中文
gradient	梯度
optimizer states	优化器状态：Adam 为每个参数记的 first / second moment（一阶、二阶矩）
data parallelism (DP)	数据并行：切 batch，每卡一份模型
model parallelism	模型并行：把模型本身拆到多卡
tensor / pipeline / expert parallelism	张量并行（切矩阵） / 流水线并行（切层） / 专家并行（分 expert）
shard / partition	分片
communication cost	通信开销：卡与卡之间传数据的时间
HBM (high bandwidth memory)	高带宽显存：GPU 上大而相对慢的那块内存
SRAM	片上静态存储：紧挨计算单元，小而快
IO-aware	把内存读写也算进算法设计里
tiling	分块
exact attention	精确注意力：结果与标准算法完全一致，没有近似
recomputation	重计算：forward 不存激活，backward 时再算一遍
quantization	量化：把数换成更低精度的表示
sign / exponent / mantissa	符号位 / 指数位 / 尾数位
mixed precision training	混合精度训练
SFT (supervised fine-tuning)	监督微调
instruction tuning	指令微调
data mixture	数据配比：各类数据按什么比例混在一起
refusal / hedging	拒答 / 留有余地的表述
helpful / harmless	有帮助 / 无害
out of distribution	分布外：和训练时见过的数据差得远
benchmark	基准测试
auxiliary training set	benchmark 附带的辅助训练集
alignment	对齐：SFT 加 preference tuning
preference tuning	偏好调优（第 5 讲）
mid-training	中期训练：预训练之后、SFT 之前，换数据不换目标
parameter-efficient fine-tuning (PEFT)	参数高效微调
low-rank adaptation (LoRA)	低秩适配
rank	秩：这里指 B、A 两个矩阵中间那一维的大小
NF4 (4-bit NormalFloat)	按正态分布的分位数划分的 4-bit 格式

English	中文
quantile	分位数
double quantization	双重量化：把量化常数再量化一次
VRAM	显存

字幕勘误

"Shervin / Shvin" → Shervine; "Afin / Ashen / Ain" → Afshine; "MOE" → MoE; "based LLMs" → MoE-based LLMs; "flops"（两处含义不同）→ FLOPs 与 FLOPS; "scaling loss" → scaling laws; "chinchila" → Chinchilla; "atom optimizer" → Adam optimizer; "zero / 0 1, 02, 03" → ZeRO、ZeRO-1 / 2 / 3; "HPM / HPN" → HBM; "SROM / SRAMM" → SRAM; "submax" → softmax; "red from" → read from; "flaps" → FLOPs; "mantisa" → mantissa; "contisation" → quantization; "P32" → FP32; "lama 35 billion" → Llama 3 405B; "LMA 3" → Llama 3; "apps quantization" → absmax quantization; "GP5" → GPT-5; "sentiment instruction" → sentiment extraction; "span detection" → spam detection; "Laura / LoRa / Lora" → LoRA; "BNA" → B and A; "prom distribution" → prompt distribution; "cured by experts" → curated by experts; "JetBot Arena" → Chatbot Arena; "Chad GPT" → ChatGPT; "grad school math" → grade school math (GSM8K); "VRAMm" → VRAM。

带走的问题

- 按 Chinchilla 的 20 倍经验值，GPT-3 的 175B 参数该配多少 token？反过来，课上说 Llama 3 用了 15T token——它最小的 8B 版本每个参数摊到近 2,000 个 token（课外事实），远超 20 倍。Chinchilla 优化的只是训练算力；把推理成本也算进去，“最优”会往哪边移？
- 用第 4 节的四类显存给一个 7B 模型记账：参数、梯度、Adam 状态各占多少？ZeRO-1 / 2 / 3 分别把哪一项除以卡数？为什么 ZeRO-3 通信最多，而 data parallelism 省不掉“每卡一份模型”？
- FlashAttention 多做了 FLOPs 却更快，说明标准 attention 的瓶颈是什么？换成一个本来就受限于计算的算子，recomputation 还能“两头都赚”吗？

4. SFT 为什么不在指令部分算 loss? 如果算上, 模型会多学到什么、可能带来什么副作用? mid-training 的目标函数和预训练一样, 那它和 SFT 的界线到底在哪?
5. "两个模型要么都在测试任务上训练过、要么都没有"才可比——闭源模型的训练数据不公开, 这个条件怎么保证? Chatbot Arena 的投票偏差 (喜欢 emoji、讨厌拒答) 如果被拿来当优化目标, 会把模型推向哪里 (第 5 讲 preference tuning、第 8 讲评测)?

CME295 第 5 讲 | LLM 微调 (LLM Tuning)

Stanford CME295: Transformers & Large Language Models (2025 秋) · 第 5 讲, 2025 年 10 月 31 日 · 期中之后的第一讲, 期末考覆盖第 5—9 讲
视频: https://www.youtube.com/watch?v=PmW_TMQ3IOI (1:47:41, 英文字幕是自动生成的, 缩写和人名错得多, 见文末勘误) 讲者: Afshine Amidi (开场到 Best-of-N) · Shervine Amidi (1:29:47 起的 DPO 部分) 课程大纲: <https://cme295.stanford.edu/syllabus/>

一句话: SFT 只能教模型"该怎么答", 没法告诉它"别这样答"; 偏好微调 (preference tuning) 改用"两个回答里哪个更好"这种便宜得多的成对数据来补这一课。经典做法 RLHF 分两步: 先用 Bradley-Terry 公式把偏好对训成一个给回答打分的奖励模型, 再用 PPO 让 LLM 去追高分, 同时用 KL 惩罚和 clip 把它拴在 SFT 模型附近, 防止 reward hacking。嫌 RL 太重, 可以只拿奖励模型在推理时做 Best-of-N, 或者用 DPO 把两步并成一个直接作用在偏好对上的监督 loss。

时间轴

时间	内容
0:06	开场: 期中之后的安排; 回顾第 4 讲的 pretraining、SFT、LoRA
4:00	今天的主题: 训练的第三步, 让模型对齐人的偏好
4:50	Preference tuning: 泰迪熊例子中的偏好对; 为什么 SFT 之后还需要这一步
10:05	问答: LoRA 和偏好微调是什么关系; 偏好对能注入负信号
11:31	数据收集: pointwise / pairwise / listwise 三种标注形式
13:55	造偏好对的配方: 同一 prompt 采样两次再比较; 二元还是分级; 改写日志里的坏回答
17:43	RLHF overview: RL 的基本词汇 (agent、state、action、policy、reward)
19:47	把这些词一一对应到 LLM 上
22:27	问答: 贵不贵; 奖励信号稀疏; 奖励按整条回答给
26:02	RLHF 的两个阶段; RLHF 与 RLAIIF 的区别
28:24	Reward model: 要一个能分清好坏回答的打分模型
29:46	Bradley-Terry 公式与 sigmoid
32:20	训练 RM: 从最大似然一步步推出 pairwise loss
38:44	成对训练、单点打分; 数据量、模型选型、RewardBench
41:50	问答: 奖励分维度; 标注指南; 回归还是分类; 分数归一化

时间	内容
46:02	RL 阶段：rollout → RM 打分 → 更新 LLM
48:29	为什么不能离初始模型太远：遗忘、reward hacking、不稳定
53:54	PPO：奖励项加 KL 惩罚；KL divergence 是什么
57:56	advantage 与 value function；GAE
1:03:39	PPO-Clip：公式、三个易混点、两张图的直觉
1:10:20	问答：不可导点；为什么和上一轮比；ratio 怎么算
1:12:24	PPO KL-penalty；现在常见的混合写法
1:14:24	PPO 要同时跑四个模型；GRPO 预告
1:16:20	RL 路线的挑战
1:19:58	On-policy vs off-policy；问答：为什么不直接拿偏好数据做 SFT
1:22:43	Best-of-N：做法、例子、代价
1:27:17	问答：N 条全差怎么办；分数尺度在这里无所谓
1:29:47	DPO (Shervine 接手)：对 PPO 和 Best-of-N 的两点抱怨
1:32:09	DPO 的 loss；论文副标题的含义
1:35:19	DPO 是怎么推出来的
1:39:00	DPO 对比 RLHF；为什么 PPO 整体仍更强：分布偏移
1:42:05	问答：两套权重；reference 能不能换成别的模型
1:44:13	泰迪熊进洗衣机：偏好微调改语气不改事实；问答：实践中怎么选

核心内容

1. 为什么 SFT 之后还要第三步

- **前两步回顾（第 4 讲）**：预训练用海量文本教模型语言和代码的规律，又慢又贵，产物只会续写下一个 token；SFT 用一份小而精的数据教它"怎么表现"，比如变成聊天助手，LoRA 让这一步不必动全部权重。
- **第三步要解决什么**：SFT 之后的模型已经像个助手了，但语气、友好度、安全性未必合意。课上的例子：让模型推荐能和泰迪熊一起做的活动，它却劝用户少花时间在泰迪熊上——格式没毛病，只是不是我们想要的回答。把它改写成想要的样子，两条配在一起就是一个**偏好对** (preference pair)：同一个 prompt，一条想要、一条不想要。让模型向"想要"的那一侧靠拢，就叫偏好微调，也常叫对齐 (alignment)。
- **为什么不把这些修正直接补进 SFT 数据**，讲者给了四条理由：

1. 比较比创作容易。从零写一首好诗很难，两首诗里挑一首好的却很容易。SFT 需要示范，偏好数据只需要判断，收集成本低得多。
 2. SFT 数据的 prompt 分布要小心配平，某类 prompt 多了，模型就会偏向那种答法。模型每出一次错就往 SFT 集里补一条，很容易把分布带歪。
 3. SFT 数据要求质量极高，把模型的每个失误都改写成完美示范，时间上耗不起。
 4. SFT 只有正信号：它告诉模型该生成什么，却没法表达"别生成这个"。偏好对里输的那条正好提供负信号。
- 讲者也提醒：偏好微调不是万灵药。SFT 后的模型如果错得很多，先回头检查 SFT 数据。
 - **问答：**LoRA 在偏好微调里的对应物是什么？——两者不在一个层面：LoRA 决定"调哪些参数"，偏好微调换的是"目标函数"，完全可以叠在一起用。

2. 偏好数据怎么来

- **三种标注形式**
 - *pointwise*: 给单个回答打一个绝对分。人很难校准——0.9 和 0.2 之间到底差在哪，说不清。
 - *pairwise*: 两个回答里挑更好的那个。最容易标，业界基本都用它，本讲后面也只用这一种。
 - *listwise*: 给 n 个回答排序。不用说"好多少"，比 *pointwise* 容易，但比 *pairwise* 麻烦。
- **标准配方：**取一个 prompt（来自线上日志，或一份希望覆盖的 prompt 清单，总之要贴近用户真实提问的分布）→ 用大于 0 的 temperature 把它喂给模型两次，得到两个不同的回答（第 3 讲的采样）→ 找评审比较。评审可以是人，可以是 LLM-as-a-judge（讲者说后面的课会细讲，应在第 8 讲），也可以是 BLEU、ROUGE 这类规则指标（现在用得少）。
- **量表：**最简单的是二元的"更好 / 更差"；也可以分成"好很多、好、略好、略差、差、差很多"几级。但很多任务本身就主观，分级越细越难标得一致，所以实践中多用二元。
- **另一种来源：**从日志里找出不满意的回答，人工改写成好的——开头的泰迪熊例子就是这样来的。代价是又回到了"创作"，比单纯比较贵。

小注：InstructGPT 实际是按 listwise 收集、按 pairwise 训练的：每个 prompt 让标注员给 4—9 个回答排序，再拆成两两配对去训奖励模型。

3. RLHF 全景：先把 RL 的词翻译成 LLM 的词

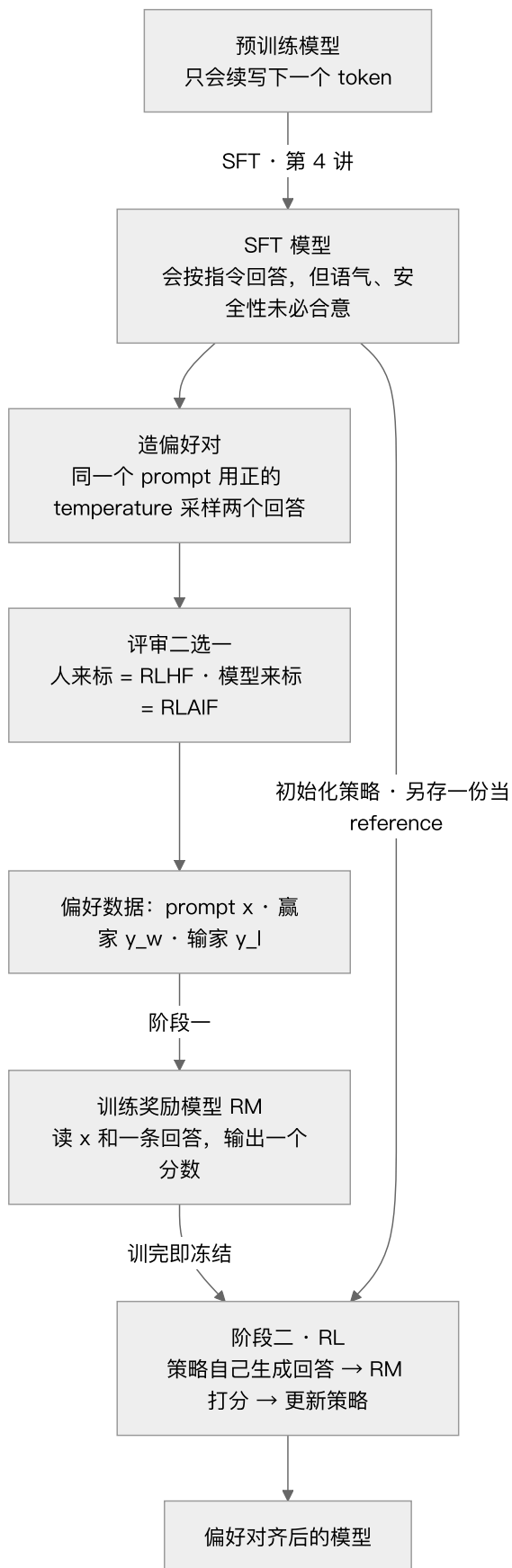


图 5-1 | 从 SFT 模型到偏好对齐模型：偏好数据、奖励模型、RL 三步怎么接起来（自绘示意）· ▶ 看原幻灯片 26:32 · 出处：Ouyang et al., 2022 (InstructGPT；课上没点名，推断)

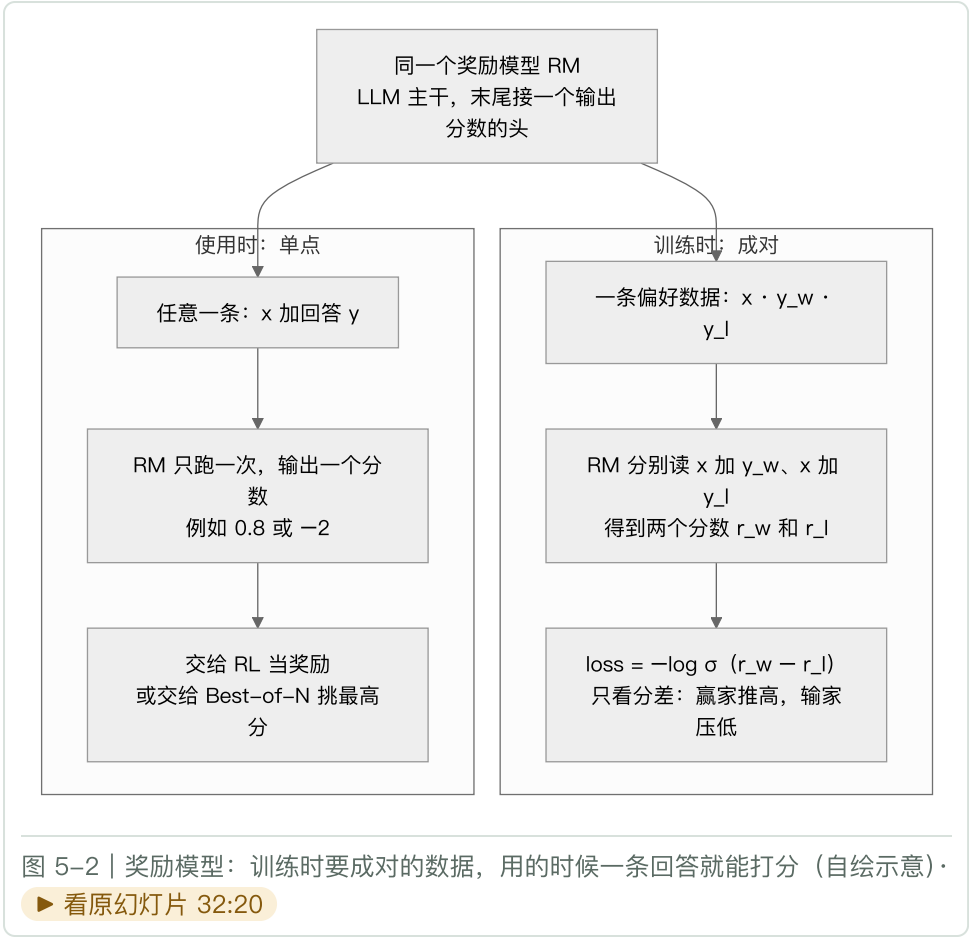
- **RL 的基本词汇**（讲者问有没有 RL 专家，没人举手，于是从零讲）：
一个 agent 在时刻 t 处于状态 s_t ，按照策略 $\pi_\theta(a_t | s_t)$ 选一个动作 a_t ，环境随后给它一个奖励。策略无非是"给定状态，每个动作的概率"。
- **搬到 LLM 上，每个词都有对应物：**

RL 概念	在 LLM 里对应什么
agent	LLM 本身
state s_t	到目前为止的输入：prompt 加已经生成的 token
action a_t	生成下一个 token
environment	讲者的说法：可以看成词表，也就是所有可选的 token
policy π_θ	LLM 前向一次输出的 next-token 概率分布， θ 就是 LLM 的权重
reward	对生成结果好坏的打分，从偏好数据里来

- 目标：学一组 θ ，让 π_θ 和偏好一致。
- **RLHF** (reinforcement learning from human feedback) 分两个阶段（图 5-1）：阶段一训练**奖励模型**，输入是 prompt 加回答，输出一个分数，全部偏好对都用在这里；阶段二是 RL，用这个分数当奖励去调 LLM。名字里的 human feedback 指的是奖励模型的训练标签来自人；标签若来自模型，就叫 RLAIFF。
- **问答**
 - 奖励是每个 token 给一次，还是整条回答给一次？——整条给一次。
 - 这点信号够吗？——SFT 在每个 token 上都有监督；这里一条回答大约只有一个信号，所以人们常说 RLHF 的信号是稀疏的 (sparse)。
 - 会不会很贵？——照常按 batch 训练，本质上和别的训练过程一个量级，只是比 SFT 多出几样东西（见第 6 节的四个模型）。

小注：这套"SFT → 奖励模型 → PPO"的三步流程因 InstructGPT（ChatGPT 的前身）而广为人知。那篇论文把这里的环境称为 bandit 环境：给一个 prompt，收一条回答，RM 给分，回合结束——环境几乎是空的。到了 agent 场景（第 7 讲），环境才变成真实的工具和网页，上表里变的主要是 environment 和 reward 两行。

4. 阶段一：奖励模型与 Bradley-Terry



- **要什么：**一个打分函数 $RM(x, y)$ ，回答好就给高分，差就给低分。它必须同时读 prompt 和回答——脱离问题，没法判断回答好不好。
- **难点：**手里只有"谁赢谁输"的成对标签，没有绝对分数。Bradley-Terry 公式把两者接了起来：假设每个回答背后有一个隐藏分数 r ，回答 i 胜过回答 j 的概率只由两者的分差决定。

$$P(y_i \succ y_j) = \frac{e^{r_i}}{e^{r_i} + e^{r_j}} = \sigma(r_i - r_j), \quad \sigma(z) = \frac{1}{1 + e^{-z}}$$

r_i 、 r_j 是两个回答各自的分数， σ 是 sigmoid 函数（把任意实数压到 0 和 1 之间）； r_i 比 r_j 高得越多，这个概率越接近 1。

- **训练：**同一个模型分别读"x 加 y_w "和"x 加 y_l "，得到两个分数。
loss 是讲者带着全班从最大似然推出来的：假设各个偏好对相互独立，要找的是让"观察到这些胜负结果"的联合概率最大的参数；联合概率是一串连乘，数值会小到不稳定，于是取 log 变成连加；机器学习习惯做最小化，再添一个负号。有学生先答了 binary cross-entropy，讲者确认方向是对的——要的就是这个概率的负对数似然。

$$\mathcal{L}_{RM}(\theta) = -\mathbb{E}_{(x, y_w, y_l)} \left[\log \sigma(r_\theta(x, y_w) - r_\theta(x, y_l)) \right]$$

y_w 、 y_l 是同一个 prompt x 下赢和输的回答， r_θ 是参数为 θ 的奖励模型；赢家的分数比输家高得越多，loss 越小。

- **成对训练，单点使用**（图 5-2，讲者说这是他觉得最漂亮的地方）：
loss 要两条回答才能算，但训出来的模型读一条"prompt 加回答"就能输出一个数，用的时候不需要配对。
- **工程细节**：数据通常是几万对或更多，标签就是偏好（RLHF 里来自人）。模型现在的通行做法是拿一个 decoder-only 的 LLM，在序列末尾接一个输出分数的分类头；也可以用 BERT 这类 encoder-only 模型，在 [CLS] 的表示上做投影（第 2 讲）。奖励模型的好坏有专门的 benchmark，讲者举了 RewardBench。
- **问答**
 - 不同任务里"好"的标准不一样怎么办？——奖励总是沿某个维度定义的：有用、友好、安全……可以每个维度一个 RM，也可以做一个综合分。
 - 人工偏好对标注指南非常敏感。指南要尽量客观、清楚，否则标签噪声会很大。
 - 这算回归还是分类？——讲者倾向于把它看成一个概率模型：标签是"赢 / 输"，像分类；最后用的却是那个连续分数。分数没有固定量纲（好回答可能是 1，差回答可能是 -2、-3），所以进入 RL 的 loss 之前通常要做归一化。

小注：Bradley-Terry (1952) 原本是给成对比较排名用的统计模型，国际象棋的 Elo 和 Chatbot Arena 的排行榜是同一思路。这个 loss 只约束分差，所有分数整体平移不改变 loss，所以 InstructGPT 在做 RL 之前先加一个偏置，把分数的均值挪到 0。另外，这里的 RM 给整条回答打一个分，相当于 CS329A 笔记里的 ORM (outcome reward model)。

5. 阶段二：让 LLM 追高分，但要拴住它

- **一轮 RL 长什么样**：取一个 prompt $\rightarrow$ LLM 生成一整条回答（叫 completion，也叫 rollout） $\rightarrow$ prompt 和回答一起送进 RM $\rightarrow$ 得到一个分数，比如 -2 $\rightarrow$ 拿这个分数去更新 LLM。此时 RM 是冻结的，被训练的只有 LLM。数据量通常在十万条 prompt 以上，"标签"由 RM 现场给出，起点是 SFT 模型。
- **目标有两半**：奖励要高；同时不能离初始模型太远。为什么要后一半，课堂上讨论出三条理由：

1. SFT 模型里已经有大量知识和能力，走远了会灾难性遗忘 (catastrophic forgetting)。
 2. **Reward hacking**。RM 只是"我们真正想要的东西"的一个不完美代理。讲者的比方：目标是把课讲得有信息量，却拿下课时的掌声大小当奖励；优化过了头，就会发现讲笑话最能换掌声——奖励拉满，目标落空。LLM 也一样，对 RM 优化得太狠，学到的是钻 RM 的空子。
 3. 训练稳定性。
- 两半写在一起，就是 RLHF 的目标函数：

$$\max_{\theta} \mathbb{E}_{x, y \sim \pi_{\theta}(\cdot|x)} [r(x, y)] - \beta \text{KL}(\pi_{\theta}(\cdot|x) \parallel \pi_{\text{ref}}(\cdot|x))$$

第一项是 RM 给当前策略 π_{θ} 自己生成的回答 y 打的平均分；第二项衡量 π_{θ} 的输出分布离 reference (冻结的 SFT 模型 π_{ref}) 有多远， β 决定罚多重。

- **KL divergence**：衡量两个概率分布 P 、 Q 差多远的量， $\text{KL}(P \parallel Q) = \sum p_i \cdot \log(p_i / q_i)$ 。讲者特意不叫它"距离"。它恒大于等于 0 (用 Jensen 不等式证明)，当且仅当 P 和 Q 完全相同时等于 0。

小注：不叫距离，是因为它不对称 ($\text{KL}(P \parallel Q)$ 一般不等于 $\text{KL}(Q \parallel P)$)，也不满足三角不等式。实现上，InstructGPT 把 KL 惩罚直接折进奖励：每个 token 扣掉 $\beta \cdot \log(\pi_{\theta} / \pi_{\text{ref}})$ ，RM 的分数只在回答结束时加一次。数据量上它也比课上说的量级小：SFT 约 1.3 万条 prompt，RM 约 3.3 万条，PPO 约 3.1 万条。

6. PPO (上)：advantage、value function 与四个模型

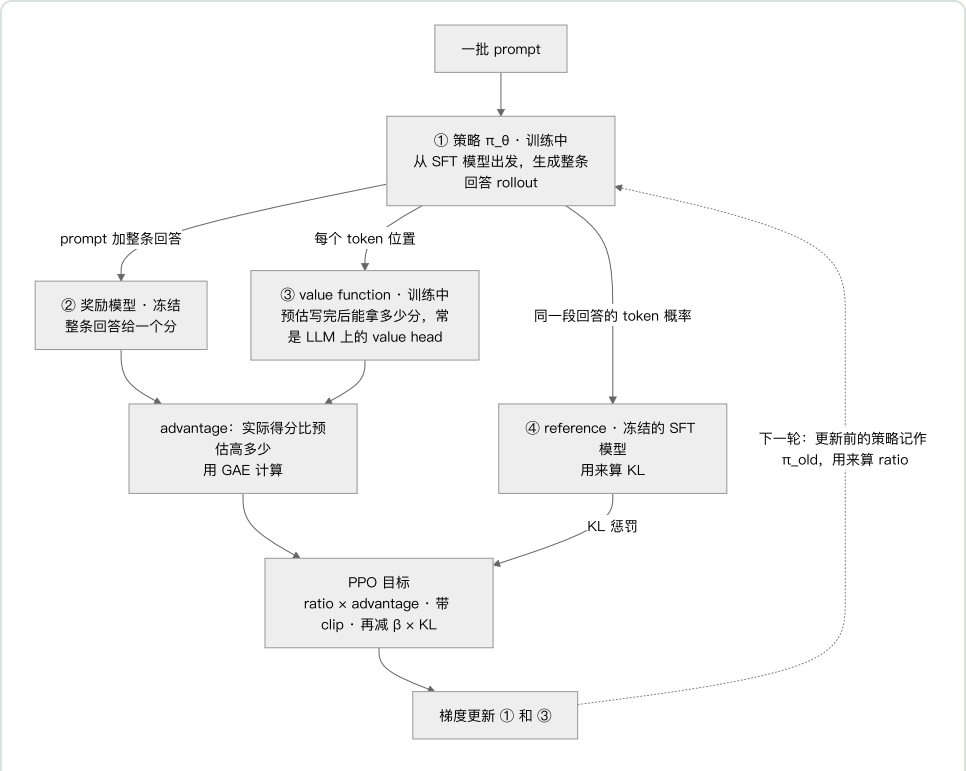


图 5-3 | PPO 一次迭代里的四个模型：两个在训练，两个冻结（自绘示意）·

► 看原幻灯片 1:14:24 · 出处: Schulman et al., 2017

- **PPO** (proximal policy optimization) 是 RL 阶段最常用的算法。proximal 是"邻近"的意思：每次更新都别走远。
- 实际最大化的不是原始奖励，而是 **advantage**（讲者自嘲前面"撒了个小谎"）。advantage 衡量的是：这次的输出比"预期水平"好多少。同样拿到 0.8 分，如果这个 prompt 平时都能拿 0.9，那这次其实算差的。这样做的原因是：从奖励里减掉一个基线（baseline）能降低估计的方差，训练更稳、更快。
- "预期水平"由 **value function** 来估。它和奖励模型很容易混，对比如下：

	奖励模型 RM	value function
读什么	prompt 加完整回答	prompt 加写到一半的回答
输出	这条回答的分数	照当前策略继续写完，预计最终能拿多少分
粒度	整条回答一个数	每个 token 位置一个数
怎么来	阶段一训好，RL 时冻结	RL 时和策略一起训（回归问题），通常是 LLM 上另接的一个 value head

- advantage 具体怎么从奖励和 value 算出来，用的是 **GAE** (generalized advantage estimation) ——一个带几个超参数的大公

式。讲者说这超出了本课范围，只给了论文。

小注：GAE 的两个超参数是折扣因子 γ 和在偏差与方差之间做权衡的 λ 。

- 所以 PPO 要同时伺候四个模型（图 5-3）：策略（训练）、value function（训练）、RM（冻结）、reference（冻结的 SFT 模型）。显存和工程负担主要就来自这里。真的需要这么多吗？讲者的回答是“也许不用”：GRPO 就是朝这个方向改的，留到第 6 讲，想预习可以看 DeepSeekMath 论文。

小注：GRPO 砍掉的正是 value function：对同一个 prompt 采样一组回答，用组内的平均奖励当基线。CS329A 笔记里 PPO 与 GRPO 的对比，说的就是图 5-3 里 ③ 这个框要不要。

7. PPO（下）：Clip 与 KL-penalty 两种写法

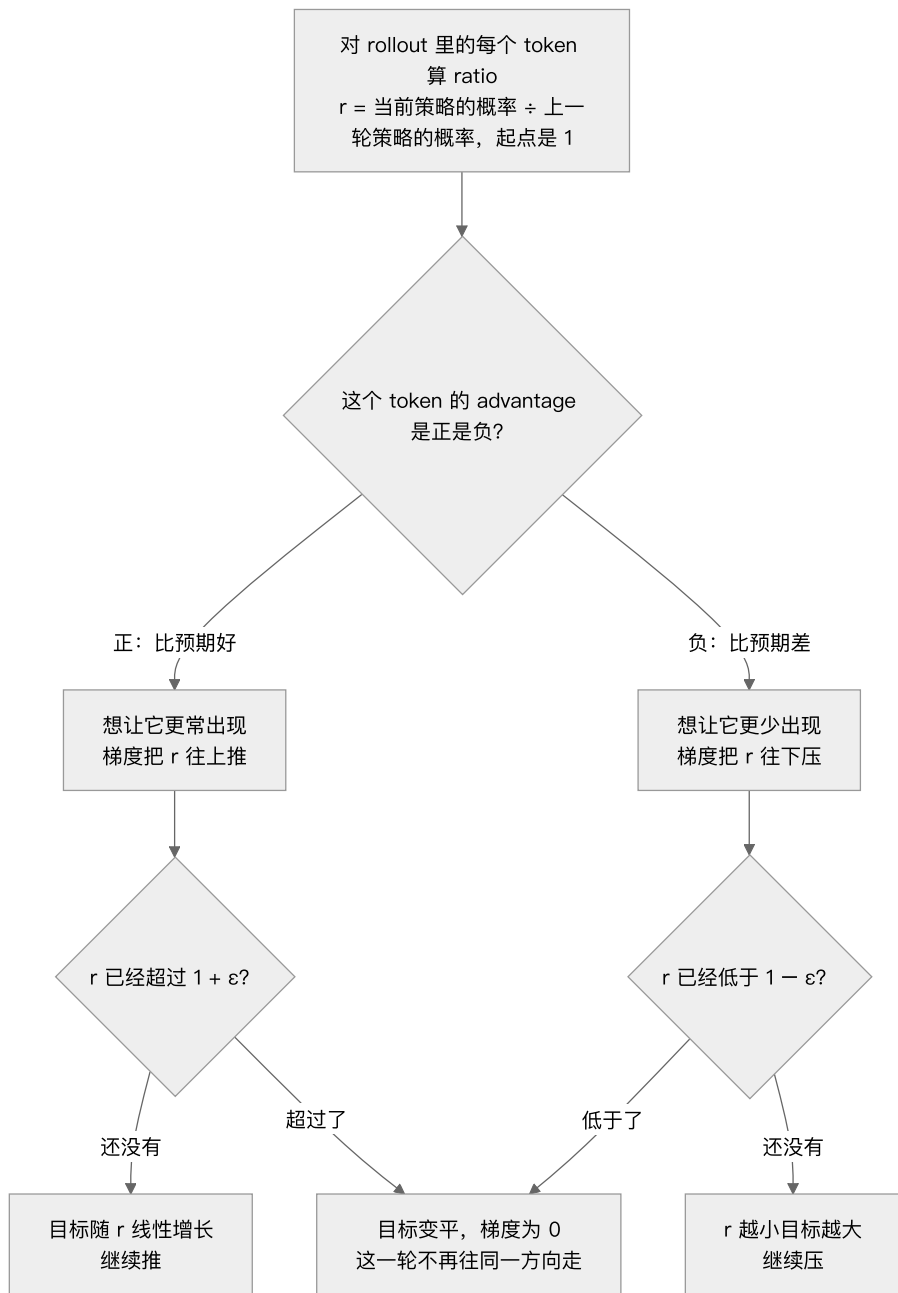


图 5-4 | PPO-Clip 的判断流程：好的多来一点，坏的少来一点，但每轮只许挪一小步（自绘示意）· [▶ 看原幻灯片 1:06:42](#) · 出处：Schulman et al., 2017

PPO 论文给了两种目标函数，区别在“别走远”这件事怎么写。先看用得最多的 PPO-Clip：

$$L^{\text{CLIP}}(\theta) = \mathbb{E}_t \left[\min \left(r_t(\theta) A_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) A_t \right) \right], \quad r_t(\theta) = \frac{\pi_{\theta}(a_t | s_t)}{\pi_{\theta_{\text{old}}}(a_t | s_t)}$$

A_t 是第 t 个 token 的 advantage， $r_t(\theta)$ 是同一个 token 在当前策略和上一轮策略下的概率之比， ϵ 是允许这个比值偏离 1 的幅度。

- 三个容易绊倒的地方（讲者说他自己当初也被绕过）：

1. 虽然记作 L ，这是一个要最大化的目标，不是要最小化的 loss。

2. $r_t(\theta)$ 不是 reward，是概率比 (ratio)。

3. π_{old} 不是 SFT 模型，而是 RL 过程中上一次迭代的策略。

- **直觉** (图 5-4)：A 大于 0，说明这个 token 值得强化，想把它概率调高，也就是让 r 变大；目标随 r 线性增长，但过了 $1 + \epsilon$ 就变平，再推也没有收益，所以一步不会推太多。A 小于 0 则反过来，想让 r 变小，低于 $1 - \epsilon$ 之后同样变平。

小注：min 的作用是只在“朝有利方向走过头”时才截断。如果 A 大于 0 而 r 已经掉到 $1 - \epsilon$ 以下，取 min 会选没截断的那一项，梯度照样把它拉回来。PPO 论文的实验里 $\epsilon = 0.2$ 效果最好，后来成了常用默认值；CS329A 笔记里 DAPO 的 clip-higher，改的就是这里的上界 $1 + \epsilon$ 。

- **KL-penalty 写法**：不做硬截断，改成减去一个惩罚项，目标是 $E[r_t(\theta) \cdot A_t - \beta \cdot \text{KL}(\pi_{old} \parallel \pi_\theta)]$ 。2017 年的原论文里 old 指上一轮迭代；到了 LLM 时代，KL 里放的一般是 reference (SFT 模型)。现在常见的是两者混用——对 reference 的 KL 惩罚，加上对上一轮策略的 clip。讲者也强调各家 loss 写法很多，这不是定式。
- **问答**
 - clip 的折点不可导怎么办？——和 ReLU 一样处理。
 - 为什么和上一轮比，而不是和 base 模型比？——两个约束都要：不能离 base 太远 (KL 项管)，每一轮更新也不能太猛 (clip 管)，后者是为了训练稳定。
 - ratio 具体怎么算？——就是 LLM 输出分布里，那个 token 在给定输入下的概率，新旧两个策略各算一次再相除。

8. RL 路线的代价；on-policy 与 off-policy

• 讲者列的挑战

1. 两阶段流水线有依赖：阶段二做完才发现 RM 有问题，就得全部重来。
2. 超参数多：KL 的 β 、clip 的 ϵ 、GAE 里至少还有两个……换一组就要重训。
3. 不稳定：限制了步长也不总是够。
4. 不好监控：能看的主要是平均奖励，它不像预训练和 SFT 的交叉熵那样，直接反映模型学得怎么样。
5. 需要探索 (exploration)：同一个 prompt 多次生成如果都差不多，模型就没机会发现哪种回答更好，得设法保证采样的多样性。

6. 门槛：多数人不熟 RL，而且这一步为什么非用 RL 不可，并不显然。

- **on-policy 与 off-policy**（论文里会反复出现的一对词）：SFT 时，模型模仿的是一批现成的"prompt 加回答"，不一定是它自己写的，这叫 off-policy。RL 阶段每一轮都让当前的模型自己生成，再按这些生成的好坏更新自己，这叫 on-policy。PPO 是 on-policy 算法。
- **问答**：为什么不直接拿偏好数据做 SFT？——SFT 只能说"这样答"，没法说"别这样答"，除非把坏回答全都改写成好回答。不过讲者顺势埋了个伏笔：确实有一种监督式的办法能做到 RL 做的事，就是第 10 节的 DPO。

9. Best-of-N：有奖励模型，但不想做 RL

- **做法**：SFT 模型原封不动。每来一个 prompt，采样 N 条回答（比如 4、5 条），每条交给 RM 打分，把分最高的那条返回给用户。泰迪熊的例子三条回答的分数是 0.8、-2 和一个居中的分，返回 0.8 那条。
- **代价**
 - 模型本身得有点水平。N 条全都差，挑出来的也差；调高 temperature 能增加多样性，但救不了差模型。
 - 主要问题是把训练的活全推给了推理：每个请求的生成成本乘以 N。流量大的服务要先算账——推理侧的流量有多大，一次性的训练成本又是多少。
 - Shervine 后来补了一个思想实验：就算钱无限、N 条并行生成，也得等最慢的那条；N 个延迟里取最大值，分布整体右移，延迟一定比单次生成差。
- **问答**：分数的尺度在这里无所谓——任何保持顺序的缩放都不改变谁最高。尺度只在分数进入 loss（也就是 RL）时才要紧。

小注：这就是 CS329A 笔记里"重复采样加验证器"的最简形态：RM 充当 ORM，N 就是花在推理时的算力。

10. DPO：把两阶段压成一个监督 loss

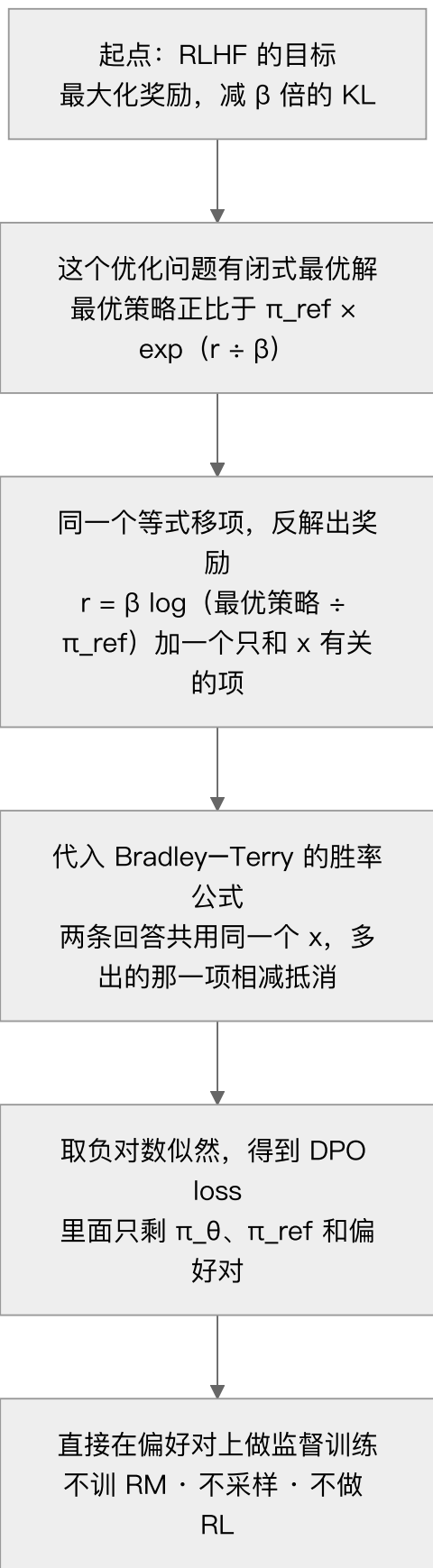


图 5-5 | DPO 的推导链：从 RLHF 的目标出发，把奖励模型“消掉”（自绘示意）·

► 看原幻灯片 1:35:19 · 出处：Rafailov et al., 2023

- **动机**就是前面攒下的抱怨：PPO 的 loss 背后要带四套权重（当前策略、old 或 reference，advantage 里还藏着 RM 和 value function）；Best-of-N 又有成本和延迟问题。DPO（direct preference optimization）的回答是：全程用监督学习，一个 loss 直接更新 LLM。

$$\mathcal{L}_{\text{DPO}}(\theta) = -\mathbb{E}_{(x, y_w, y_l)} \left[\log \sigma \left(\beta \log \frac{\pi_{\theta}(y_w | x)}{\pi_{\text{ref}}(y_w | x)} - \beta \log \frac{\pi_{\theta}(y_l | x)}{\pi_{\text{ref}}(y_l | x)} \right) \right]$$

$\pi_{\theta}(y | x)$ 是正在训练的模型生成整条回答 y 的概率， π_{ref} 是冻结的 SFT 模型， β 和第 5 节目标函数里的 β 是同一个（讲者给的量级是 0.1 左右）。

- **怎么读这个式子**：形状和第 4 节 RM 的 loss 一模一样，都是 $-\log \sigma(\text{赢家的分} - \text{输家的分})$ ，只是“分”换成了 $\beta \cdot \log(\pi_{\theta} / \pi_{\text{ref}})$ ——模型相对 reference 把这条回答的概率抬高了多少。里面没有 RM，没有采样，输入只有偏好对。论文副标题 Your Language Model is Secretly a Reward Model 说的就是这件事：那个相当于奖励的量，完全由策略自己表达。
- **怎么推出来的**（图 5-5）。讲者强调全程没有引入新假设，只是代数推导：
 1. 从第 5 节的 RLHF 目标（最大化奖励，减 β 倍 KL）出发。
 2. 这个优化问题有闭式的最优解，见下式左半。
 3. 同一个等式移项，把奖励写成最优策略的函数，见下式右半。
 4. 把它代进 Bradley-Terry 的胜率公式，再像第 4 节那样取负对数似然，就得到上面的 loss。

$$\pi^*(y | x) = \frac{1}{Z(x)} \pi_{\text{ref}}(y | x) \exp\left(\frac{r(x, y)}{\beta}\right) \iff r(x, y) = \beta \log \frac{\pi^*(y | x)}{\pi_{\text{ref}}(y | x)} + \beta \log Z(x)$$

π^* 是最优策略， $Z(x)$ 是只依赖 prompt 的归一化常数（partition function）；左式的意思是“最优策略等于 reference 按 $\exp(r / \beta)$ 重新加权”。

小注： $Z(x)$ 要对所有可能的回答求和，实际算不出来。但 Bradley-Terry 只看同一个 x 下两个回答的分差， $\beta \cdot \log Z(x)$ 相减时正好抵消——DPO 能落地全靠这一步，课上一带而过。

- **和 RLHF 比**：两阶段变一阶段；四个模型变两个（一套在训，一套冻结）；不需要在线采样，也就没有 RL 那一堆稳定性问题。

- **那为什么不是人人都用 DPO：**讲者引了一篇系统对比的论文，结论是 PPO 调好了整体仍然更强。DPO 的软肋是**分布偏移**（distribution shift）：它可以直接拿一份现成的偏好数据集来训，省掉奖励建模，但这些回答不是当前模型自己生成的，和模型的输出分布对不上。补救办法是先在偏好数据上做一遍 SFT，或者自己采样、自己标注——后者又把成本加了回来。

小注：这篇应为 Xu et al., 2024 (ICML 2024)：Is DPO Superior to PPO for LLM Alignment? 用第 8 节的词说，DPO 是 off-policy 的，PPO 是 on-policy 的，分布偏移就是 off-policy 的代价。

- **问答**
 - 两套权重分别是什么？——冻结的 reference 只出现在 loss 里；被更新的那一套就是最终的偏好微调模型。
 - reference 能不能换成另一个更大的模型？——那就成了另一种算法。偏好微调的本意是从上一个 checkpoint 出发、微调回答的分布，换掉 reference，loss 就失去了这层含义。讲者没把话说死，但直觉上不看好。
- **收尾的例子（贯穿整门课的泰迪熊）：**问"能把泰迪熊放进洗衣机吗"，SFT 后的回答事实正确——会洗坏，建议手洗——但口气生硬；偏好微调后的回答内容不变，语气温和得多。**偏好微调不教新事实**，它只是在模型已有的回答分布里挪动概率，往人更喜欢的那一侧靠。

11. 三条路线放在一起看

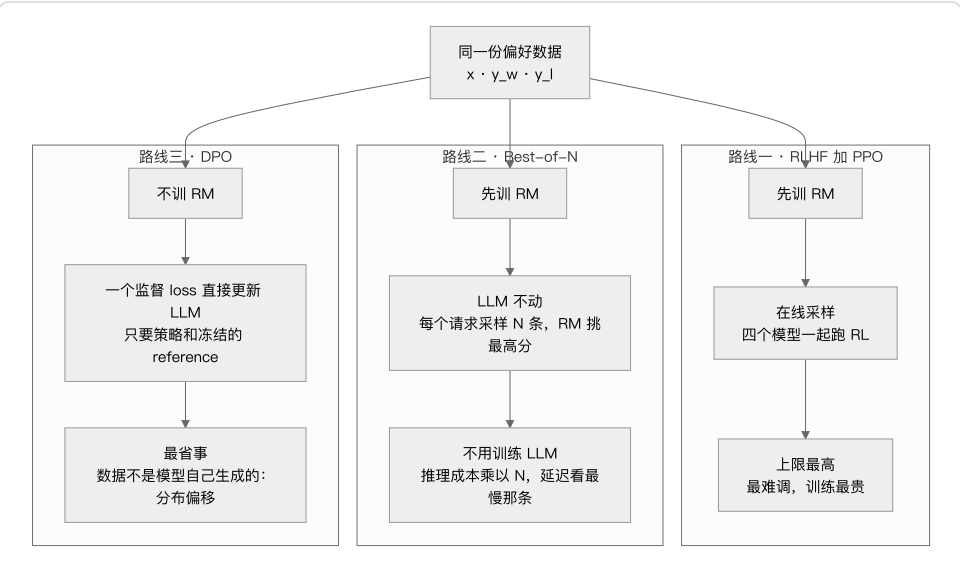


图 5-6 | 同一份偏好数据的三种用法，成本分别落在哪里（自绘示意）·

► 看原幻灯片 1:39:00

	RLHF + PPO	Best-of-N	DPO
要不要奖励 模型	要	要	不要
改不改 LLM 权重	改	不改	改
训练时同时 要带的模型	4 个：策略、value function、RM、reference	不训练	2 个：策略、reference
训练数据从 哪来	on-policy：当前模型现场生成，RM 打分	—	off-policy：现成的偏好对
成本花在哪	训练：最贵、最难调	推理：每个请求乘以 N，延迟变差	训练：和一次 SFT 差不多
讲者的评价	上限最高，适合懂 RL、要榨出最后一点性能的团队	先算推理流量和训练成本的账	花小得多的力气拿到接近的效果，但未必最好

- **实践中怎么选**（全场最后一个问答）：看算力预算，也看你愿意花多少精力"照看"训练过程。PPO 难调但上限最高；想快速做一次偏好微调、效果又要过得去，DPO 是首选。

关键图表速查（点时间戳跳到原幻灯片）

图	看什么	跳转	出处
泰迪熊偏好对	同一个 prompt 下，一条"像助手但不想要"的回答和改写后的回答怎么配成一对	5:19	—
RL 回路映射到 LLM	agent 与 environment 的回路上，每个量被换成 LLM 里的对应物	21:55	—
RLHF 两阶段	两个阶段各自的输入输出：RM 吃 prompt 加回答、吐分数；RL 吃 prompt、吐更符合偏好的回答	26:32	InstructGPT（推断，课上没点名）
Bradley–Terry 与 sigmoid 曲线	胜率只取决于分差；sigmoid 两端分别趋于 0 和 1	29:46	Bradley & Terry, 1952
RM loss 的手推过程	从"偏好对相互独立、最大似然"到负的期望 $\log \sigma(\text{分差})$ ，每一步怎么来	34:52	—
PPO loss 的两部分	奖励项和 KL 项各管什么；reference 就是 SFT 模型	54:15	PPO
advantage 与 value function	value 是 token 级的"预计最终得分"，由接在 LLM 上的 value head 输出	59:03	GAE
PPO–Clip 的两张折线图	A 为正时目标在 $1 + \epsilon$ 之后变平；A 为负时在 $1 - \epsilon$ 以下变平	1:06:42	PPO
PPO 需要的四个模型	策略、value function、RM、reference，哪两个在训、哪两个冻结	1:14:24	—

图	看什么	跳转	出处
Best-of-N 例子	三条回答各自的 RM 分数 (0.8、-2、居中)，返回最高的那条	1:24:11	—
DPO loss	认出 $\log \sigma$ (分差) 的 Bradley-Terry 结构；"分"换成了 $\beta \cdot \log(\pi_\theta / \pi_{\text{ref}})$	1:32:41	DPO
DPO 推导四步	目标 → 闭式最优策略 → 反解奖励 → 代入 Bradley-Terry	1:35:19	同上
PPO 与 DPO 的实证对比	讲者据此说 PPO 总体更强、DPO 受分布偏移拖累	1:40:33	应为 Xu et al., 2024
泰迪熊进洗衣机	SFT 后与偏好微调后的两个回答：事实相同，语气不同	1:44:13	—

提到的工作

名称	在本讲里的作用
pretraining、SFT、LoRA、ZeRO (第 4 讲)	开场回顾：偏好微调接在这两步之后；LoRA 可以和它叠加用
ChatGPT 这类聊天助手	SFT 把预训练模型变成助手的例子
RLHF	本讲主线：奖励模型加 RL 的两阶段偏好微调
RLAIF	对照：偏好标签来自模型而不是人（术语应出自 Constitutional AI，课上只提了名字）
Bradley-Terry 模型 (1952)	把"谁赢"的成对标签和"每个回答一个分数"连起来的概率模型；RM 和 DPO 的 loss 都建在它上面
LLM-as-a-judge	偏好标注的一种来源，讲者说后面的课细讲（应在第 8 讲）
BLEU / ROUGE	规则类指标，也能用来比较两个回答，现在用得少
BERT 与 [CLS] (第 2 讲)	RM 的另一种选型：encoder-only 模型，在 [CLS] 表示上接投影
RewardBench (Lambert et al., 2024)	评测奖励模型好坏的 benchmark
Jensen 不等式	证明 KL divergence 非负
PPO (Schulman et al., 2017)	RL 阶段的主力算法；Clip 和 KL-penalty 两种目标都出自这篇
GAE (Schulman et al., 2015)	计算 advantage 的标准方法，课上只给了引用
GRPO / DeepSeekMath (Shao et al., 2024)	"PPO 真要四个模型吗"的后续答案，留到第 6 讲
Best-of-N (BoN)	有 RM、不做 RL 的推理时方案
DPO (Rafailov et al., 2023)	把 RLHF 的两阶段并成一个监督 loss
Is DPO Superior to PPO? (Xu et al., ICML 2024；应为这篇)	"PPO 总体更强、DPO 有分布偏移"的依据

术语对照

English	中文
preference tuning / alignment	偏好微调 / 对齐：让输出更合人（或某个指标）的偏好
preference pair	偏好对：同一个 prompt 下一好一坏两个回答
pointwise / pairwise / listwise	单点打分 / 成对比较 / 列表排序
winning / losing response (y_w / y_l)	赢家回答 / 输家回答
RLHF	基于人类反馈的强化学习
RLAIF	基于 AI 反馈的强化学习：偏好标签由模型给
agent / environment / state / action	智能体 / 环境 / 状态 / 动作
policy π_θ	策略：给定状态下各个动作的概率；在这里就是 LLM 的 next-token 分布
reward model (RM)	奖励模型：读 prompt 加回答，输出一个分数
Bradley–Terry model	成对比较的概率模型：胜率等于分差的 sigmoid
negative log-likelihood	负对数似然
classification head / value head	接在 LLM 末端的小输出层：前者给回答打分，后者预估最终得分
completion / rollout	模型针对一个 prompt 生成的一整条回答
sparse reward	稀疏奖励：整条回答只给一次分
reward hacking	奖励投机：分数涨了，真正的目标没达到
catastrophic forgetting	灾难性遗忘
KL divergence	KL 散度：两个概率分布差多远（不对称，不是距离）
reference model π_{ref}	参考模型：冻结的 SFT 模型
PPO (proximal policy optimization)	近端策略优化
advantage	优势：这次输出比预期好多少
value function	价值函数：从写到一半的回答预估最终奖励
baseline	基线：从奖励里减掉的"平均水平"，用来降方差
GAE (generalized advantage estimation)	广义优势估计
probability ratio $r_t(\theta)$	概率比：当前策略与上一轮策略对同一个 token 的概率之比（不是 reward）
clipping	截断：ratio 超出 $1 \pm \epsilon$ 的区间后不再带来收益

English	中文
on-policy / off-policy	用当前模型自己生成的数据训练 / 用别处来的数据训练
exploration	探索：采样要够多样，才能发现更好的回答
Best-of-N (BoN)	N 选一：采样 N 条，RM 挑最高分
DPO (direct preference optimization)	直接偏好优化
partition function $Z(x)$	配分函数：让概率之和为 1 的归一化常数
distribution shift	分布偏移：训练用的回答不是模型自己会生成的那种

字幕勘误

"PO""PPU" → PPO；"RHF" → RLHF；"RL ... Aif" → RLAIIF；"Kale / Kyle / scale divergence" → KL divergence；"Laura" → LoRA；"zero" → ZeRO；"blur rouge" → BLEU、ROUGE；"BERS" → BERT；"Reward bench" → RewardBench；"Bradlary" → Bradley-Terry；"binary cony" → binary cross-entropy；"parise""pair wise" → pairwise；"perference" → preference；"bun" → BoN；"DPU" → DPO；"deepse math" → DeepSeekMath；"raiders" → raters（标注员）；"overfeeding""overfeit" → overfitting；讲 KL 正负时的 "luck" → log；"80""AD and H" → a_t 、 a_t 和 s_t ；"Afinen""Ein""Afin" → Afshine；"Shervin" → Shervine。

带走的问题

- RM 的 loss 只约束分差，不约束绝对值。这对 Best-of-N 无所谓，对 PPO 却要先做归一化——为什么？如果换一个分数尺度大 10 倍的 RM，同一个 β 下 KL 这根绳子是变松了还是变紧了？
- PPO 里有两根绳子：对 reference 的 KL，和对 π_{old} 的 clip。它们各自防的是什么？去掉其中一根会出什么事？（DAPO 去掉了 KL 项、放宽了 clip 的上界——在奖励可验证的推理任务上，为什么敢这么做？）
- DPO 的隐式奖励是 $\beta \cdot \log(\pi_{\theta} / \pi_{ref})$ 。训练中如果 y_w 和 y_l 的概率一起下降、只是 y_l 降得更快，loss 同样会变小。这对生成质量意味着什么？on-policy 采样的 PPO 为什么不容易出这个问题？
- 在你做的 agent 产品里，偏好信号可以从哪里来（重试、编辑、点踩、任务是否完成）？它们更像 pointwise、pairwise 还是 listwise？

标注口径不清带来的噪声，会怎样从 RM 一路传到策略？

5. Best-of-N 把成本放在推理侧，PPO 和 DPO 放在训练侧。什么样的流量和延迟要求下 BoN 反而划算？能不能把 BoN 挑出来的回答再拿去做 SFT，把推理成本"蒸馏"回训练里？

CME295 第 6 讲 | LLM 推理 (LLM Reasoning)

Stanford CME295: Transformers & Large Language Models (2025 秋) · 第 6 讲, 2025 年 11 月 7 日 视频: <https://www.youtube.com/watch?v=k5Fh-UgTuCo> (1:47:10, 英文字幕是自动生成的, 缩写和专名错得比较多, 见文末勘误) 讲者: Afshine Amidi (开场到 1:29:38: 推理模型、评测指标、GRPO 及其改进) · Shervine Amidi (1:29:38 起: DeepSeek-R1 配方与蒸馏) 课程大纲: <https://cme295.stanford.edu/syllabus/>

小注: 大纲页现在挂的是 2026 秋季版课表, 讲次已经重排 (那一版的第 6 讲是 AI Agents)。这套录像对应的 2025 秋课表在 <https://cme295.stanford.edu/syllabus/2025/>, 那一页也挂着本讲的幻灯片 PDF。本笔记里说的"第 N 讲"都指 2025 秋录像的编号。

一句话: 推理模型就是先写出一段推理链、再给答案的 LLM——把第 3 讲的 chain of thought 从提示技巧变成训练出来的习惯。衡量它用数学题、编程题这类能自动判对错的基准, 核心指标是 pass@k; 训练它不靠人写推理链, 而是拿"格式对不对、答案对不对"这两个可验证奖励做 RL, 算法是 GRPO: 同一道题采一组回答, 用组内均值当基线, 省掉 PPO 那个要一起训练的价值模型。GRPO 按回答长度取平均的写法会让答错的长回答少挨罚, 于是有了 DAPO 和 Dr. GRPO 的修补。最后, DeepSeek-R1 把这些零件拼成完整配方: R1-Zero 用纯 RL 做概念验证, R1 走"冷启动 SFT → 推理 RL → 拒绝采样 SFT → 全场景 RL"四步, 再把能力蒸馏进小模型。

时间轴

时间	内容
0:06	开场与回顾: 预训练 → SFT → 偏好微调; 用 RL 的词汇描述 LLM
5:16	RLHF 损失的两部分; PPO-Clip 与 PPO-KL penalty
8:56	vanilla LLM 的长处与四个短板; 后三个留给第 7、8 讲
12:43	本讲的两个目标; reasoning 的工作定义; 知识题与推理题
15:43	核心想法: 把 chain of thought 做大; 问答: 日期这类上下文放在 preamble
18:53	两个直觉: 拆成见过的小问题; 多写 token 就是多用算力
21:24	vanilla LLM 与推理模型的输出对比; 推理模型时间线
24:30	怎么认出推理模型: Thinking 与思考摘要; 推理 token 按输出计费

时间	内容
27:49	基准：编程（HumanEval、Codeforces、SWE-bench）与数学（AIME、GSM8K），怎么自动判对错
32:04	pass@k 的定义与动机；和 Best-of-N 的关系
35:32	为什么要采 n 个而不是 k 个；板书推导估计式
44:21	pass@1；温度怎么影响 pass@k；cons@k
48:07	怎么训出推理模型：不从 SFT 起步的三个理由
51:32	两个可验证奖励（格式 + 正确性）；R1-Zero 的 AIME 曲线
54:10	控制思考量：思考预算、上下文感知、budget forcing、连续思考
57:44	GRPO：名字、目标，和 PPO 在 advantage 上的分歧
1:03:30	问答：为什么要换掉价值函数；组内基线的直觉
1:06:03	GRPO 与 PPO 的流程对照；哪些模型冻结、哪些要训练
1:11:44	两个目标函数并排看：相同点与不同点；小结
1:16:14	长度偏置：RL 越训回答越长，准确率却先走平了
1:19:31	从 GRPO 目标函数里找原因：按回答长度取平均的那个因子
1:25:00	DAPO 与 Dr. GRPO 的修法和效果
1:27:14	另外两处改动：标准差带来的偏置；非对称的裁剪上下界
1:29:38	换 Shervine：DeepSeek-R1 配方总览；R1-Zero 在 base 模型上直接做 RL
1:34:31	R1-Zero 的问题：推理链语言混杂、难读
1:36:11	R1 的前两个阶段：冷启动 SFT；加了语言一致性奖励的推理 RL
1:38:42	后两个阶段：拒绝采样造数据的第二轮 SFT；全场景 RL
1:42:20	结果：推理模型与非推理模型分成两簇
1:43:24	蒸馏到小模型；为什么不直接对小模型做 RL

核心内容

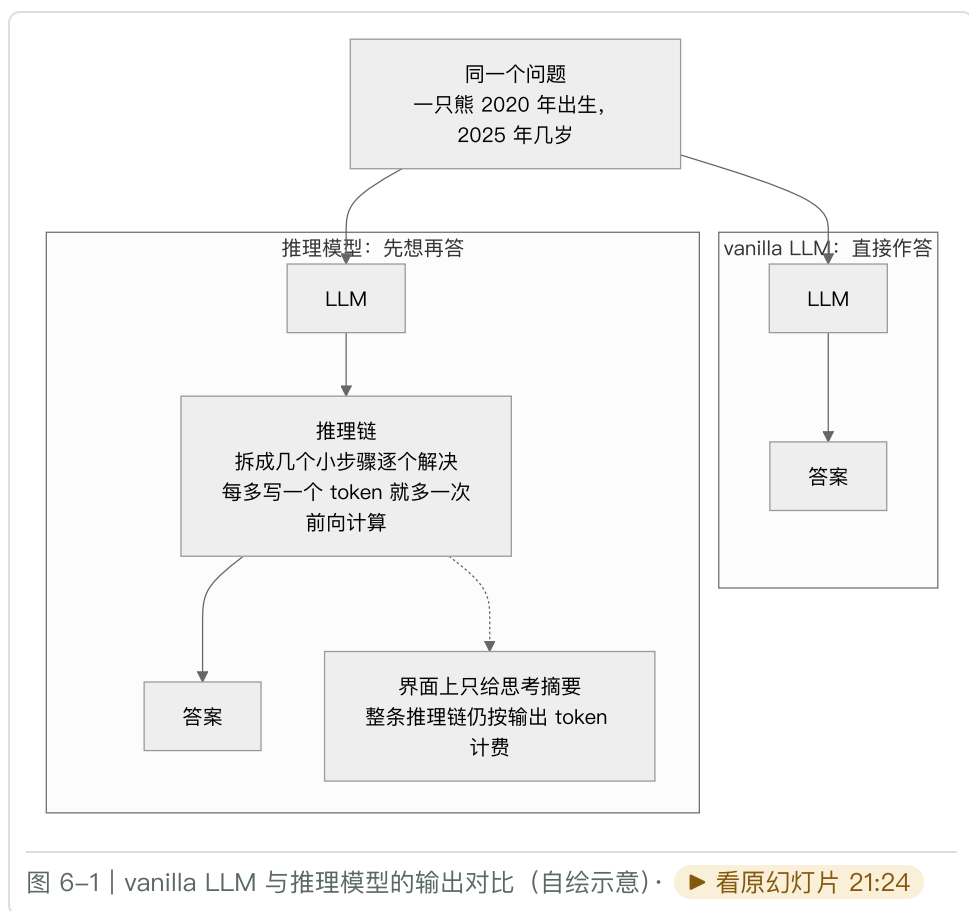
1. 承上启下：第 5 讲的 RL 工具箱，和 vanilla LLM 的四个短板

- **三步训练回顾**：预训练最烧算力，模型学到文本和代码的结构，但只会续写（第 4 讲）；SFT 用精选的高质量数据教它按要求办事，比如当助手；偏好微调让它贴近人的偏好，做法是 RLHF——先用人类偏好数据训一个奖励模型，再进入 RL 阶段（第 5 讲）。今天要借用的就是这个 RL 阶段。
- **把 LLM 套进 RL 的词汇表**：agent 是 LLM 自己；状态是到目前为止的输入（prompt 加已生成的 token）；动作是选下一个 token；策略

(policy) 就是模型给出的下一个 token 的概率分布；一段回答写完后拿到一个奖励，第 5 讲里这个奖励代表人的偏好。

- **RL 阶段的损失有两部分**：一是最大化 advantage——奖励减去一个基线，意思是"这个回答比预期好多少"，减基线是为了降低梯度的方差；二是别跑太远，既不要离上一轮迭代的旧策略太远，也不要离起点的 SFT 模型太远。理由是模型已经学会了很多，这一步只想校准，不想重塑。
- **PPO 的两个变体**：PPO-Clip 盯着新旧策略对同一个 token 的概率比（幻灯片里记作 r ，它不是 reward），超出阈值就裁掉，不鼓励一步迈太大；PPO-KL penalty 用 KL 散度当罚项，原论文里比的是上一轮的策略，如今的 RLHF 通常比的是 SFT 模型。实际训练一般是两者混着用。
- **vanilla LLM 的四个短板**：讲者把"输入 prompt、直接吐答案"的模型叫 vanilla LLM。它写代码、找 bug、写文章都很强，但有四个弱点：推理能力有限，稍复杂的数学题就容易半途走丢；知识是静态的，停在预训练数据的截止日期（例子：问它几天前的选举结果）；只会说不会做，没法替你下单；输出是自由文本，翻译用的 BLEU、摘要用的 ROUGE 这类规则指标不再适用，评测很难。后三个留给第 7、8 讲，本讲只管第一个。

2. 什么是推理模型：把 chain of thought 做成模型的习惯



- **先对齐用词：**中文里 inference（运行模型做生成）和 reasoning（分步求解问题）都常被译成“推理”。本讲标题里的推理是 reasoning。
- **工作定义：**讲者先承认学界对 reasoning 没有公认的定义，然后给了一个够用的：解决问题的能力，而这类问题通常要经过多步的求解过程，典型的是数学题和编程题，并希望这种能力能外溢到其他领域。对照的例子：问“这门课的课号是什么”是知识题，知道就是知道；问“一只熊 2020 年出生，到 2025 年几岁”是推理题——当然真正的推理题要难得多。
- **核心想法来自 chain of thought（第 3 讲）：**CoT 在 prompt 里放几个把推理过程写出来的示例，引导模型也先写过程、再给答案。推理模型做的是同一件事，只是规模大得多，而且是训练出来的：输出不再只是答案，而是“推理链 + 答案”。
- **先写过程为什么有用，讲者给了两个直觉：**
 - LLM 靠 next-token prediction 作答，本质是在挑“看起来最像样”的续写。一道很难的题几乎不可能在训练集里原样出现过；拆成小步之后，每一小步都更像训练时见过的模式，可以各个击破——和学生把新题往做过的题上靠是一回事。

- 每生成一个 token 都要做一次完整的前向计算，多写 token 等于给这道题多分配算力。后面反复出现的 compute budget（思考预算）指的就是愿意让模型为一次回答花多少 token。
- **问答：题目依赖上下文怎么办**（学生指出熊的例子得知道今天的日期）。模型前面通常有一段 preamble，日期这类信息就放在那里；需要去外部取的信息留给第 7 讲。讲者把这些看作推理模型的扩展，而不是它的根基。
- **时间线**：2024 年 9 月 OpenAI 的 o1-preview 是起点，之后各家都在猜它是怎么做的；12 月 Google 出了 Gemini 2.0 Flash Thinking；2025 年 1 月 DeepSeek-R1 的论文引起轰动，因为它追平了 OpenAI 的推理成绩，而且把方法写了出来；此后 xAI、Anthropic、Mistral 等的模型也陆续有了推理能力。这个方向只有一年左右的历史，本讲几乎所有内容都出自 2024—2025 年。
- **怎么认出推理模型**：界面上有 Thinking 字样和思考耗时（那段时间就是在生成推理链），ChatGPT 还能在标准和加长思考之间切换。界面显示的是思考摘要，不是原始推理链。讲者猜了三个原因：原始链条人未必读得懂；用户不想读好几页；拿到原始链条的人可以用它训练出模仿这种能力的模型——第 7 节的蒸馏正是这么做的。
- **计费**：各家 API 文档里都有一句，推理 token 计入输出 token 收费，看不到也要付钱。所以用户想要的是“用最少的推理 token 换最强的推理能力”，这为第 6 节的长度问题埋了伏笔。

3. 怎么衡量：能自动判对错的基准，和 pass@k

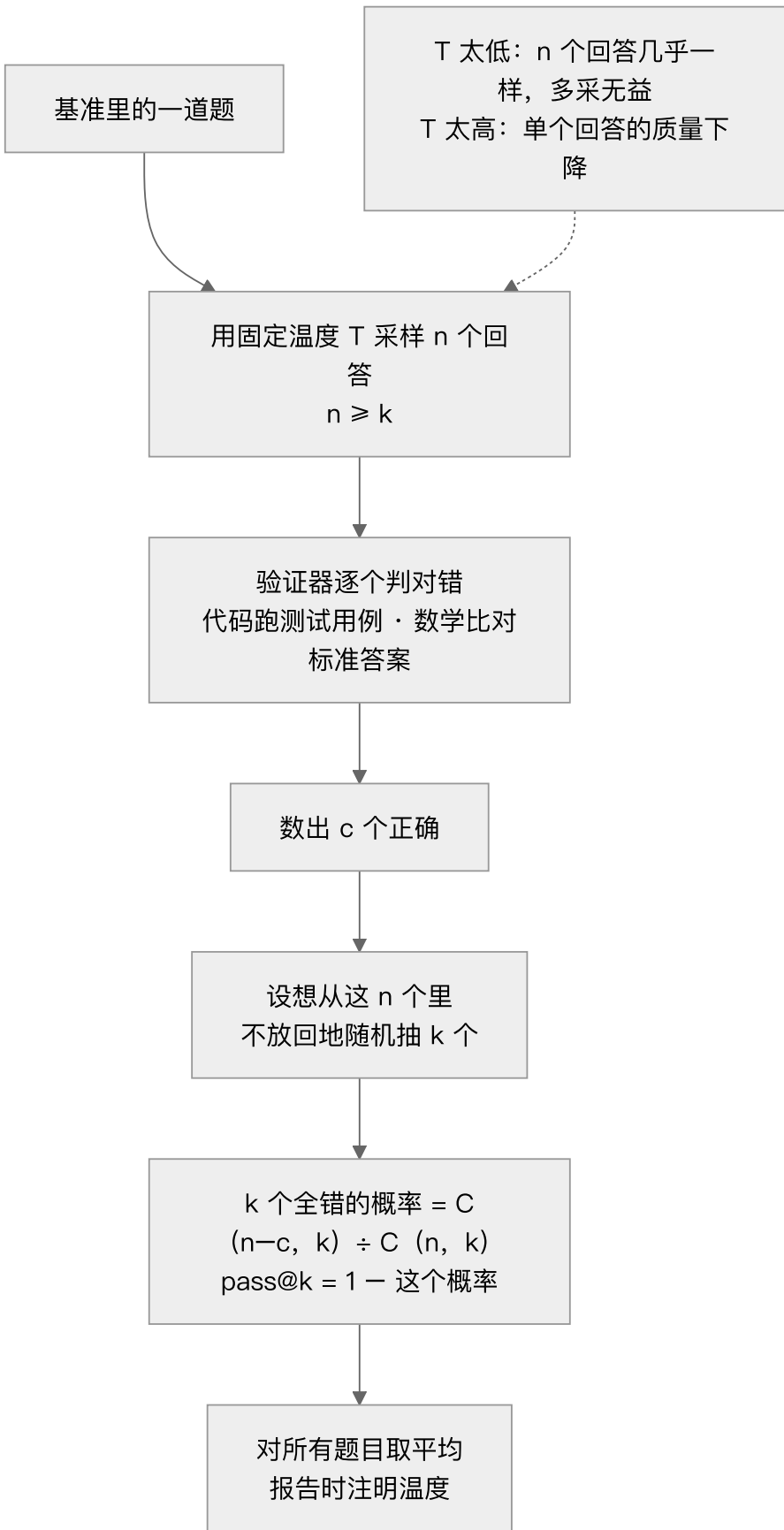


图 6-2 | pass@k 的估计流程: 多采、验证、再用组合数折算 (自绘示意) ·

- **编程基准**：给一个题目或一个 bug，模型交出的代码去跑测试用例，全过才算对。HumanEval 是一百多道人工写的编程题；Codeforces 来自竞赛编程网站；SWE-bench 的题目取自真实的 GitHub issue。

小注：HumanEval 共 164 题，出自 OpenAI 的 Codex 论文 (Chen et al., 2021)；下面的 pass@k 估计式也是这篇论文提出的。

- **数学基准**：在 prompt 里要求模型把最终答案放进固定格式（比如一个 box 里），解析出来和标准答案比对。AIME 是美国数学奥赛的资格赛，题目并不简单，推理模型的报告里几乎都有它；GSM8K 是小学水平的应用题。
- 这两类任务的共同点是对错可以自动判定，不需要人，也不需要另一个模型。到下一节，这一点会变成训练信号。
- **pass@k 的定义**：让模型对同一道题作答 k 次，至少有一次正确的概率。它有意义的前提是你付得起多次生成、又有办法检查答案——这时多花算力换更高的命中率是划算的。它和第 5 讲的 Best-of-N 很像，区别是这里用确定性的验证器来挑，而不是用奖励模型打分。
- **怎么估**（讲者在黑板上推了一遍）：只生成 k 个、看里面有没有对的，这样估出来的值波动很大。做法是生成 n 个 ($n \geq k$)，数出其中 c 个正确，然后问：从这 n 个里不放回地随机抽 k 个，至少抽到一个正确的概率是多少？用“至少一个对 = 1 - 全都错”来算：第一个抽到错的概率是 $(n-c)/n$ ，第二个是 $(n-c-1)/(n-1)$ ，一直乘到第 k 个；把这串乘积写成阶乘，正好是两个组合数之比。

$$\text{pass}@k = 1 - \prod_{j=0}^{k-1} \frac{n-c-j}{n-j} = 1 - \frac{\binom{n-c}{k}}{\binom{n}{k}}$$

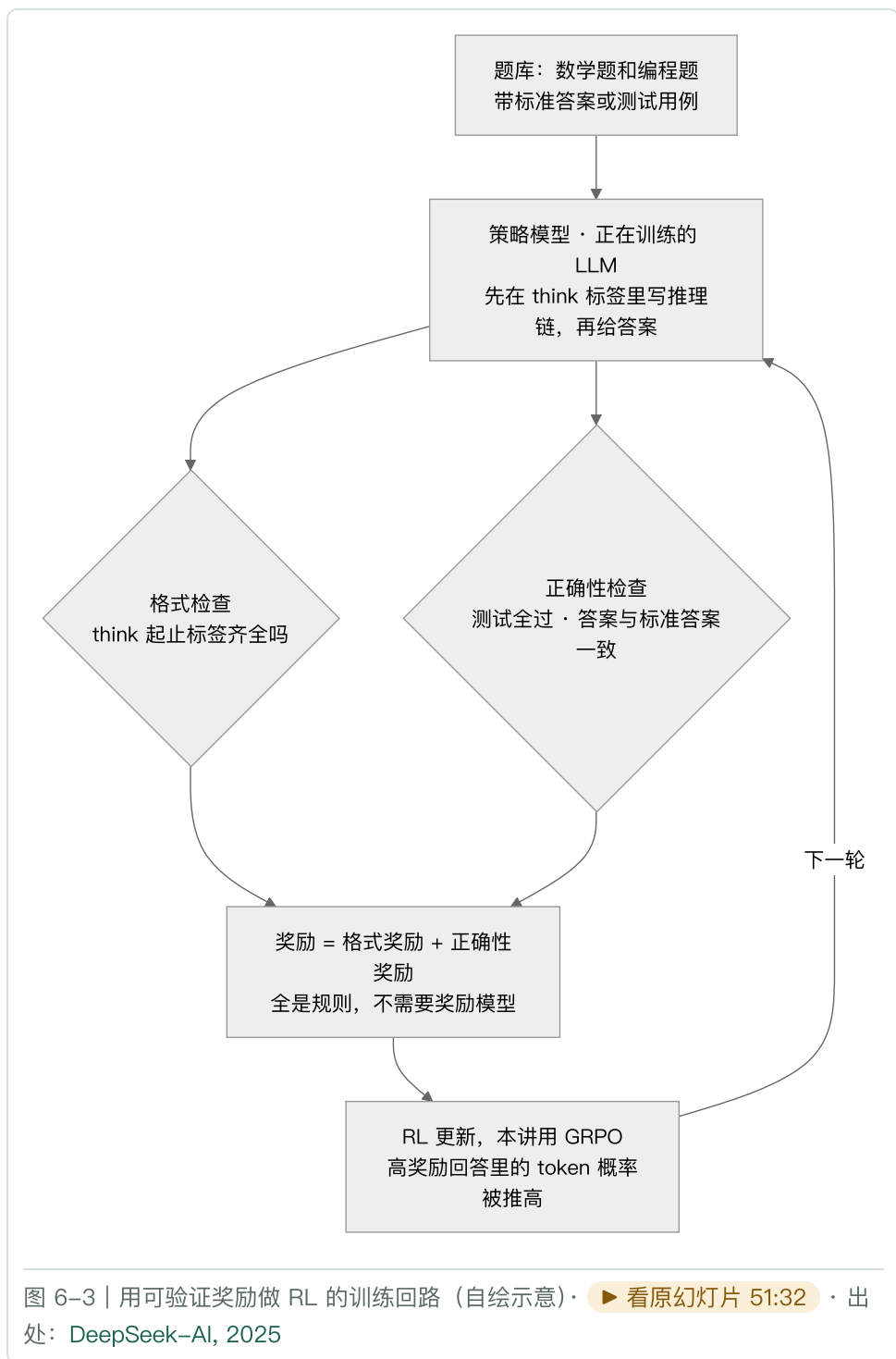
其中 n 是每道题采样的回答总数，c 是其中判为正确的个数，k 是指标允许的尝试次数；分子是“只从错的里面挑 k 个”的挑法数，分母是“从全部 n 个里挑 k 个”的挑法数。

- $k = 1$ 时公式化简成 c/n ，也就是单次作答的正确率 pass@1。
- **温度和 pass@k**（回答前面一位学生的提问）：多次采样的收益来自回答的多样性，而多样性由温度控制。T = 0 时每次输出都一样，k 再大曲线也是平的；T 稍高（图里的 0.2）曲线开始随 k 上升；T 过高（图里的 1.2）虽然够多样，但原本不太可能的 token 被频繁选中，单个回答的质量下降，效果也不好。图里 k 较大时最好的是 0.8，k 较小时偏低的温度（比如 0.4）更好。所以论文报基准成绩时都会注明温度。

小注：Codex 论文做过同样的实验（幻灯片上的图推断就出自那里），结论是 pass@1 的最优温度为 0.2、pass@100 为 0.8——允许的尝试次数越多，越值得用高温度换多样性。

- **其他指标**：cons@k (consensus) 取 k 个回答里出现次数最多的那个答案，再判对错，和第 3 讲的 self-consistency 是同一个思路；此外还有常见的 accuracy、exact match。
- **和 CS329A 的关系**：CS329A 第 2 讲的 coverage 曲线画的就是 pass@k 随 k 的变化，那边讲的是它的幂律和“采得到不等于挑得出” (generation-verification gap)；cons@k 就是 CS329A 第 6 讲里的 maj@K。那几讲讲的是研究前沿，这里补的是基础：这个数该怎么估，温度为什么要一起报。

4. 怎么训出来：为什么是 RL，奖励从哪来



- **目标：**让模型在回答之前自己写出推理链，而且这件事要能规模化。
- **为什么不从 SFT 起步**（前提是手里没有现成的推理链）：
 1. SFT 要高质量的示范数据，也就是得有人把很长的推理链一条条写出来，又难又贵。
 2. 模型“想问题”的方式未必和人一样，人写的推理过程不一定是教它的最好教材。
 3. 上一节刚看到，这类任务天生带着可验证的奖励：代码能不能通过测试，数学答案和标准答案是否一致。有现成的奖励信号，第 5 讲

又刚学了 LLM 上的 RL，那就不用 RL。

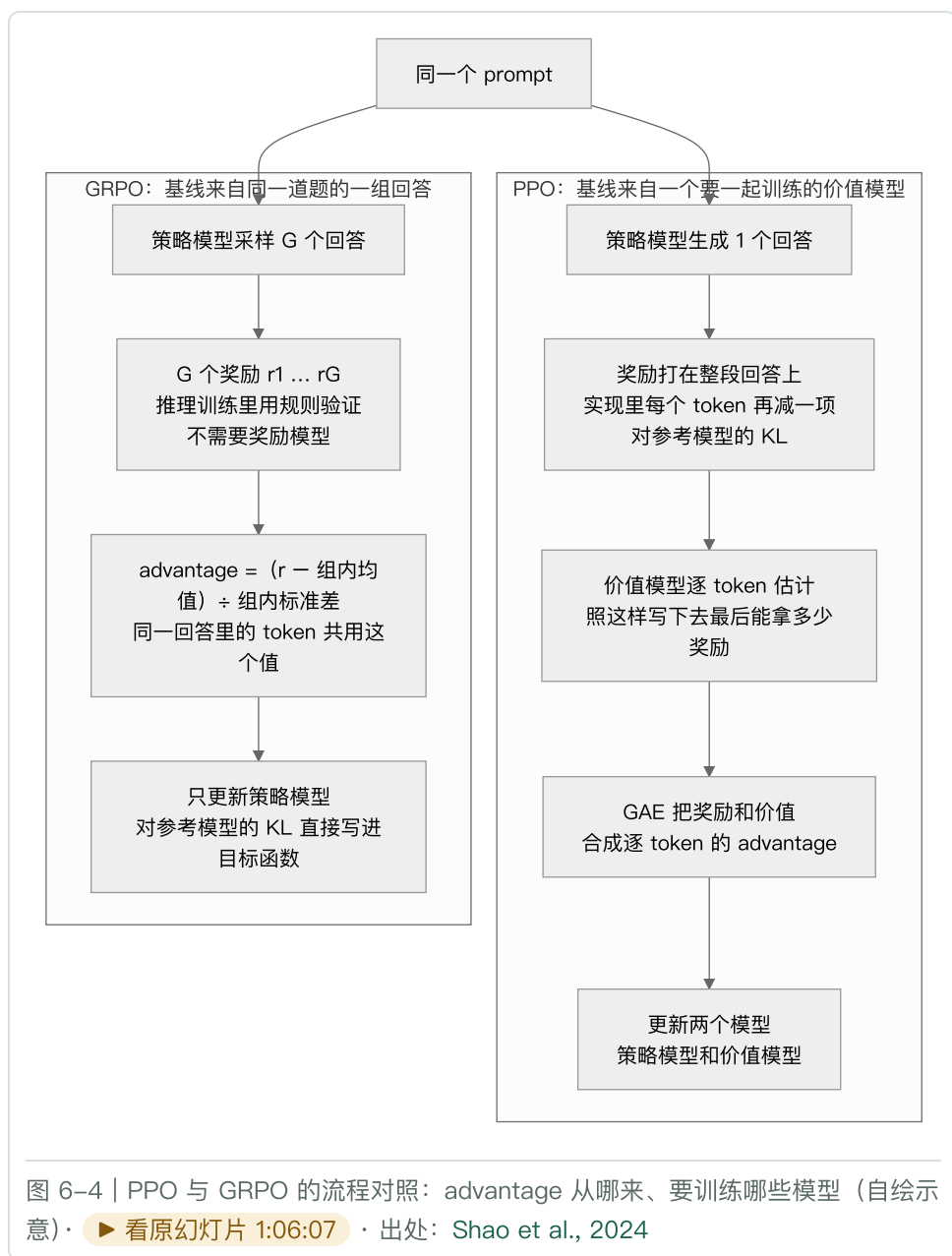
- **两个奖励，都是规则：**格式奖励检查模型有没有把推理过程放在约定的 think 起止标签之间；正确性奖励检查最终答案——代码过没过全部测试，数学答案对不对。两者合起来就是训练用的奖励，全程不需要奖励模型。
- **效果：**只靠这两个奖励做 RL，模型在 AIME 上的准确率随训练步数稳步上升，幅度很大。图来自 DeepSeek-R1-Zero 的训练曲线（第 7 节细讲）。

小注：按论文 arXiv 第一版，R1-Zero 的 AIME 2024 pass@1 从 15.6% 升到 71.0%；64 个样本取多数票 (cons@64) 是 86.7%。

- **和 CS329A 的关系：**CS329A 第 3 讲讲的是对错要靠训练出来的验证器来判的情形 (ORM / PRM)；本讲挑的是规则就能判的任务，所以整个绕开了奖励模型。CS329A 第 6 讲的 STaR 是另一条路：让模型自己生成推理链，答对才留下来做 SFT——R1 配方的第三阶段用的是同一种思路。
- **思考量怎么控制**（讲者说这些还是开放问题）：
 - 不是每个 prompt 都值得长考，要避免在简单问题上想太多 (overthinking)。一种可能的做法是先用一个轻量分类器判断这道题需要多少思考，再分配预算。
 - 上下文窗口有限，模型思考时得知道还剩多少空间。
 - **budget forcing** (s1 论文)：想让模型多想，就在它准备收尾时插入 Wait 这样的词，它往往会换一条思路再查一遍；想让它停，就插入一句“时间到，我的答案是”，逼它作答。
 - **连续思考** (continuous thought)：推理不一定要落在语言 token 上，也可以在隐藏表示里进行，信息更密、也更省；这条线很活跃，讲者说几天前还看到新论文。

小注：幻灯片上链接的那篇 2024 年论文应为 Coconut (Hao et al.)，做法是把最后一层的隐藏状态直接当作下一步的输入，不解码成 token。另外，s1 本身只用 1,000 条带推理链的样本对 Qwen2.5-32B-Instruct 做 SFT——手里已有推理链时 SFT 依然好用，这和讲者“从零开始才用 RL”的前提并不矛盾。

5. GRPO：用同一道题的一组回答，代替 PPO 的价值模型



- **定位:** GRPO (Group Relative Policy Optimization) 2024 年提出, 如今是推理训练里默认的 RL 算法。它和 PPO 要做的两件事一样: 最大化 advantage; 不让模型离旧策略和参考模型太远。分歧只在 advantage 怎么算。

小注: GRPO 出自 DeepSeekMath (Shao et al., 2024 年 2 月)。那篇论文里奖励还来自训练出来的奖励模型, 到 DeepSeek-R1 才换成纯规则奖励。

- **PPO 怎么算:** 奖励是打给整段回答的, 但更新要落到每个 token 上。PPO 为此另外训练一个价值函数 (value function, 实现上是另一个要和策略一起训练的模型): 在每个 token 的位置上, 预测"按当前策略继续写下去, 最后能拿多少奖励"。再用 GAE (generalized

advantage estimation, 课上没有展开的一个公式) 把奖励和逐 token 的价值合成 advantage。讲者说瓶颈就在这个昂贵的价值模型上。

- **GRPO 怎么算**: 对同一个 prompt 采样 G 个回答, 各自算出奖励; 拿每个回答的奖励和这一组的平均水平比, 再除以组内标准差, 就是它的 advantage, 这个回答里的所有 token 共用这一个值。没有价值模型, 代价只是每道题要多采几个回答。

$$\hat{A}_i = \frac{r_i - \text{mean}(r_1, \dots, r_G)}{\text{std}(r_1, \dots, r_G)}$$

其中 r_i 是第 i 个回答的奖励, G 是同一个 prompt 采样的回答数, mean 和 std 是这 G 个奖励的均值和标准差。

- **问答: 为什么要这么换**。PPO 是 2017 年的算法, 为通用的 RL 场景设计, 那时还没有 LLM; 价值函数在那些场景里很自然, 放到语言任务里却意味着多养一个昂贵的模型。GRPO 的想法是有一个"相对好坏"就够了, 而组内均值恰好是一个懂题目难度的基线: 简单题大家都答对, 奖励高也不稀奇; 难题上难得有一个回答做对, 就应该大力推高它用到的 token。
- **一个实现细节**: PPO 对参考模型的 KL 罚项通常并进奖励里——每个 token 都带一项 KL, 只有最后一个 token 再加上整段回答的奖励; GRPO 则把 KL 项直接写在目标函数里。
- **谁冻结、谁训练**: 两种算法里参考模型和奖励模型都是冻结的; 推理训练因为奖励可验证, 干脆没有奖励模型。要训练的, GRPO 只有策略模型, PPO 是策略模型加价值模型。
- **两个目标函数并排看**: 相同点是都围绕新旧策略的概率比做文章, 都用 clip 把一次更新限制在小范围内 (第 5 讲按 advantage 的正负画过那两张图); 不同点是 KL 放在哪, 以及 advantage 怎么来。

$$J_{\text{GRPO}}(\theta) = \mathbb{E} \left[\frac{1}{G} \sum_{i=1}^G \frac{1}{|o_i|} \sum_{t=1}^{|o_i|} \left(\min(\rho_{i,t} \hat{A}_i, \text{clip}(\rho_{i,t}, 1 - \varepsilon, 1 + \varepsilon) \hat{A}_i) - \beta D_{\text{KL}}[\pi_\theta \parallel \pi_{\text{ref}}] \right) \right]$$

其中 o_i 是第 i 个回答, $|o_i|$ 是它的 token 数; ρ 是新旧策略对第 i 个回答第 t 个 token 的概率比 (幻灯片沿用 PPO 的记号写作 r , 这里改用 ρ , 免得和奖励混淆); ε 是裁剪范围, β 是 KL 罚项的系数, π_{ref} 是参考模型。

- **讲者的小结**: PPO 主要用在偏好微调, GRPO 主要用在推理训练; GRPO 不需要价值函数, advantage 靠和同组的其他回答比出来。他也坦言这是全课最难的一段, 建议回看录像。

- 和 CS329A 的关系：CS329A 第 6 讲是顺着 DeepSeekMath 论文讲 GRPO 的（省显存、online 胜过 offline、整组全对或全错时没有信号）；这里补的是基础：目标函数逐项怎么读，和 PPO 在实现层面差在哪。

6. 长度偏置：GRPO 为什么越训越长，DAPO 和 Dr. GRPO 怎么修

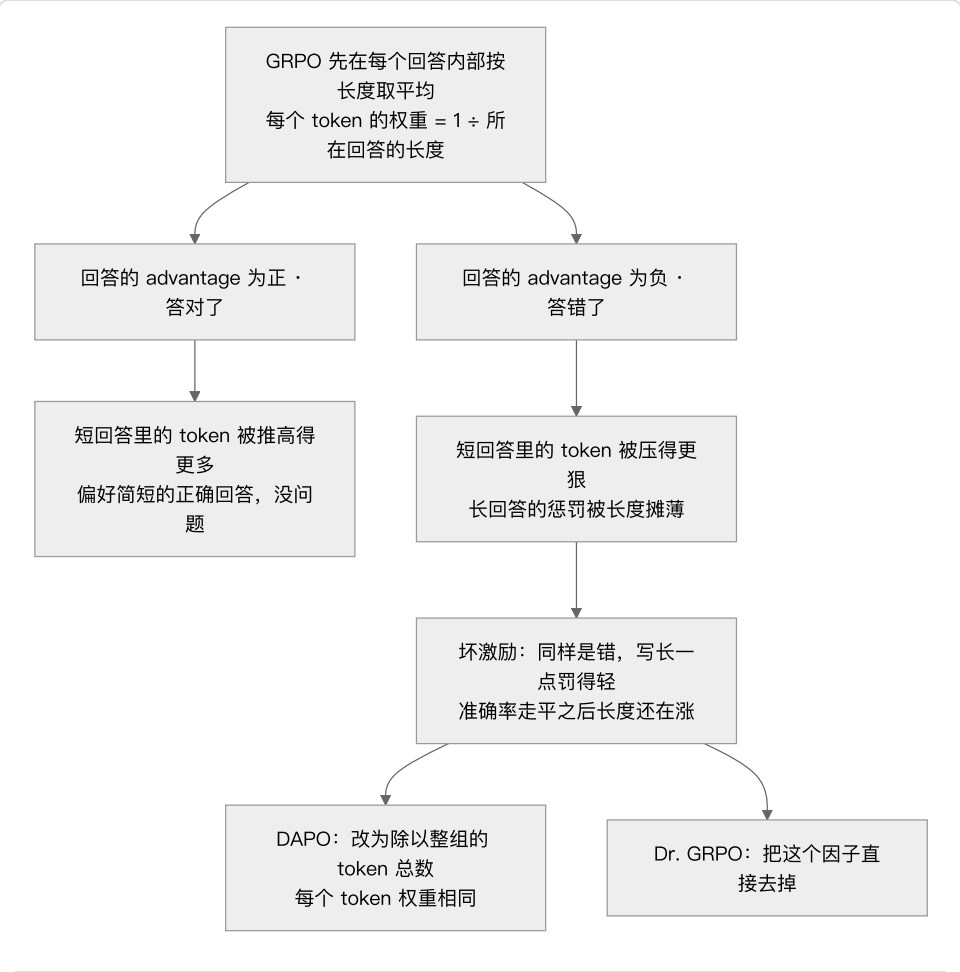


图 6-5 | 长度偏置是怎么从"按回答长度取平均"里长出来的（自绘示意）·

► 看原幻灯片 1:21:04 · 出处：Liu et al., 2025

- 为什么在意长度：推理 token 按量收费，对用户是钱，对服务方是算力，双方都希望同样的效果用更短的输出达到。
- 现象：RL 训练过程中，平均回答长度随步数一路上涨，主要涨在推理链上。起初这和准确率的上升同步，看着是好事；但到某个阶段准确率已经走平，长度还在涨——说明推着长度往上走的不只是"题目需要更长的思考"。
- 从目标函数里找原因：上一节的式子有两层求和，外层遍历组里的 G 个回答，内层遍历回答里的每个 token，内层前面有一个 $1/|o_i|$ 。把

这个因子挪到求和号里面看，它只取决于 token 所在的回答有多长：回答越短，每个 token 分到的权重越大。

- advantage 为正（答对）时，短回答里的 token 被推高得更多——偏好简短的正确答案，这没问题。
- advantage 为负（答错）时，短回答里的 token 被压得更狠，长回答的惩罚却被长度摊薄了。对模型来说，同样是错，写得长反而罚得轻。这个坏激励被认为是长度失控的原因。
- **两种修法：**DAPO（2025 年 3 月；不是第 5 讲的 DPO，字幕把它也识别成了 DPO）把归一化因子改成整组回答的 token 总数，对所有 token 一视同仁；Dr. GRPO（全称 GRPO Done Right）干脆把这个因子去掉。DAPO 的目标函数：

$$J_{\text{DAPO}}(\theta) = \mathbb{E} \left[\frac{1}{\sum_{i=1}^G |o_i|} \sum_{i=1}^G \sum_{t=1}^{|o_i|} \min(\rho_{i,t} \hat{A}_i, \text{clip}(\rho_{i,t}, 1 - \varepsilon_{\text{low}}, 1 + \varepsilon_{\text{high}}) \hat{A}_i) \right]$$

和上一节的式子比，变了两处：分母从每个回答各自的长度 $|o_i|$ 换成整组的 token 总数；裁剪的上下界拆成 ε_{low} 和 $\varepsilon_{\text{high}}$ 两个数（见下面的非对称裁剪）。

小注：DAPO 的目标函数里没有 KL 项——论文认为长推理链训练本来就该让模型远离初始分布；论文取 $\varepsilon_{\text{low}} = 0.2$ 、 $\varepsilon_{\text{high}} = 0.28$ 。Dr. GRPO 出自 Liu et al., 2025，实现上是把 $1/|o_i|$ 换成一个固定常数。

- **效果：**修正之后回答长度不再无休止地涨。拆开看更有说服力：答对的回答，长度和原版 GRPO 差不多；答错的回答，平均长度比原版短得多——改动正好作用在该作用的地方。
- **顺带提到的另外两处改动：**
 - advantage 里除以标准差，会带来和题目难度有关的偏置：一道很难的题，采出来的回答大多失败，组内奖励的标准差很小，拿它当分母会出问题。
 - clip 的上下界不该对称。裁剪限制的是概率比，所以一个 token 的新概率最多涨到 $(1 + \varepsilon) \times$ 旧概率；旧概率本来就真的很低的 token，这点空间几乎涨不动。但也不能把 ε 整体调大，否则高概率的 token 可能一步掉到接近零。于是让上下界不对称，只放宽上界。

小注：去掉标准差是 Dr. GRPO 的另一半改动——论文的说法是，标准差小的题（太难或太容易）除过之后 advantage 被放大，等于被过度加权，称为题目难度偏置，与长度偏置并列。非对称裁剪是 DAPO 的 Clip—

Higher。DAPO 还有两个补丁本讲没讲：dynamic sampling 和超长样本的处理。

- **和 CS329A 的关系：**CS329A 第 6 讲把 DAPO 的四个补丁连同消融实验过了一遍；这里补的是基础：token-level loss 到底在修什么，从公式上一步步推出来。

7. DeepSeek-R1 配方：R1-Zero 验证想法，R1 拼成流水线，再蒸馏给小模型

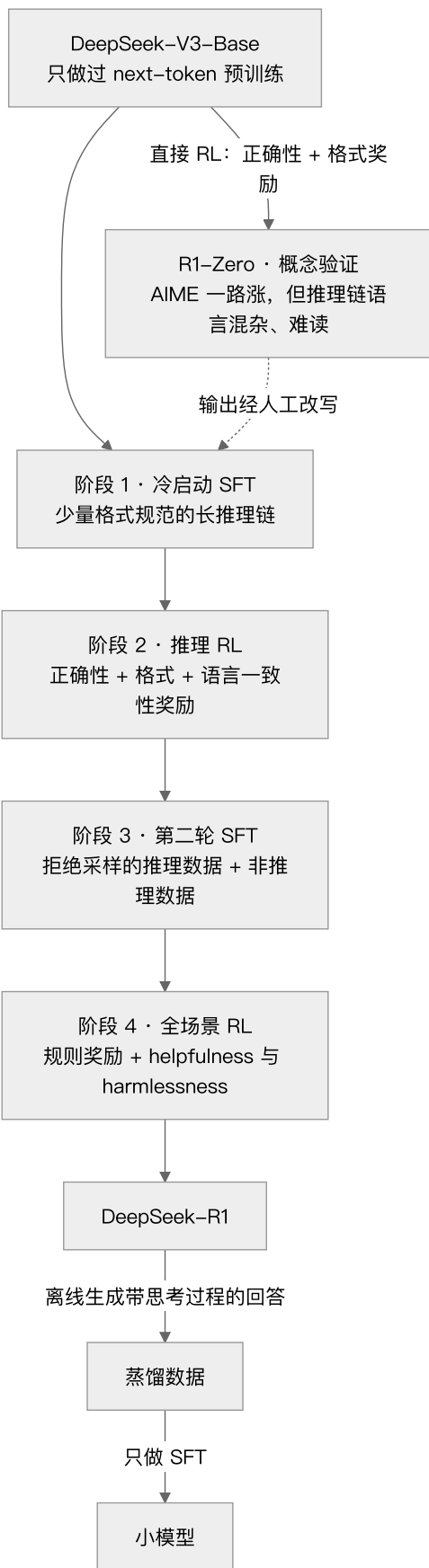


图 6-6 | 从 V3-Base 到 R1-Zero、R1, 再到蒸馏小模型的整条流水线 (自绘示意) ·

- **对照组：传统 LLM 的流水线**是预训练的 base 模型 → SFT（指令微调）→ 面向偏好的 RL。R1 论文分两步走：先用 R1-Zero 证明只靠可验证奖励的 RL 能走多远，再根据观察到的问题设计出完整的 R1。
- **R1-Zero**
 - 起点是 DeepSeek-V3-Base：只做过 next-token 预训练、没见过任何监督数据的 base 模型；架构上是 MoE（第 3 讲），注意力用的是 DeepSeek-V2 引入的 MLA。
 - 不做 SFT，直接上第 4 节的 RL，奖励只有答案正确性和格式两项。prompt 模板很简单：开头几句交代这是用户和助手的对话，然后用大白话说明格式要求（思考过程放进 think 标签，最终答案放进 answer 标签），中间留一个位置填题目，接着让模型往下写。
 - 结果是 AIME 准确率随训练一路上涨，一条 SFT 数据都没用。但作者去读推理链时发现两类问题：思考过程里多种语言混用；行文不通顺、难读。讲者的解释是，模型此前没受过任何强监督，RL 又只优化被奖励的东西，推理链"该长什么样"没有锚点。

• **R1 的四个阶段**（同样从 V3-Base 出发）

阶段	做什么	数据或奖励	针对什么
1 · 冷启动 SFT	先做一小轮 SFT	少量长推理链：取自 R1-Zero 的输出，由人改写成格式规范、语言一致的样子	给后面的 RL 一个"推理链该长什么样"的锚点
2 · 推理 RL	和 R1-Zero 同样的 RL	正确性 + 格式，再加语言一致性奖励：推理链里目标语言的词占多大比例	提升推理能力，同时压住语言混杂
3 · 第二轮 SFT	规模大得多的 SFT	非推理数据约 200k 条，沿用 V3 的 SFT 数据；推理数据是它的 3 倍，靠拒绝采样得到	模型不能只会做题，还得覆盖写作、问答这些日常用途
4 · 全场景 RL	最后一轮 RL	推理题照旧用规则奖励；通用题用 helpfulness 和 harmlessness 两种偏好奖励	对齐成一个有用且无害的助手

- **几个细节**
 - 冷启动数据有多少，讲者说论文没给确切数字，从措辞看比其他阶段少几个数量级，大概是几千条。
 - **拒绝采样**（rejection sampling）在这里的意思是：拿一批推理类的 prompt，让阶段 2 训出来的模型生成回答，再用规则和 LLM judge 筛选，只留好的，其余丢弃。数据量太大，筛选只能自动做。

- 阶段 4 里 harmless 看的是整段输出，包括 think 里的推理过程；helpfulness 只看用户最终读到的那部分回答。

小注：论文的原话是 thousands of cold-start data，来源除了整理过的 R1-Zero 输出，还有用长 CoT 做 few-shot、直接提示模型写出带反思和验证的详细解答，再由人工后处理。语言一致性奖励在论文的消息里会让成绩略降，作者为了可读性接受了这个代价。阶段 3 的数据是约 600k 推理 + 约 200k 非推理；这一轮 SFT 是从 V3-Base 重新训的，阶段 2 的 RL 模型只用来造数据。

- **结果：**在推理类基准上，模型明显分成两簇，非推理模型一簇，推理模型一簇，后者高出一大截；R1 和宣称具备推理能力的闭源模型不相上下——这也是它在 2025 年 1 月引起轰动的原因。
- **蒸馏：手里没有 600B 的模型怎么办**
 - 回顾经典蒸馏（第 2 讲末尾讲 DistilBERT 时）：训练文本是固定的，学生模型在每个位置上拟合的不是 one-hot 的"下一个 token"，而是老师模型给出的整个概率分布。
 - 推理能力的蒸馏是另一种做法：现成的推理 SFT 数据并不存在，所以先让老师（R1）离线生成一批带思考过程的完整回答，再拿这些序列对小模型做普通的 SFT——拟合的是老师写出来的 token 序列，不是概率分布。名字仍叫蒸馏。
 - 蒸馏出来的小模型，成绩和 o1-mini 这类闭源的小号推理模型相当。
 - 为什么不直接对小模型做同样的 RL？作者的结论是：在小尺寸上，蒸馏比从头做 RL 更有效。
 - 回头看第 2 节：厂商不展示原始推理链，防的正是这种蒸馏。

小注：V3 / R1 总参数 671B，每个 token 激活 37B。蒸馏用的就是阶段 3 那约 800k 条数据，学生是 Qwen2.5 和 Llama 3 系列的 1.5B 到 70B，只做 SFT、不做 RL。论文的对照实验：Qwen-32B 基座直接做上万步 RL（R1-Zero-Qwen-32B）只和 QwQ-32B-Preview 相当，而从 R1 蒸馏出来的 32B 在各项基准上都明显更高。

关键图表速查（点时间戳跳到原幻灯片）

图	看什么	跳转	出处
vanilla LLM 与推理模型对照	输入相同，输出从"答案"变成"推理链 + 答案"	21:24	讲者自绘
推理模型时间线	o1-preview (2024 年 9 月) → Gemini 2.0 Flash Thinking (12 月) → DeepSeek-R1 (2025 年 1 月) → 各家跟进	22:26	讲者汇总
pass@k 板书推导	从"1 - 全错的概率"出发，连乘式怎样整理成两个组合数之比	37:03	Codex 论文
温度与 pass@k 曲线	$T = 0$ 的线是平的； k 越大，最优温度越高； $T = 1.2$ 已经过头	45:54	推断出自 Codex 论文
R1-Zero 的 AIME 训练曲线	横轴是 RL 步数，准确率稳步上升；整个过程只有两个规则奖励	53:05	DeepSeek-R1
budget forcing 示例	推理链中途插入 Wait 之后，模型换了一条思路继续查	55:40	s1
GRPO 流程图	一个 query 采 G 个回答 → G 个奖励 → 组内归一化的 advantage；KL 项单独接到参考模型	1:06:07	DeepSeekMath
PPO 流程图	逐 token 的奖励（只有末位带回答的奖励，每位都带 KL）、价值模型、GAE	1:07:38	同上
冻结与训练的模型对照	GRPO 只训练策略模型；PPO 多训练一个价值模型；推理训练里没有奖励模型	1:10:14	同上
两个目标函数并排	找相同：概率比、clip；找不同：KL 的位置、advantage 的来源	1:11:44	PPO · DeepSeekMath
回答长度与准确率两条曲线	长度一直涨；准确率到后段走平，两条线从这里分道	1:17:57	长度曲线应出自 DeepSeek-R1
挪动长度因子的那一页	把" $1 \div$ 回答长度"移进内层求和之后，每个 token 的权重一目了然	1:21:04	讲者推导
修正前后的对比图	右下角那张：答错的回答明显变短，答对的长度基本不变	1:25:39	应出自 Dr. GRPO 论文
R1-Zero 的 prompt 模板	格式要求就是几句大白话；红色部分是填题目的位置	1:32:58	DeepSeek-R1

提到的工作

名称	在本讲里的作用
RLHF、PPO (Schulman et al., 2017)	第 5 讲的回顾；PPO-Clip 与 KL penalty 是 GRPO 的对照
GAE (Schulman et al., 2015)	PPO 用价值函数算 advantage 的方法，课上只提了名字
BLEU、ROUGE	传统 NLP 的规则指标，说明自由文本为什么难评测
Chain of thought (Wei et al., 2022)	推理模型的思想来源：先写过程再作答
Self-consistency (Wang et al., 2022)	cons@k 指标背后的思路
Best-of-N (第 5 讲)	pass@k 的近亲：用奖励模型而不是验证器来挑
OpenAI o1-preview、o1-mini	推理模型的起点；R1 和蒸馏小模型的比较对象
Gemini 2.0 Flash Thinking；xAI、Anthropic、Mistral 的推理模型	时间线上的后继者
HumanEval / Codex 论文 (Chen et al., 2021)	编程基准；pass@k 估计式与温度曲线的出处（后者为推断）
Codeforces、SWE-bench (Jimenez et al., 2023)	竞赛编程题、真实 GitHub issue 两类编程基准
AIME、GSM8K (Cobbe et al., 2021)	数学基准：奥赛资格赛、小学应用题
s1 (Muennighoff et al., 2025)	budget forcing：用 Wait 延长思考，或强制收尾
Coconut (Hao et al., 2024, 应为)	连续思考：在隐藏表示里推理
DeepSeekMath / GRPO (Shao et al., 2024)	本讲的核心算法
DAPO (Yu et al., 2025)	按整组 token 总数归一；非对称裁剪
Dr. GRPO (Liu et al., 2025)	去掉长度因子和标准差
DeepSeek-R1 / R1-Zero (2025)	后半场的主角：纯 RL 的概念验证、四阶段配方、蒸馏
DeepSeek-V3、DeepSeek-V2	R1 的 base 模型；MLA 的出处
经典蒸馏与 DistilBERT (第 2 讲)	对照：拟合老师的概率分布，而不是老师写出的序列

术语对照

English	中文
reasoning model	推理模型：先输出推理链再作答的 LLM（这里的推理是 reasoning，不是 inference）
reasoning chain / chain of thought (CoT)	推理链 / 思维链

English	中文
vanilla LLM	讲者的叫法：输入 prompt、直接出答案的普通 LLM
knowledge cutoff	知识截止日期
preamble	前置提示：放日期等背景信息的那段系统文字
compute / thinking budget	思考预算：愿意为一次回答花多少 token
reasoning tokens	推理 token，计入输出 token 收费
thought summary	思考摘要：界面上展示的、经过概括的推理过程
verifiable reward	可验证奖励：用规则就能判定对错的奖励
test case / ground truth	测试用例 / 标准答案
pass@k, pass@1	k 次尝试里至少一次正确的概率；单次正确率
cons@k (maj@k)	k 个回答取多数票之后的正确率
sampling without replacement	不放回抽样
temperature	温度：控制采样的随机程度
format reward / accuracy reward	格式奖励 / 正确性奖励
overthinking	过度思考：在简单问题上写很长的推理链
budget forcing	预算强制：插入特定词来延长或终止思考
continuous thought	连续思考：在隐藏表示而不是 token 上推理
policy / old policy / reference model	策略 / 上一轮迭代的旧策略 / 参考模型（通常是 SFT 模型）
advantage / baseline	优势 / 基线：回答比预期好多少，以及那个"预期"
value function (value model, critic)	价值函数：预测从当前位置写下去最终能拿多少奖励
probability ratio, clipping	新旧策略的概率比；把它裁剪在一个区间里
KL divergence	KL 散度：衡量两个概率分布差多远
group	组：同一个 prompt 采样出的 G 个回答
length bias	长度偏置
token-level loss	按 token 计的损失：每个 token 等权
Clip-Higher	只放宽上界的非对称裁剪
cold start	冷启动：RL 之前先用少量数据做一轮 SFT
language mixing / language consistency reward	语言混杂 / 语言一致性奖励
rejection sampling	拒绝采样：多生成、只留通过筛选的
helpfulness / harmlessness	有用性 / 无害性
distillation, teacher / student	蒸馏，老师模型 / 学生模型

字幕勘误

"RHF / RLF" → RLHF; "PO / PPU / PO clip" → PPO / PPO-Clip; "kale / scale divergence" → KL divergence; "blur" → BLEU; "Chenn thought / chain of dots" → chain of thought; "01 preview / 01 mini" → o1-preview / o1-mini; "DeepSync / DeepSc" → DeepSeek; "R10" → R1-Zero; "clouds" → Claude; "Chad GPT" → ChatGPT; "human evil" → HumanEval; "swbench" → SWE-bench; "AIM" → AIME; "pass at K" → pass@k; 讲 budget forcing 时的 "weights" → Wait; 1:25 处的 "DPO" → DAPO (不是第 5 讲的 DPO); "GRP1" → GRPO one; "too white" → too wide; "multi-latent attenu attention" → Multi-head Latent Attention (MLA); "a porm" → 推断为 RMSNorm; "coots" → CoTs; "in the O of case" → 推断为 on the order of thousands; "huristic" → heuristic; "Shervin" → Shervine, "Afin / Afen" → Afshine。

带走的问题

1. 估计 pass@k 时，为什么要采 $n \geq k$ 个再用组合数折算，而不是直接采 k 个看有没有对的？ $n = k$ 时估计式退化成什么？最优温度为什么随 k 变大而升高？
2. GRPO 的组内基线在什么情况下给不出学习信号（提示：整组全对或全错）？这和“除以标准差会过度加权太难或太容易的题”是什么关系？
3. 修掉长度偏置之后，答错的回答变短了，答对的长度没变。原先的长度增长里，有多少是题目真的需要更长的推理，有多少只是目标函数的副作用？怎么设计实验把两者分开？
4. R1-Zero 纯 RL 就能涨分，R1 为什么还要在前面加冷启动 SFT、后面再来一轮 SFT 和 RL？语言一致性奖励让推理链更好读、成绩却略降——对人可读和对模型最优发生冲突时该怎么取舍（回想“不从 SFT 起步”的第二个理由）？
5. 小模型上蒸馏胜过直接做 RL，这说明 RL 更像是在发掘 base 模型已有的能力，还是在教新能力？和 CS329A 第 6 讲“maj@K 涨、pass@K 不涨”的结论放在一起，你会怎么判断？

CME295 第 7 讲 | Agentic LLM (Agentic LLMs)

Stanford CME295: Transformers & Large Language Models (2025 秋) · 第 7 讲 (2025 年 11 月 14 日) 视频: <https://www.youtube.com/watch?v=h-7S6HNq0Vg> (1:49:22, 英文字幕是自动生成的, 专名错得比较多) 讲者: Afshine Amidi (回顾与 RAG, 到 59 分钟为止) · Shervine Amidi (tool calling、MCP、agent、安全) 课程大纲: <https://cme295.stanford.edu/syllabus/>

小注: 大纲页面现在默认显示 2026 秋的排课 (页面上可切换 2025 / 2026), 编号和这个视频对不上——2026 版里 AI Agents 是第 6 讲。本笔记按视频的编号。

一句话: 训练好的 LLM 还有两块短板: 知识停在截止日期, 也不会替人做事。这一讲分三层给它接上外部世界。RAG 把检索到的相关片段拼进 prompt, 功夫全在检索上: 切块建库, bi-encoder 加 BM25 粗召回, cross-encoder 重排, 用 NDCG / MRR / precision@k / recall@k 量好坏。Tool calling 让模型只管预测函数参数, 由运行时去执行, 再把结构化结果交回模型转述; 工具太多就先用 tool selector 筛, 接口各家不同就用 MCP 统一。Agent 是在工具调用外面再套一个 observe-plan-act 循环 (ReAct), 多个 agent 之间靠 A2A 通信。模型能动手之后, 数据外泄这类安全问题, 加上每一步都可能出错并且累积, 成了 agent 大规模落地的主要限制。

时间轴

时间	内容
0:06	开场: 本讲三件事 RAG / tool calling / agent; 回顾第 6 讲 (推理模型、GRPO、length bias、DAPO 与 Dr. GRPO)
5:18	vanilla LLM 剩下的两块短板: 知识不更新、不能行动
6:38	RAG 的动机: knowledge cutoff; 为什么不接着训练、为什么不全塞进上下文 (长度、needle in a haystack、按 token 计费)
14:33	RAG 的定义与三步: retrieve → augment → generate; 问答: 检索坏了怎么办
19:22	建知识库: chunk 与 embedding; 三个超参 (embedding 维度、chunk size、overlap)
23:36	两阶段检索: candidate retrieval 保 recall, ranking 求精; 问答: 按 token 硬切合理吗

时间	内容
27:39	语义相似检索：cosine similarity、ANN、bi-encoder、Sentence-BERT
34:25	BM25 关键词打分与 hybrid 检索（Cuddly 和 Huggy 的例子）
37:54	HyDE：先让 LLM 写一篇假想文档再去检索；contextual retrieval：给每个 chunk 补一段上下文
41:00	prompt caching：相同前缀只算一次；缓存命中的输入按十分之一计价
45:24	reranking：cross-encoder 把 query 和 chunk 一起送进 encoder
47:49	检索评测：NDCG、MRR、precision@k、recall@k；MTEB；问答（reranker 从哪来、contrastive loss）
59:28	换 Shervine 讲。Tool calling：结构化数据看成函数；定义；"找附近的泰迪熊"例子
1:05:22	函数定义长什么样：模型只看签名和文档，看不到实现
1:07:54	三步机制：预测参数 → 执行函数 → 把结果转述成回答
1:11:00	怎么训练：两类 SFT 样本，样本要多样
1:15:05	不训练的路线：few-shot 的局限；拿评测集和 reasoning model 迭代出一份工具使用说明
1:20:30	工具的三类用途；多工具并存的麻烦；小结与三个缺陷
1:26:22	tool selection：先凭名字和极短说明筛工具，再放完整 API；问答：这不就是 RAG 吗
1:29:17	MCP：server、tools、prompts、resources、client；荐诗集的例子
1:31:56	agent 的定义；ReAct 的 observe-plan-act 循环（"泰迪熊冷了"的例子）
1:39:06	多个 agent 与 A2A 协议；问答：每个 agent 都是一个 LLM 吗、token 预算
1:42:16	安全：数据外泄的例子、ToolSword、训练期与推理期两道防线、Agent-SafetyBench、Anthropic 刚披露的攻击
1:46:04	收尾：错误逐步累积；起步建议；AI 辅助编程与判断力

核心内容

1. 引子：回顾上一讲，和 vanilla LLM 剩下的两块短板

- 回顾第 6 讲：reasoning model 先写一段推理链（通常不给用户看）再作答，换来数学、代码任务上的提升。训练用的 GRPO 不训练 value function：对同一个 prompt 采一组回答，每个回答的 advantage 是它的 reward 相对组内其他回答的好坏；奖励只看有没有写出推理链、答案对不对。训练中 AIME 成绩上升，输出却越来越长，成绩趋平后还在涨，根源是 loss 里同一个 token 在短回答和长回答中贡献不同（length bias）。DAPO 把归一化改成与 token 所在序列无关，Dr. GRPO 干脆去掉这一项。

- **本讲补什么：**上一讲补的是推理能力。剩下的短板，一是知识不会更新，对应 RAG (Afshine 讲)；二是不能行动，对应 tool calling 和 agent (Shervine 讲)。

2. RAG 为什么存在：两条省事的路都走不通

- **问题：**模型只知道训练数据里有的东西。各家 model card 都写着 knowledge cutoff (知识截止日期)，课上举的 GPT-5 是 2024 年 9 月 30 日。问它这之后的事，裸模型答不了，或者答错。
- **路一：接着训练，把新知识训进去。**改动模型的知识而不让别的能力倒退很难，大家通常避免这么干。再者，如果你已经基于这个模型为好几个用例各做了微调，每注入一次知识就得全部重做一遍，维护不起。
- **路二：把截止日期之后的资料全塞进 prompt。**三个麻烦：
 - 上下文有限。常见量级是几十万 token (GPT-5 是 400,000)。按 1 token 约 4 个字符粗算，相当于几百页、一本很厚的书，装下一个不断增长的资料库。
 - 就算装得下，无关内容越多，模型越容易被带偏。needle in a haystack 测试量的就是这个：在一段很长的 prompt (haystack) 里埋一条事实 (needle)，再问模型这条事实是什么。课上放的是 GPT-4 的热力图，横轴是 prompt 长度，纵轴是事实埋在文中的深度；超过一定长度后模型开始找不回来，事实落在前半段时尤其明显。
 - 调用按 token 计费。GPT-5 的输入价在每百万 token 1 美元的量级，每条请求都塞满就积少成多。

小注：这张热力图应为 Greg Kamradt 2023 年 11 月对 GPT-4-128K 做的测试 (代码仓库)；课上只说“有人做了这个测试”。

- **RAG 的答案：**只把和问题相关的资料找出来放进 prompt。名字就是三步：retrieve (从知识库取回相关文档) → augment (把取回的内容拼到问题后面) → generate (把拼好的 prompt 交给 LLM)。效果上等于把答案递到模型眼前，让它照着读。
- **问答：**检索这一步做坏了怎么办？讲者的回应是，正因如此，谈 RAG 基本就是在谈怎么把检索做好，后面的内容全部围绕这一步。

小注：这里讲的是开头检索一次的标准 RAG。把检索做成一个工具，放进第 11 节的循环里，由模型决定何时查、查什么，就是 CS329A 第 7 讲拿

来和 Search-o1 对比的 agentic RAG。

3. 建知识库，和两阶段检索

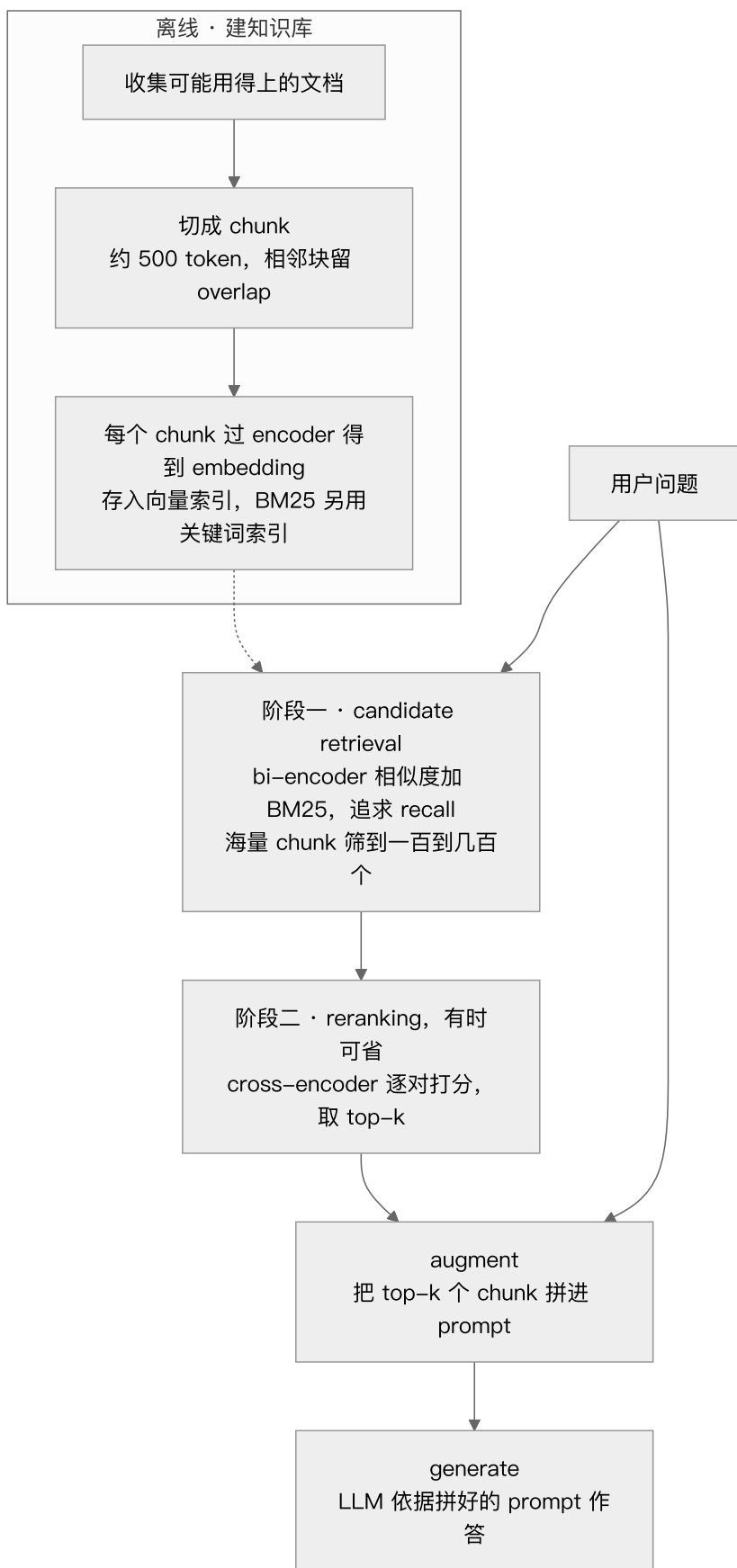


图 7-1 | RAG 全流程：离线建库，在线两阶段检索，再拼接与生成（自绘示意）·

- **建库**：先把可能用得上的文档收集起来，切成 chunk（文本块，每块有 token 数上限，量级是几百），再给每个 chunk 算一个 embedding（把一段文本压成一个定长向量，意思相近的文本，向量也相近）。讲者的提醒：听到 retrieval 就该想到 embedding。
- **三个超参**：
 - embedding 维度：文档越细腻复杂，越想要大一点；但维度大了占空间，检索时计算也更多。常见在一千多，比如 1,500 左右。
 - chunk size：太小，文字脱离上下文读不懂；太大，一个向量概括不了里面的内容。常见取 500 token 上下。
 - overlap：相邻 chunk 之间重叠一段，因为读懂当前块往往需要上一块末尾的内容。课上给的量级是低几百 token。
- **embedding 模型从哪来**（问答）：可以直接用预训练好的（多数人这么做），也可以自己训。它的目标就是让相关的文本在向量空间里挨得近，好把相关文档取回来。
- **两阶段检索**，做法借自搜索和推荐系统：
 - 阶段一 candidate retrieval（候选召回）：从海量 chunk 里粗筛出一小批可能相关的候选，一百到几百个。这一步追求 recall，宁可多拿也不能漏，所以只能用便宜的运算。
 - 阶段二 ranking，也叫 reranking（重排，有时可以省）：候选已经很少，付得起更贵的模型，把真正相关的排到最前面，最后取 top-k 拼进 prompt。
- **问答**：按 token 数硬切，会不会把内容切得支离破碎？会，第 5 节的 contextual retrieval 就是为此而来。讲者另外提醒：JSON、Markdown 这类自带结构的文件，切块时要顾及它的结构，课上没展开。

4. 阶段一：语义检索（bi-encoder）加关键词检索（BM25）

- **语义相似检索**：把 query 也编码成 embedding，和库里所有 chunk 的 embedding 比相似度，留下分数最高的一批。相似度通常用 cosine similarity。

$$\cos(q, c) = \frac{q \cdot c}{\|q\| \|c\|}$$

q 是 query 的向量，c 是 chunk 的向量，分子是两者的点积，分母是两个向量长度的乘积；结果越接近 1 越相似。

- **换一种距离行不行（问答）**：各种实现里也会见到 L2 距离等变体，讲者让大家自己想想它们之间的关系。向量都归一化到长度 1 时，展开平方可得 $\|q - c\|^2 = 2 - 2 \cdot \cos(q, c)$ ，按 L2 从小到大排和按 cosine 从大到小排是同一个顺序，所以这些变体本质上差不多。
- **ANN**：知识库可能极大，逐个比一遍（线性扫描）太慢。实际用 approximate nearest neighbor（近似最近邻）方法：建库时先把向量分区组织好，查询时只看其中一小部分。课上只点了名字。

小注：常见实现有 FAISS（IVF 分桶、PQ 压缩）、HNSW 图索引、ScaNN。

- **bi-encoder**：这种结构的名称。query 过一次 encoder，chunk 过一次 encoder，两边互不相干，最后只比两个向量。encoder 通常是 BERT 一类的 encoder-only 模型（讲者说是第 2、3 讲的内容）。讲者强烈推荐读 Sentence-BERT：它扩展了 BERT，让模型为整段文本输出一个适合做相似度检索的向量，训练目标是让相关的一对 cosine 高、不相关的一对 cosine 低。

小注：SBERT 论文比较的是 classification、regression、triplet 三种训练目标。学生问到的 contrastive loss（带 in-batch negatives 的对比损失）是今天训练 embedding 模型更常见的做法。

- **语义相似不保证关键词命中**：embedding 检索找的是意思相近，取回的文档可以和 query 没有一个词相同。有时你恰恰要求结果里必须出现 query 里的词。课上的例子：两只泰迪熊，一只叫 Cuddly，一只叫 Huggy，问 Cuddly 在哪。这两个名字语义上很近，纯 embedding 检索可能取回讲 Huggy 的文档；BM25 取回的文档按定义一定和 query 有词重叠。
- **BM25**：一个启发式的相关性分数，由 query 和文档之间词的重叠情况算出来，不需要训练。

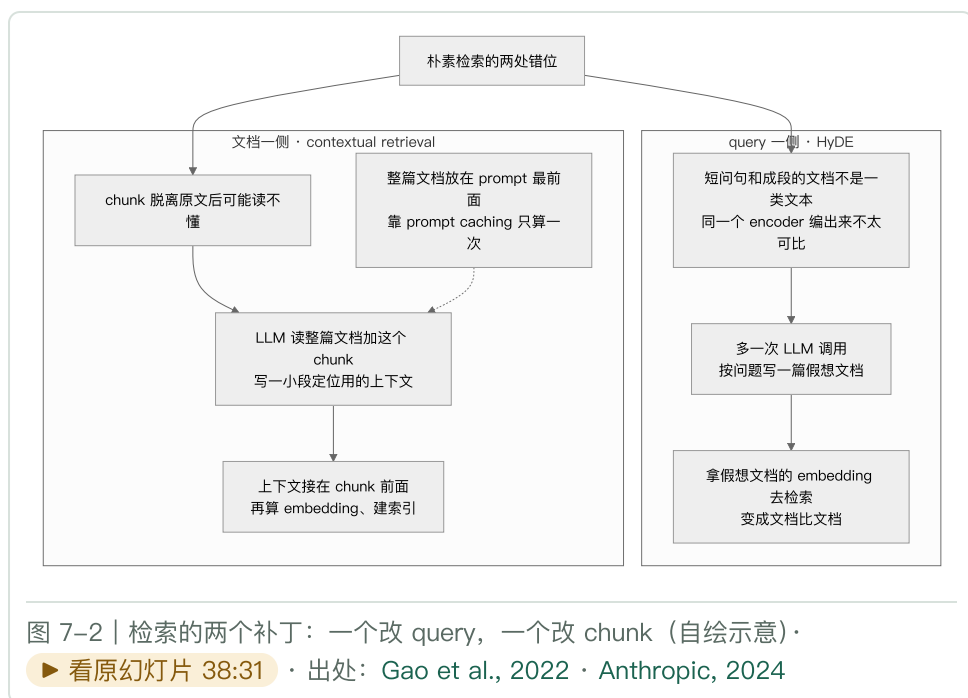
$$\text{BM25}(q, d) = \sum_{t \in q} \text{IDF}(t) \cdot \frac{f(t, d) (k_1 + 1)}{f(t, d) + k_1 \left(1 - b + b \frac{|d|}{\text{avgdl}}\right)}$$

小注：课上没有给公式，上式是 BM25 的标准形式。f(t,d) 是词 t 在文档 d 里出现的次数，|d| 是文档长度，avgdl 是平均文档长度；k1（常取 1.2–2.0）让同一个词重复出现的收益递减，b（常取 0.75）惩罚长文档，IDF 让罕见词权重更高。

- **hybrid**：所以现在的做法是看用例，决定要不要在相关性分数里掺一份启发式分数。不少人用 embedding 检索和 BM25 的某种组合。

小注：课上没说怎么合并。常见的是两路分数加权，或用 reciprocal rank fusion (RRF) 按名次融合。

5. 两个补丁：HyDE 与 contextual retrieval (附 prompt caching)



- **HyDE，修 query 一侧的错位**：用户的 query 通常很短，是个问句；知识库里的文档是成段的陈述。用同一个 encoder 去编码这两种性质不同的文本，得到的向量并不那么可比。HyDE 的办法是多一次 LLM 调用：先让模型根据 query 写一篇假想的文档，再拿这篇假文档的 embedding 去检索，于是变成文档和文档比。讲者的评价很克制：不一定有效，也不是人人都用，值得试一下。

小注：HyDE 原文允许假想文档包含事实错误，作者的论点是 encoder 的稠密瓶颈会滤掉细节，只留下“相关文档大概长什么样”。

- **另一条路**：干脆给 query 和文档各训一个 encoder。理论上可行，但要维护两个模型，实际很少这么做。
- **contextual retrieval，修文档一侧的错位**：chunk 脱离原文后可能读不懂。办法是在每个 chunk 前面补一小段文字，概括读懂这一块需要知道的背景。这段文字由 LLM 写，输入是整篇文档加上这个 chunk，提示它给一段简短的说明。之后用“上下文加 chunk”去算 embedding、建索引。

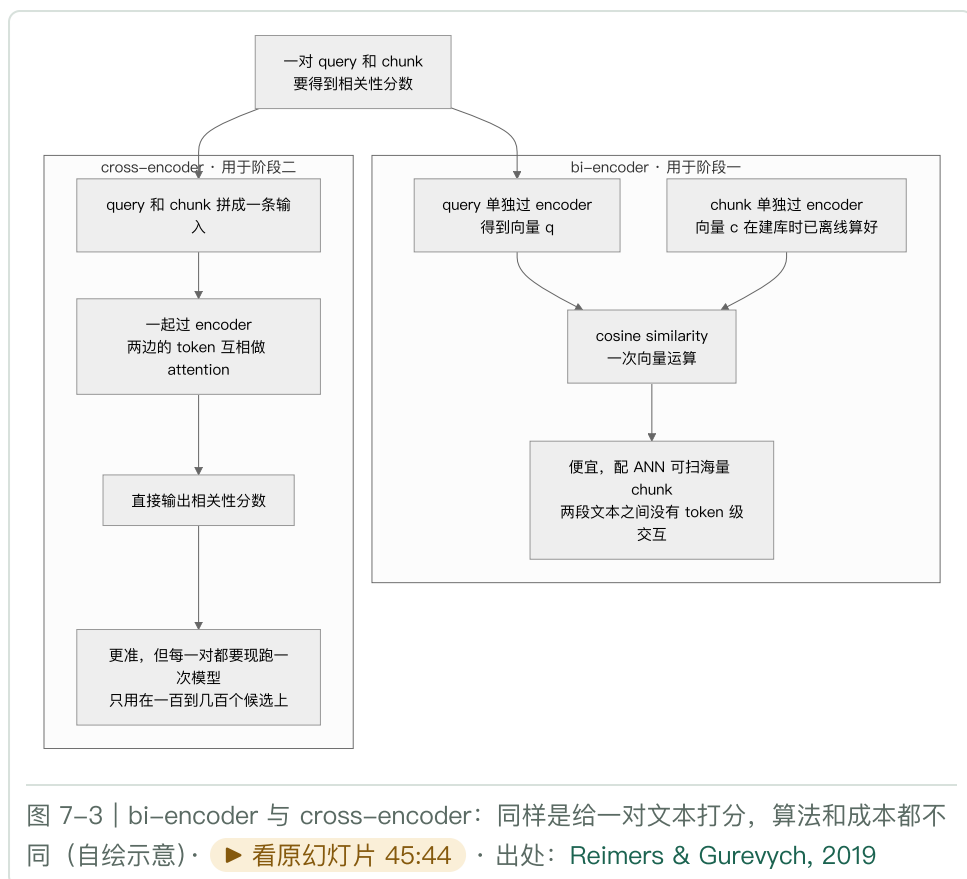
小注：这是 Anthropic 2024 年 9 月公开的做法，补的上下文约 50—100 token，embedding 和 BM25 索引两边都用。他们报告的检索失败率（1

— recall@20): 5.7% → 3.7% (只用带上下文的 embedding) → 2.9% (再加带上下文的 BM25) → 1.9% (再加 reranking)。

- **代价与 prompt caching**: 每个 chunk 一次 LLM 调用, 每次还都带着整篇文档, 听上去很贵, 这里可以用 prompt caching。原理来自 decoder-only 模型的因果性: 每个位置只看它左边的内容, 所以两条 prompt 只要前缀相同, 前缀部分算出的激活值就完全相同, 算一次存起来, 之后直接查。闭源模型的提供商把它做成了计费选项: 定价页上单列一栏缓存命中的输入 token, 课上展示的 OpenAI 模型是正常输入价的十分之一。
 - 用在这里: 把整篇文档放在 prompt 最前面, 变化的 chunk 和指令放在最后, 同一篇文档的所有 chunk 共用一份缓存。
 - 通用的建议: 把多条 prompt 之间会重复的内容尽量挪到开头。

小注: 被存下来的就是前缀各层的 key 和 value (KV cache)。命中要求从第一个 token 起逐 token 完全一致, 所以可变内容必须放在后面; 各家另有最短长度和缓存存活时间的限制。

6. 阶段二: 用 cross-encoder 重排



- **做法**：不再分别编码，而是把 query 和 chunk 拼在一起送进同一个 encoder，直接输出一个相关性分数。模型同时看到两段文本，query 的 token 和 chunk 的 token 之间有 attention（学生追问，讲者确认），能捕捉到阶段一里两个独立向量表达不了的交互。这种结构叫 cross-encoder，和阶段一的 bi-encoder 相对。
- **为什么只放在第二阶段**：分数依赖具体的 query，没法像 chunk 向量那样离线算好，每一对都要现跑一次模型。对几十万、上百万个 chunk 不现实，对一百到几百个候选付得起。
- **模型从哪来（问答）**：有现成的预训练 reranker，也可以自己训一个。讲者推荐看 [SBERT 的文档站](#)，bi-encoder 和 cross-encoder 两种用法都有。
- 之所以叫 reranking，是因为阶段一按相似度其实已经排过一次序。

7. 检索质量怎么量：NDCG、MRR、precision@k、recall@k

- **设定**：和二分类一样需要真值标签（ground truth），也就是对每个 query 事先标好哪些 chunk 真的相关。系统跑完两个阶段，把 chunk 按预测分数从高到低排，取前 k 个；RAG 里正是这 k 个会被拼进 prompt。指标回答两个问题：相关的有没有进前 k；进来之后是不是排在靠前的位置。
- **NDCG@k**：顾及位置的指标。

$$DCG@k = \sum_{i=1}^k \frac{rel_i}{\log_2(i+1)} \quad NDCG@k = \frac{DCG@k}{IDCG@k}$$

rel_i 是系统排在第 i 位的那个 chunk 的真值标签（相关为 1，不相关为 0）；分母 log₂(i+1) 是随名次增大的折扣；IDCG 是把所有真正相关的 chunk 排在最前面时能得到的 DCG。

- **把名字拆开来**：cumulative 是把前 k 位的相关性累加起来；discounted 是位置越靠后折扣越大，相关文档排在第 1 位比排在第 k 位得分高；normalized 是因为 DCG 的取值范围取决于这个 query 有多少相关文档，除以理想排序的 IDCG 之后，排得完美恰好得 1，不同 query 之间才可比。

小注：折扣函数课上只说是“名次的某个函数”，这里写的是最常见的 log₂(i+1)。

- **MRR**：更简单的指标。只看第一个相关文档排在第几位，取倒数，再对一批 query 取平均。

$$\text{MRR} = \frac{1}{|Q|} \sum_{q \in Q} \frac{1}{\text{rank}_q}$$

Q 是评测用的 query 集合，rank_q 是 query q 的结果里第一个相关 chunk 的名次（排在第 2 位就贡献 1/2）。它完全不管第一个相关文档之后还有没有别的相关文档，但通常和其他指标走势一致，所以常用。

- **precision@k 与 recall@k**：分类指标的排序版。

$$\text{Precision@}k = \frac{|\text{Rel} \cap \text{Top}_k|}{k} \quad \text{Recall@}k = \frac{|\text{Rel} \cap \text{Top}_k|}{|\text{Rel}|}$$

Rel 是真正相关的 chunk 的集合，Top_k 是系统排出的前 k 个。precision@k 问的是选进来的有多少是对的，recall@k 问的是该选的有多少被选进来了。

小注：自拟算例，不是课上的数字。库里共有 3 个相关 chunk，系统的 top-5 按名次的真值是 1, 0, 1, 0, 0。DCG@5 = 1 + 0.5 = 1.5，IDCG@5 = 1 + 0.631 + 0.5 = 2.131，NDCG@5 ≈ 0.70；RR = 1；precision@5 = 0.4；recall@5 ≈ 0.67。把前两位对调成 0, 1, 1, 0, 0：NDCG@5 降到约 0.53，RR 降到 0.5，precision 和 recall 不变——后两者只看进没进前 k，不看排第几。对应到两阶段（推断）：阶段一主要看 recall@100 这类指标，因为漏掉的后面补不回来；最终的 top-k 看 NDCG 和 MRR。

- **MTEB**：Massive Text Embedding Benchmark，常用的公开基准。想知道自己的 retriever 行不行，就拿到上面跑出这些指标，和别的方案比。这四个指标在检索论文里反复出现，讲者建议把思路和公式都记熟。

8. Tool calling：一次调用的三步，每一步上下文里有什么

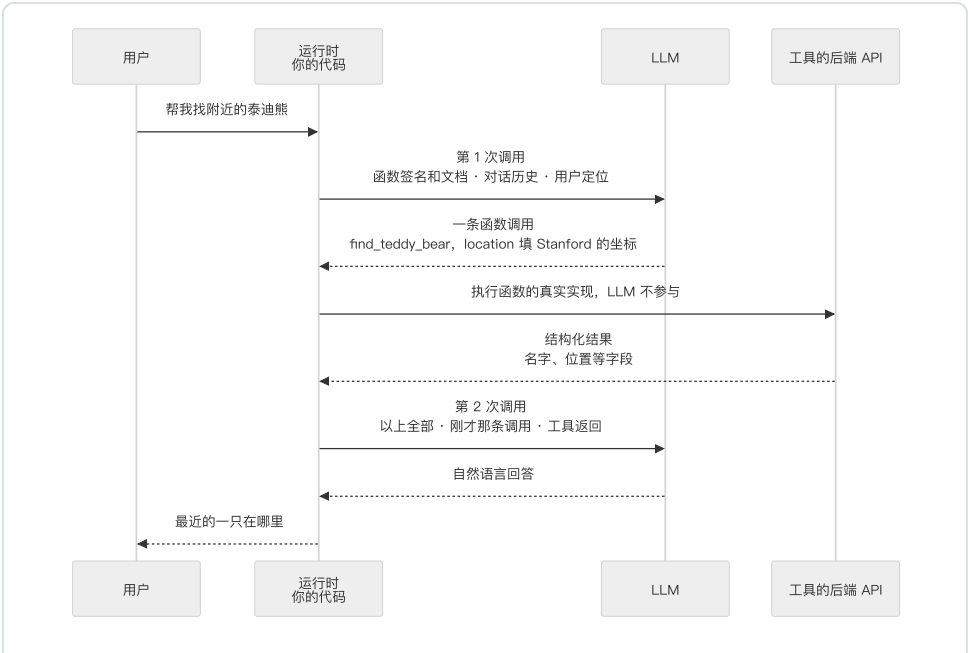


图 7-4 | 一次工具调用里谁在什么时候看到什么（自绘示意）。

▶ 看原幻灯片 1:08:26

- **和 RAG 的分工：**RAG 处理的是非结构化的文字资料。如果要接入的数据是结构化的，像一张表，给定某几列的值就得到一个输出，就可以把它看成一个函数：输入是参数，输出是返回值。让模型使用这种函数叫 function calling，也叫 tool calling。课上用 Python 写例子只是因为好读，换别的语言也行。
- **定义：**讲者取了 IBM 一篇文章的说法，要点两个。一是为了完成某个任务；二是可以动态地访问外部资源，并对其施加动作（是"可以"，不是"必须"）。第二点正好补上开头说的知识缺口。
- **工具是事先写好的（问答）：**函数由开发者事先定义和实现，不是模型临场生成的。
- **函数长什么样：**例子是 find_teddy_bear，按位置去调某个后端 API，返回附近的泰迪熊。它有三个部件。签名：函数名、参数、返回类型。文档字符串 docstring：说明这个函数干什么、参数是什么意思，模型全靠它理解工具，所以至关重要。实现：里面是真正的后端调用，返回一个带结构的对象（课上用一个 class 规定了名字、位置等字段）。**模型只看得到签名和文档，看不到实现。**
- **三步（图 7-4）：**

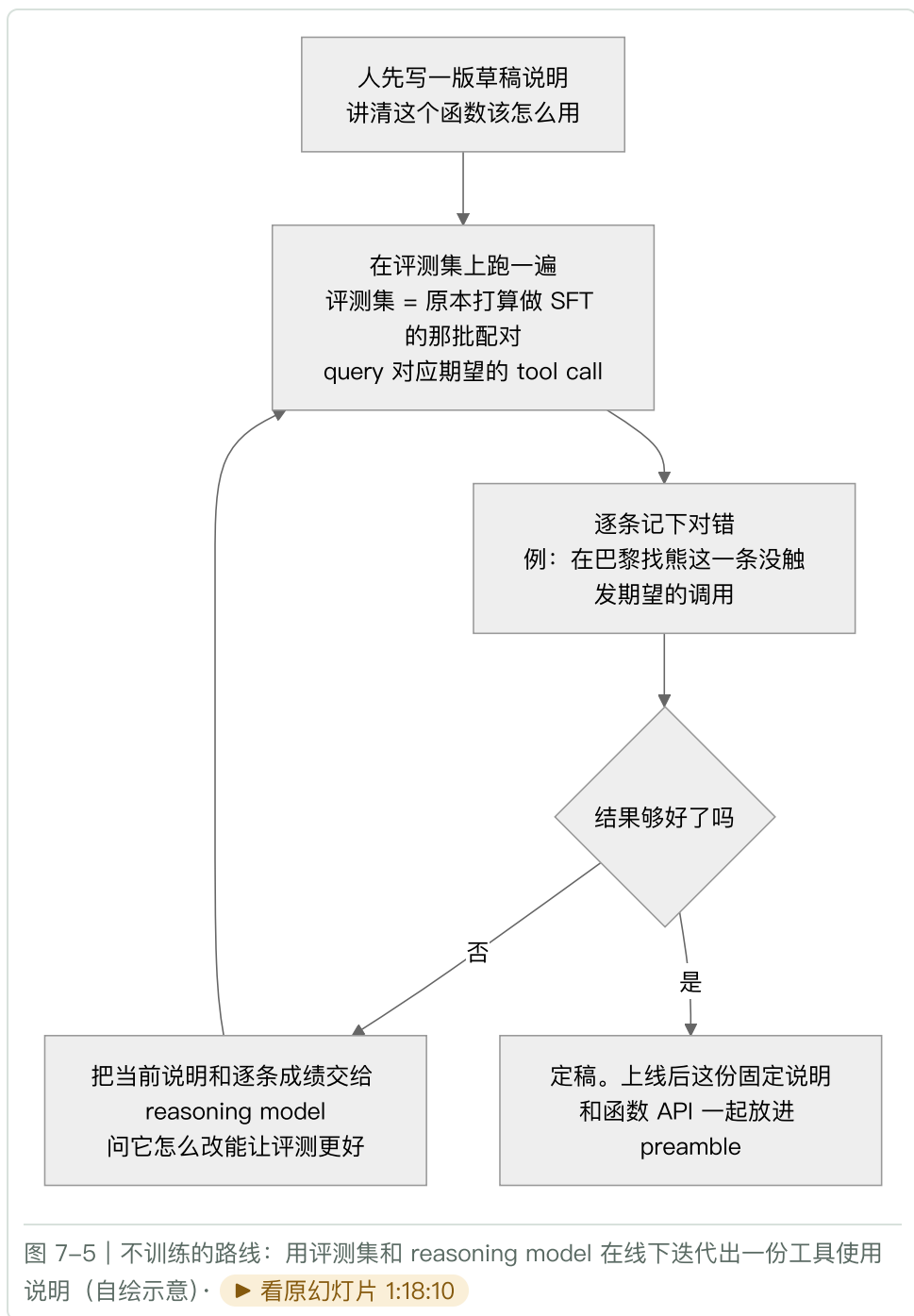
步骤	谁来做	上下文里有什么	产出
1 · 预测参数	LLM	preamble 里的函数 API（签名加文档，不含实现）；对话历史和用户这句话；已知的环境信息，如定位权限给出的坐标	一条函数调用：函数名加填好的参数，比如把

步骤	谁来做	上下文里有什么	产出
			Stanford 的坐标填进 location
2 · 执行	你的运行时代码，和 LLM 无关	不涉及上下文：解析出函数名和参数，真正去跑	结构化的返回值（名字、位置等字段）
3 · 转述	LLM	第 1 步的全部内容，加上模型刚发出的那条调用，加上工具返回	给用户的自然语言回答

- 第 1 步里模型的任务不是推断函数怎么实现，只是决定往里填什么参数。第 3 步还要再过一次模型，是因为用户要的是一句人话而不是 JSON，而且你往往希望回答遵循特定的格式。

小注：对应到今天的 chat API：工具定义（名字、描述、JSON Schema 形式的参数）经 tools 字段进入上下文；第 1 步的产出是一条结构化的 tool call 消息；第 2 步的返回作为 tool 角色的消息追加进对话，再发起第 2 次模型调用。

9. 怎么让模型学会用一个工具：做 SFT，或者只写一份说明



- **路线一：SFT。**三步里有两步是模型在做，所以要两类训练样本。输入都是到此刻为止的全部对话历史，输出是模型在这一刻该说的话。
 - 第一类（tool prediction）：输入是函数 API 加用户 query，输出是带参数的函数调用。
 - 第二类：输入是原始 query、已经发出的调用、工具返回这一整串历史，输出是最终回答。需要它，是因为你通常希望回答按特定方式组织；模型也得知道最初问的是什么、这份结果对应哪一次调用。

- 样本要多样，贴近真实用户的分布：位置没明说、要从用户定位里取的；位置直接写在话里的；多轮对话聊到一半才提出要找熊的。学生问工具不止一个怎么办：SFT 数据里放多工具的样本就行，选工具的问题见第 10 节。
- **路线二：不训练，只靠提示。**如今的模型在预训练和指令微调阶段见过大量代码，读 Python 函数、写函数调用本来就熟练，专门为此做 SFT 未必必要。
 - few-shot（在上下文里放几组输入输出的示例）可行，也是公认的做法。但示例只覆盖具体的几个点，未必能泛化到用户千变万化的说法。
 - 讲者更推荐在 preamble 里放一份详细的文字说明，讲清这个工具该怎么用。这种说明很难写好，所以不要自己从头写到尾：起个草稿，其余交给 reasoning model 按图 7-5 迭代——拿原本打算做 SFT 的那批配对当评测集，逐条看对错，让它据此改写。讲者说实际效果好得出人意料。
 - 迭代都发生在线下（问答里澄清过）。上线后，这份固定的说明和函数 API 一起放进 preamble，对任何 query 都一样。

小注：课上只讲了 SFT 和纯提示两条路。用 RL 学工具使用，见 CS329A 第 4 讲的 RLEF（执行结果当奖励）和第 5 讲的 SWiRL（judge 逐步打分）。图 7-5 的循环，是自动提示优化（automatic prompt optimization）的一个最小版本。

10. 工具多了之后：tool selection 与 MCP

- **工具的三类用途：**取信息（搜索、天气、股价；“今天有什么新闻”靠的是搜索工具，不是模型自己的知识）；做计算（把问题转成代码，执行后读出结果，比让模型在推理链里硬算可靠）；替用户办事（例如发邮件：模型填好抬头和正文，连发送也替你按）。
- **讲者在这里停下来小结的三个缺陷：**
 - 真实产品不会只有一个工具：找熊、抱熊、看熊的心情、给熊送礼。不知道用户要哪个，只好全放进 preamble。工具一多又回到 needle in a haystack：某个工具的 API 淹没在上下文里，还可能功能相近、互相打架的 API。什么都想支持，结果什么都做不好。
 - 上下文终究有限。几亿用户、五花八门的需求，不可能把所有工具同时放进去。

- 工具是手写的，每家 LLM 定义和使用工具的方式可能都不一样，同一个工具要为每个模型重写一遍。
- **tool selection (对付前两条)**：讲者引了一篇 Google DeepMind 的技术论文（没给题目），里面管它叫 tool selector，文献里也叫 router。分两步。第一步给 LLM 看用户 query 和一份工具清单，清单可以很长，但每个工具只有名字和一两个词的说明，让它挑出可能用得上的。第二步只把被选中工具的完整 API 和 query 一起放进上下文，走第 8 节的流程。
- **问答**：这不就是 RAG 吗？可以用 RAG 来做（把工具描述当文档来检索），但不是非得这样，让一个 LLM 按指令来挑也行。
- **MCP (对付第三条)**：Model Context Protocol, Anthropic 提出的协议，统一工具暴露给模型的方式。工具的提供方按协议实现一次，支持 MCP 的 LLM 应用都能用，不必再为每个模型各写一份。课上只过了一遍词汇：

MCP 里的角色	含义	课上"给爱读诗的泰迪熊荐书"的例子
host	跑着 LLM 的应用	Claude
MCP client	host 里的一个基础设施组件，和一个 server 一对一连接	课上没展开
MCP server	对外提供工具的服务实例	由图书供应商来实现，他们最懂怎么提供这类内容
tools	供模型调用的函数实现	找书、荐书
prompts	一类模板，示范这些工具怎么用	按书名查找、按用户口味推荐
resources	完成任务时可以依托的外部数据	泰迪熊自己的藏书、畅销榜

小注：MCP 于 2024 年 11 月发布，消息格式是 JSON-RPC 2.0。client 用 tools/list 拿到每个工具的 name、description 和 inputSchema (JSON Schema)，用 tools/call 执行——进入模型上下文的仍然只是签名加文档，和第 8 节一致。规范里三种原语的控制方不同：tools 由模型决定调用，resources 由应用决定取用，prompts 由用户选用。

11. Agent：在工具调用外面套一个循环 (ReAct)

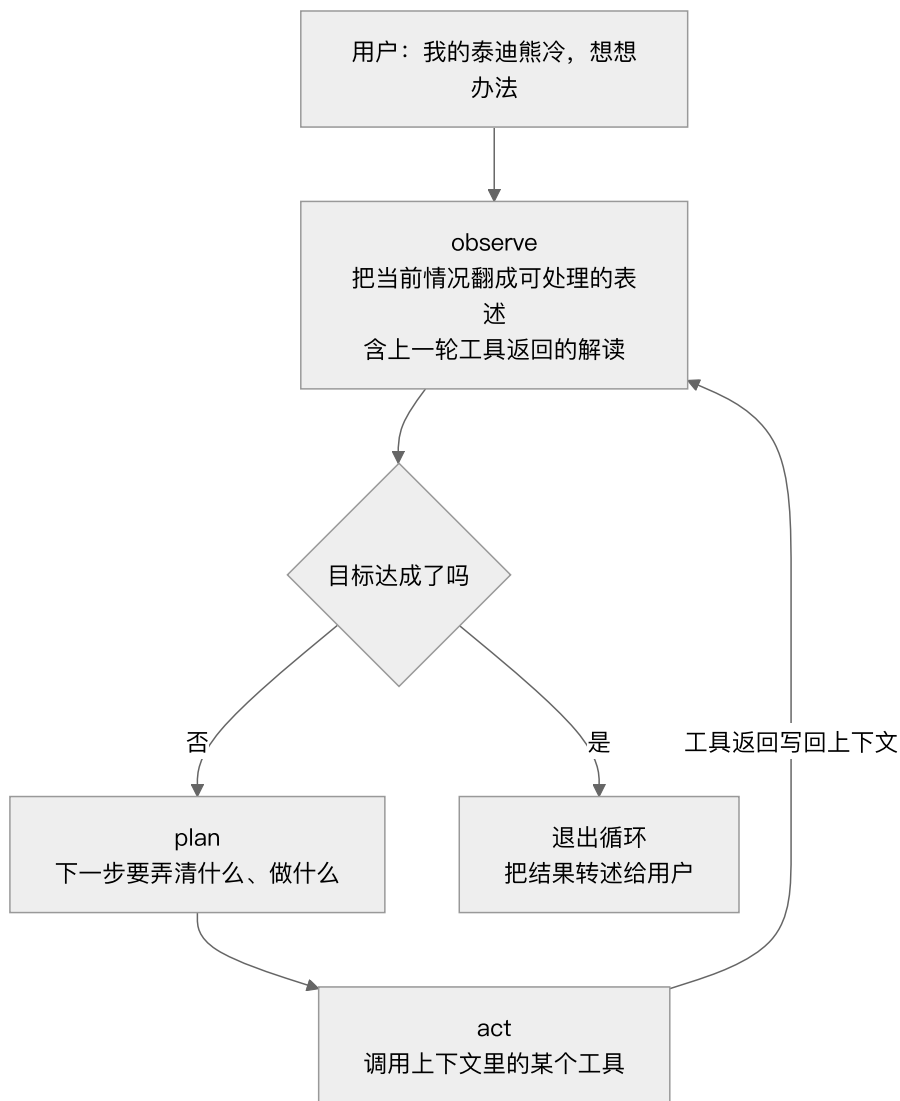


图 7-6 | 课上的 observe—plan—act 循环，退出与否由模型自己在 observe 时判断（自绘示意）· [▶ 看原幻灯片 1:35:26](#) · 出处：Yao et al., 2022

- **定义：**agent 是一个自主地追求目标、代表用户完成任务的系统。和单次工具调用相比多了两样东西：推理，以及多轮迭代。通俗地说，人们讲 agent 时指的就是这层循环。它和上一讲的推理链并不互斥，循环内部完全可以有 reasoning chain；骨架是工具调用加迭代。
- **ReAct (reason + act)：**讲者称之为里程碑式的论文。很多请求没法一步完成，得把目标拆成可以执行的小步，逐步做完再给答案。ReAct 把这个过程组织成若干轮循环，每轮由几个原子阶段组成。课上分成 observe、plan、act，论文里的叫法是 thought、action、observation；各处的名称和顺序会有出入，直觉是同一个。
- **走一遍例子**（用户说：我的泰迪熊冷，想想办法）：

- 第 1 轮。observe: 把这句话翻成可处理的表述——熊冷, 可能和室温有关, 而室温现在未知。plan: 先弄清室温。act: 上下文里恰好有读取室温的工具, 调用它。
 - 第 2 轮。observe: 工具返回 65°F, 解读为比预期的冷。plan: 需要把温度调高。act: 调用设定温度的工具, 参数是升高 5 度。
 - 第 3 轮。observe: 温度已经设到位, 目标达成。退出循环, 把"已调高 5 度"转述给用户。
- **什么让一个 workflow 成为 agentic:** 有一个初始请求, 有一组可执行的动作, 每一轮由 LLM 自己判断目标是否达成, 达成就输出, 否则再来一轮。工具返回的内容必须是模型读得懂的, 因为下一轮的 observe 全靠它。

小注: CS329A 第 4 讲细讲过 ReAct 原文: 纯 prompting (few-shot 轨迹), 在 HotpotQA、FEVER、WebShop 上验证; 如今打开 thinking 的模型会自动"想一步、调一次工具"。CS329A 第 5 讲的 LATS, 是在这条单轨迹循环上再加树搜索和反思。

12. 多个 agent 与 A2A

- **场景:** 家里除了调温的 agent, 还可以有管能源分配的、管空气质量的。用户只对其中一个说话, 它们之间需要互相沟通。host 和工具之间需要 MCP, agent 和 agent 之间同样需要一套标准, 于是 Google 在 2025 年早些时候发布了 Agent2Agent (A2A) 协议。
- **协议让开发者填什么:** 一是 agent 对外声明自己的 skills (会做什么事), 并附上示例, 让别的 agent 知道能找它干什么。二是这个 agent 怎样执行一个请求: 执行过程中向其他 agent 发出什么状态; 收到"停下"时怎样取消正在做的事 (cancel 方法)。

小注: 在 A2A 规范里, 这份自我介绍叫 Agent Card (一个 JSON 文档), 每个 skill 有 id、name、description、tags、examples; 一次请求对应一个 task, 状态包括 submitted、working、input-required、completed、failed、canceled 等。

- **问答:** 每个 agent 都是一个带着自己上下文的 LLM 吗? 可以这么理解, 课上的例子就是这样。它们各自独立运行自己的推理循环, 彼此只看到对方的输入和输出。有学生担心 token 开销失控: 可以设预算上限; 各 agent 不会挤占彼此的上下文, 花的是你的钱。

13. 安全, 与落地的建议

- **新能力带来新风险**：模型能替你执行动作，恶意的一方就可能让它替自己执行。课上的例子是数据外泄（data exfiltration）：agent 手里有一个能向外写数据的工具（比如发邮件），又接触得到用户的敏感信息，只要有一条 prompt 让它把密码发到某个地址，数据就出去了。更多的风险类型，讲者推荐读 ToolSword。

小注：这类攻击现在通称 prompt injection；恶意指令藏在工具返回、网页或邮件正文里时，叫 indirect prompt injection。ToolSword 把工具学习分成输入、执行、输出三个阶段，各给两个安全场景（恶意请求与越狱、噪声误导与风险提示、有害反馈与错误冲突）。

- **两类补救**：
 - 训练期：和 R1 等模型的训练流程一样，在 SFT 和 RL 的数据配比里放入覆盖安全（harmlessness）的数据。
 - 推理期：穿过了训练期防线的请求还可以再拦一道，例如一个安全分类器，读到目前为止的对话，判断模型即将给出的输出是否安全。
- **基准**：Agent-SafetyBench 汇总了 agent 可能出现的各类安全隐患，并给了一整套评测。

小注：该基准有 349 个交互环境、2,000 个测试用例、8 类风险、10 种失败模式；论文评测的 16 个 agent，安全得分都不到 60%。

- **一条当时的新闻**：上课前一天，Anthropic 发了一份很详细的报告，披露有攻击者借助 Claude 的工具和 agent 能力发动了大规模网络攻击，逐步还原了攻击者的做法和可能的补救。讲者想传达的是：攻防双方都会越来越强，这不是一场注定要输的仗，但防御措施必须到位。

小注：报告题为 [Disrupting the first reported AI-orchestrated cyber espionage campaign](#)（2025 年 11 月 13 日）。按报告，Anthropic 不是被攻击的一方，而是滥用行为的发现方：一个被它评估为有国家背景的组织利用 Claude Code 攻击了约 30 个机构，少数得手，80—90% 的操作由 AI 完成；绕过防护的办法是把任务拆成看似无害的小步，并谎称在做防御性安全测试。

- **为什么还看不到大规模自主运行的 agent**：循环的每一步都可能跑偏，比如没有正确理解工具返回、参数填错，而且错误会沿着后续步骤累积。能力上的缺口可以用 SFT 补，但讲者认为推理上的缺口最好

靠模型自身的能力，而不是靠 SFT 打补丁。agent 怎么评测留到下一讲（第 8 讲）。

小注：CS329A 第 8 讲的 METR 结果是同一件事的量化版：要求 80% 成功率时，模型能独立完成任务长度只有 15 分钟量级。

- 动手建议：
 - start small：先挑最简单的用例（比如找最近的熊），确认现有的实现和 prompt 跑得通，再往外扩。
 - start smart：先用能力最强的模型，摸清现有模型能做到的上限，之后再优化延迟和成本。先求对，再求快。
 - 调试：模型会输出推理链，出问题时先去读它。
- 讲者最喜欢的用法是 AI 辅助编程：把繁琐的管线类工作交出去，腾出脑子。前提是自己懂代码的基本功。生成代码变得廉价之后，判断一段代码对不对、是不是在做该做的事，才是难的部分，你的判断力（taste）从此最值钱。

关键图表速查（点时间戳跳到原幻灯片）

图	看什么	跳转	出处
needle in a haystack 热力图	横轴 prompt 长度，纵轴事实埋入的深度；看长 prompt 一侧、前半段深处成片的失败	13:03	应为 Kamradt 的测试
两阶段检索	海量 chunk → 一百到几百个候选 → top-k；哪一段追求 recall，哪一段用贵的模型	24:06	—
bi-encoder 示意	query 和 chunk 各过各的 encoder，最后只比两个向量	30:46	Sentence-BERT
HyDE	多出来的那次 LLM 调用在哪：query → 假想文档 → embedding	38:31	HyDE
contextual retrieval	整篇文档加 chunk 进 LLM，产出的 一小段上下文接在 chunk 前面	40:35	Anthropic, 2024
cross-encoder	两段文本拼成一条输入，出来的直接是分数；和 bi-encoder 那张对照着看	45:44	SBERT 文档
NDCG 公式	分子是真值相关性，分母随名次增大；再看 IDCG 怎样把分数归一到 1	49:52	—
find_teddy_bear 的函数定义	三块：签名、docstring、带后端调用的实现；模型看得到的只有前两块	1:05:52	—

图	看什么	跳转	出处
工具调用三步	每一步的输入和输出；第 2 步完全没有 LLM	1:08:26	—
两类 SFT 样本	输入总是到此为止的对话历史，输出分别是 tool call 和最终回答	1:12:30	—
tool selector 两步	第一步的清单里每个工具只有名字和极短说明，第二步才出现完整 API	1:26:41	Google DeepMind 的一篇技术论文（课上没给题目）
MCP 词汇与荐书例子	server、tools、prompts、resources、client 各对应例子里的什么	1:29:49	MCP
ReAct 走查	observe → plan → act 转了几轮，哪一次 observe 触发退出	1:35:26	ReAct
数据外泄例子	三个条件凑齐：能向外写的工具、读得到的敏感数据、一条恶意指令	1:42:54	ToolSword

提到的工作

名称	在本讲里的作用
GRPO、DAPO、Dr. GRPO	开场回顾第 6 讲：组内相对 advantage、length bias 与两种修法
DeepSeek-R1	回顾里的训练曲线应出自它；讲安全时又提到它训练流程里的 harmlessness 数据
GPT-5 的 model card 与定价页	knowledge cutoff、400,000 token 上下文、按 token 计费、缓存输入十分之一价
needle in a haystack 测试	说明长上下文里无关内容会拖累模型
RAG (Lewis et al., 2020)	本讲前半的主题；课上只讲了方法，没提这篇论文，列在这里备查
BERT 与 Sentence-BERT (Reimers & Gurevych, 2019)	bi-encoder 用的 encoder；讲者强烈推荐读 SBERT，多种损失函数的比较也在里面
approximate nearest neighbor (ANN)	阶段一避免线性扫描的手段，只点了名
BM25	关键词启发式打分，与 embedding 检索组成 hybrid
HyDE (Gao et al., 2022)	用假想文档替 query 去检索
Contextual Retrieval (Anthropic, 2024)	给每个 chunk 补一段由 LLM 写的上下文
prompt caching	让 contextual retrieval 付得起；也是排布 prompt 的通用技巧
MTEB (Muennighoff et al., 2022)	检验 retriever 与 embedding 模型的公开基准

名称	在本讲里的作用
IBM 的 What is tool calling	tool calling 定义的出处
Google DeepMind 的一篇技术论文	tool selector 的出处；课上没给题目，未能确认是哪一篇
MCP (Anthropic)	统一工具暴露给模型的方式
ReAct (Yao et al., 2022)	agent 循环的代表作
A2A (Google, 2025)	agent 之间通信的协议
ToolSword (Ye et al., 2024)	工具使用各阶段的安全问题
Agent-SafetyBench (Zhang et al., 2024)	agent 安全的评测基准
Anthropic 2025 年 11 月 13 日的披露报告	工具与 agent 能力被用于真实攻击的案例

术语对照

English	中文
knowledge cutoff	知识截止日期
context window / context length	上下文窗口 / 上下文长度
needle in a haystack	大海捞针测试：在长 prompt 里埋一条事实，再问模型
retrieval-augmented generation (RAG)	检索增强生成
knowledge base	知识库
chunk / chunk size / overlap	文本块 / 块大小 / 相邻块的重叠
embedding	嵌入向量：把一段文本压成一个定长向量
candidate retrieval	候选召回，即粗筛
ranking / reranking	排序 / 重排
semantic similarity	语义相似：意思相近，不要求有相同的词
cosine similarity	余弦相似度
approximate nearest neighbor (ANN)	近似最近邻搜索
bi-encoder	双编码器：query 和文档各自独立编码，再比向量
cross-encoder	交叉编码器：query 和文档拼在一起编码，直接出分数
BM25	基于词重叠的启发式相关性打分
hybrid search	混合检索：embedding 加关键词

English	中文
HyDE (hypothetical document embeddings)	假想文档嵌入
contextual retrieval	带上下文的检索：给每个 chunk 补一段背景
prompt caching	提示缓存：相同前缀的计算结果复用
ground truth	真值标签
DCG / IDCG / NDCG	折损累计增益 / 理想排序下的 DCG / 归一化后的 DCG
MRR (mean reciprocal rank)	平均倒数排名
precision@k / recall@k	前 k 个里的精确率 / 召回率
tool calling / function calling	工具调用 / 函数调用
preamble	放在对话最前面的固定内容，如 system prompt 和工具定义
docstring	函数的文档字符串
few-shot	少样本示例：在上下文里放几组输入输出
tool selector / router	工具选择器 / 路由
Model Context Protocol (MCP)	模型上下文协议
host / MCP client / MCP server	跑 LLM 的应用 / 其中的连接组件 / 提供工具的服务
agent	智能体
observe / plan / act	观察 / 规划 / 行动
Agent2Agent (A2A)	agent 之间的通信协议
skill (A2A 语境)	agent 对外声明的一项能力
data exfiltration	数据外泄
harmlessness	无害性
safety classifier	安全分类器
headroom	上限空间：现有模型最多能做到什么程度

字幕勘误

"rag" / "rack" / "drag" → RAG；"gRPO" → GRPO；"AIM data set" → AIME；"DAPO" → DAPO；"GRPO done right" 指的是 Dr. GRPO；"haststack" → haystack；"GP5" → GPT-5；"cosign" → cosine；"by encoder" → bi-encoder；"birds" / "sentence bird" / "espt" / "expert paper" → BERT / Sentence-BERT / SBERT paper；"BF20" → BM25；"huristic" / "heristic" → heuristic；"the height" → HyDE；"short 16 context" → 应为 short, succinct context

(推断); "one10enth" → one tenth; "Ashin" / "Afin" / "Shervin" → Afshine / Shervine; "deep minds" → DeepMind; "with only training" (1:16 处) → 应为 with only prompting (推断, 上下文在说省掉 SFT); "fshot" → few-shot; "preputed" → pre-computed; "maybeformational" → maybe informational; "cloud" → Claude (MCP 例子和攻击披露两处都是); "react" → ReAct; "teddy bear is called" → is cold; "65 fah" → 65°F; "is not set at the correct temperature" → 应为 is now set (推断, 随后就退出了循环); "agentto agent" → Agent2Agent; "a tool sword" → ToolSword; "nearest pair" → nearest bear; 1:00:41 处一长串重复的 "rest" 是识别故障, 可忽略。

带走的问题

1. 两阶段检索里, 阶段一只管 recall, 阶段二才管排序。最终答案错了, 你怎么分辨是没召回、排错了, 还是模型没用好拼进去的 chunk? 三种情况各看哪个指标?
2. NDCG、MRR 都需要"哪些 chunk 真的相关"的真值标签。自己的业务语料没有标注时怎么办: 人工标、让 LLM 标, 还是用下游回答的质量反推? LLM 标注会带来什么偏差 (第 8 讲会讲 LLM-as-a-judge) ?
3. tool selector 本身就是一次检索, 也会漏选。漏掉一个必需的工具, 和多放几个无关的工具, 哪种错更贵? 另外它和 prompt caching 有冲突: 每个 query 选出的工具集不同, 前缀就变了, 怎么排布上下文才能两头兼顾?
4. 数据外泄的例子里, 恶意指令从哪来并不重要, 只要进了上下文, 模型就可能照办。训练期的 harmless 数据和推理期的安全分类器各自拦得住哪一类、拦不住哪一类? 对不可逆的动作 (发邮件、付款), 还需要哪些模型之外的机制? 可对照 CS329A 第 5 讲 LATS 对不可逆动作的讨论。
5. 循环的每一步都可能出错, 并且会累积。假设单步成功率是 95%, 一个 20 步的任务整体成功率大约是多少? 这对"先用最强的模型摸清上限"这条建议意味着什么?

CME295 第 8 讲 | LLM 评测 (LLM Evaluation)

Stanford CME295: Transformers & Large Language Models (2025 秋) · 第 8 讲 (2025 年 11 月 21 日) 视频: <https://www.youtube.com/watch?v=8fNP4N46RRo> (1:49:25, 英文字幕为自动生成, 专名错得很多) 讲者: Afshine Amidi (前 60 分钟: 人工评分、规则指标、LLM-as-a-judge、事实性) · Shervine Amidi (后 50 分钟: agent 评测、benchmark、数据污染) 课程大纲: <https://cme295.stanford.edu/syllabus/>

小注: 大纲页现在显示的是 2026 秋的新课表——评测挪到了第 7 讲, 第 8 讲换成了 Diffusion LLMs。本笔记沿用 2025 秋录像的编号。

一句话: 自由文本没有万能指标, 这一讲按"人参与得越来越少"走了三级台阶: 请人逐条打分 (贵、慢、主观, 要用 Cohen's kappa 这类扣掉"碰巧一致"的指标盯住评分者之间的一致性) → 人只写一次参考答案, 用 METEOR / BLEU / ROUGE 做 n-gram 比对 (换个说法就失灵, 和人的判断相关性一般) → 让另一个 LLM 当裁判 (不要参考答案、分数自带理由, 但有位置、冗长、自我偏好三种偏差, 必须拿人工评分来校准)。后半讲把 agent 的一次工具调用拆成"预测—执行—综合"三步, 列出七种常见失败, 再按知识、推理、代码、安全、agent 五类各讲一个代表 benchmark (MMLU、AIME 与 PIQA、SWE-bench、HarmBench、 τ -bench 及其 pass^k), 最后提醒数据污染和 Goodhart 定律。

时间轴

时间	内容
0:06	开场: 量不出来就没法改进; 回顾第 7 讲 (RAG、tool calling、agentic workflow)
4:45	"评测"的多种含义; 本讲只谈输出质量; 自由文本没有通用指标
6:21	理想方案: 每条输出都请人打分
7:08	人工评分的主观性 (生日礼物例子); inter-rater agreement
10:01	板书: 两个人闭着眼睛打分, 也有 50% 的时候一致
14:43	Cohen's kappa; Fleiss' kappa、Krippendorff's alpha; 对齐会
17:50	人工评分的第二个局限: 慢、贵
18:24	规则指标的设定: 人只写一次参考答案

时间	内容
21:00	METEOR: $F\text{-score} \times (1 - \text{penalty})$
25:07	BLEU 与 brevity penalty; ROUGE
26:42	规则指标的三个局限
28:00	LLM-as-a-judge: 输入、输出、prompt 模板
31:25	技巧: 先写 rationale 再给 score
33:44	structured output: 用约束解码保证输出能解析
36:48	两种形态: pointwise 与 pairwise; 顺带合成偏好数据
38:47	三种偏差: position、verbosity (40:21 起)、self-enhancement (42:27 起)
45:03	问答: 还有别的偏差吗; judge 要不要更大
47:22	最佳实践: 标准写明确、二元量表、人工校准、低温度
52:48	评测维度: 任务表现 vs 形式与对齐
54:06	事实性: 拆成事实、逐条核对、加权汇总
1:00:15	换 Shervine: agent 评测; 一次工具调用的三步
1:02:42	工具预测阶段的四种失败
1:14:12	工具执行阶段的两种失败
1:18:19	综合作答阶段的失败
1:21:52	七种失败的共性; 要系统地给错误分类
1:23:50	benchmark 分类总览
1:25:12	知识: MMLU
1:29:34	推理: AIME 与 PIQA
1:32:35	代码: 为什么人人都该看这一项; SWE-bench (1:33:57 起)
1:36:15	安全: 为什么发布对比表里少见; HarmBench
1:40:51	agent: τ -bench; pass ^k (1:43:22 起)
1:45:12	对回现实: Gemini 发布的对比表; 模型画像与 Pareto frontier
1:47:48	数据污染; Goodhart 定律; 结语

核心内容

1. 先说清楚"评测"指什么

- 讲者开场就说这可能是整门课最重要的一讲: 量不出模型的好坏, 就不知道该往哪里改。第 6、7 讲都在补 LLM 的短板 (推理、取外部知识、执行动作), 这一讲回答"补完之后到底好了多少"。

- "评测一个 LLM"可以指很多事：输出质量（连贯性、事实性.....），也可以是系统指标（延迟、价格、可用率）。本讲只谈**输出质量**——给定一条回答，怎么量化它有多好。
- 难点：LLM 是 text-to-text 模型，输出可以是自然语言、代码、数学推理，形式完全自由，不存在一个通用指标。前半讲按"人参与得越来越少"的顺序走了三条路线：

路线	人要做什么	好处	课上点出的问题
人逐条打分	模型每改一版，都重新评每条输出	最接近想要的"真值"	贵、慢；任务主观时评分者之间不一致
参考答案 + 规则指标	给固定的一批 prompt 写一次参考答案	模型怎么迭代都能自动重算	容不下换说法；与人工评分相关性一般；起步仍要人写参考
LLM-as-a-judge	写评分标准；抽样做人工校准	不要参考答案；分数带解释	位置、冗长、自我偏好等偏差；它只是人工评分的近似

2. 人工评分，以及怎么量评分者之间一不一致

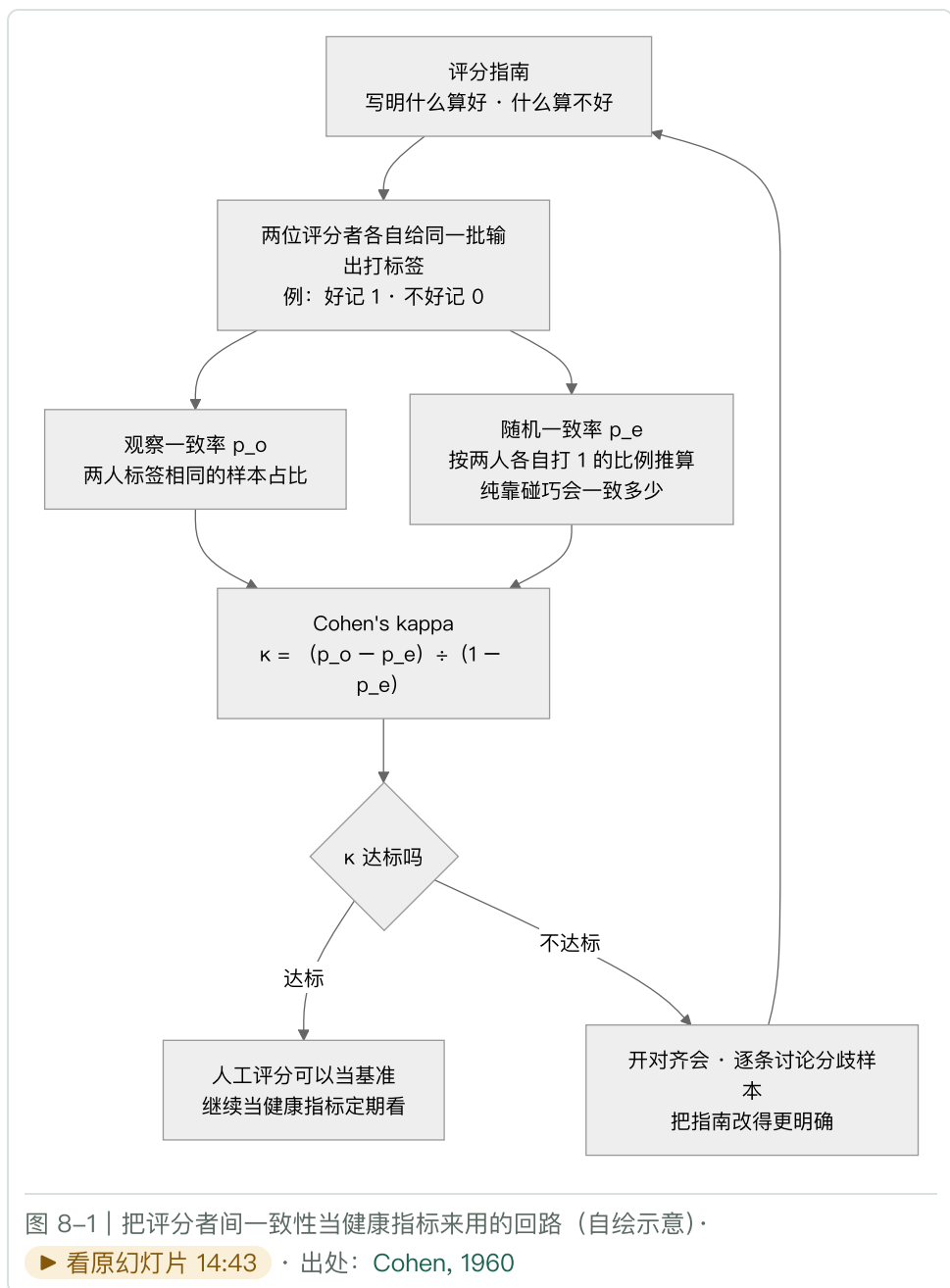


图 8-1 | 把评分者间一致性当健康指标来用的回路（自绘示意）·

► 看原幻灯片 14:43 · 出处：Cohen, 1960

- **理想方案和它的第一问题：**最理想是每条输出都请人打分，再汇总。但评分任务本身可能是主观的。课上的例子：用户问该送什么生日礼物，模型答"泰迪熊几乎总是贴心的礼物，挑一个你觉得合适的就行"。按"有用性"评：一位评分者觉得有用（方向明确），另一位觉得没用（到底选哪种毛绒玩具并没有说）。于是需要 inter-rater agreement（评分者间一致性）这类指标，确认大家用的是同一把尺子。
- **为什么不能只看 agreement rate：**最直观的指标是"两人给出相同标签的比例"。问题是单看这个数，判断不了它算高还是算低。板书推导：假设 Alice 和 Bob 根本不看内容，各自以概率 p_A 、 p_B 随机打"好"（记 1），否则打"不好"（记 0），互相独立；两人一致 = 同时打

1 或同时打 0。代入 $p_A = p_B = 0.5$ ，碰巧一致的概率已经是 50%。这个"碰巧一致率"还随两人的打分比例变化：标签越偏向一边，它越高。

- **Cohen's kappa**：把观察到的一致率放到"碰巧一致率"这条基线上重新标定。

$$p_e = p_A p_B + (1 - p_A)(1 - p_B), \quad \kappa = \frac{p_o - p_e}{1 - p_e}$$

p_A 、 p_B 是两位评分者各自打"好"的比例， p_e 是两人按这个比例独立随机打分时碰巧一致的概率， p_o 是实际观察到的一致率； κ 的分子是"比碰巧多出来的一致"，分母是"最多还能多出来多少"。

- **读法与变体**：完全一致时 $\kappa = 1$ ；和瞎打一样时 $\kappa = 0$ ；比瞎打还差时 κ 为负。Cohen's kappa 针对两位评分者；Fleiss' kappa、Krippendorff's alpha 是它的推广，思路相同——都以"随机打分会怎样"为基线。

小注：自己代个数：两人都有 90% 的概率打"好"时， $p_e = 0.9 \times 0.9 + 0.1 \times 0.1 = 0.82$ ；此时观察到 85% 的一致率， κ 只有 $(0.85 - 0.82) \div (1 - 0.82) \approx 0.17$ 。agent benchmark 的通过率常常很偏，只报一致率会很好看，所以要报 κ 。Fleiss' kappa 面向多位评分者的类别标签；Krippendorff's alpha 还容许缺失标注和有序、连续量表。常见的经验读法 (Landis & Koch, 1977)：0.41–0.60 中等，0.61–0.80 较强，0.81 以上近乎完全一致。

- **实践中的用法**：把一致性当成健康指标长期跟踪；数值不理想，就让评分者开对齐会，对一对各自是怎么评的，把评分指南写得更明确（图 8-1 的回路）。
- **第二个问题**：慢、贵——请人评 1,000 条输出要很久。结论是"每条都请人评"不现实；但人工评分没有被丢掉，后面它是校准 LLM judge 的基准。

小注：CS329A 第 8 讲里，METR 的人类基线样本少，讲者给的通行做法同样是看 inter-rater agreement、分歧大就补样本；GDPval 的专家盲评彼此一致率 71%。主观任务上，人与人的一致率就是任何自动评审的天花板。

3. 规则指标：METEOR、BLEU、ROUGE

- **设定**：不再让人评每条输出，而是请人为固定的一批 prompt 各写一次参考答案 (reference)，之后模型每迭代一版，都拿输出和参考自

动比对。同一个意思可以有多种说法，所以这类指标都想留一点弹性。

- **METEOR**（机器翻译用；全称 Metric for Evaluation of Translation with Explicit ORdering）：一个 F-score 乘上一个词序惩罚。

$$\text{METEOR} = F_{\text{mean}}(1 - \text{Pen}), \quad F_{\text{mean}} = \frac{PR}{\alpha P + (1 - \alpha)R}, \quad \text{Pen} = \gamma \left(\frac{c}{u_m} \right)^\beta$$

P 是预测里的 unigram（单个词）有多少比例能在参考里匹配上，R 是参考里的 unigram 有多少比例被预测匹配上，F_mean 是两者的加权调和平均（常见的 F1 是等权特例，这里权重 α 可调）；c 是匹配上的词连成的连续片段（chunk）数，u_m 是匹配上的 unigram 总数， γ 、 β 是人为选定的超参数。

- METEOR 惩罚项的直觉：词序和参考完全一致时，匹配的词连成一段，c 最小、惩罚最小；词都对、顺序却被打散，c 变大、惩罚变大——名字里的 explicit ordering 指的就是这个。
- "匹配"不限于字面相同：同义词、同词根的词也算。
- 讲者对 METEOR 的两点批评： α 、 β 、 γ 一串超参数，更像一份配方；匹配条件再放宽，仍然容不下风格上的改写。

小注：原论文（Banerjee & Lavie, 2005）的取值相当于 $\alpha = 0.9$ （recall 的权重是 precision 的 9 倍）、 $\gamma = 0.5$ 、 $\beta = 3$ ；匹配分三轮：先字面，再 Porter 词干，再 WordNet 同义词。课上用的是后来带可调参数的写法。

- **BLEU**（机器翻译用；Bilingual Evaluation Understudy）：偏 precision——预测里的 n-gram 有多少出现在参考里。只看 precision 的话，译得很短就能刷分，所以再乘一个 brevity penalty 惩罚过短的输出。课上只讲到这里，没有展开公式；下面是原论文的定义。

$$\text{BLEU} = \text{BP} \cdot \exp \left(\sum_{n=1}^N w_n \log p_n \right), \quad \text{BP} = \min \left(1, e^{1-r/c} \right)$$

p_n 是 n 阶 n-gram 的 precision，w_n 是各阶的权重，BP 是 brevity penalty，c 是生成文本的长度、r 是参考的长度：比参考短就按指数打折，不短则不罚。

小注：这个式子课上没有出现，取自 Papineni et al., 2002。通常 N = 4、各阶等权；p_n 用"截断计数"——同一个 n-gram 最多按它在参考里出现的次数计，防止靠重复一个词刷分。

- **ROUGE**（摘要用）：同样的思路，有多个变体。

小注：ROUGE 全称 Recall-Oriented Understudy for Gisting Evaluation，和 BLEU 相反，偏 recall：ROUGE-N 看参考里的 n-gram 有多少被生成文本覆盖，ROUGE-L 用最长公共子序列。

- **三个共同局限**

1. 容不下风格变化。课上把"毛绒泰迪熊能在睡前安抚孩子"这个意思换了三种说法，意思相同、用词几乎不重叠，这些指标会给后两种很低的分。
2. 与人工评分的相关性并不高——那些超参数本来就是为了凑相关性调出来的。
3. 起步仍然要人写参考答案，有的项目负担不起。

4. LLM-as-a-judge：让另一个 LLM 来打分

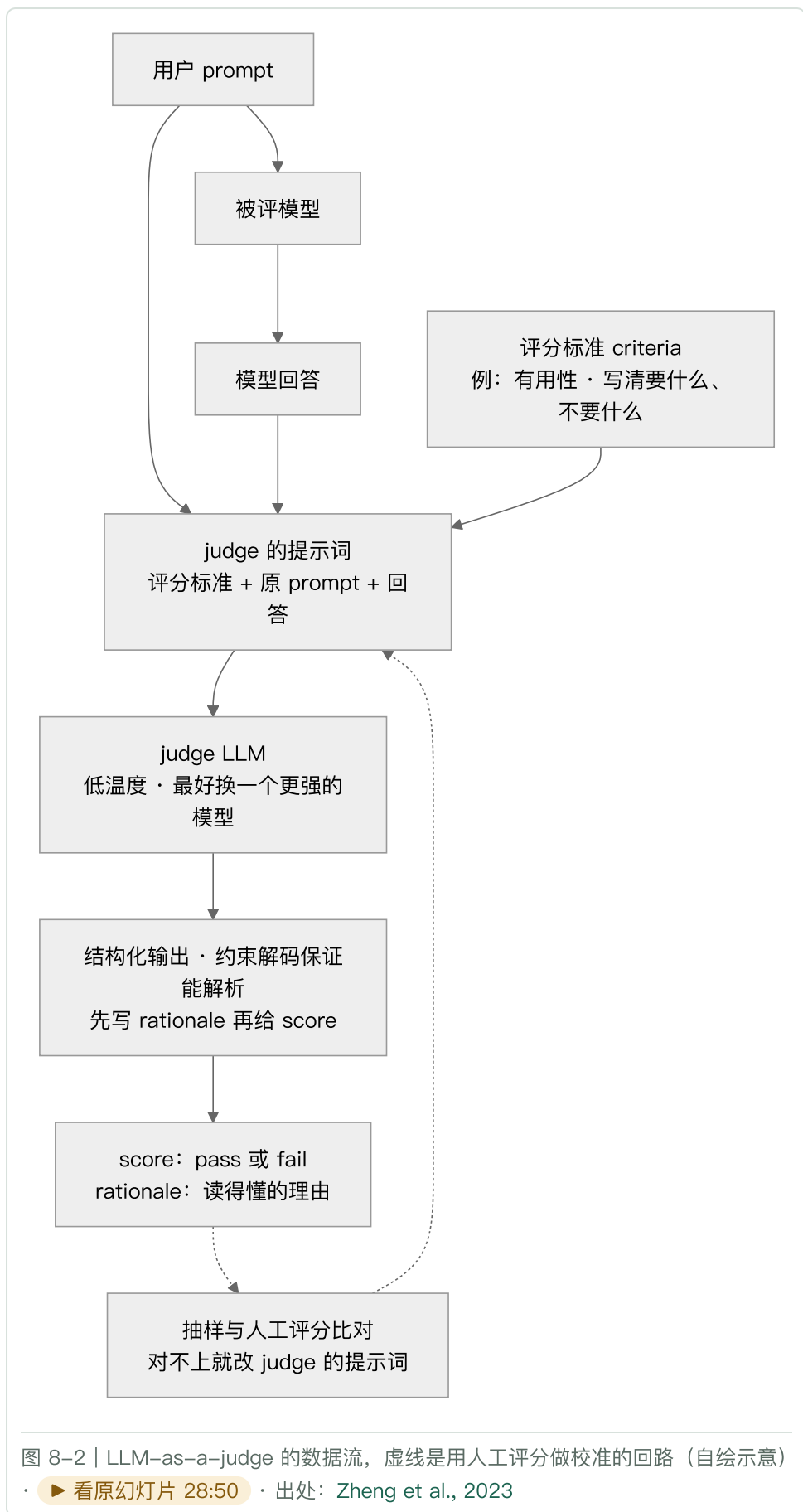


图 8-2 | LLM-as-a-judge 的数据流，虚线是用人工评分做校准的回路（自绘示意）

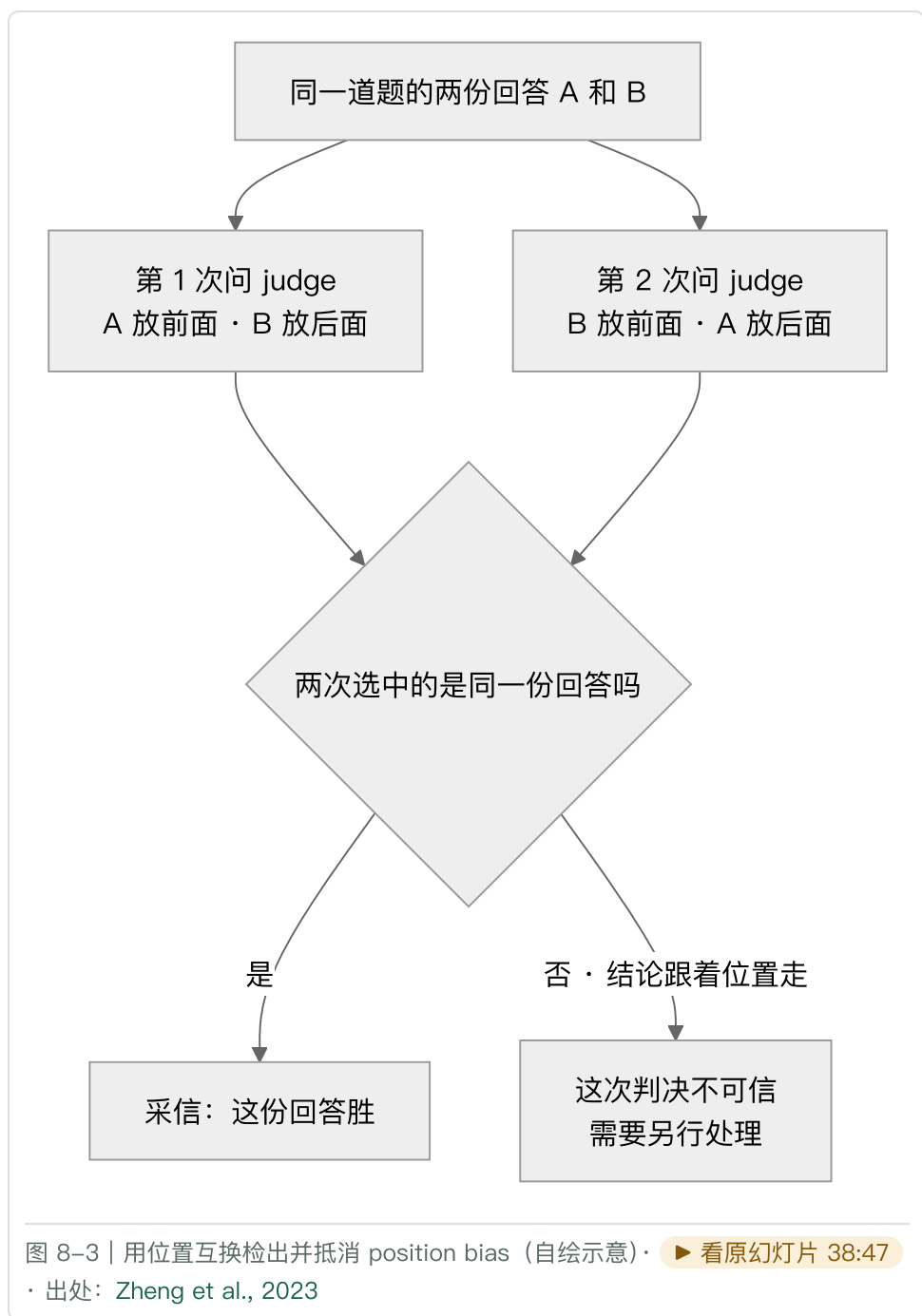
· [▶ 看原幻灯片 28:50](#) · 出处：Zheng et al., 2023

- **动机**：前七讲的 LLM 在海量数据上预训练，又按人类偏好调过（第 5 讲），既有知识，也"知道"人喜欢什么样的回答——那就让它来评。这个叫法出自两年前的一篇论文（应为 Zheng et al., 2023）。
- **输入三样**：生成回答时用的 prompt、模型的回答、评分标准（criteria，沿哪个维度评）。**输出两样**：分数（例如 pass / fail）和 rationale（理由）。和规则指标的关键区别在后者：公式乘出来的数字说不清"为什么是这个分"，judge 能解释。
- **技巧：先 rationale、后 score**。经验上能提高评分质量。讲者把它和第 6 讲的推理模型类比：先输出 chain of thought 再作答，等于让模型在给分之前把判断过程说出来。

小注：机制上，自回归生成时先写出的理由会成为给分时的上下文；反过来先给分，理由就成了事后找补。用结构化输出时字段顺序也要 rationale 在前——OpenAI 的文档说明输出按 schema 里键的顺序生成。

- **保证能解析：structured output**。课上提问：照这样写 prompt，能保证每次都解析出 rationale 和 score 吗？不能，采样是概率性的。办法是第 3 讲讲过的 constrained / guided decoding：解码时只允许从"合法 token"里采样，强制输出符合指定格式（如 JSON）。OpenAI、Gemini、Anthropic 的 API 都把它叫 structured output：把想要的格式写成一个类（这里两个字段 rationale、score），通过参数传进去（讲者记得 OpenAI 的参数名是 text_format，别家大同小异）。
- **两个好处**：不需要参考答案和人工评分就能起步；分数可解释。
- **两种形态**：pointwise——给一个回答，问好不好；pairwise——给两个回答，问哪个更好。pairwise 还有一个副产品：第 5 讲训练 reward model 需要成对的偏好标签，可以让 judge 来合成。

5. judge 的三种偏差，以及怎么缓解



偏差	表现	课上给的缓解办法
position bias (位置偏差)	pairwise 比较时, 倾向选先出现的那份回答, 与内容无关	同一对回答按两种顺序各问一次, 结论一致才采信 (图 8-3); 也有论文去改位置编码, 但不是开箱即用的做法
verbosity bias (冗长偏差)	更长、更啰嗦的回答更容易被判为更好, 哪怕并不更正确	在评分指南里明说不要因为长就加分; 放几个 in-context 例子做示范; 改成 pointwise 打分, 再按输出长度加惩罚
self-enhancement bias (自我偏好)	judge 偏爱由它自己 (同一个模型) 生成的回答	生成和评审不要用同一个模型; 最好选更大、推理更强的模型当 judge

- **自我偏好的直觉**：模型会生成某段文字，说明在它自己的概率分布里这段文字本来就“很可能”，回头再看到时自然也倾向于认为它好。讲者也承认，如今各家模型的训练数据高度重合，“换一个模型”做不到完全隔离，只是把风险压低。
- **两次结论不一致怎么办**：课上只说“可能要另做处理”。

小注：Zheng et al. 的保守做法是两种顺序下都胜才算胜，不一致就记平局；大规模评测时也可以随机分配位置，让偏差在期望上抵消。该论文报告 GPT-4 当 judge 与人的一致率超过 80%，和人与人之间的一致率同一水平。

• 问答

- judge 的偏好和人 (ground truth) 不一致、系统性地偏向某个标签，算不算偏差？算。讲者强调这三种远不是全部。
- 换了不同的模型当 judge，它还会偏爱“像 LLM 写的”回答吗？会，取决于 judge 有多强。通行做法是选容量大得多、推理能力强的模型，不容易被“听起来像自己会说的话”带走。judge 比被评模型大不是硬性要求，只是常见选择。

6. judge 的最佳实践：把它当成需要校准的仪器

- **评分标准写得干脆**：明确要什么、不要什么。标准含糊，judge 和人都会各评各的。
- **用二元量表** (pass / fail)，不用细粒度分数：judge 更好判；人工评分者也更好判，拿人的评分去对齐 judge 时噪声更小；多出来的档位未必带来有用的信号。
- **先 rationale、后 score**；对已知偏差逐项设防（第 5 节）。
- **用人工评分校准**：虽说不靠人工评分就能起步，但最终想逼近的量就是人的判断。预算允许时，收一批人工评分，和 judge 的分数做相关性分析，对不上主要靠改 judge 的 prompt。

小注：二元标签下，judge 与人的一致程度同样应该用第 2 节的 κ 来报，并和人与人之间的 κ 放在一起看。CS329A 第 8 讲的两个实例：GDPval 的自动评审与专家一致率 66%（专家之间 71%）；DeepScholar-Bench 的自动指标与人工标注一致率 70–80%。

- **低温度**（常见取 0.1 或 0.2）：今天评一次、两天后再评一次，分数不该差很多——评测要可复现。
- **不要对代理指标过度优化**：judge 分数只是人工评分的近似。模型越改、judge 分越高，不等于人也觉得越好；所以在迭代模型的过程中

要持续确认两者没有走散。这和本讲结尾的 Goodhart 定律是同一件事。

7. 评哪些维度；事实性要拆开评

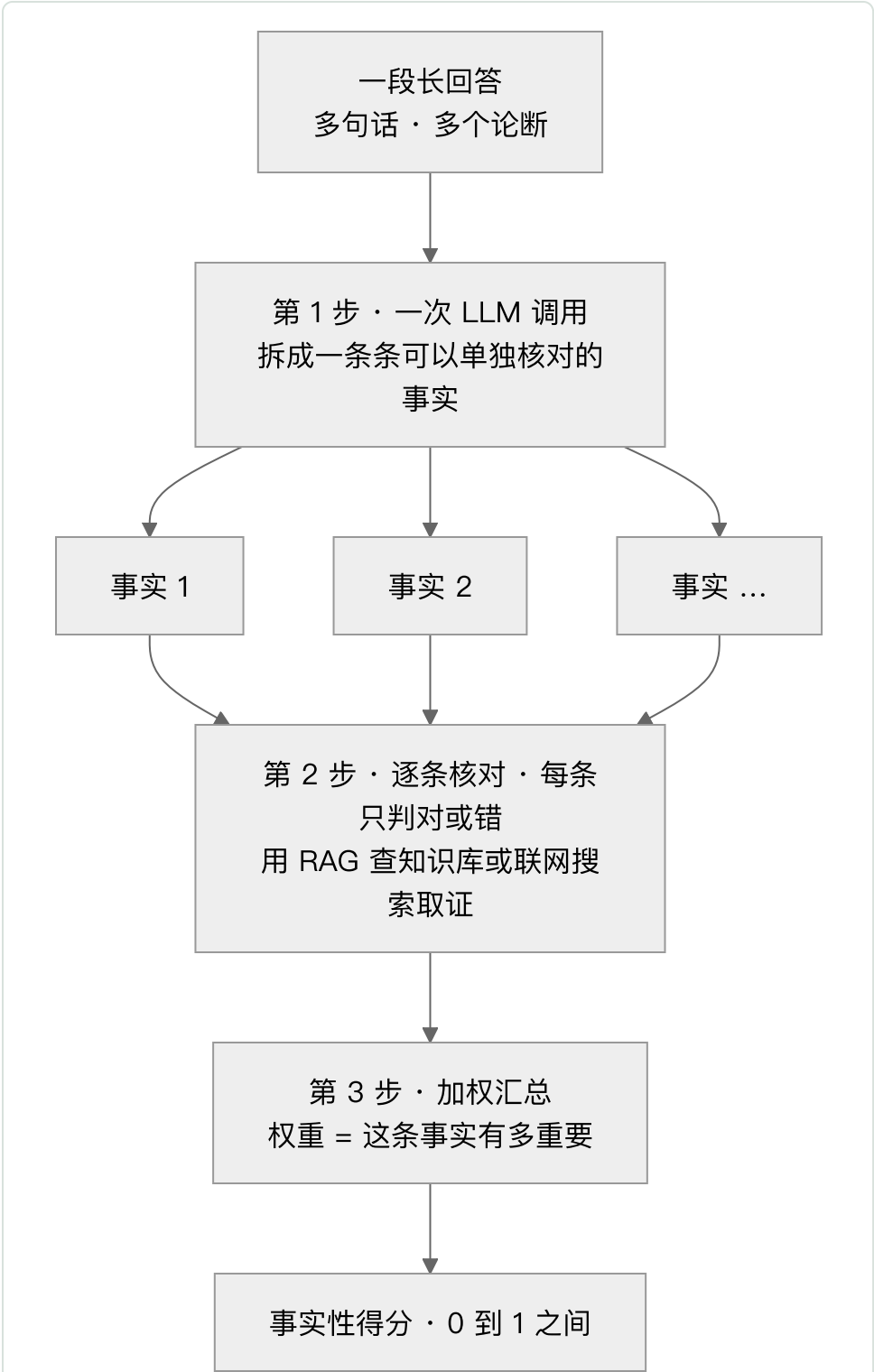


图 8-4 | 事实性评测的三步流水线（自绘示意） · [▶ 看原幻灯片 55:57](#) · 出处（推断）：Min et al., 2023

- **维度分两大类**：任务表现（有用性、事实性、相关性.....）；形式与对齐（语气、风格、有没有不安全的内容）。每个维度各写一份评分标准。
- **事实性为什么特殊**：前面建议用二元量表，但一段长文本可能只错一小处，整段判"不事实"太粗，需要量出"错了多少"。课上的例子是一段讲泰迪熊来历的话，里面藏了两处错：年代应为 1900 年代而不是 1920 年代；罗斯福总统是拒绝开枪，而不是得意地想开枪。
- **三步流水线**（图 8-4）
 1. 一次 LLM 调用把文本拆成事实列表（例子里是 4 条）。
 2. 逐条核对，每条只判对 / 错——单条事实基本上非对即错，不必再搞复杂。核对通常要用上一讲的 RAG 去查知识库，或者联网搜索，所以这一步本身也是若干次 LLM 调用。
 3. 加权汇总。权重 α_i 表示每条事实的重要性（"以谁的名字命名"比"总统当时的态度"重要）；图省事可以全取相等。

$$\text{Factuality} = \frac{\sum_{i=1}^m \alpha_i \cdot \mathbf{1}[\text{fact}_i \text{ correct}]}{\sum_{i=1}^m \alpha_i}$$

m 是拆出来的事实条数， α_i 是第 i 条事实的重要性权重，方括号里的指示函数在第 i 条事实核对为正确时取 1、否则取 0；权重已经归一（和为 1）时分母就是 1。这是按讲者的口头描述写出来的形式。

- **例子的结果**：第 2、3 条正确，汇总得 0.6（字幕只剩一个"6"，按 0—1 的量纲推断为 0.6；4 条里对 2 条却不是 0.5，推断幻灯片上用了不相等的权重）。讲者强调怎么量化这种"部分正确"仍是开放问题，这只是目前常见的做法。

小注：这套"拆成原子事实—逐条查证—汇总"的做法应出自 FActScore (Min et al., 2023, 等权, 即被可靠知识源支持的原子事实占比) 和 SAFE (Wei et al., 2024, 用 LLM 加 Google Search 逐条查证, 与众包标注者 72% 一致、成本低 20 多倍), 课上没有点名。它量的是 precision (说出来的有多少是对的), 不惩罚"该说的没说"; CS329A 第 8 讲 DeepScholar-Bench 的 nugget coverage 补的正是 recall 这一侧, citation precision / claim coverage 则是同一思路用在"引用是否撑得住论断"上。

8. Agent 评测：把一次工具调用拆成三步，逐步找失败

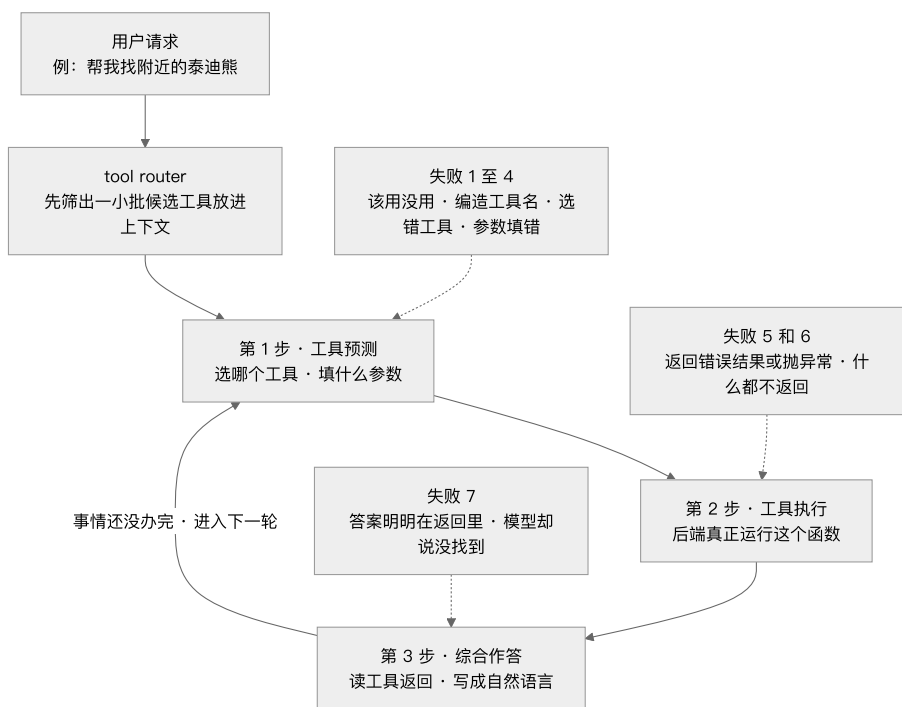


图 8-5 | 一次工具调用的三步，以及七种失败各落在哪一步（自绘示意）。

► 看原幻灯片 1:01:38

- 为什么单拿出来讲：agent 是一个循环（第 7 讲的 ReAct：observe → plan → act），最终结果不好，可能错在任何一环。想让评测结果有意义，先得知道每一环能怎么错。Shervine 只看其中一轮：一次工具调用 = ① 工具预测（选对工具、填对参数）→ ② 工具执行 → ③ 根据返回综合作答（第 7 讲的三步图）；agent 就是把这三步串很多轮。
- 七种常见失败（讲者说是他自己最常遇到的，不是全部）
 1. 该用工具却没用（步骤 ①）。模型直接回“抱歉，我不知道去哪找”——assistant 的行话叫 punt（放弃作答）。
 - 可能是 tool router 没把这个工具选进上下文。工具一多，不可能每次都把全部 API 塞进 preamble，通常先用 router 筛一遍（第 7 讲）；router 应当偏 recall——省上下文可以，但不能漏掉要用的。对策：调 router。
 - 也可能工具就在上下文里，模型却没想到用。对策：回到“教模型用工具”那一步，在 SFT 数据里补这种模式，或者改提示词。
 2. tool hallucination：调用了一个不存在的函数（定义的是 find_teddy_bear，模型却去调 find_bear）。
 - 先查是不是普遍现象：横跨所有工具的总指令（horizontal instructions）有没有讲清“只能用给定的函数”。总指令很关

键，格式和逻辑都要严谨，可以让 LLM 帮着迭代。

- API 写得不好：模型在 SFT 阶段见的都是高质量 API，你的写法不够“像样”，它就容易对不上。模型能看到的只有函数名、参数、描述这三样，它们就是你能拧的三个旋钮。
- 模型太弱、不 ground（不以给定内容为依据，自己编）：确认属实再换更强的模型。

3. **用错了工具**：本该调查找工具，模型却去“发消息找人要一只熊”——也说得通，但不是你要的行为；让模型知道你偏好哪种做法，是你的责任。对策：检查 router 的召回；检查两个工具的 API 描述是否范围重叠，写清各自负责什么情形。

4. **工具对了、参数错了**：坐标填成 (0, 0)，落在大西洋里。多半是上下文里根本没有位置信息，模型只好编——让上下文带上位置，或者先跑一个定位工具，拿不到权限就给用户一个可操作的报错，而不是塞假参数。也可能是模型不懂参数该填什么：重训，或重写 API。

5. **工具返回了错的结果，或直接抛异常**（步骤 ②）。这是纯软件工程问题：修工具。一个细节：报错未必是坏事（没给定位权限时，模型可以据此告诉用户）；但裸抛异常时，模型常把它当成自己的内部错误，只会笼统道歉、说不清发生了什么。更好的做法是返回结构化的、把状况讲明白的正常输出。

6. **工具什么都不返回**。对执行动作的工具尤其危险：第 7 讲“把恒温器调高”的例子，工具不吭声，模型不知道成没成，可能直接回一句“已经调好了”——虚假确认。原则：每次工具调用都要返回有意义的输出。连“没找到”也一样：空 JSON 表示“找过了，没有”，None 则什么都没说。

7. **结果在手却不会用**（步骤 ③）：工具明明返回了一只叫 Teddy、离你一英里的熊，模型却说没找到。依次排查：模型 grounding 差（早期模型常见，如今少了）；工具返回太多，关键信息被淹没——回去精简工具输出；返回的是难以解读的原始数据——改成带字段名的结构化对象（比如含 name、distance 的 TeddyBear 对象）。

- **共性**：毛病分两大类。模型侧——推理与 grounding 的能力、上下文里内容的相关度、tool router 和工具 API 的“建模”（SFT、prompt，以及函数名、参数、docstring 写得合不合理）；工具侧——实现本身有 bug。

- **方法论：**工具一多，错误五花八门，每个 case 查起来都像一次探险。要有条理地给错误分类，再成组地解决。

小注：CS329A 第 8 讲里 METR 对 GPT-4 和 o1 的失败轨迹做人工归类，讲者说"提出难 benchmark 就该配失败模式分析"，是同一条方法论。两者的粒度互补：METR 按整条轨迹归类（过早放弃、重复已失败的动作……），这里按单次工具调用的环节归类。

9. Benchmark 的几大类：知识、推理、代码、安全

类别	代表	题型	怎么判分	主要在量什么
知识	MMLU	近 60 个学科的四选一	抽出模型输出的选项字母，与答案比对	预训练把知识记住了多少
推理 · 数学	AIME	竞赛题，答案是一个整数	与标准答案精确匹配	多步推演的质量
推理 · 常识	PIQA	物理常识二选一，约 2 万题	选项比对	对日常物理世界的理解
代码	SWE-bench	真实的 GitHub issue 加代码库，要求给出 patch	打上 patch，跑该 PR 引入的测试	在真实仓库里读写代码、修 bug
安全	HarmBench	四类有害请求	训练出来的分类器判断模型是否"照做了"	面对有害请求会不会配合
agent	τ-bench (第 10 节)	多轮对话加工具调用	对话结束后的数据库终态	守着政策把事办成，而且次次办成

- **一条主线：能硬编码判分，就不要用 judge。**MMLU、AIME、PIQA 都把答案的形式卡死（一个字母、一个整数、二选一），好处是可以用写死的规则提取和比对，不必再叠一层 LLM judge——judge 自己会错，多一层就多一层误差。HarmBench 是这几个里唯一靠分类器判分的，读它的分数时要记得判分器本身也会错。
- **MMLU**（Massive Multitask Language Understanding）：学科从日常话题到法律、医学。题目多数要先有该领域的知识，单靠逻辑推不出来（课上的医学例题：给一串临床指标，问损伤在哪）。讲者的定位：它主要量的是预训练的成效（但不只是预训练）。

小注：原论文（Hendrycks et al., 2020）是 57 个学科。

- **AIME：**美国高中生通往奥赛选拔的考试，很难；题干往往只有一两句话，但要写出完整推理才能得到答案——正好考 chain of thought（或推理模型的 thinking tokens）。答案格式固定，对评测很友好。课上展示的是 2025 年的题。

小注：AIME 的答案是 0—999 的整数（课上说成“三位数”）。每年两套卷、每套 15 题，一年只有 30 题，一题就是 3.3 个百分点，比较模型时要留意方差。

- **PIQA** (Physical Interaction QA)：扎根日常物理世界的常识推理。论文里的例子：东西掉在地毯上找不到，该给吸尘器口套一个实心的封盖，还是套一个发网？实心封盖不透气，什么也吸不了；发网透气，又能把小东西兜住。对人是常识，对模型未必显然。
- **为什么人人都该看代码成绩**：一是 AI 辅助编程本身就是最成功的用途（第 7 讲）；二是 agent 的工具多半写成 Python 函数，模型得会读写代码，才能正确发起工具调用、读懂返回。
- **SWE-bench 怎么造出来的**：从热门 Python 仓库里筛出“解决了某个 issue、同时引入了测试”的 pull request。可以合理假设这些测试在修复前不过、修复后通过，于是“测试状态的前后变化”就是修复质量的客观度量（test-driven development 的思路）。评测时给模型代码库和 issue，要它产出 patch，打上之后看测试过不过。名字应是 software engineering benchmark 的缩写，论文没有明说。

小注：原论文（Jimenez et al., 2023）是 12 个仓库、2,294 个任务，当时最好的 Claude 2 只解决 1.96%。现在厂商报的多是 SWE-bench Verified（OpenAI 组织人工筛过的 500 题子集）。

- **HarmBench 与安全评测的特殊性**
 - 为什么模型发布时的对比表里很少有安全 benchmark：什么该拒、什么不该拒取决于各家的政策，那是某些人做出的决定，未必普世；各家也并不都宣称要把某个安全 benchmark 做到 100%，横向比较意义有限。model card 里通常有安全章节，讲做了哪些工作，但不拿同一个 benchmark 互相比。自己跑安全 benchmark 之前，先看它的内容和你的政策是否一致，分数才有意义。
 - 四类：standard（普通的有害行为）、copyright（诱导生成受版权保护的内容）、contextual（带一段文本上下文）、multimodal（上下文是文本以外的模态）。课上的例子：撬开不该开的门；左右别人的选举投票。
 - 判分：有害输出是开放式的，正则匹配解决不了，论文训练了一个分类器。一个值得学的设计是把“能力”和“安全”分开：只要模型试着去做有害的事，哪怕做得很差、没做成，也算攻击成功。

小注：HarmBench (Mazeika et al., 2024) 共 510 个行为（四类依次 200 / 100 / 100 / 110），指标是 attack success rate；非版权类用微调过的 Llama 2 13B 当分类器，版权类用基于哈希的比对。"harmful behavior benchmark"是讲者对名字的猜测，论文标题里没有这个展开。

10. τ -bench 与 pass^k: agent 要的是次次都成



图 8-6 | τ -bench 的一次 trial：LLM 扮用户，结束后按数据库终态判分（自绘示意） · [▶ 看原幻灯片 1:42:40](#) · 出处：Yao et al., 2024

- **设置：** τ 取自 Tool-Agent-User。两个领域（航空、零售），每个领域给一组工具 API、一份政策（agent 能做什么、不能做什么）和一批任务。任务是交给"用户"的一段问题陈述，用户要通过 agent 把事办成。
- **为什么要用 LLM 模拟用户：**多轮对话里，用户下一句说什么取决于 agent 上一步做了什么，没法预先写死脚本，所以让另一个大模型来扮演用户。
- **怎么判分：**对话结束后，看数据库是不是处在目标状态（比如机票确实改签了），和 / 或是否执行了任务要求的动作（比如取消），据此给 reward。

小注：论文里 $\text{reward} = r_{\text{action}} \times r_{\text{output}}$ ，取 0 或 1：前者要求最终数据库与标注的唯一目标数据库完全一致，后者要求 agent 的回复里包含任务规定要告知用户的信息。零售 115 个任务、航空 50 个；用户由 gpt-4-0613 扮演。

- **pass^{^k}** (读作 pass hat k): 同一个任务独立跑 k 次, k 次**全部**成功的概率; 对照此前的 pass@k——k 次里**至少一次**成功的概率。为什么要换: 航空、零售正是想拿 agent 顶替人工坐席的场景, 要的是可靠和一致, "十次里能成一次"没有用。

$$\text{pass}@k = \mathbb{E}_{\text{task}} \left[1 - \frac{\binom{n-c}{k}}{\binom{n}{k}} \right], \quad \text{pass}^k = \mathbb{E}_{\text{task}} \left[\frac{\binom{c}{k}}{\binom{n}{k}} \right]$$

n 是每个任务实际跑的次数, c 是其中成功的次数, k 是要考察的尝试次数 ($k \leq n$), 期望是对所有任务取平均; 两式在 $k = 1$ 时都等于 c/n , k 增大时 pass@k 只升不降、pass^{^k} 只降不升。

小注: 讲者说 pass@k 的推导是"上一讲"做的, 实际在第 6 讲 (板书从 37:03 起)。pass^{^k} 用的是同一套"从 n 次里无放回抽 k 次"的组合论证: 抽到的 k 次全是成功的概率为 $C(c,k) \div C(n,k)$ 。直觉上, 单次成功率为 p 时 pass^{^k} 约等于 p 的 k 次方——单次 90% 的 agent, 连续 8 次全成只有约 43%。论文的数字: 当时最强的 GPT-4o 的 pass^{^1} 为零售 61.2%、航空 35.2%, 零售的 pass^{^8} 不到 25%。

小注: CS329A 第 8 讲里 METR 的 50% 与 80% time horizon 问的是同一件事的另一面: METR 固定可靠度、看能做多长的任务; pass^{^k} 固定任务、看能不能次次做成。该课第 2 讲重复采样里的 coverage 则对应 pass@k。

11. 读榜、选模型, 以及数据污染

- **对回现实**: 上课前几天 Gemini 发布了新模型, 发布材料里的对比表和上面的分类基本对得上: 推理一栏有 AIME 和 PIQA 的多语言版 Global PIQA, 代码一栏是 SWE-bench 的一个变体, 工具使用一栏是 τ -bench 的后继 τ^2 -bench。不少经典 benchmark 如今以加了多语言的变体出现。

小注: 指 2025 年 11 月 18 日发布的 Gemini 3 Pro, 表里代码一栏是 SWE-bench Verified。Global PIQA (Chang et al., 2025) 覆盖 100 多种语言; τ^2 -bench (Barres et al., 2025) 新增电信领域, 用户一侧也能动手操作环境 (dual control)。

- **benchmark 画的是模型的画像, 不是总分**: 没有全好或全坏的模型, 各有长短。讲者的个人经验 (他声明不是普适建议): 写代码用 Claude Sonnet 系列, 要快、要便宜时用 Gemini Flash。
- **Pareto frontier**: 把性能对另一个你在意的维度画出来——价格最常见, 也可以是安全、上下文长度——每个价位上最好的模型连成的边界就是 Pareto 前沿, 选模型就在前沿上选。

- **数据污染**：benchmark 成绩成立的前提是模型没见过题目和答案。课上提到三种防范：给数据集加哈希值（推断指 canary string 一类的标记串，便于从训练语料里识别和剔除）；评测能上网的 agent 时，用 blocklist 挡住可能含答案的网站；数学上可以直接用模型训练之后才出的新卷子（比如当年的 AIME）。

小注：CS329A 第 8 讲的 DeepScholar-Bench 只收主流模型训练截止期之后的 arXiv 论文、每月换新题，是"用新题"这条路的系统化版本。本讲没有谈 benchmark 饱和，那一讲正是从"传统 benchmark 饱和"讲起的。

- **Goodhart 定律**：一个指标一旦被当成目标，就不再是好指标。
benchmark 成绩要和你真正想要的东西对照着看，它不能直接告诉你某个模型对你好不好用。此前课上提过的 Chatbot Arena 可以补一点"真实使用中的表现"，但最终还是得自己拿任务去试。

关键图表速查（点时间戳跳到原幻灯片）

图	看什么	跳转	出处
板书：随机一致率	Alice、Bob 各以 p_A 、 p_B 随机打分；一致 = 同为 1 或同为 0；代入 0.5 得到 50%	10:01	板书
Cohen's kappa 公式	观察一致率高于、等于、低于碰巧水平时， κ 分别为正、零、负	14:43	Cohen, 1960
METEOR 公式	$F\text{-score} \times (1 - \text{penalty})$ ；penalty 里"连续片段数 ÷ 匹配 unigram 数"怎么体现词序	21:29	Banerjee & Lavie, 2005
BLEU 与 ROUGE	BLEU 偏 precision，所以要 brevity penalty；ROUGE 用于摘要	25:07	Papineni et al., 2002 · Lin, 2004
同一句话的三种说法	意思相同、用词几乎不重叠——规则指标为什么失灵	26:42	—
judge 的 prompt 模板	三个输入（标准、原 prompt、回答）；要求先返回 rationale 再返回 score	30:45	Zheng et al., 2023
structured output 代码	用一个类声明 rationale、score 两个字段，经 API 参数传入	34:34	—
三种偏差	位置、冗长（40:21）、自我偏好（42:27）各一页，每页下方是缓解办法	38:47	同上 Zheng et al.
事实性流水线	原文 → 4 条事实 → 逐条对错 → 加权得分	55:57	应出自 FActScore
工具调用三步图	工具预测 → 工具执行 → 综合作答；后面七种失败都挂在这张图上	1:01:38	第 7 讲的幻灯片
七种失败汇总	按三步归类的总表；下一页是模型侧 / 工具侧两类共性	1:21:22	—

图	看什么	跳转	出处
SWE-bench 流程图	代码库 + issue → 模型给 patch → 打上后看测试状态的前后变化	1:35:50	Jimenez et al., 2023
τ -bench 改签例子	用户、agent、工具三方的多轮往来；最后按数据库终态给 reward	1:42:40	Yao et al., 2024
Gemini 发布对比表	找 AIME、Global PIQA、SWE-bench 变体、 τ^2 -bench 各在哪一栏	1:45:27	Gemini 3 Pro 发布材料（推断）

提到的工作

名称	在本讲里的作用
RAG (bi-encoder 召回 + cross-encoder 重排)、tool calling、ReAct (第 7 讲)	开场回顾；事实核对要用 RAG；agent 评测沿用三步图、tool router、恒温器例子
Cohen's kappa (Cohen, 1960)、Fleiss' kappa、Krippendorff's alpha	扣掉"碰巧一致"之后的评分者间一致性指标
METEOR (Banerjee & Lavie, 2005)	翻译指标：加权 F-score $\times$ 词序惩罚
BLEU (Papineni et al., 2002)	翻译指标：n-gram precision $\times$ brevity penalty
ROUGE (Lin, 2004)	摘要指标，偏 recall，多个变体
LLM-as-a-judge (应为 Zheng et al., 2023)	本讲的核心方法；三种偏差的出处
constrained / guided decoding，即各家 API 的 structured output (第 3 讲)	保证 judge 的输出一定能解析
reward model 与偏好数据 (第 5 讲)	pairwise judge 可以用来合成偏好标签
推理模型与 chain of thought (第 6 讲)	"先 rationale 后 score"的类比；pass@k 也在第 6 讲
FActScore、SAFE (课上未点名，推断)	"拆事实—逐条核对—汇总"的事实性评测
MMLU (Hendrycks et al., 2020)	知识类 benchmark
AIME	数学推理 benchmark，可以用当年新题避开污染
PIQA (Bisk et al., 2019) 与 Global PIQA	物理常识推理；后者是多语言变体
SWE-bench (Jimenez et al., 2023)	代码类 benchmark：真实 issue + 测试判分
HarmBench (Mazeika et al., 2024)	安全类 benchmark：四类行为、分类器判分
τ -bench (Yao et al., 2024) 与 τ^2 -bench	agent 类 benchmark：LLM 扮用户、数据库终态判分、pass^k
pass@k (Chen et al., 2021)	pass^k 的对照：至少成功一次 vs 次次成功

名称	在本讲里的作用
Gemini 新模型的发布对比表	说明这些 benchmark 类别在真实发布里怎么出现
Claude Sonnet、Gemini Flash	讲者的个人选型经验：前者写代码，后者快而便宜
Chatbot Arena	此前课上提过；用真实用户的偏好补 benchmark 的不足
Goodhart's law	指标一旦成为目标，就不再是好指标

术语对照

English	中文
output quality	输出质量（本讲的评测对象，区别于延迟、价格等系统指标）
rater / human rating	评分者 / 人工评分
inter-rater agreement	评分者间一致性
agreement rate	一致率（两人标签相同的比例）
chance agreement	随机（碰巧）一致率
Cohen's kappa	科恩 kappa 系数（两位评分者、扣除碰巧一致）
agreement session	对齐会（评分者一起统一口径）
reference	参考答案（人写的理想输出）
rule-based metric	规则指标（按公式比对输出与参考）
unigram / n-gram	单个词 / 连续 n 个词的片段
precision / recall	精确率 / 召回率
harmonic mean	调和平均
chunk	连续匹配片段（METEOR 里用来衡量词序）
brevity penalty	过短惩罚（BLEU）
stylistic variation	风格变化（同一意思的不同说法）
LLM-as-a-judge	用 LLM 当裁判
criteria	评分标准
rationale	理由（judge 对分数的解释）
structured output	结构化输出
constrained / guided decoding	约束解码（只允许采样合法 token）
pointwise / pairwise	单点评分 / 两两比较
preference data	偏好数据
position bias	位置偏差

English	中文
verbosity bias	冗长偏差
self-enhancement bias	自我偏好偏差
calibration	校准（让 judge 的评分向人工评分看齐）
proxy	代理指标
factuality	事实性
fact-checking	事实核对
tool prediction / tool execution	工具预测（选工具、填参数） / 工具执行
punt	放弃作答（助手直接说做不了）
tool router / tool selector	工具路由器（先筛出候选工具再放进上下文）
tool hallucination	工具幻觉（调用不存在的函数）
grounding	以给定内容为依据（而不是自己编）
horizontal instructions	横跨所有工具的总指令
docstring	函数的文档字符串
false confirmation	虚假确认（没做成却说做成了）
benchmark	基准测试
commonsense reasoning	常识推理
pull request / issue / patch	合并请求 / 问题单 / 补丁
test-driven development	测试驱动开发
model card	模型卡（随模型发布的说明文档）
policy	政策（agent 或模型被允许和禁止的行为）
user simulator	用户模拟器（LLM 扮演用户）
pass@k / pass^k	k 次里至少成一次 / k 次全部成功的概率
reliability / consistency	可靠性 / 一致性
Pareto frontier	帕累托前沿
data contamination	数据污染（评测题进了训练数据）
blocklist	屏蔽名单
Goodhart's law	古德哈特定律

字幕勘误

"raider(s)" → rater(s); "interrator" → inter-rater; "coins scapa" / "cohen scappa" → Cohen's kappa; "fly scapa" → Fleiss' kappa; "crypendors alpha" → Krippendorff's alpha; "meteor" /

"mter" / "meter" → METEOR; "blur" / "BL" / "blue" → BLEU; "under study" → Understudy; "rouge" → ROUGE; "unigs" / "engrams" → unigrams / n-grams; "fcore" → F-score; "LM as a judge" / "LS judge" / "L me" / "LM messages" → LLM-as-a-judge (及其复数); "rework model" → reward model; "facility dimension" → factuality dimension; "facteing" → fact-checking; "a score of 6" → 应为 0.6 (推断); "tool wer" / "tool wrers" → tool router(s); "duck string" → docstring; "agent tick loop" → agentic loop; "find a beer" → find a bear; "AIM" / "aim" → AIME; "PAS" / "pika" / "pa" → PIQA; "SWEbench" / "swbench" → SWE-bench, "Swiss" → SWE; "toao bench" / "Towbench" / "TaoBench" → τ -bench, "towel squared bench" → τ^2 -bench; "pass hat K" 即 pass^k ; "Sony models" → Sonnet models (Claude Sonnet); "parareto" / "parto" → Pareto; "a good heart's law" → Goodhart's law; "multimodel" → multimodal; "c by encoder" / "sentence spurt" → bi-encoder / Sentence-BERT; "Ashin" / "Afin" / "Shervin" → Afshine / Shervine。

带走的问题

1. 你的 agent benchmark 通过率很偏 (比如九成样本都 pass) 时, judge 与人工标注"95% 一致"到底说明了什么? 换成 κ 大约是多少? 该怎样抽样 (分层、补难例) 才能让一致性指标有信息量?
2. 位置互换能干净地检出位置偏差, 冗长偏差和自我偏好却没有这么现成的对照。你会怎样设计实验去量自己的 judge 有多偏爱长回答 (例如同内容的长短改写对)? 当被评模型已经是最强的模型时, "换一个更强的 judge"这条建议还剩什么?
3. 事实性流水线量的是 precision: 只说一句绝对正确的废话也能拿满分。它该怎样和有用性、recall 类的指标组合? "拆事实"这一步自己出错 (漏拆、拆错) 时, 误差会怎样传到最终分数里?
4. τ -bench 让 LLM 扮用户、按数据库终态判分。用户模拟器自己出错 (没按剧本说、提前结束对话) 会不会被算成 agent 的失败? 终态判分看不见"过程中违反了政策、结果却碰巧正确"的情形, 该怎么补? pass^k 的 k 取多大, 才对得上你的业务对可靠性的要求?
5. 本讲的七种失败按"单次工具调用的环节"归类, METR 按"整条轨迹的行为"归类。你的 benchmark 的失败分析需要哪一层? 两层之间怎么

对上——比如"重复已失败的动作"往下拆，会落到七种里的哪几种？

CME295 第 9 讲 | 回顾与当前趋势 (Recap & Current Trends)

Stanford CME295: Transformers & Large Language Models (2025 秋) · 第 9 讲，也是全课最后一讲（视频简介标注的授课日期：2025 年 12 月 5 日） 视频：<https://www.youtube.com/watch?v=Q86qzJ1K1Ss> (1:51:31，只有自动生成的英文字幕，专名识别错误较多) 讲者：Afshine Amidi（回顾与两个趋势）· Shervine Amidi (1:23:38 起的收尾部分) 课程大纲：<https://cme295.stanford.edu/syllabus/>

一句话：最后一讲分三段。先用 48 分钟把前八讲串成一条线：模型长什么样（第 1–3 讲）、怎么训出来（第 4–6 讲）、怎么用和怎么量（第 7–8 讲）。再讲两个方向相反的趋势：Transformer 走出文本（ViT 把图像切成 patch 当 token，VLM 让 LLM 就图作答），图像领域的 diffusion 走进文本（拿 MASK 充当噪声的 diffusion LLM，用少量并行步数换速度）。最后 Shervine 盘点还没定型的东西——架构细节、数据、成本、硬件、agent 的落地和几个开放问题——并给出课后跟进这个领域的办法。

时间轴

时间	内容
0:06	开场：本讲三部分——回顾、趋势、收尾
1:12	回顾第 1 讲：tokenization、word2vec、RNN、self-attention、Transformer
6:35	回顾第 2 讲：RoPE、GQA、pre-norm；BERT / GPT / T5 三个家族
11:17	回顾第 3 讲：decoder-only LLM、MoE、采样与 temperature
15:05	回顾第 4 讲：scaling laws、20 倍经验法则、FlashAttention、并行、三段式训练
24:09	回顾第 5 讲：LLM 即 policy、Bradley–Terry、reward model、RL 目标的两道约束
29:41	回顾第 6 讲：reasoning chain、GRPO 对比 PPO、verifiable reward、长度偏差
38:37	回顾第 7 讲：RAG 的两步检索、tool calling 的两步、agentic workflow
44:09	回顾第 8 讲：规则指标、LLM-as-a-judge 及其偏差、benchmark 的维度
47:35	期末范围：第 5–8 讲（期中考的是第 1–4 讲），之后的内容不考
48:57	趋势一的动机：self-attention 处理的只是向量，向量不一定非得来自文字
52:46	ViT 的做法，以及"归纳偏置很低、数据够多就能赢 CNN"的发现

时间	内容
55:49	ViT 端到端走一遍：patch → 向量 → CLS → encoder → 类别
59:01	VLM 的两种接法：图像 token 拼进输入 (LLaVA)、走 cross-attention (Llama 3)
1:01:34	Transformer 在文本之外：DiT、MM-DiT、推荐、语音
1:04:02	趋势二的动机：自回归生成在推理时没法并行（训练时可以）
1:07:12	2025 年的信号：Google 的实验性文本 diffusion 模型、Inception；难点是文本离散
1:08:44	两分钟讲 diffusion：为什么从噪声出发；雕塑家的类比；前向加噪与反向去噪
1:13:24	搬到文本：MASK 之于文本如同噪声之于图像；MDM 与 dLLM 两个名字
1:16:28	推理过程：从全 MASK 出发逐步还原；写演讲稿的直觉
1:18:31	为什么快：forward pass 次数等于步数；参考文献 LLaDA
1:20:06	优点（速度、fill-in-the-middle）、现状（在追赶）、待做（把自回归的配套技巧搬过来）
1:23:38	收尾开始：两种模态互相借架构、借输入（DeepSeek-OCR）、借技巧（2D RoPE）
1:28:35	Transformer 的设计仍在迭代：optimizer、归一化、attention、激活函数、MoE、超参
1:32:12	数据：互联网被 LLM 文本充斥、data curation、mid-training、model collapse
1:34:49	架构本身是不是最优；从刷 benchmark 转向成本与质量的 Pareto 前沿；SLM
1:36:52	硬件：GPU 只擅长矩阵乘；把 attention 直接做进模拟电路的概念验证
1:39:58	今天的用例：写代码、text-to-query、可视化、通用助手、创作起稿、辅助学习
1:43:05	往前看：agent 大众化、AI 浏览器与 prompt injection、OS 级助手、客服这块试金石
1:46:39	开放问题：持续学习、“幻觉”、个性化、可解释性、安全
1:47:40	怎么保持更新：arXiv、会议、代码库、Hugging Face Trending Papers、X、YouTube、课程 study guide
1:50:16	致谢与告别

核心内容

1. 全课一页图：八讲是怎么接起来的



图 9-1 | 全课一页图：八讲分三层，最后一讲从这条线上往外探两步（自绘示意）·

回顾部分没有新内容，讲者的目的是让各块拼得上。下表把每一讲压成三件事，想重看哪一段，点时间戳跳过去。

讲	要记住的三件事	跳转
1 ·	① 文本先切成 token (subword 最常用，词根可以复用)，再变成向量；Tra word2vec 靠"预测中心词或上下文词"这类代理任务学向量，但同一个词nsf 在任何句子里向量都一样 ② RNN 逐个读 token，隔得远的信息留不住；orm self-attention 让任意两个 token 直接相连 ③ Q / K / V：拿 query 和所er 有 key 算相似度，softmax 之后对 value 加权平均；Transformer 由 encoder 和 decoder 组成，最初为翻译而设计	1:12
2 ·	① 位置：原版给每个绝对位置配一个向量加到 token 上；RoPE 改成分成 2 部分，attention 内部旋转 Q 和 K，分数只取决于两个 token 的相对距离 ②改进 GQA：多个 head 共用一组 K / V 投影矩阵；归一化从子层之后 (post-norm) 挪到子层之前 (pre-norm) ③ 三个家族：encoder-only 衍生 (BERT，取 CLS 向量做分类，不能生成文本)、decoder-only (GPT)、生 encoder-decoder (T5)，后两类都能自回归生成	6:35
3 ·	① LLM 就是放大的 decoder-only 模型，文本进、文本出 ② MoE：把 LL FFN 换成多个 expert，由 gate 按 token 稀疏路由，一次前向只动用一部M 分参数；expert 还能分放到不同 GPU 上并行 ③ 解码：greedy 每次取概率最高的 token，sampling 按分布抽；temperature 低则分布尖、输出稳定，高则更随机、更有变化	11:17
4 ·	① scaling laws：算力、数据、参数越多，test loss 越低；但算力有限训 时，当年多数模型相对数据量而言太大、没训够，经验法则是训练 token 数至少为参数量的 20 倍 (100B 参数对应 2T token) ②FlashAttention：GPU 有又大又慢的 HBM 和又小又快的 SRAM，把计算切块送进 SRAM、少读写 HBM；结果是精确的，还靠"丢掉中间结果、用时重算"换来更快的速度；另有 data parallelism 与 model parallelism ③三段式：pre-training 学会续写 → SFT 学会按要求作答 → preference tuning 用成对偏好数据注入负信号	15:05
5 ·	① 把 LLM 看成 RL 里的 policy：state 是到目前为止的输入，action 是调 预测下一个 token，reward 来自人的偏好 ② 人工偏好标注只覆盖很少的优 数据，所以训练 reward model；它按 Bradley-Terry 形式成对训练 ③RL 目标：最大化奖励，同时别离 SFT 模型太远 (奖励不完美，会被钻空子，即 reward hacking)，每轮更新也别离上一轮太远	24:09
6 ·	① reasoning model 先输出一段 reasoning chain 再给答案，思路来自第推 3 讲的 chain-of-thought 提示；训练过程中 AIME 准确率一路上升 ②理 GRPO 对比 PPO：不要 value model，同一道题采样一组回答、在组内互相比得到 advantage；答案可验证时连 reward model 也省了，只需保留 policy 和 reference 两个模型 ③ 原版 GRPO 的归一化项让错误回答越写越长；Dr. GRPO 和 DAPO 是针对性的修正	29:41
7 ·	① RAG：模型的知识停在 knowledge cutoff，现实中也不会天天重训，Age 所以先检索、再作答 ② 检索分两步：bi-encoder 做向量召回 (query 向量对预先算好的文档向量求 cosine 相似度) → cross-encoder 把 query 和文档一起送进模型精排 → 取 top-K 拼进 prompt ③ tool calling 也是两步：模型选定 API 并填好参数 → 外部执行 → 结果回填，模型再生成最终回答；agentic workflow 是这两样的多步组合	38:37

8 · ① BLEU / ROUGE / METEOR 这类规则指标认不出"说法不同但同样正确"② LLM-as-a-judge: 输入 prompt、模型回答和评判标准, 先写 rationale 再给分; 现在多用 pass / fail 二值, 因为更容易判③ 三种偏差: position (先出现的占便宜)、verbosity (偏爱长回答)、self-enhancement (偏爱自己的输出); benchmark 覆盖知识、推理、代码、安全等维度

44:09

全课的中心思想是 self-attention, 讲者在回顾里把这条式子又念了一遍:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V$$

Q、K、V 分别是所有 token 的 query、key、value 向量排成的矩阵, d_k 是 key 向量的维度; 写成矩阵形式, 正好是今天的硬件最擅长的运算。这条式子后面还会出现两次: 趋势一里它处理的向量换成了图像 patch, 收尾部分的硬件话题则围着 $QK^\top$ 这一步的开销转。

$$P(y_i \succ y_j) = \frac{e^{r_i}}{e^{r_i} + e^{r_j}} = \sigma(r_i - r_j)$$

这是第 5 讲的 Bradley-Terry 形式: r_i 和 r_j 是 reward model 给两个回答打的分, 左边是"回答 i 比回答 j 更好"的概率, σ 是 sigmoid。

- 讲者特意放慢讲的三处 (期末只考第 5-8 讲, 所以这几讲回顾得更细)
 - reward model 的一个细节: 训练时成对喂数据 (这个好、那个差), 让模型对两个回答各出一个分数; 推理时却只给它一个回答、拿一个分数。成对训练、单个使用。
 - RL 损失里的两道约束针对不同的风险: 贴近 SFT 模型是防 reward hacking (SFT 模型本身已经不差, 相当于正则化); 贴近上一轮是防单步更新过大。
 - PPO 和 GRPO 的差别落在"advantage 怎么来"上。value function 估计的是照当前 policy 走下去预期能拿到的奖励, 给奖励提供一个参照; PPO 要训练并维护一个 value model, 再用 GAE 把奖励和 value 预测合成 advantage。GRPO 嫌这太贵, 改用组内比较。它又常用在数学这类答案可验证的任务上, 于是 reward model 也省了。

小注: 那个让回答变长的归一化项, 是 GRPO 损失里按回答长度取平均的 $1/|o|$ ——同样答错, 长回答摊到每个 token 上的惩罚更小。Dr. GRPO (论文副标题里的 GRPO Done Right, Liu et al., 2025) 去掉了它, 也去掉了

advantage 里除以组内标准差的那一项。20 倍经验法则应出自 Chinchilla (Hoffmann et al., 2022)，讲者在回顾里只说是 2020 年代初的一篇论文。

2. 趋势一：Vision Transformer——把图像切成"词"

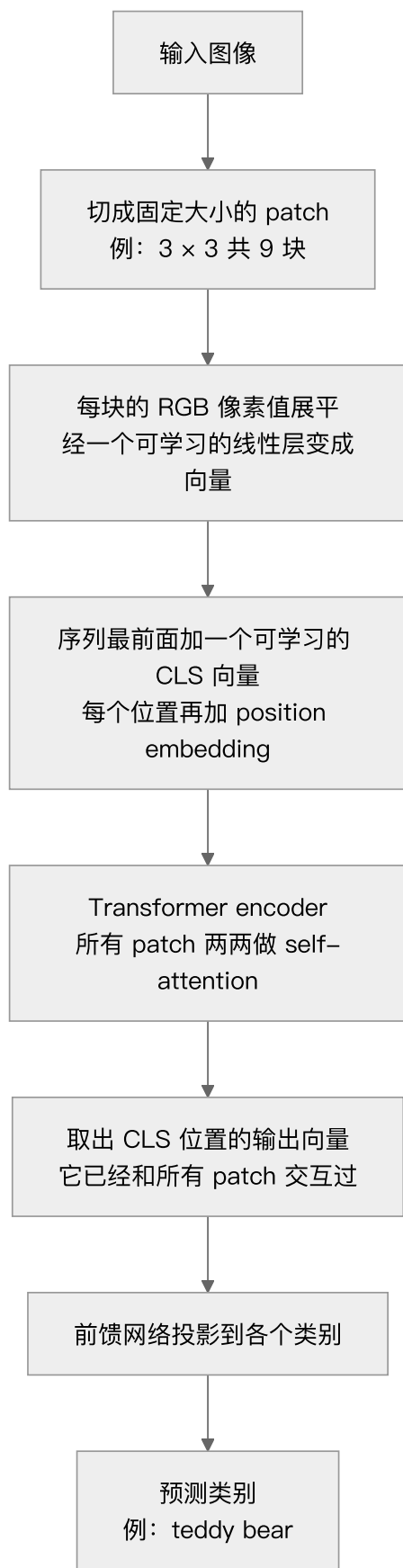


图 9-2 | ViT 做图像分类的数据流（自绘示意）· [▶ 看原幻灯片 55:49](#) · 出处：
Dosovitskiy et al., 2020

- **问题从哪来：**Transformer 先在机器翻译上成功，随后在各种文本任务上都表现很好，自然要问它能不能用在文本之外。讲者的切入点是 self-attention 的定义：一个 query 去看一堆 key 和 value，判断哪些元素和自己相关、据此算出自己的新向量。这个过程里，token 只是向量——如果向量代表的是图像的一小块，机制照样成立。
- **为什么只用 encoder：**任务是图像分类（给一张图，判断属于哪一类），需要的是“读懂”，不是“生成”。课上见过的模型里最适合分类的是 BERT：encoder-only，算出有意义的向量，再投影到类别上。ViT（2020 年）照搬了这个思路。
- **做法（图 9-2）：**把图像切成固定大小的 patch；每个 patch 是一组像素，每个像素有 RGB 三个值，展平后经过一个可学习的线性层变成向量；再加上位置信息，让模型知道每块在图上的哪里；序列最前面放一个可学习的 CLS 向量。整条序列送进 encoder，所有位置互相 attention。最后取 CLS 位置的输出——一来这是沿用 BERT 的习惯例，二来它已经通过 self-attention 看过所有 patch——接一个前馈网络预测类别。

$$z_0 = [x_{\text{cls}}; x_p^1 E; x_p^2 E; \dots; x_p^N E] + E_{\text{pos}}$$

x_p^i 是第 i 个 patch 展平后的像素向量， E 是所有 patch 共用的线性投影矩阵， x_{cls} 是可学习的 CLS 向量， E_{pos} 是位置向量， N 是 patch 的个数； z_0 就是送进 encoder 的输入序列。课上是用话描述的这一步，记号取自 ViT 论文。

- **值得记的发现：归纳偏置不是必需的。**inductive bias（归纳偏置）指模型结构里预先写死的假设，它引导模型往某个方向找规律。CNN 的卷积核在图上滑动、每次只看一小片邻域，这和人看图的方式相近，大家一直认为视觉任务需要这种先验。ViT 让每个 patch 从第一层起就能看到所有其他 patch，几乎没有这种先验。论文的结论是：只要训练的图像数据足够多，ViT 反而胜过 CNN。讲者称这个结果很了不起——先验可以靠数据学出来。

小注：ViT 论文的标准设置是 16×16 像素的 patch，课上的 3×3 指把整张图切成 9 块，只是为了画得下。“数据足够多”在论文里指在 ImageNet-21k（约 1,400 万张）或 JFT-300M（约 3 亿张）上预训练；只在 ImageNet-1k 这种中等规模数据上从头训练时，ViT 比同量级的 ResNet 低几个点。

3. 从“看懂图”到“就图作答”：VLM 的两种接法

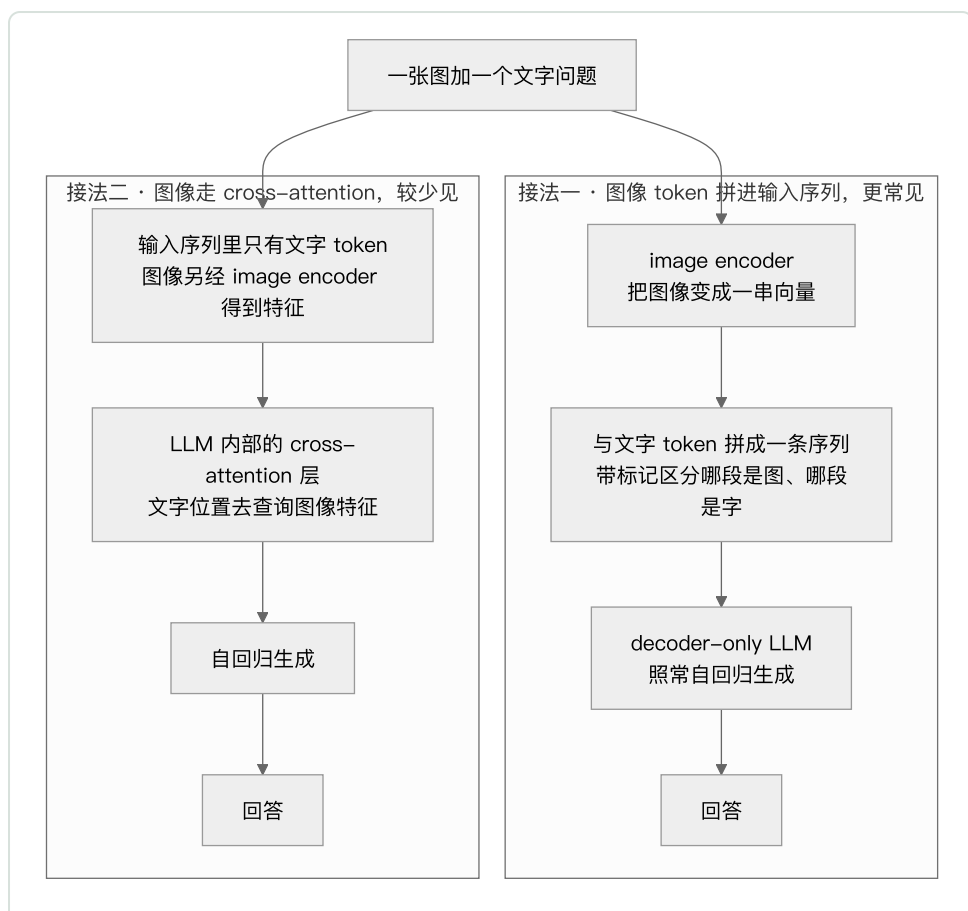


图 9-3 | 让 LLM 就一张图回答问题的两种接法（自绘示意）·

► 看原幻灯片 1:00:03 · 出处：LLaVA、Llama 3

- **场景：**在 ChatGPT 里上传一张图再提问。输入有两种：图像（上一节已经会表示）和文字（token）。这类模型叫 vision language model（VLM）。
- **接法一，更常见：**全部当输入。图像先过一个 encoder 变成一串 token，和文字 token 拼在一起，再用某种标记让模型知道哪些是图、哪些是字；之后就是普通的 decoder-only 自回归生成。讲者举的例子是开放权重的 LLaVA。
- **接法二，较少见：**图像不进输入序列，而是在 cross-attention 层和文字 token 交互。cross-attention 是第 1 讲里 decoder 去看 encoder 输出的那一层，这里被看的对象换成了图像特征。讲者说 Llama 3 的论文里画的是这种接法。

小注：LLaVA (Liu et al., 2023) 用现成的 CLIP 视觉编码器（本身就是一个 ViT）加一个投影层，把图像特征映射到 LLM 的词向量空间。Llama 3 论文 (2024) 里的多模态部分当时注明仍在开发，后来发布的 Llama 3.2 Vision 沿用了 cross-attention 适配层。对照第 1 讲：接法一把图像

当成"更长的 prompt", LLM 的结构不用动, 代价是占用上下文长度; 接法二要在 LLM 里加新层, 但图像不挤占输入序列。

- **文本之外还有哪些:** 课程只讲了文本到文本, 但 Transformer 的用武之地远不止于此。图像理解有 ViT; 图像生成里, diffusion transformer (DiT) 和 multimodal diffusion transformer (MM-DiT) 同样建立在 self-attention 之上; 推荐、语音等领域也在用。讲者的建议是课后对非文本的 Transformer 应用保持开放, 这几个名字可以当入门的线索。

4. 趋势二: diffusion 语言模型 (上) ——把噪声换成 MASK

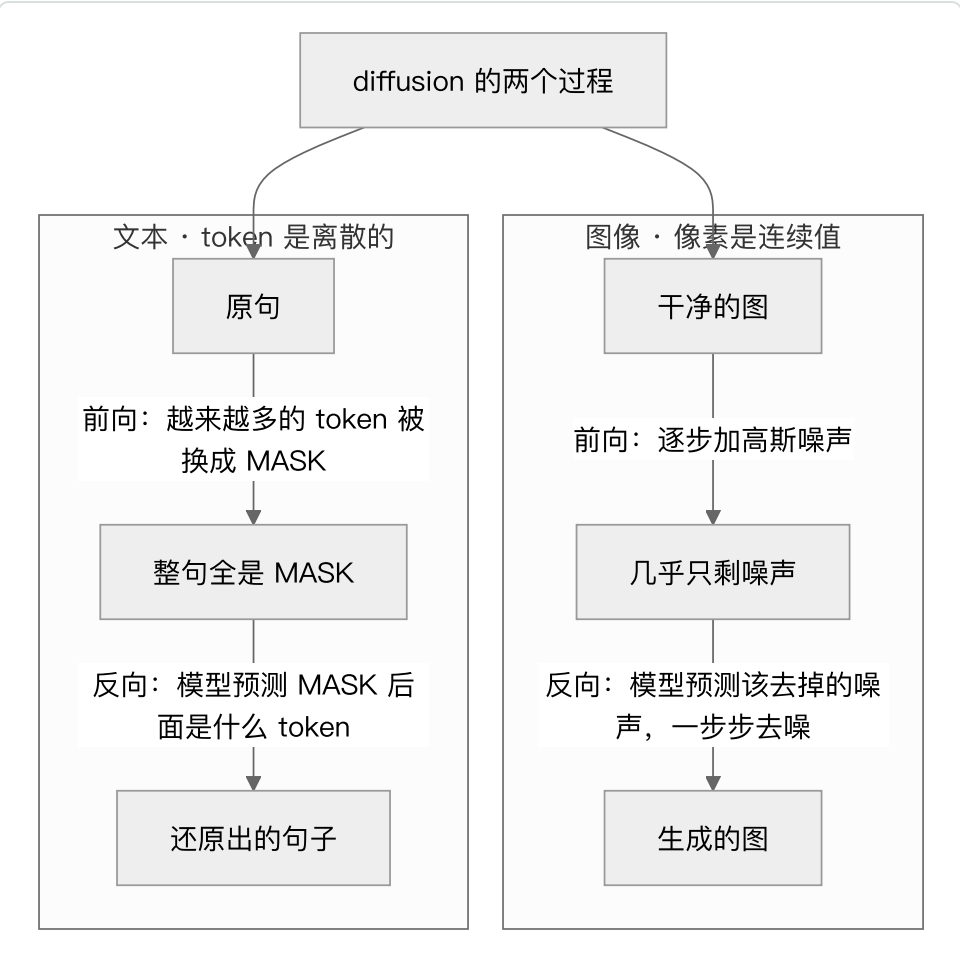


图 9-4 | 图像 diffusion 与文本 masked diffusion 的对应关系: MASK 扮演噪声的角色 (自绘示意) · [▶ 看原幻灯片 1:13:55](#) · 出处: Nie et al., 2025

- **动机: 自回归的推理没法并行。**到这里为止, 课上一直默认 LLM 是自回归的 (文献里记作 ARM, autoregressive model): 根据已有的全部内容预测下一个 token, 拼回去, 再预测下一个, 直到输出 EOS。后一个 token 依赖前面所有 token, 所以推理时只能一个一个出。讲者特意做了区分: 训练是可以并行的——整段目标文本一次喂进去,

causal mask（第 1 讲）保证每个位置看不到后面的 token、没法作弊。不能并行的只是推理。

- **2025 年的信号：**Google 在 I/O 上展示了一个实验性的文本 diffusion 模型，提速很明显；创业公司 Inception 也在走这条路，讲课前约一个月刚上过头条。讲者说这个方向最早的论文出在 2020 年代初，到现在才开始真正好用。

小注：Google 那个模型即 Gemini Diffusion（2025 年 5 月的 I/O 上公布）。Inception 的模型叫 Mercury，[论文摘要](#)称其代码模型在 H100 上每秒输出 1,109 和 737 个 token，比速度优化过的前沿模型平均快至多 10 倍；"一个月前的头条"推断指 2025 年 11 月 6 日宣布的 5,000 万美元融资。

- **两分钟讲 diffusion：**图像生成的通行做法是从噪声出发。为什么是噪声？讲者给了几条理由：逐像素做自回归不可行，像素太多；噪声可以用高斯分布描述，数学性质好，也容易采样；随机的起点带来多样性，同一个要求能生成不同的图。目标是学一个变换，把噪声分布变成目标图像的分布。训练涉及两个过程（图 9-4 左）：前向过程把干净的图逐步加噪，直到几乎只剩噪声；模型学的是反向过程——预测该去掉的噪声，一步步把图还原出来。
- **雕塑家的类比：**石料相当于噪声，每块都不一样；成品相当于目标数据分布；雕塑家要学的是该凿掉哪些部分。讲者借了米开朗基罗的说法来对应：雕像本来就在石头里，要做的只是去掉多余的部分。
- **搬到文本：难点与对应关系。**像素是连续值，可以只加一点点噪声；token 是离散的，没有"加一点噪声"这回事。目前研究给出的对应是：MASK token 之于文本，如同噪声之于图像——MASK 的含义就是这个位置的信息被拿掉了。于是前向过程变成把越来越多的 token 换成 MASK，直到整句全是 MASK；模型学的是把 MASK 还原成原来的 token（图 9-4 右）。这类模型叫 masked diffusion model（MDM），也叫 diffusion LLM（dLLM）；讲者提醒这些叫法还没有固定下来。数学推导课上略过了，想深入可以读 LLaDA。

小注：LLaDA（[Nie et al., 2025](#)）的训练目标很朴素：随机抽一个掩码比例 t ，把每个 token 以概率 t 独立换成 MASK，只在被盖住的位置上算交叉熵，再按 $1/t$ 加权。和第 2 讲 BERT 的 masked language modeling 相比，区别在于掩码比例从 0 到 1 随机取，而不是固定在 15% 左右，所以它能从全 MASK 开始采样，是一个真正的生成模型。摘要称 LLaDA 8B 在 in-context learning 上与 LLaMA3 8B 相当。

5. diffusion 语言模型（下）——推理时怎么跑、快在哪、卡在哪

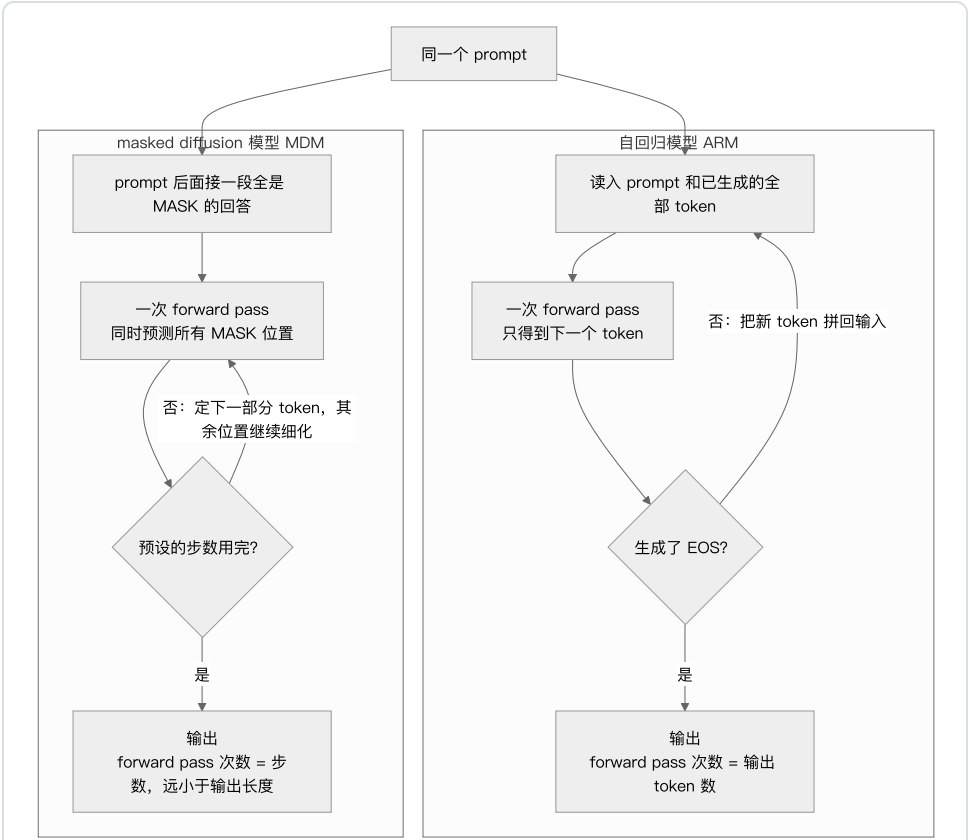


图 9-5 | 同一个 prompt，自回归解码与 masked diffusion 解码各要做多少次 forward pass（自绘示意） · [▶ 看原幻灯片 1:16:28](#) · 出处: Nie et al., 2025

- **推理过程**（图 9-5 右）：prompt 作为条件放在前面，回答部分一开始全是 MASK，模型预测这些 MASK 后面是什么 token，分若干步完成。它可以先定下靠后位置的 token，而靠前的某些位置还空着——这一点和自回归根本不同。
- **直觉：写演讲稿**。讲者坦言自己一开始也想不通：人写东西明明是一个词一个词写的，文本为什么适合用 diffusion 来生成？他的办法是换个例子：写演讲稿不会从第一个字线性地写到最后一个字，而是先列提纲（先讲什么、再讲什么），有了草稿再逐段打磨。diffusion 的生成就是这样由粗到细（coarse-to-fine）的过程。
- **快在哪**：自回归生成要做的 forward pass 次数等于要输出的 token 数；diffusion 只需要做"步数"那么多次，而步数可以事先定好。步数越多质量越高，但通常远小于输出长度——这是它快的根本原因。
- **两个优点**
 - **速度**。输出越长越明显，有的基准测出来大约快 10 倍。讲者点名 coding 场景：一个任务要调很多次模型，用户一直在等，延迟降下

来体验差别很大。多步调用模型的 agent 面对的是同一笔账。

- 生成时把文本当成一个整体来考虑，能同时利用两边的上下文，所以天然适合 fill-in-the-middle 这类任务：给定前后的代码，补出中间缺的那一段。

- 卡在哪：**有一段时间性能赶不上自回归的前沿模型，不过课上提到的几篇论文里差距在缩小。另一条工作线是把围绕自回归模型发展出来的技巧搬过来，比如 reasoning chain（第 6 讲）在 diffusion 里该怎么做——很多技巧天生是为自回归设计的。

小注：课上没有展开每一步具体怎么“定下一部分 token”。LLaDA 的做法是每步对所有 MASK 位置同时给出预测，保留置信度高的，把其余位置重新盖上 MASK 留给后面的步骤（low-confidence remasking）；回答长度要预先给定，是一个超参数。

- 两个趋势合起来看：**课上学的东西可以拿到别的领域去用（Transformer 进入视觉），别的领域的东西也可以拿回文本来用（diffusion 进入 LLM）。讲者说这只是当下动向的一小部分。

6. 收尾（上）：两种模态互相借东西，Transformer 的细节也没有定型

从这里起换 Shervine 讲。他先接着上一段的话头，把文本和图像之间的互相借鉴列了一遍：

借的是什么	方向	课上的例子
架构	图像 → 文本	diffusion 用到文本生成上，换来更低的延迟
架构	文本 → 图像	图像 diffusion 模型里原本用卷积网络，换成 Transformer 后效果更好；新近的图像 diffusion 论文基本都用 Transformer（幻灯片上链接的是 DiT）
输入表示	图像 → 文本	DeepSeek-OCR：用很少的 vision token 就能还原出对应的文字 token
内部技巧	文本 → 图像与多模态	RoPE 改写成 2D：在二维网格上分配位置，文字 token 也摆进同一套坐标，相对位置的计算依然成立

- DeepSeek-OCR 为什么值得一提：**名字里的 OCR（光学字符识别，把扫描图像转成文字）容易误导。讲者说这篇论文的看点不在 OCR 任务本身，而在它表明：可以学到一个函数，从很少的 vision token 重建出文本 token——图像 patch 当 token 用，表示能力很强。有研究

者顺势给出理由：tokenizer 本来就不是理想的工具；图像块天然承载文字的含义，比如 emoji，用文本来表示反而要更多的 token。

小注：DeepSeek-OCR (2025 年 10 月) 摘要里的数字：文本 token 数在 vision token 的 10 倍以内时，解码精度约 97%；压到 20 倍时仍有约 60%。论文把它定位成"用视觉模态压缩长上下文"的初步探索。"有研究者"推断指论文发布后 Andrej Karpathy 等人的公开评论。

- **2D RoPE 为什么能成立**：RoPE 的要点是 attention 分数只取决于相对位置（第 2 讲）。图像 patch 的位置有行和列两个坐标，只要分别按行距和列距去旋转，相对关系照样保得住；图文混排时，再给文字 token 在同一套坐标里安排一个位置即可。幻灯片上的图画的就是这种分配方式。

小注：那张图课上没有点名出处。这个思路的一个代表是 Qwen2-VL 的 M-RoPE (Wang et al., 2024)：把旋转用的维度分给时间、高、宽几个分量；纯文字 token 的几个分量取相同的值，于是退化回普通的 1D RoPE。

- **还在迭代的设计决定**。Shervine 的意思是：研究远没有结束，今天用的每个细节都还在被改。
 - optimizer：Adam 长期是默认选择，现在受到挑战。讲者举的是 Kimi K2 论文里的 Muon 及其新版本 MuonClip，认为它有可能成为新的标准。

小注：Muon 最早由 Keller Jordan 等人在 2024 年提出；Kimi K2 (2025 年 7 月) 的贡献是 MuonClip——在 Muon 上加 QK-clip 压住训练不稳定。摘要称这个 1T 总参数（激活 32B）的 MoE 模型用它训了 15.5T token，没有出现 loss spike。
 - 归一化：位置从 post-norm 变成 pre-norm；种类也在变，原版用 LayerNorm，现在常见参数更少的 RMSNorm。背后的理论还没有定论。
 - attention：GQA 之后并没有统一的设计，各家论文各用各的；有的模型在某些层用一种 attention，到别的层又换一种。
 - 激活函数：深度学习里长期用简单好用的 ReLU；LLM 转向了"像 ReLU 但不完全是"的函数，比如 GELU，新的还在不断出现。
 - 要不要做成 MoE，多少层、多少个 head、FFN 多宽——都还在争论。
- **连架构本身也未必是终点**：Transformer 是不是最好的架构并不清楚，这本身就是一个研究方向，未来的突破可能来自重新设计它。

7. 收尾（中）：数据、成本与硬件

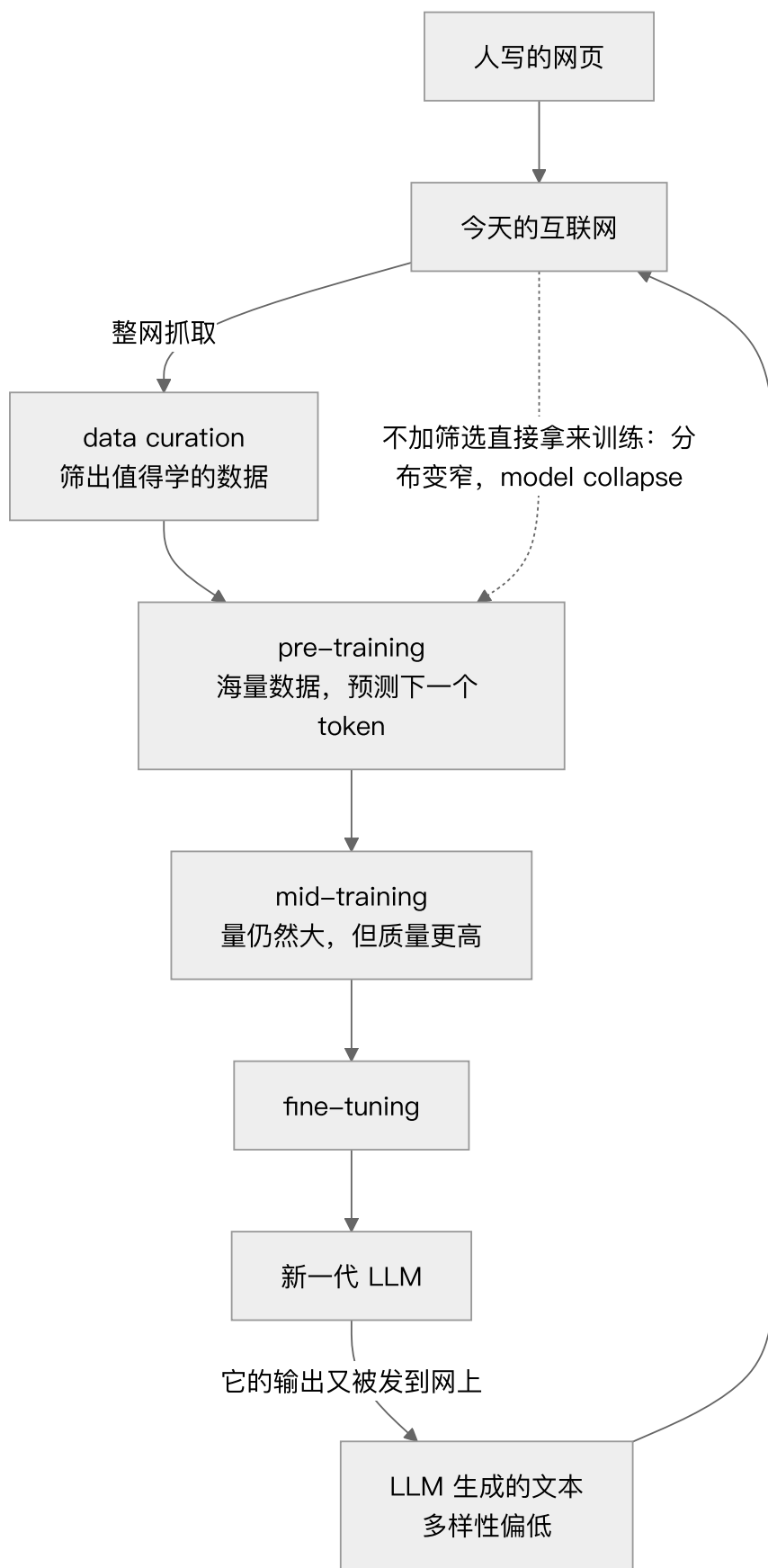


图 9-6 | LLM 生成的文本回流进训练数据的回路，以及 data curation 和 mid-training 所处的位置（自绘示意）· ▶ 看原幻灯片 1:33:15 · 出处：Shumailov et al., 2023

- **数据：**早期的 LLM 占了一个便宜——爬下整个互联网，拿到的基本都是人写的内容，格式不干净，但来源可靠。现在随便搜点什么，排在前面的结果里已经有大量 LLM 生成的文本（讲者随口估了个八成）。他认为并非没救，理由有两条。一是 data curation（数据的筛选与整理）越来越受重视，已经有公司专门做这件事。二是训练流程多了一段：从 pre-training → fine-tuning 变成 pre-training → mid-training → fine-tuning，mid-training 的语料量仍然很大，但质量更高。幻灯片上链接的论文讲的是 model collapse：LLM 生成的文本多样性偏低，拿它来训练，模型见到的数据分布就变了，能学到的东西变少（图 9-6 的虚线）。

小注：这篇应为 Shumailov 等人的工作（arXiv 版，2023；2024 年发表于 Nature）。摘要的说法是：用模型生成的内容训练会造成不可逆的缺陷，原始分布的尾部会消失。

- **成本：**过去几年的研究主线是把 benchmark 刷得更高。假如我们关心的用例都被解决了，下一步是什么？Shervine 的判断是 Pareto 前沿的另一条边会变得重要：在保持高质量的前提下，让每次推理更便宜。迹象有两个：small language model (SLM) 这个叫法的出现；LLM 供应商自己说，连最高档的订阅也在亏钱——推理侧的算力必须花得更聪明。
- **硬件：**全课几乎没碰的一块。GPU 真正擅长的只有矩阵乘法，而 Transformer 有自己的特殊需求： QK^T 这一步很贵；FlashAttention 为了少搬数据宁可重算；大量优化工作研究的都是数据在 GPU 各级内存之间怎么流动。这说明也许需要更贴合的硬件。讲者介绍了一篇概念验证论文：把这些运算直接做进硬件，用模拟信号实现输入输出，数组里的数值用脉冲表示，计算结果是电路物理性质的副产品（比如基尔霍夫定律让电流自然相加），最后只要把结果读出来。论文在仿真里看到延迟和能耗都明显下降——计算不用自己做，是硬件顺带给的。

小注：应为 Leroux 等人的 analog in-memory computing attention（arXiv 2409.19315）：用 gain cell 这种基于电荷的存储单元存放 K、V 投影，并就地以模拟方式并行完成 attention 的点积。arXiv 摘要称 attention 的延迟和能耗相对 GPU 最多可降两个和五个数量级。预印本出

在 2024 年 9 月，正式版 2025 年刊于 Nature Computational Science；讲者说的"9 月"应指正式版。

8. 收尾（下）：LLM 今天用在哪、接下来往哪走、怎么跟进

• 今天的用例

- 写代码：AI 编程助手的 agent mode 把自然语言变成代码。
- 很多问题可以转化成 text-to-query 或 text-to-code，连可视化也是：讲者提到 Google 最近的一次发布，能按一些原则即时生成可视化界面。

小注：推断指 2025 年 11 月随 Gemini 3 推出的 generative UI (Gemini app 里的 dynamic view 和 visual layout)。

- 通用助手：问常识，让它替你浏览网页、再用自然语言转述。
 - 创作类工作的起稿：从草稿改，比从白纸写容易。
 - 学习：Shervine 说他看到有学生边听课边在 ChatGPT 里追问相关概念，他很赞成——及早拿到反馈回路对掌握概念很有用，现在是学东西的好时候。
- **明天**：agentic workflow 目前还局限在懂行的人手里，下一步是大众化——不用写代码，用自然语言就能搭出对自己有用的东西。讲者说幻灯片上写的是"明天"，可就在两天前已经有一个发布朝这个方向走了。

小注：按日期推断是 Google Workspace Studio (2025 年 12 月 3 日发布，用自然语言搭建 Workspace 里的 agent)。

• 再往后

- AI 替你浏览网页。现在人做任务的方式太"微观"，这类琐碎操作正适合交给助手。代表产品是 10 月发布的 ChatGPT Atlas。讲者猜它的使用量还不小，障碍在安全：任何人都可能在网页里埋一段恶意 prompt (prompt injection)，借此把你的数据偷走。他的类比是 HTTPS——当年用它表明连接是安全的，将来也许会有某种证书，表明一个网站可以放心让 AI 助手浏览。
- 再高一层：LLM 进到操作系统这一级，在桌面或手机上替你操作。第 7 讲说过 agent 不够可靠：步骤越多，出错的概率越大，怎么让预测稳定下来是个重要课题。

小注：一个粗略的算法：假设每步成功率是 p 且相互独立， n 步全对的概率是 p 的 n 次方； $p = 0.95$ 、 $n = 20$ 时只剩约 36%。

- 更长远的试金石：AI 客服是否真的有用。Shervine 说自己每次打电话听到 AI 客服，第一反应都是赶紧转人工。人能带来的价值维度远多于 LLM：同理心、对现实的把握，还有很多没写进 system prompt、但大家都觉得理所当然的东西。

• 当前架构下的开放问题

- 持续学习：训练结束后权重就固定了，RAG 和工具（第 7 讲）都是绕开这个限制的办法。能不能有一个持续学习的系统，是开放问题。
- "幻觉"：Shervine 给这个词打了引号。模型被训练来预测下一个 token，而不是把陈述对应到事实；从这个角度看，"幻觉"是这种设计的直接后果，算不上故障。
- 还有个性化、可解释性、安全，清单很长。

- 怎么保持更新：arXiv 上有最新的论文；NeurIPS 这样的会议（讲课那周正在开）会帮你筛出值得看的；除了论文，强烈建议读作者放出来的代码，附上实现如今已是惯例；Papers with Code 已经被 Hugging Face 的 Trending Papers 取代；X 上有活跃的社区；YouTube 上推荐 Yannic Kilcher（2017 年就细讲过 Transformer 论文）和 Andrej Karpathy（约十年前在 Stanford，讲者眼里最好的教育者之一）；各公司的技术博客；还有课程配套的 study guide，两位讲者打算至少每年更新一次，并且已有多种语言的译本。

关键图表速查（点时间戳跳到原幻灯片）

图	看什么	跳转	出处
self-attention 的 Q / K / V 示意与矩阵公式	一个 query 对所有 key 打分、再对 value 加权平均；同一张图在 50:12 又被拿来引出 ViT	4:49	Vaswani et al., 2017
scaling laws 三联图	纵轴是 test loss，三张分别对应算力、数据量、参数量，都是越多越低	15:51	应出自 Kaplan et al., 2020
RLHF 回路	prompt → rollout → reward model 打分 → 更新权重；损失里的两道约束各防什么	27:13	—
PPO 与 GRPO 对照图	GRPO 一侧没有 value model；一组回答的奖励互相比得到 advantage	33:01	应出自 DeepSeekMath
RAG 的两步检索	bi-encoder 召回与 cross-encoder 精排，输入的组织方式有何不同	40:22	—

图	看什么	跳转	出处
ViT 端到端示例	小熊图片被切块、变向量、加 CLS 与位置；CLS 的输出接分类头	55:49	ViT
VLM 的两种接法	图像 token 是拼在输入序列里，还是从 cross-attention 层进入	1:00:03	LLaVA、Llama 3
2025 年的发布截图	Google I/O 上的文本 diffusion 模型、Inception 的报道——这个方向热起来的旁证	1:07:12	—
图像 diffusion 的两个过程	前向逐步加噪，反向预测该去掉的噪声	1:11:51	—
文本版的前向过程与推理	MASK 逐步增多直到全 MASK；推理时带着 prompt 从全 MASK 往回还原	1:14:26	LLaDA
2D RoPE 示意	位置怎样分配到二维网格上，文字 token 又摆在哪	1:27:33	课上未点名
仍在迭代的设计决定清单	optimizer、归一化、attention、激活函数、MoE、各类超参，逐项对照今天的做法	1:28:35	Kimi K2 等
模拟电路里的 attention	运算被做进硬件，结果直接读出；延迟与能耗的对比	1:38:26	应为 Leroux et al.
保持更新的资源页	论文、代码、社区、视频、study guide 各去哪找	1:47:40	—

提到的工作

名称	在本讲里的作用
word2vec、RNN	回顾第 1 讲：不看上下文的词向量、留不住远距离信息的循环结构，引出 self-attention
Transformer (2017)	全课的地基；本讲两个趋势都从它出发
RoPE、GQA、pre-norm	回顾第 2 讲的三项改进；RoPE 在收尾部分以 2D 形式再次出现
BERT、GPT、T5	三个家族的代表；BERT 的 CLS 做法被 ViT 直接沿用
MoE	回顾第 3 讲：FFN 处的稀疏专家；收尾部分又作为仍有争议的设计选项出现
scaling laws、Chinchilla (应为)	回顾第 4 讲：越大越好，以及 20 倍 token 的经验法则
FlashAttention	回顾第 4 讲；在硬件话题里再次被引用，说明瓶颈在数据搬运
Bradley-Terry、reward model、PPO、GAE	回顾第 5、6 讲：偏好建模与带 value model 的 RL

名称	在本讲里的作用
GRPO (DeepSeekMath)、Dr. GRPO、DAPO	回顾第 6 讲：组内比较的 advantage、长度偏差及其修正
chain-of-thought、AIME	reasoning chain 的思路来源；衡量推理训练进展的数学 benchmark
RAG、bi-encoder 与 cross-encoder、tool calling	回顾第 7 讲；收尾部分说它们是在绕开"权重训练后就固定"这个限制
BLEU / ROUGE / METEOR、LLM-as-a-judge	回顾第 8 讲：规则指标的局限与 judge 的三种偏差
ViT (2020)	趋势一的主角：patch 当 token、encoder-only、CLS 分类
CNN	ViT 的对照：自带很强的视觉归纳偏置
LLaVA	VLM 接法一的例子：图像 token 与文字 token 拼接
Llama 3	VLM 接法二的例子：图像经 cross-attention 进入
DiT、MM-DiT	图像生成里的 Transformer；也是"文本借给图像"的架构例子
Google 的实验性文本 diffusion 模型 (Gemini Diffusion)	2025 年 dLLM 升温的信号之一
Inception (Mercury)	做 dLLM 的创业公司，另一个信号
LLaDA	讲者推荐的 dLLM 入门论文，含课上略过的数学
DeepSeek-OCR	很少的 vision token 就能还原文字——图像 patch 作为输入表示的潜力
Adam、Muon / MuonClip (Kimi K2)	optimizer 这种基础部件也还在变
LayerNorm、RMSNorm、ReLU、GELU	归一化与激活函数的更替
model collapse (Shumailov et al., 应为)	用 LLM 生成的数据训练会怎样；data curation 的动机
SLM	成本—质量前沿上的小模型路线
analog in-memory attention (Leroux et al., 应为)	为 attention 定制硬件的概念验证
ChatGPT Atlas	AI 浏览器；引出 prompt injection 的安全问题
Hugging Face Trending Papers、Papers with Code	跟进论文与代码的去处，后者已被前者取代
Yannic Kilcher、Andrej Karpathy	讲者推荐的两位 YouTube 讲解者

术语对照

English	中文
subword tokenizer	子词分词器（把词拆成可复用的片段）
long-range dependency	长程依赖（隔得很远的 token 之间的关系）
rotary position embeddings (RoPE)	旋转位置编码
grouped-query attention (GQA)	分组查询注意力（多个 head 共用 K / V）
post-norm / pre-norm	归一化放在子层之后 / 之前
mixture of experts (MoE)、gating	混合专家、门控路由
greedy decoding / sampling / temperature	贪心解码 / 采样 / 温度
undertrained	训练不足（模型相对数据量太大）
HBM / SRAM	GPU 上又大又慢 / 又小又快的两级内存
recomputation	重计算（不存中间结果，用时再算）
data / model parallelism	数据并行 / 模型并行
pre-training / mid-training / SFT / preference tuning	预训练 / 中期训练 / 监督微调 / 偏好调优
policy、rollout、reward hacking	策略、一次完整的生成、钻奖励的空子
value function、advantage	价值函数（预期奖励的参照）、优势（比参照好多少）
verifiable reward	可验证奖励（答案能直接核对，无需 reward model）
knowledge cutoff	知识截止日期
bi-encoder / cross-encoder	双塔编码器（各自编码后比相似度） / 交叉编码器（query 与文档一起编码）
rationale	评分理由（judge 在给分前先写出来）
position / verbosity / self-enhancement bias	位置 / 冗长 / 自我偏好偏差
patch	图像块（ViT 里的 token）
inductive bias	归纳偏置（写死在模型结构里的先验假设）
vision language model (VLM)	视觉语言模型
cross-attention	交叉注意力（一路序列去查询另一路）
autoregressive model (ARM)	自回归模型（逐 token 生成）
forward / reverse process	前向过程（加噪或加 MASK） / 反向过程（去噪或还原）
masked diffusion model (MDM)、diffusion LLM (dLLM)	掩码扩散模型、扩散语言模型
coarse-to-fine refinement	由粗到细的逐步细化
fill-in-the-middle	中间填空（给定前后文补中段，多见于代码）

English	中文
data curation	数据筛选与整理
model collapse	模型坍缩（反复用模型生成的数据训练导致分布变窄）
Pareto frontier	帕累托前沿（这里指成本与质量无法同时再改进的边界）
small language model (SLM)	小语言模型
analog in-memory computing	模拟存内计算（在存储单元里直接用模拟电路做运算）
prompt injection、exfiltrate	提示注入、窃取数据外传
continual learning	持续学习（部署后还能继续更新知识）

字幕勘误

"CM295" → CME295；"wordtovec" → word2vec；"birds" → BERT；"rope" → RoPE；"HPM""SRAMM" → HBM、SRAM；"PO""PPU" → PPO；"GRPL" → GRPO；"AIM benchmark" → AIME；"GRPO done rights" → GRPO Done Right（即 Dr. GRPO）；"depo dapo" → DAPO；"blur, rouge, meor" → BLEU、ROUGE、METEOR；"by encoder" → bi-encoder；"lava" → LLaVA；"LADA" → LLaDA；"mass tokens" → mask tokens；"paralyzable""paralyze" → parallelizable、parallelize；"indictive bias" → inductive bias；"Kim K2" → Kimi K2；"atom optimizer" → Adam optimizer；"the LLM as ane" → as an MoE；"Parto Frontier" → Pareto frontier；"kirhoff" → Kirchhoff；"JGPT" → ChatGPT；"used as an eight" → as an aid；"new rips" → NeurIPS；"Yanik Kilchshire" → Yannic Kilcher；"Andrish Karpath" → Andrej Karpathy；"Afin""Ashin""Ein" → Afshine；"Shervin" → Shervine。

带走的问题

- ViT 几乎没有归纳偏置，靠大数据赢了 CNN。数据不够多的时候怎么选？"少先验、多数据"这条经验的边界在哪——在你自己的任务上，数据量落在哪一侧？
- VLM 的两种接法各自的代价是什么：哪一种占用上下文长度，哪一种要改动 LLM 本体？输入是多张图或一段视频时，账会怎么变？为什么

多数模型选了把图像 token 拼进输入这条路？

3. dLLM 的步数是速度与质量之间的旋钮。同一步里同时定下的多个 token 彼此看不到对方，这对要求严格前后一致的输出（代码、数学推导、工具调用的 JSON）意味着什么？回答长度要预先给定，又会带来什么限制？
4. 围绕自回归模型的一整套配套——KV cache、reasoning chain、RL 后训练、流式输出——有多少能平移到 dLLM？对一个多步 agent 来说，单步延迟降到十分之一、单步质量略降，总体成功率是升还是降（对照第 8 节的成功率小注）？
5. 互联网被 LLM 文本充斥之后，data curation 和 mid-training 能从根本上避开 model collapse 吗？后训练里又在大量主动使用合成数据（第 5、6 讲），这两件事的区别在哪？
6. Shervine 说权重训练后固定、RAG 和工具只是绕路，又说“幻觉”是 next-token 目标的直接后果。如果接受这两个判断，RAG、verifiable reward、LLM-as-a-judge 各自补的是哪一块？一个真正持续学习的系统至少要改掉什么？